

NESHBOY SOC PROFESSIONAL LIBRARY

THE SOC HOME LAB HANDBOOK

Building a Personally-Operated, Isolated Practice Lab —
SIEM, Endpoints, Network Monitoring, Honeypots, and Safe
Attack Simulation

Build date: 2026-09-15

Table of Contents

CHAPTERS

Section A — Foundations, Scope & the Safety Architecture

- Part 1 — The SOC Home Lab: What It Is, and What It Deliberately Isn't
- Part 2 — Lab Architecture Patterns and Topology Decisions
- Part 3 — Hardware and Resource Budgeting
- Part 4 — Network Isolation and Segmentation Architecture

Section B — Core Platform Build

- Part 5 — Choosing and Installing a Hypervisor
- Part 6 — Perimeter Firewall and Segmentation Build
- Part 7 — Internal DNS and Core Network Services
- Part 8 — Building the SIEM Platform

Section C — Endpoint Telemetry

- Part 9 — Linux Endpoints and auditd Deployment
- Part 10 — Windows Endpoints and Sysmon Deployment
- Part 11 — The Windows Domain Lab Problem: Scope, Cost, and an Honest Failure

Section D — Network Security Monitoring

- Part 12 — Packet Capture and Traffic Visibility Fundamentals
- Part 13 — Deploying Zeek for Network Security Monitoring
- Part 14 — Deploying Suricata for Intrusion Detection

Section E — Deception and External Signal

- Part 15 — Honeypots and HoneyNet Architecture
- Part 16 — Threat Intelligence Feed Integration

Section F — Practice: Simulation, Dashboards & Log Generation

- Part 17 — Dashboards and Visualization Build
- Part 18 — Safe Attack Simulation with Atomic Red Team-Style Tooling
- Part 19 — Generating Practice Telemetry and Purple-Team Drills

Section G — Operating, Scaling & Closing

- Part 20 — Lab Maintenance, Patching, and Snapshot/Backup Strategy
- Part 21 — Storage Growth, Retention, and Long-Term Cost Management
- Part 22 — From Lab to Practice: Closing Synthesis and the Series Map

CHAPTERS

Section A — Foundations, Scope & the Safety Architecture

neshbooy

Part 1 — The SOC Home Lab: What It Is, and What It Deliberately Isn't

Why this part exists

[CONCEPT] The other three volumes in the NESHBOY SOC Professional Library assume telemetry already exists: a SIEM to search, endpoints producing logs, alerts to triage, a team to staff. This book is where a reader who has none of that starts. It is a construction manual for a personally-operated, safely isolated practice environment — a SIEM, Windows and Linux endpoints producing real Sysmon and auditd telemetry, a segmented network, Zeek and Suricata watching the wire, honeypots collecting unsolicited attacker traffic, threat-intel feeds enriching what comes in, dashboards worth looking at, and safe, reversible attack-simulation tooling to generate known-cause telemetry on demand. Every part after this one builds one piece of that environment, in an order chosen so each piece has somewhere to send its output before it exists.

This part does three things before any build content starts. First, it draws the boundary between what this book covers and what the other three volumes cover, so a reader doesn't go looking for detection-query syntax or incident-response procedures here and come away thinking the book is thin — it isn't thin, it's scoped. Second, it states the one rule every later part depends on: you build a lab to attack and defend systems you own, on a network segment that cannot reach, and cannot be reached by, anything you don't own. Third, it introduces the vocabulary — six content tags, eight recurring callouts, four evidence classes — that every subsequent part uses without re-explaining.

Safety Gate *Every part after this one assumes a lab network with zero route back to a network holding real credentials or reachable production systems, and zero unsolicited-inbound exposure to the open internet beyond what a deliberately-exposed decoy service needs. This part doesn't build that isolation — Part 4 does, and Appendix A4 is the pre-flight checklist you run before powering on anything intentionally vulnerable — but nothing past this page assumes you'll get to it "eventually." If you're tempted to skip ahead to Part 9, 10, 15, or 18 because that's where the interesting tools are, read Part 4 first. A honeypot or an attack-simulation tool built on top of unverified isolation isn't a shortcut; it's the specific failure mode this rule exists to prevent.*

1. What this book actually is

[CONCEPT] This is a hands-on build book for a single operator — a hobbyist, a student, or a working analyst — standing up a lab on hardware they own, sized realistically against what that hardware can actually run. It does not assume a budget, a team, or prior virtualization experience, but it does assume you're willing to install operating systems, edit configuration files, and read an error message instead of clicking past it. Every part states what to install, what to click, what to check, and what breaks if you skip

a step — that's the promise a construction manual makes, and the parts that follow are graded against whether they keep it, not against how interesting the subject sounds.

The four volumes of the NESHBOY SOC Professional Library split by a different axis than most security-book series: not beginner/intermediate/advanced, but what job the reader is doing right now. *The SOC Home Lab Handbook* (this book) builds the environment. *The Detection Engineering Handbook V2* teaches what the telemetry that environment produces means, and how to turn it into working detection logic. *SIGNAL TO ACTION: The Complete SOC Playbook Handbook* teaches what to do when a detection fires — the triage and response procedure. *The SOC Manager's Operating Handbook* teaches how to run the team, budget, and process around all of that. A reader can enter at any volume, but this one has to come first for anyone with no lab at all, because the other three assume the telemetry is already flowing.

This book is written by, and partly evidenced from, an author who actually operates a lab of this kind — a Proxmox-based environment with a Pi-hole resolver, a honeynet, and several monitored Linux containers, cited throughout *The Detection Engineering Handbook V2* as REAL LAB EXAMPLE evidence. Where a part draws on that running lab directly, it says so and tags the figure REAL LAB EXAMPLE. Where it doesn't — most of the Windows and Active Directory content — it says that too, rather than blur the line. Section 8 below covers this in full, because getting it wrong in either direction (fabricating evidence, or hiding a real result behind vague language) is the specific defect this book's own review checklist exists to catch.

2. What this book deliberately isn't

[CONCEPT] Three things this book does not teach, on purpose, because another volume in the series already owns that content and duplicating it would cost a reader two inconsistent explanations of the same idea instead of one good one:

- **Not a detection-logic book.** Once Part 9 has your Linux endpoint forwarding auditd events, or Part 10 has Sysmon running on a Windows host, this book stops at "the log arrives and is searchable." What a specific Sysmon Event ID 1 (Process Create) entry means, how to write a correlation rule against it, how to baseline normal behavior against attacker behavior — that's *The Detection Engineering Handbook V2*'s job, and it's a much deeper treatment than a paragraph here could give it.
- **Not a playbook-response book.** Part 15's honeynet will generate real, unsolicited attack sessions, and Part 18's attack-simulation tooling will generate real, on-demand technique executions. Neither part tells you how to run an incident-response procedure against what you catch. *SIGNAL TO ACTION: The Complete SOC Playbook Handbook* owns that.
- **Not a people/ops book.** Nothing here covers staffing a SOC, budgeting for one, or running a vendor RFP. Part 3's hardware budgeting is a hobbyist's dollars and watts, not an enterprise capital-planning process — *The SOC Manager's Operating Handbook* owns the real version of that conversation.

The table below states, part-scope granular, what each of the four volumes owns and where this book hands off to it — worth bookmarking, since every later part repeats a version of this hand-off at the specific point it applies.

Volume	Owns	Doesn't cover here	This book hands off at
This book (SOC Home Lab Handbook)	Building the SIEM, endpoints, network segmentation, NSM sensors, honeypots, and safe attack-simulation tooling	What a fired alert means; how to respond to one; staffing and budget	Parts 9–19, once telemetry exists to interpret
Detection Engineering Handbook V2	Detection rule logic, correlation, baselining, threat-intel scoring, hunt methodology	How to install the SIEM producing the telemetry it analyzes	Its own Parts 3, 8–9, 22–36 — cited by part number throughout this book
SOC Playbook Handbook	Alert triage and response procedures, the Playbook Library	How the telemetry behind an alert was generated or collected	Cited wherever this book's honeypot or simulation output would, in a real SOC, trigger a response
SOC Manager's Operating Handbook	Staffing, budget, vendor contracts, drill design, Field Test framing	How to physically build any of the components a manager is budgeting for	Part 22's closing synthesis, and Part 3's loose budgeting analogy

3. The one non-negotiable rule: own it, isolate it

[SAFETY] Everything else in this book is a recommendation you can weigh against your own hardware, budget, and curiosity. This rule isn't one of those. You build a lab to attack and defend systems you own, on a network segment that cannot reach, and cannot be reached by, anything you don't own. No part in this book instructs you to point a scanner, an exploit, or an "attack simulation" tool at a system you don't own or don't have explicit authority to test — Part 18 restates this rule on its own, in full, because it's the part most likely to get skipped to directly.

3.1 What "own" means here

[SAFETY] "Own" means hardware and virtual machines under your direct administrative control, on a network you control end to end. It doesn't mean a shared university lab, a friend's home network you have Wi-Fi access to, or a cloud account where "the lab" is one VM on a provider's shared infrastructure with terms of service you haven't read closely. Part 21's provenance notes explain why this book keeps cloud lab-building out of the main part list entirely: a cloud tenant introduces real billing and a real provider's acceptable-use policy, and the "isolated home network" safety story this book is built around doesn't transfer cleanly to a shared platform you don't administer at the hardware layer.

3.2 What "isolated" means here

[SAFETY] "Isolated" means the lab segment has no route to any network holding a real credential, a real financial account, or a device you'd be upset to lose — and, for anything intentionally vulnerable (a honeypot, an unpatched practice target, a live exploit run against a lab endpoint), no route to the open internet except the one specific inbound path a decoy service is designed to expose. Part 4 builds this for real, with VLANs, an isolated virtual switch, and default-deny egress; Part 6 builds the firewall rules that enforce it; Appendix A4 is the checklist you run to confirm it actually holds before anything risky goes live. Figure 1.1 previews the shape of that architecture before any of those parts build it.

Figure 1.1 — Target lab isolation topology (preview). *CONCEPTUAL.* Illustrates the segmentation this book builds toward, not a capture of a running system — Part 4 designs it, Part 6 builds the firewall rules that enforce it, and Figure 6.1 shows the same topology after a real pfSense/OPNsense build. The two dashed paths are the ones this book's Safety Gate boxes exist to keep honest: no route between the home network and the lab firewall in either direction, and an inbound-only path from the internet to the honeynet VLAN limited to the specific ports its decoy services listen on. Diagram ID [FIG-01-01](#).

```

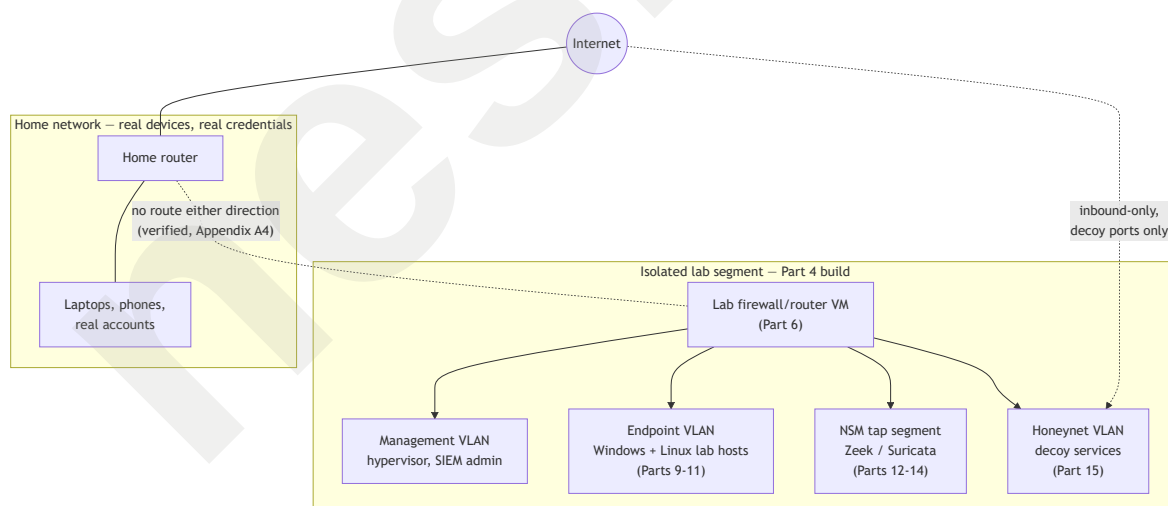
flowchart TB
    subgraph HOME["Home network – real devices, real credentials"]
        ROUTER[Home router]
        REALDEV["Laptops, phones,\nreal accounts"]
        ROUTER --- REALDEV
    end

    INTERNET(("Internet"))

    subgraph LAB["Isolated lab segment – Part 4 build"]
        LABFW["Lab firewall/router VM\n(Part 6)"]
        MGMT["Management VLAN\nhypervisor, SIEM admin"]
        ENDPOINT["Endpoint VLAN\nWindows + Linux lab hosts\n(Parts 9-11)"]
        NSM["NSM tap segment\nZeek / Suricata\n(Parts 12-14)"]
        HONEY["Honeynet VLAN\ndecoy services\n(Part 15)"]
        LABFW --> MGMT
        LABFW --> ENDPOINT
        LABFW --> NSM
        LABFW --> HONEY
    end

    INTERNET --- ROUTER
    ROUTER -. "no route either direction\n(verified, Appendix A4)" .-> LABFW
    INTERNET -. "inbound-only,\ndecoy ports only" .-> HONEY

```



What Would Change My Mind This part treats "own it, isolate it" as a hard, non-negotiable rule rather than a strong default a reader can weigh against convenience. The one thing that would change that framing is a documented case where a partially-isolated lab (say, a honeypot on a shared VLAN with a bandwidth cap but no true segmentation) caused zero real harm across a large enough sample of readers to say the risk was overstated — and even then, the fix would be narrowing the rule's scope with evidence, not loosening the "never target a system you don't own" half of it, which isn't really a home-lab-specific claim at all.

4. Six tags, one build phase at a time

[CONCEPT] This book's readers differ by what phase of building they're in, not by job title — a hobbyist and a working analyst standing up the identical Wazuh install need the same information in the same order. Every paragraph in this book carries one of six bracketed tags marking which phase it belongs to:

Tag	Answers the question	Example from later in this book
[CONCEPT]	Why does this component exist, and what's the architecture behind it?	Why Zeek watches a mirrored switch port instead of sitting inline
[SETUP]	What do I click, install, or edit, in order?	Installing Wazuh's manager and indexer packages (Part 8)
[HANDS-ON LAB]	What's the exercise, and how do I know it worked?	Running one Atomic Red Team technique and confirming the matching Sysmon event (Part 18)
[TROUBLESHOOTING]	What specifically broke, and what's the fix?	Zeek's <code>conn.log</code> staying empty because the vSwitch isn't in promiscuous mode (Part 12)
[SAFETY]	What must be true about the network before I proceed?	Confirming a honeynet VLAN has no route back to the management network (Part 15)
[COST/RESOURCE]	What does this cost in RAM, disk, or dollars, and what breaks below that?	Budgeting 8GB for Wazuh's indexer alone, before adding a single endpoint (Part 8)

A single `##` section in this book usually carries several of these across its subsections — a section on deploying Zeek plausibly opens **[CONCEPT]**, sizes itself **[COST/RESOURCE]**, walks the install **[SETUP]**, verifies it **[HANDS-ON LAB]**, and closes with the two most common install failures under **[TROUBLESHOOTING]**. Part 1 is the exception: it's almost entirely **[CONCEPT]** and **[SAFETY]**, because there's nothing to install yet.

5. Eight callouts, and one that's never optional

[CONCEPT] Layered on top of the six tags are eight recurring callout boxes, each with one fixed format defined in `STYLE-GUIDE.md`. Seven of them are opportunistic — used where a section genuinely has that kind of thing to say, skipped where it doesn't.

Callout	Fires when
Build Autopsy	A real or realistic build shipped broken — the plan, why it seemed reasonable, how it failed, the fix (or the honest admission there wasn't one)
Lab Note	A tactical, practitioner-voice shortcut worth saying out loud
Engineering Reality	The gap between documented tool behavior and what actually happens on real hardware
Resource Reality	A concrete CPU/RAM/disk/dollar number, and what breaks specifically below it
Validation Test	A reproducible action confirming a build step worked, with the exact expected result
Blind Spot	A specific, named gap in what a built component can see or teach
What Would Change My Mind	A falsifiability statement — the specific evidence that would revise a claim
Safety Gate	Mandatory, every part, no exceptions

Safety Gate is the one deliberate break from "opportunistic." If it were just one of eight optional boxes, a part could ship with build content and no explicit isolation statement at all — which is exactly the failure this book exists to prevent. Every part, including this one, carries a Safety Gate immediately after its "Why this part exists" section and before any numbered content begins. A part missing one isn't a style nitpick for the reviewer to note; per the review checklist in [STYLE-GUIDE.md](#) §11, it's a rejection.

Lab Note *If you're the type who skips front matter, skip everything in this part except the Safety Gate above and Part 4 itself. Nothing else here is required reading before you touch a keyboard — it's required reading before you touch a keyboard and then forget you meant to isolate the network "after I get it working." That's the point it exists to make, said plainly instead of buried in a callout box you might also skip.*

6. The build order: seven sections, twenty-two parts

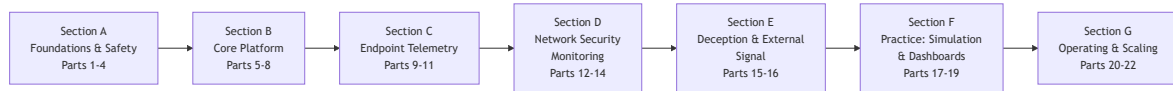
[CONCEPT] The book runs Parts 1–22 across seven labeled sections, in a build order chosen so each part has somewhere to send its output before the next part needs it — segmentation before anything sits on the network, a SIEM before an endpoint has anywhere to forward logs, a packet-capture point before Zeek or Suricata has anything to watch.

Figure 1.2 — Build order across the book's seven sections. *CONCEPTUAL.* Illustrates the dependency order this book's part sequence follows, not a fixed reading requirement — a reader retrofitting one component onto an existing lab can enter at the relevant section directly, per Section 9 below, but a reader building from zero gets a working, ingesting pipeline earliest by following this order. Diagram ID

FIG-01-02.

flowchart LR

```
A["Section A\nFoundations & Safety\nParts 1-4"] --> B["Section B\nCore Platform\nParts 5-8"]
B --> C["Section C\nEndpoint Telemetry\nParts 9-11"]
C --> D["Section D\nNetwork Security\nMonitoring\nParts 12-14"]
D --> E["Section E\nDeception & External\nSignal\nParts 15-16"]
E --> F["Section F\nPractice: Simulation\n& Dashboards\nParts 17-19"]
F --> G["Section G\nOperating & Scaling\nParts 20-22"]
```



Section A (Parts 1–4) is this part, plus topology decisions, hardware budgeting, and the network-isolation design every later Safety Gate box points back to. Section B (Parts 5–8) is the hypervisor, the perimeter firewall, internal DNS/DHCP, and the SIEM itself — the platform everything else plugs into. Section C (Parts 9–11) is endpoint telemetry: Linux first, deliberately, because the author's real running lab is Linux-only and Part 9 can lean on real captured evidence immediately, then Windows, then an honest look at why a full Active Directory domain is a much harder build than a standalone host. Section D (Parts 12–14) is network security monitoring — capture placement, then Zeek, then Suricata. Section E (Parts 15–16) is deception and enrichment: honeypots, then threat-intel feed integration. Section F (Parts 17–19) is where the lab starts producing something to look at and something to shoot at: dashboards, safe attack simulation, and purple-team drill design. Section G (Parts 20–22) is keeping the thing running — patching without erasing deliberate vulnerabilities, storage growth, and a closing chapter that walks a full "lab built, now use it" path back into the other three volumes.

7. What to do with what you build

[CONCEPT] Every build-heavy part in Sections C through F produces output that this book deliberately stops short of interpreting — that's not a gap, it's the hand-off point to whichever of the other three volumes owns what comes next. Concretely, once you've built:

- **Part 9's auditd telemetry or Part 10's Sysmon telemetry** — go to *Detection Engineering Handbook V2*, Part 3 (host telemetry) or Parts 8–9 (Windows/Sysmon detection engineering) to learn what a specific event actually means and how to build detection logic against it. This book gets you a searchable log; that book gets you a rule.
- **Part 13's Zeek logs or Part 14's Suricata alerts** — *Detection Engineering Handbook V2*, Parts 14–15 cover what a suspicious `conn.log` entry or a fired Suricata rule actually indicates, and how to correlate it against other telemetry rather than react to it in isolation.
- **Part 15's honeynet sessions** — this is the part with the most going on downstream. *Detection Engineering Handbook V2* is where you'd learn to interpret a correlated attack session and tag it against MITRE ATT&CK; *SOC Playbook Handbook's* Playbook Library is where you'd practice the triage and response procedure you'd run if that session had hit something real instead of a decoy.
- **Part 18's attack-simulation runs** — pair each simulated technique with a specific Detection Test from *Detection Engineering Handbook V2* (did the detection you built actually fire, for the right reason?) and a specific playbook from *SOC Playbook Handbook* (could you have triaged this correctly if it were real?). Part 19 formalizes this pairing into a repeatable purple-team drill.

- **A repeatable drill built from any of the above** — *SOC Manager's Operating Handbook's* Field Test framing (its Part 8) is where you'd learn to structure that drill for a team instead of a solo run, including how to score it and what to report afterward.

Part 22 walks this full path once, end to end, as the book's closing synthesis. This section exists so the hand-off isn't a surprise twenty parts later — every build chapter between here and there names its specific hand-off again, at the point it applies, rather than making you remember this list.

8. Evidence discipline: what's real here, and what isn't yet

[CONCEPT] This book reuses *The Detection Engineering Handbook V2's* four-class evidence system verbatim rather than inventing its own, because the two books frequently cite the same physical lab and the same underlying capture files: **CONTROLLED LAB EXAMPLE** (captured from a build run specifically to generate that evidence), **REAL LAB EXAMPLE** (captured from a real, continuously-operated environment), **OFFICIAL REFERENCE** (vendor documentation or a public standard, cited), and **CONCEPTUAL** (an illustrative diagram with no claim of being a capture — both figures in this part are tagged this way). A figure gets one of these four tags, always, and the tag describes what backs it *right now* — never what will back it once a future capture happens.

Here's the honest state of the author's own lab, because Part 11 exists specifically to hold this rather than let it leak into a hedge somewhere else: it's a Proxmox host running a Pi-hole resolver, a honeynet with multiple decoy services and session correlation, and several monitored Linux containers. It's real, it's running, and it's Linux and network content only — no Windows host, no Active Directory domain, no cloud tenant, and no commercial SIEM or EDR product is deployed in it as of this writing. That means Part 9 (Linux endpoints) and Part 15 (honeynet architecture) can and do cite real captured evidence throughout. It also means Part 10 (Windows/Sysmon) and Part 11 (the domain-lab problem) mostly can't — those parts are honest, conceptual, or vendor-documented walkthroughs, not build-tested results, and Part 11 specifically documents a real abandoned attempt at a Windows/AD forensics lab as a Build Autopsy rather than fabricating a domain-lab walkthrough that would misrepresent what was actually built.

Blind Spot *An evidence-classification system only protects a reader if every author and reviewer actually applies it instead of reaching for **CONTROLLED LAB EXAMPLE** because a screenshot "will obviously get captured eventually." This part can state the rule; it can't enforce it in Parts 2 through 22 on its own. That enforcement is the independent reviewer's job on every single unit, per the checklist in **STYLE-GUIDE.md** §11, item 10 — and it's the specific check this series was built to pass after both companion volumes learned the cost of skipping it.*

9. How to read this book

[CONCEPT] Two reading paths, depending on what you're starting from. Building from zero: read Sections A and B in order — skipping the isolation and platform work to get to "the fun parts" in Sections E and F is the single most common way this book's Safety Gate warnings turn out to matter in

practice, not in theory. Retrofitting one component onto a lab you already have: use the series map above and the expanded lookup table in Appendix A5 to jump straight to the relevant section, but still read Part 4 first if you haven't verified your existing network is actually isolated the way this book assumes — an existing lab built without that verification is exactly the gap Part 4's own Engineering Reality callout describes: isolation that quietly breaks through a bridged NIC or a forgotten port-forward is worse than no isolation, because it looks safe.

Either way, the six tags, eight callouts, and four evidence classes introduced above don't get re-explained again. From Part 2 forward, this book assumes you know what [SAFETY] means, what a Safety Gate box requires you to verify, and what REAL LAB EXAMPLE versus CONCEPTUAL is telling you about how much to trust a given figure. That's the entire point of putting them here first.

Cross-references: Part 2 (lab architecture patterns and topology decisions) and Part 4 (network isolation and segmentation architecture) for the safety backbone this part previews; Part 11 (the Windows domain lab problem) for the full Build Autopsy behind this part's evidence-discipline claims; Part 22 (from lab to practice) for the closing, full-length version of Section 7's hand-off path; Appendix A5 (cross-series quick reference) for the exhaustive topic lookup across all four NESHBOY volumes; *Detection Engineering Handbook V2* Part 1 (detection engineering foundations) and *SOC Manager's Operating Handbook* Part 8 (assessment design) for the companion volumes' own foundational parts.

Part 2 — Lab Architecture Patterns and Topology Decisions

Why this part exists

Part 1 stated the book's non-negotiable rule: you build a lab to attack and defend systems you own, on a network segment that can't reach — and can't be reached by — anything you don't own. This part is where that rule starts turning into hardware and cabling decisions. Before you buy a mini PC, spin up a cloud VM, or repurpose the desktop under your desk, you need to know what *shape* a home SOC lab can take, which shape fits your budget and your household, and the three logical networks every later part in this book assumes exist regardless of which shape you pick. This part doesn't install anything — no hypervisor, no SIEM, no firewall rule. It's the decision framework and the shared vocabulary that Section B's build chapters (Parts 5 through 8) and the network-isolation chapter right after this one (Part 4) all lean on without re-explaining it.

If you already know you're running Proxmox on a single box and just want the isolation build, skip to Part 4. If you're still deciding whether one box, several boxes, or a cloud account is the right starting point, this is the part that answers that question.

Safety Gate Every topology pattern compared in this part — single-host, multi-host, or cloud-hosted — has to end up with the same three logically separate networks defined in §2: a management network, a lab/victim network, and, if you build one, a honeynet segment with zero route back to your home network or any host holding real credentials. Nothing in this part is a substitute for that boundary; it's a menu of hardware shapes the boundary gets built on top of, in Part 4. Before you pick a pattern because it looks cheaper or quieter, confirm the hardware can actually host the VLAN or virtual-switch isolation Part 4 requires — a mini PC with a single NIC and no VLAN-capable switch downstream can't enforce a honeynet segment's isolation no matter how the guests inside it are laid out. Full procedure: Part 4 and Appendix A4.

1. Three shapes a home lab can take

[CONCEPT] Almost every home SOC lab you'll read about online turns out to be a variation on three underlying shapes. None is objectively "the" right answer — each trades physical footprint, cost, and noise against how much isolation and realism it can actually deliver.

1.1 Single-host all-in-one

[CONCEPT] One physical machine — a repurposed desktop, a mini PC, or a dedicated home-server box — runs a hypervisor, and every lab component (SIEM, endpoints, network security monitoring sensor, firewall/router VM, honeypots) lives on that one box as separate virtual machines or containers, connected through virtual switches instead of physical cabling. This is the pattern this book's own build chapters (Parts 5 through 21) default to, because it's the pattern the author actually runs.

[COST/RESOURCE] One box means one power bill, one set of firmware to patch, and one fan noise profile to negotiate with whoever else lives in the house. It also means one hardware failure takes down every lab component at once, and one undersized CPU or disk controller becomes a shared bottleneck across everything running on it.

[CONCEPT] The isolation between components on a single-host build is still real, not cosmetic, as long as the hypervisor's virtual-switch configuration is actually correct — a vSwitch with no uplink to a physical NIC is exactly as isolated as an air-gapped machine, and three separate vSwitches with no bridging between them enforce the same "no route" boundary a physical cable pull would. What breaks that isolation on a single-host build is almost always a configuration mistake (a guest with two network adapters, one on the lab network and one bridged to the home network for "convenience") rather than a hardware limitation — Part 4 names the specific failure modes in detail.

1.2 Multi-host distributed

[CONCEPT] Two or more separate physical machines split the lab's roles — for example, one box runs the hypervisor hosting endpoints and the SIEM, a second, cheaper box runs the router/firewall and network security monitoring sensor inline on real Ethernet between them, and a third handles the honeynet segment on its own physical NIC. This is closer to how a real small-business network is actually laid out — physical hops between segments, a real switch enforcing VLANs instead of a vSwitch config file, and a genuine single point of failure removed from any one box.

[COST/RESOURCE] More boxes means more up-front hardware cost, more power draw, more fan noise, and more firmware/OS instances to keep patched — three or four small boxes running around the clock will cost more in electricity over a year than one larger box sized to do the same total work, even before you account for the extra hardware purchase itself.

[CONCEPT] The payoff for that extra cost is that isolation stops being a config file and becomes a cable you can trace with your hand. A honeynet segment on its own physical NIC, connected to a managed switch port with its own VLAN and no trunk to the management switch, survives a hypervisor misconfiguration that would otherwise bridge two vSwitches by accident on a single-host build — the failure mode in §1.1's last paragraph simply has no physical path to occur. That's the real argument for multi-host, and it's worth paying for only if the extra maintenance surface (§3.1) is a cost you're willing to carry every month, not just on build day.

1.3 Cloud-hosted and hybrid

[CONCEPT] Some or all of the lab runs as cloud VMs and managed services instead of hardware you own — a SIEM in a cloud account, endpoints as cloud instances, a virtual network instead of a physical one. A hybrid variant keeps intentionally vulnerable and attack-simulation components on owned, physically isolated hardware at home while pushing lower-risk components (a SIEM's indexer, a dashboard) to the cloud.

[COST/RESOURCE] Cloud removes the up-front hardware cost and the noise/power problem entirely, replacing them with a recurring bill that scales with usage and doesn't stop when you're not actively using the lab unless you remember to shut instances down. It also replaces "isolated home network

with no route to the internet" with "a cloud provider's network and billing account, governed by that provider's terms of service" — a materially different safety story that this book doesn't treat as equivalent to a home network build.

[SAFETY] A cloud-hosted intentionally-vulnerable service is not automatically isolated the way a home-network segment can be — it typically has a public IP and is reachable from the entire internet unless you explicitly lock it down, and a misconfigured security group can expose a deliberately unpatched honeypot to real attackers with real consequences for your cloud bill and your provider relationship. This book's Safety Gate framework assumes a home-network build; a reader choosing the cloud-hosted pattern owns the extra work of translating "isolated segment" into that provider's specific network-ACL and security-group model before deploying anything intentionally vulnerable, and this book doesn't walk through that translation (per the series map — cloud lab-building is deliberately out of this book's main scope, noted again in Part 22).

The table below compares the three patterns directly, to support choosing one before moving to Part 3's hardware tiers.

Table 2.1 — Home lab topology patterns compared. (*CONCEPTUAL SAMPLE — figures are representative ranges for a hobbyist build, not a specific vendor's quote.*)

Pattern	Physical footprint	Approx. up-front cost	Noise/power profile	Isolation model	Best fit for
Single-host all-in-one	1 box	\$300–\$1,200 (mini PC to dedicated server)	1 fan, 1 PSU, easiest to tuck away or run silent	Virtual switches/VLANs inside 1 hypervisor	Apartment/shared-household readers, budget-constrained readers, anyone starting out
Multi-host distributed	2 to 4 boxes plus a managed switch	\$500–\$2,000+	Multiple fans/PSUs; noticeably louder and warmer as a group	Real switch VLANs plus physical segment boundaries	Readers with a dedicated space (garage, closet, office), readers wanting hardware-fault isolation
Cloud-hosted/hybrid	None locally (or a small on-prem component in the hybrid case)	\$0 up-front, recurring monthly bill	None locally	Cloud provider's VPC/security-group model, not a home network	Readers with no space/power tolerance for hardware, readers already fluent in 1 cloud provider's networking model

2. The vocabulary every later part assumes

[CONCEPT] Whichever shape from §1 you choose, this book describes the resulting lab using three logical network names, not physical ones. A "network" in this vocabulary is a boundary defined by routing and firewall policy, not a specific piece of hardware — on a single-host build all three might be virtual switches inside one hypervisor; on a multi-host build they might be VLANs spanning a real switch. Part 4 builds the actual segmentation; this part just names the pieces so Part 4 (and every part after it) doesn't have to re-derive them.

2.1 Management network

[CONCEPT] The network you use to administer the lab itself — the hypervisor's own web console, SSH to a firewall VM's configuration interface, the SIEM's admin login. Nothing on the management network should be reachable from the lab/victim network or the honeynet segment; if a compromised lab endpoint can reach your hypervisor's management interface, the isolation between "the thing being attacked" and "the thing running the attack surface" has already failed.

2.2 Lab/victim network

[CONCEPT] The network holding the endpoints and services this book teaches you to build and monitor — Windows and Linux hosts running Sysmon and auditd, the SIEM's ingest-facing interface, Zeek and Suricata's monitored segment. This is where Part 18's attack-simulation tooling runs against hosts you own, and where most of this book's hands-on exercises live.

2.3 Honeynet segment

[CONCEPT] A network holding intentionally vulnerable, deliberately exposed decoy services (Part 15) that exist purely to attract and log unsolicited contact. This segment carries the strictest isolation requirement in the book: it's designed to be attacked, its inbound path is intentionally left open on specific ports, and it must have zero route to anything you're not fully prepared to lose if a decoy is fully compromised.

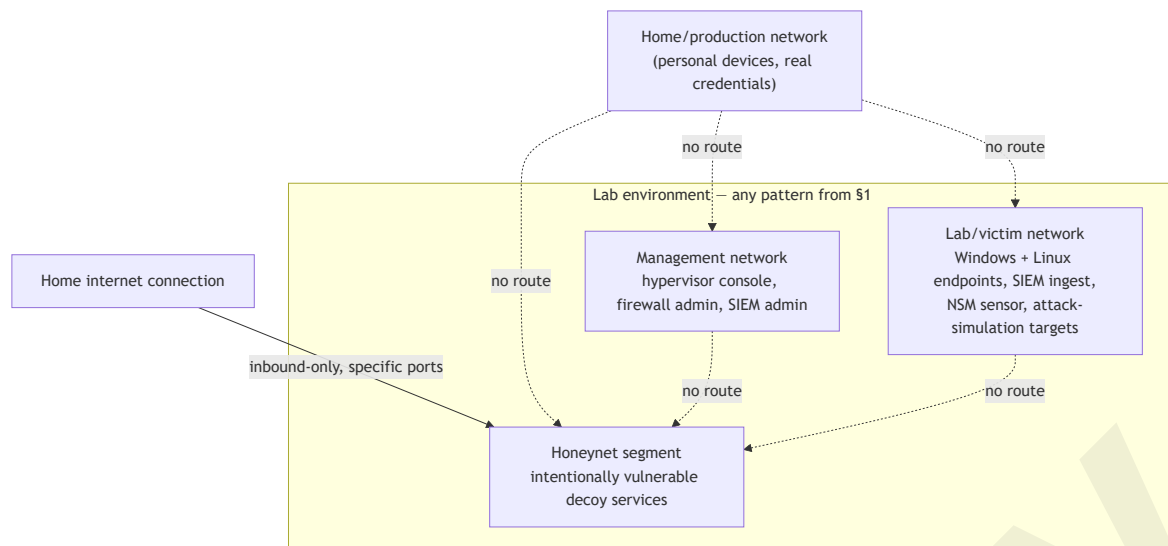
Figure 2.1 — The three logical networks and what does not connect to what. *CONCEPTUAL.*

Illustrates the vocabulary this part defines and the isolation posture Part 4 builds — a lab environment with a management network, a lab/victim network, and a honeynet segment, none of which routes to the home/production network, and none of which (management or victim) routes to the honeynet segment either. This is an architecture sketch of the target state, not a capture from a specific running build; Part 4 renders the same relationships as actual VLAN/vSwitch rules, and Part 6 as a real pfSense/OPNsense rule table. Diagram ID [FIG-02-01](#).

```
flowchart TB
    INTERNET["Home internet connection"]
    HOME["Home/production network<br/>(personal devices, real credentials)"]

    subgraph LAB["Lab environment – any pattern from §1"]
        MGMT["Management network<br/>hypervisor console, firewall admin, SIEM admin"]
        VICTIM["Lab/victim network<br/>Windows + Linux endpoints, SIEM ingest,<br/>NSM sensor, attack-simulation targets"]
        HONEY["Honeynet segment<br/>intentionally vulnerable decoy services"]
    end

    INTERNET -->|inbound-only, specific ports| HONEY
    HOME -.->|no route| MGMT
    HOME -.->|no route| VICTIM
    HOME -.->|no route| HONEY
    MGMT -.->|no route| HONEY
    VICTIM -.->|no route| HONEY
```



Lab Note If you can't yet answer "which of these three networks is this VM on" for every guest in your lab, stop before adding anything else. The single most common way isolation quietly breaks (Part 4 covers the mechanisms) is a guest nobody remembers assigning to a network, sitting on whatever interface the hypervisor defaulted it to.

3. Choosing a pattern: budget, noise, and how much you want to own

[CONCEPT] Three axes decide which pattern from §1 fits a given reader — none of them is "which tool is best," because the tools in Section B run on any of the three patterns. The pattern question is about the platform underneath the tools, not the tools themselves.

3.1 Budget

[COST/RESOURCE] A single-host build has the lowest floor: one piece of hardware, sized against Part 3's tiers, running everything. A multi-host build multiplies that floor by however many boxes you split roles across, plus a managed switch capable of VLAN tagging if you want the multi-host pattern's isolation benefit to be real rather than cosmetic. Cloud removes the floor but replaces it with a ceiling that keeps rising the longer instances stay running — a lab left on by accident for a month behaves financially nothing like a home lab left powered on by accident, which just adds a few dollars to one electric bill.

3.2 Household noise and power tolerance

[COST/RESOURCE] This axis gets skipped in most online lab-building guides and shouldn't be. A dedicated server-grade box with several fans and a real power supply is audibly a server — fine in a garage or a dedicated closet, genuinely disruptive in a bedroom or a shared apartment. A mini PC or a repurposed laptop draws a fraction of the power and can sit on a desk silently. If you don't have a room to put a running-24/7 box in that nobody minds hearing or paying the power bill for, that constraint alone should push you toward the single-host, low-power end of §1's spectrum before cost does.

3.3 Own versus simulate

[CONCEPT] The last axis is how much of the stack you want to genuinely build and operate long-term versus stand up briefly to run one exercise and tear down. A persistent single-host lab that stays running for months is what most of this book assumes and what makes Part 9's real telemetry examples possible in the first place. A short-lived, torn-down-after-use lab — spun up in the cloud for a weekend, or rebuilt from a snapshot for one drill — trades that persistence for lower ongoing cost and no long-term patching burden. Part 11 revisits this axis specifically for Windows domain labs, where "persistent" turns out to carry a maintenance cost most single-operator readers underestimate.

3.4 A worked walkthrough of the three axes

[CONCEPT] The table below isn't a formula — no single number decides this for you — but it's the shape of the reasoning worth running through before buying anything, matching a rough reader profile to the pattern from §1 that profile's constraints point toward.

Table 2.2 — Matching reader constraints to a topology pattern. (*CONCEPTUAL SAMPLE — illustrative profiles, not a substitute for your own budget/noise/ownership answers.*)

Reader profile	Budget signal	Noise/power signal	Own-vs-simulate signal	Pattern this points to
Apartment, shared household, first lab build	Under \$500 to start	Must run silently, ideally out of sight	Wants it running long-term to learn incrementally	Single-host all-in-one (§1.1)
Dedicated room/garage, some hardware already on hand	\$500–\$1,500, spread over time	Fan noise and heat acceptable in that space	Wants hardware-fault isolation, willing to patch more boxes	Multi-host distributed (§1.2)
No space or tolerance for always-on hardware	Comfortable with a recurring bill instead of a purchase	No local noise/power constraint at all	Plans short, scenario-based sessions rather than a persistent build	Cloud-hosted/hybrid (§1.3)
Already fluent in 1 cloud provider, wants to keep intentionally-vulnerable services at home	Mixed — some recurring cloud cost, some hardware cost	Only the honeynet/attack-simulation piece needs to be physically isolated at home	Wants the SIEM/dashboard layer always reachable without running a home box 24/7	Hybrid split (§1.3, hybrid variant)

4. A real lab example: what a "distributed" topology looks like once it's actually running

[CONCEPT] *REAL LAB EXAMPLE.* The author's own home lab is a useful case study precisely because it doesn't cleanly match any single row of Table 2.1 — it's a single physical Proxmox host (matching §1.1's footprint and cost profile) running enough separately-networked guests that it behaves, logically, like §1.2's multi-host distributed pattern. One box hosts a Pi-hole/DNS resolver, a Tor relay, a honeynet spanning several containers, a vulnerability-scanner platform, and a monitored network-visibility service, each assigned to its own role and network segment rather than dumped onto one flat guest network. That's a legitimate fourth answer to "which pattern do I pick": buy for §1.1's cost and noise profile, then design the guest network layout as if you'd bought for §1.2.

[CONCEPT] This didn't happen as a single planned build. The lab grew role by role — DNS first, then a honeynet, then a vulnerability scanner, then network-visibility tooling — with each new guest assigned to a network segment matching §2's vocabulary at the time it was added, rather than a from-scratch redesign every time something new went on the host. That incremental pattern is realistic for a hobbyist lab and worth planning for deliberately: decide the segment-naming and VLAN-numbering scheme in Part 4 before the first guest goes on the network, so guest number nine doesn't require renumbering guests one through eight to fit.

Resource Reality *The author's real lab is also proof that this pattern's flexibility has a ceiling. Running enough guests on one physical host to cover DNS, a Tor relay, a multi-container honeynet, a vulnerability scanner, and a network-monitoring service has pushed that single host to CPU and RAM oversubscription and a root filesystem sitting at 100% full — not a hypothetical "performance may degrade" warning, but the actual current state of a lab built exactly the way §4 describes. Section 1.1's "one hardware failure takes down everything" and "one CPU becomes a shared bottleneck" caveats aren't abstract risks; they're what happens when a single-host lab's guest count outgrows the box it started on, and the fix is the same one Part 3 states for any tier: budget headroom before you add the next guest, not after the host is already this full.*

Blind Spot *A single physical host running several logically separated guests teaches you the network-segmentation half of the multi-host pattern — VLANs, firewall rules between segments, distinct roles per guest — but not the hardware-fault-isolation half. If that one Proxmox host's disk fails, every guest on every segment goes down at once, including the honeynet and the DNS resolver the rest of the lab depends on. A genuinely distributed multi-host build (separate physical boxes per major role) is the only pattern in this part that actually removes that single point of failure; a single host with well-designed virtual networking removes the logical flatness but not the physical one.*

5. Which pattern the rest of this book builds against

[CONCEPT] Part 5 walks through installing a hypervisor and states plainly which platform the author actually runs (Proxmox, on the single-host pattern from §1.1) — every hands-on build chapter from Part 5 through Part 21 defaults to that pattern's assumptions: one hypervisor, virtual switches standing in for the physical segment boundaries Part 4 designs. If you've chosen the multi-host pattern from §1.2 instead, the segmentation concepts in Part 4 and the product installs in Parts 5 through 21 still apply — a VLAN is a VLAN whether it spans one hypervisor's virtual switches or a real managed switch between three boxes — but the specific "click here in the Proxmox web console" instructions won't map one-to-one onto a different hypervisor or a physically distributed layout, and this book doesn't re-derive a parallel walkthrough per platform (Part 5 states this limitation explicitly rather than silently assuming Proxmox is the only valid choice).

[CONCEPT] If you've chosen the cloud-hosted pattern from §1.3, expect a larger translation gap: Parts 6 through 8 and Parts 12 through 16 assume a hypervisor-and-virtual-switch model that doesn't map directly onto a cloud provider's VPC/security-group model, and this book doesn't provide that translation (§1.3's Safety Gate note above). Part 22's closing synthesis points toward where cloud-specific lab-building content would eventually live, but it isn't part of this book's 22 parts.

6. From topology to telemetry: what the other three volumes do with what you build here

[CONCEPT] This part produces no telemetry by itself — it's a decision, not a build step. But the pattern chosen here determines something every other NESHBOY volume depends on: how many distinct network segments and log sources will eventually exist, and what each one is *for*. That mapping is what makes the rest of the series usable once Parts 5 through 21 finish building against it.

- **Detection Engineering Handbook V2** assumes a reader already has telemetry with a known source and a known network context — its Parts 8–9 interpret Windows/Sysmon and Linux/auditd events, and its Parts 34–36 teach threat-hunting methodology, on the explicit premise that the hunter knows whether a given host sits on the lab/victim network or the honeynet segment defined in §2. Get the topology decision right here, and every detection rule and hunt built later in that volume has an unambiguous "where did this come from" answer built in from the start.
- **SOC Playbook Handbook's** Playbook Library assumes an alert arrives tagged with enough network context to triage correctly — a login-anomaly alert from the lab/victim network and the same alert shape from the honeynet segment call for entirely different playbook branches (one is a real detection to investigate, the other is the decoy behaving exactly as designed). That distinction only exists because §2's vocabulary was applied consistently when the lab was built.
- **SOC Manager's Operating Handbook** uses this book's finished lab as the practice environment for Part 8's assessment design and Part 11's onboarding/ramp-up content, and for structuring a repeatable Field Test drill (this book's Part 22 closes the loop on that explicitly). A topology nobody can clearly describe — "which network is this host on, again?" — makes a repeatable drill impossible to score consistently, which is a management problem this part's vocabulary exists to prevent before it starts.

None of that is content this part teaches directly — it's the reason getting the topology decision right, before Part 4's build, pays off three books later.

Cross-references: This book — Part 1 (the ownership/isolation rule), Part 3 (hardware tiers for whichever pattern you chose), Part 4 (building the isolation this part's vocabulary assumes), Part 5 (hypervisor install on the single-host default), Part 11 (the own-vs-simulate axis applied to Windows domain labs), Part 22 (closing synthesis and the cloud-lab forward pointer), Appendix A1 (topology diagram templates), Appendix A5 (cross-series lookup). Detection Engineering Handbook V2 — Parts 8–9, 34–36. SOC Playbook Handbook — Playbook Library. SOC Manager's Operating Handbook — Part 8, Part 11.

Part 3 — Hardware and Resource Budgeting

Why this part exists

Part 2 gave you the shapes a lab can take — single-host, multi-host, cloud-hybrid — and the vocabulary for talking about them. This part turns those shapes into numbers you can actually shop for: how many cores, how much RAM, how much disk, and how much of your electricity bill, per tier, before you spend a single dollar or repurpose a single old laptop. The goal is a hardware decision you can defend, not a hardware decision you regret three services into the build.

[CONCEPT] This is a hobbyist's budgeting problem, not an enterprise one. SOC Manager's Operating Handbook, Part 20 (Building & Defending the SOC Budget) covers headcount, licensing, and vendor contracts at organizational scale — a different problem with different stakes. The only thing worth borrowing from it here is the habit: write the number down before you buy, and write down what breaks below it. This part does that for watts, gigabytes, and cores instead of headcount and dollars per analyst.

Safety Gate *Nothing in this part's sizing guidance is a substitute for the network isolation Part 4 builds. A hardware tier tells you what a box can run — it says nothing about whether it's safe to run it. Before you provision a single VM on whichever tier you choose, confirm you have, or plan to build, enough NIC and virtual-switch capacity to keep the honeynet segment (Part 15) and any attack-simulation targets (Part 18) off the same broadcast domain as your management and endpoint traffic. A Tier 1 box with one onboard NIC can still do this correctly using the VLAN-tagged virtual switches Part 4 builds — but only if that separation exists before anything intentionally vulnerable goes on the box, not retrofitted after. Buying more RAM does not buy you isolation.*

1. Sizing is a lab problem, not a scaling problem

[CONCEPT] An enterprise buys hardware to survive growth it can't fully predict. You're buying hardware to run a fixed, known list of services you're choosing yourself, on a network nobody else depends on. That's a much easier problem, and it has a wrong answer that's worth naming up front: guessing. "I'll get a decent PC and see what fits" produces a box that runs a SIEM adequately, until you add a second Windows endpoint, at which point it swaps, the indexer restarts under memory pressure, and you spend a Saturday debugging a hardware problem you could have avoided by reading a sizing table for ten minutes first.

This part gives you three tiers — repurposed laptop or mini-PC, dedicated home-server box, multi-node cluster — with the CPU, RAM, disk, and power numbers for each, and a plain statement of what each tier cannot do. The tiers aren't marketing categories; they're checkpoints. If your ambition (a Windows domain, a full honeynet, months of PCAP retention, all running at once) exceeds your tier, the fix is

either a smaller ambition or a bigger tier — not a hope that Linux memory management will bail you out. It won't, past a point, and that point arrives faster than most first-time lab builders expect.

2. The three hardware tiers

[COST/RESOURCE] The table below maps hardware tier to what it can and can't run without swapping, so you can pick a starting point before Part 5 walks through the actual hypervisor install.

Tier	CPU	RAM	Disk	Power draw (idle/load)	Approx. cost	Can run comfortably	Cannot run
1 — Repurposed laptop/mini-PC	4 cores, no server-grade features	8–16GB	128–256GB SSD	10–25W / 35–45W	\$0 (already owned) to \$150 used	One SIEM instance, 1–2 lightweight Linux endpoints, a DNS/DHCP service	A Windows endpoint alongside the SIEM without swapping; any NSM sensor doing real packet capture; more than one or two honeypot decoys at once
2 — Dedicated home-server box	6–8+ cores, server or workstation-class	32–64GB	500GB–2TB SSD, plus HDD for cold storage	45–65W / 90–150W	\$250–700 used enterprise SFF/tower	SIEM + several Linux and Windows endpoints + Zeek or Suricata + a small honeynet + threat-intel ingestion, concurrently	A persistent multi-DC Active Directory forest and a full honeynet and months of raw PCAP retention, all at once, without adding storage
3 — Multi-node cluster	2–3 hosts, Tier-2-class each	32–64GB per node	Tier-2 disk per node, aggregated	Sum of node draw, plus switch	\$700–2,000+ used enterprise gear	Everything Tier 2 runs, plus a physically separate host for the honeynet segment, longer retention, and headroom for a Part 11-style domain experiment	Nothing, really — the limits move from hardware to your own maintenance time (Part 20)

Lab Note *Used, off-lease business desktops (HP EliteDesk/ProDesk, Dell OptiPlex) are the best dollar-per-core value in this entire book. They're quiet, they take standard ATX PSUs when the stock one dies, and a 4-core/16GB unit is routinely under \$100 on the secondhand market. You do not need ECC RAM or a rack chassis for a single-operator lab — that's Tier-3 thinking applied to a Tier-1 problem.*

[COST/RESOURCE] Tier categories describe capacity, not headroom actually left over — those are different numbers, and conflating them is the single most common hardware mistake in a lab like this.

Resource Reality *The author's own running lab sits on Tier-2-class hardware — an HP EliteDesk 800 G1 desktop, Intel Core i5-4590 (4 physical cores, no hyperthreading), 15GiB of RAM — and at last inspection (2026-09-14) it was hosting 13 guests with roughly 21 vCPUs and about 14.5GB of RAM allocated across them, on 4 physical cores and 15GiB of physical RAM. That's over 5x CPU oversubscription and RAM allocated down to the last gigabyte. It runs, because most of those guests (a DNS resolver, several honeynet decoys, a Tor relay) are idle most of the time and Linux's scheduler and memory overcommit tolerate that — but the same host's root filesystem was already at 92% full at that same inspection and fully exhausted within the following day. Oversubscription and disk exhaustion aren't hypothetical failure modes this book warns about in the abstract; they're what's actually happening on the lab this book is partly evidenced from, right now.*

3. What each lab component actually costs

[COST/RESOURCE] Tier tables tell you the ceiling. This table tells you what actually eats into it, service by service, so you can add up a realistic total before you commit to a tier.

Component	CPU	RAM	Disk	Notes
Hypervisor base overhead (Proxmox host itself)	Low, mostly idle	1–2GB	20–30GB for the host OS	Web UI, <code>pvestatd</code> , and package overhead cost more at idle than vendor "minimum requirements" pages suggest
SIEM — indexer component (Wazuh/Elastic-family)	1–2 cores under ingest	4GB heap at idle, more under load	20GB+, grows with retention	Budget 8GB total for the SIEM VM before adding a single endpoint — see Part 8
SIEM — manager/rules engine	Shares the VM above	1–2GB additional	Shares the VM above	Runs alongside the indexer on the same VM in most home-lab builds
Linux endpoint (Debian/Ubuntu + auditd + forwarding)	1 core	512MB–1GB	4–8GB	Matches the author's own real endpoints (pihole: 1 core/1GB; vulnscan: 2 cores/2GB) — Linux telemetry sources are cheap
Windows endpoint (standalone + Sysmon + forwarding)	2 cores	4GB minimum	40–60GB	Windows itself, not Sysmon, is the RAM cost here — see Part 10
Zeek sensor	1–2 cores, scales with link rate	2–4GB	Depends on log retention, not capture	CPU-bound on packet rate, not RAM-bound — see Part 13's Engineering Reality box for mirrored-port packet loss
Suricata sensor	1–2 cores, scales with rule count and link rate	2–4GB	Similar to Zeek	Loading the full ET Open rule set costs meaningfully more RAM than a trimmed rule set — see Part 14
Honeynet decoy (single service)	1 core	384–768MB	2–4GB	Matches the author's real AeroNex decoy containers — four of the five run at the low end (384MB/2GB); the one fronting a real, deliberately unpatched phpMyAdmin (CVE-2018-12613) over a full Apache/PHP/MariaDB stack runs heavier (768MB/4GB) because it's a full stack, not a single lightweight service. A decoy is deliberately cheap so you can run several
Honeynet collector/correlation backend	4 cores	3GB + swap	4–8GB	Correlation logic and a dashboard cost more than any individual decoy — see Part 15
Threat-intel feed poller	Under 1 core	256–512MB	1–2GB	Cheapest component in the book; mostly idle, polling on a timer

Engineering Reality Vendor "minimum requirements" pages describe the software running alone on an otherwise-empty machine. A Proxmox host that claims to run comfortably in a couple of gigabytes still costs you real, permanent RAM for the web UI, `corosync`, and normal package overhead before a single guest boots — treat every vendor minimum in this table as a floor for that one component in isolation, not a number you can stack without a margin.

4. Stacking exercise: what actually fits

[SETUP] Add the component table up against a tier and the picture gets concrete fast. Take a common first-timer's wish list: one SIEM, two Windows endpoints, three Linux endpoints, Zeek, Suricata, and a small honeynet, all on one box.

1. SIEM: 8GB.
2. Two Windows endpoints at 4GB each: 8GB.
3. Three Linux endpoints at ~1GB each: 3GB.
4. Zeek and Suricata sharing a sensor VM: 4GB.
5. A three-decoy honeynet plus its collector: roughly 4.5GB.
6. Hypervisor host overhead: 2GB.

That's a running total around 29.5GB before any operating-system-level swap headroom or the inevitable moment two Windows endpoints both decide to run updates at once. On a 16GB box, this list doesn't run slowly — it doesn't run. The SIEM's indexer gets OOM-killed, the Windows endpoints thrash into swap and stop producing timely Sysmon events, and the honeynet's collector either falls behind or drops correlation state. A SIEM plus a Windows domain plus a full honeynet on 16GB of RAM produces an unusable, swapping mess, not a working lab — that's not a caution, it's arithmetic from the table above.

The same list fits a 32GB Tier-2 box with room to spare, which is exactly why this book treats Tier 2 as the realistic default for a reader who wants most of Section B through Section E running at once, and treats Tier 1 as a genuinely good starting point for a reader who wants to build incrementally — SIEM and Linux endpoints first (Parts 8–9), then decide whether the next purchase is more RAM in the same box or a second box entirely.

What Would Change My Mind This section's specific claim is that 16GB of RAM cannot sustain a SIEM, two Windows endpoints, and a full honeynet running concurrently without swapping under realistic ingest load. If a reader reports running that exact combination on 16GB for a sustained multi-week period — under real event volume, not an idle demo — with no OOM kills and no swap thrashing, that specific number needs revision in this part, not just a footnote. The general "budget generously" framing would survive; the specific 16GB figure would not.

5. Beyond RAM: CPU, disk I/O, and NIC capability

[COST/RESOURCE] RAM gets the most attention because it fails loudly (a process dies) rather than quietly (a process just runs slower), but two other resources decide whether a lab is pleasant to use.

[COST/RESOURCE] CPU oversubscription is normal in a home lab and usually fine, because most of your guests are idle most of the time — the author's own host runs 21 vCPUs of allocated capacity on 4 physical cores without falling over, precisely because a DNS resolver and a Tor relay spend nearly all their time waiting on I/O, not computing. The oversubscription that actually hurts is concurrent, CPU-

bound load: a Windows endpoint installing updates at the same moment Zeek is processing a traffic spike and Suricata is matching against a large rule set. There are only 4 physical cores on a box like that, and they can't truly run 4 CPU-bound things at once regardless of how many vCPUs you've handed out on paper; when that happens, everything on the host slows down together, and a SIEM ingest pipeline that falls behind starts silently dropping or delaying events rather than crashing outright — a much harder problem to notice than an OOM kill.

[COST/RESOURCE] Disk I/O matters as much as capacity. A SIEM's indexer performs constant small writes; on a spinning hard disk instead of an SSD, ingest can fall behind badly enough that dashboards show data minutes old instead of seconds old. Reserve SSD for anything actively indexing (the SIEM, the honeynet collector's database) and it's fine to put cold storage — PCAP archives, old backups — on cheaper spinning disk, which is exactly the local/local-lvm split the author's own host uses and exactly the retention strategy Part 21 covers in depth.

[COST/RESOURCE] NIC capability is the resource first-time builders forget to check, and it bites later rather than immediately: Part 12 needs a virtual switch or physical NIC capable of promiscuous-mode mirroring for Zeek and Suricata to see anything, and not every consumer NIC chipset handles that cleanly under load.

Engineering Reality *The author's own host has crashed from exactly this kind of NIC problem — not a mirroring failure, but a specific Intel e1000e driver bug ("Hardware Unit Hang") on its primary NIC that took the whole host down four times before the fix (disabling TSO/GSO offload) was identified and made persistent. The lesson generalizes past this one chipset: a consumer NIC under sustained lab load is not guaranteed to behave like its datasheet, and a NIC-driver crash takes down every guest on the host at once, not just the network stack. Note the exact NIC chipset on any box you're considering before you buy it, and budget time to hunt down a driver quirk if the host starts crash-looping under network load — it's a hardware problem, not a config mistake, and no amount of re-reading your firewall rules will fix it.*

6. Power, noise, and heat: the constraint an enterprise budget doesn't have

[COST/RESOURCE] An enterprise SOC's power bill is somebody else's line item. Yours runs through your own electricity meter and sits in the same room you sleep in, and both of those facts should shape which tier you pick.

A Tier-1 laptop idles around 10–15W and rarely gets loud enough to notice. A Tier-2 desktop-class box under real load — SIEM ingest, a couple of endpoints, an NSM sensor all working at once — commonly draws 90–150W, which at a representative \$0.15/kWh residential rate is roughly \$10–16 a month left running continuously, before you've counted a second box. That's not a large number, but it's not zero, and it compounds with a Tier-3 cluster's per-node draw multiplied across two or three machines. Heat and fan noise scale with it: a repurposed laptop tucked on a shelf is easy to forget about; a multi-node cluster in a bedroom closet is not, and "will this be loud enough that a housemate notices" is a legitimate sizing input this book won't pretend doesn't matter.

Lab Note *If noise is the binding constraint rather than budget, a low-power NUC-class mini-PC (fanless or near-silent) running Tier-1's workload is worth the price premium over a louder used desktop of the same RAM/CPU spec — you'll actually leave it running 24/7 instead of shutting it down when it bothers you, and a lab that's off half the time produces half the telemetry.*

7. Deciding your tier: a sizing flow

[CONCEPT] The decision isn't really about your budget in isolation — it's budget, ambition, and household tolerance for noise and heat, weighed together. The flow below collapses the tables above into the same decision path this book expects most readers to actually walk.

```
flowchart TD
    A[What do you want the lab to run?] --> B{Budget under ~$150, \nusing hardware you\nalready own or can\nfind used?}
    B -- Yes --> C[Tier 1: repurposed\nlaptop or mini-PC]
    B -- No --> D{Need Windows endpoints, \nNSM, and a honeynet\nrunning concurrently?}
    D -- No --> C
    D -- Yes --> E{"Need long PCAP/log\nretention, or a\nWindows domain\nexperiment (Part 11)?"}
    E -- No --> F[Tier 2: dedicated\nhome-server box]
    E -- Yes --> G[Tier 3: multi-node\ncluster]
    C --> H[Expect to run: one SIEM\n+ 1-2 lightweight endpoints.\nNo full honeynet, no NSM sensor.]
    F --> I[Expect to run: SIEM +\nLinux/Windows endpoints +\nZeek/Suricata + small honeynet.]
    G --> J[Expect to run: everything\nTier 2 runs, plus a physically\nseparate honeynet host and\nmonths of retention.]
```

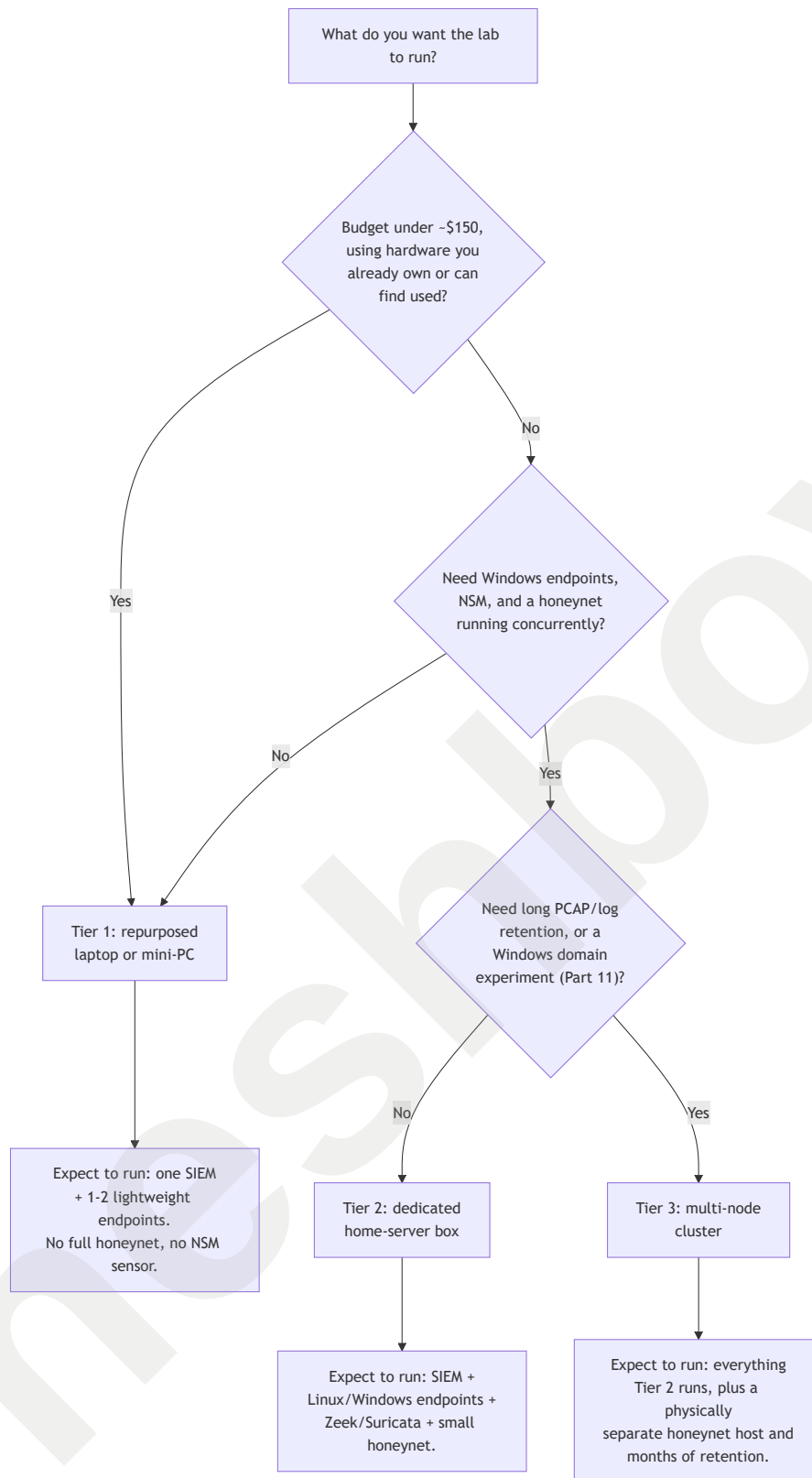


Figure 3.1 — Hardware tier decision flow. *CONCEPTUAL.* Illustrates the sizing decision this section walks through: budget and concurrency requirements route a reader to one of the three tiers defined in §2, each with the stacking expectations from §3–4 attached. This is a decision aid, not a capture from a running build — Part 5 shows the same host after an actual hypervisor install.

8. Validate your hardware before you build anything

[SETUP] Before committing to a tier, confirm the box you're considering (or already own) can actually deliver what the tables above assume. These checks target Debian/Ubuntu-family Linux and Windows 10/11 hosts — the two operating systems this book's endpoints and hypervisors run on.

On a Linux box or a Linux-based hypervisor host, confirm core count, RAM, and virtualization-extension support:

```
nproc
free -h
grep -Eoc '(vmx|svm)' /proc/cpuinfo
```

`nproc` should report the core count the listing claimed; `free -h` should show total RAM close to the advertised spec (a little less is normal — the BIOS and integrated graphics reserve some); the `grep` command should return a number greater than 0, confirming Intel VT-x (`vmx`) or AMD-V (`svm`) is present and exposed to the OS, without which no hypervisor in Part 5 will run a VM at all.

On a Windows box being considered as a host or a future endpoint, confirm the same virtualization extension from PowerShell:

```
Get-ComputerInfo -Property "HyperVRequirementVirtualizationFirmwareEnabled"
```

Validation Test Setup: a candidate box, powered on, with either a Linux live USB or its existing Linux/Windows install. **Action:** run the three Linux commands above, or the PowerShell command above, on the target box. **Expected result:** `nproc` and `free -h` match the tables in §2 for the tier you're evaluating; the virtualization-extension check returns a positive result (a nonzero count on Linux, `True` on Windows). If virtualization extensions report disabled, the fix is almost always a BIOS/UEFI setting (commonly named "Intel VT-x," "AMD-V," or "SVM Mode") rather than a hardware limitation — re-check after enabling it before concluding the box can't do the job.

9. Troubleshooting undersized hardware

[TROUBLESHOOTING] Undersized hardware rarely announces itself as "undersized hardware." It shows up as symptoms in the software layered on top, and it's worth recognizing the pattern before you spend an evening debugging a SIEM config that was never the problem.

- **SIEM indexer restarts repeatedly under load.** Almost always an OOM kill, not a software bug. Check `dmesg` or `journalctl -k` for `Out of memory: Killed process` entries matching the indexer's process name; the fix is the 8GB budget from §3, not a config tweak.
- **Sysmon events arrive minutes late, or stop arriving during load spikes.** A Windows endpoint under swap pressure delays its own event forwarding; by the time an event reaches the SIEM its timestamp is stale enough to break correlation windows in Detection Engineering Handbook V2's

detection logic. Check the endpoint's own memory pressure before assuming the SIEM pipeline is broken.

- **Dashboards show data that's minutes old instead of seconds old.** Usually disk I/O, not RAM — check whether the SIEM's storage is on spinning disk rather than SSD before touching any query or index setting.
- **The whole host crash-loops or hangs under network load.** Check the specific NIC chipset and driver for known issues (\$5's Engineering Reality box) before assuming it's a virtual-switch misconfiguration — some hardware bugs look exactly like software bugs until you check `dmesg` for driver-level errors.
- **Everything gets slower over weeks with no config changes.** Check disk capacity, not RAM — a filesystem creeping toward full (as the author's own root filesystem did) degrades write performance well before it actually fills, and a SIEM or honeynet collector that can't write new records fails quietly rather than with an obvious error.

10. What this budget buys you downstream

[CONCEPT] The hardware tier you choose here isn't just a shopping decision — it caps what the rest of the NESHBOY series can do with what your lab produces, so it's worth being explicit about the handoff before you move on to Part 4.

Detection Engineering Handbook V2's threat-hunting content (Parts 34–36) assumes you have enough retained history to hunt across — a Tier-1 box running a SIEM with a few gigabytes of headroom will force short retention windows (Part 21 covers exactly this trade-off), which means hunting exercises built from that book will realistically cover days, not months, of history until you scale up. Size for the hunting scope you actually want to practice, not just the ingest volume you have today.

SOC Playbook Handbook's playbooks assume the services a real incident touches are actually up and responsive when you work through a scenario. An underpowered box that's quietly swapping doesn't just produce noisy telemetry — it makes playbook practice unrealistic in the wrong direction, teaching you to expect sluggish tooling as normal rather than as a service-degradation incident worth escalating in its own right. If you want playbook drills (Part 19) to reflect a healthy environment, budget hardware so the lab's baseline behavior is healthy.

SOC Manager's Operating Handbook's budget-justification framing (Part 20) is worth revisiting once you've lived with a tier for a few months: the exercise of writing down "Tier 1 was fine until I added a second Windows endpoint, and here's the specific number that forced the Tier 2 upgrade" is a small, low-stakes rehearsal of the exact justify-the-spend argument that book teaches at organizational scale. Keep your own numbers from this part — they're a genuine, if miniature, case study the next time you need to make that argument for real.

Cross-references: Part 2 (Lab Architecture Patterns and Topology Decisions) for the topology shapes these tiers assume; Part 4 (Network Isolation and Segmentation Architecture) for the isolation controls this part's Safety Gate depends on; Part 5 (Choosing and Installing a Hypervisor) for the actual build on the host sized here; Part 8 (Building the SIEM Platform), Part 9 (Linux Endpoints and audit Deployment), Part 10 (Windows Endpoints and Sysmon Deployment), Part 13 (Deploying Zeek), Part 14

(Deploying Suricata), and Part 15 (Honeypots and HoneyNet Architecture) for the components priced in §3; Part 11 (The Windows Domain Lab Problem) and Part 21 (Storage Growth, Retention, and Long-Term Cost Management) for the two heaviest cost drivers this part flags but doesn't fully size; Appendix A2 (Hardware Tiers & Cost Worksheets) for a fillable version of this part's tables; SOC Manager's Operating Handbook, Part 20 for the loosely analogous enterprise budget process.

neshbooy

Part 4 — Network Isolation and Segmentation Architecture

Why this part exists

Part 2 named three logical networks every later part in this book assumes exist: a management network, a lab/victim network, and, if you build one, a honeynet segment. Part 1 stated the book's one non-negotiable rule — you build a lab to attack and defend systems you own, on a network segment that can't reach, and can't be reached by, anything you don't own. Both of those are still descriptions, not designs. This part is where the description becomes VLAN IDs, virtual-switch configuration, and a default-deny firewall posture — the actual thing Part 6 then implements with a real router/firewall VM, and the thing Part 15's honeynet, Part 18's attack-simulation tooling, and every other later part depend on being true before their own content is safe to follow.

This part adds one refinement to Part 2's three-network vocabulary rather than contradicting it: the network-security-monitoring sensor built in Parts 12 through 14 gets its own broadcast domain for its packet-capture interface, because a mirror/SPAN port carries a full copy of lab/victim traffic and shouldn't need — or get — a routable address anywhere. Its separate log-shipping interface lives on that same NSM segment and reaches the SIEM through one explicit, narrowly-scoped allow rule — not a free pass just because both ends are "admin-facing." That gives this part four VLANs to design against instead of Part 2's three names, with no conflict between them.

Nothing in this part is a full product walkthrough. Part 5 installs the hypervisor these virtual switches live inside; Part 6 installs the actual firewall product and turns the rule table below into real configuration. What this part gives you is the topology, the isolation model, and — just as important — the specific, boring ways this exact kind of isolation quietly fails on real hardware, so you're checking for the right thing instead of assuming a VLAN label alone did the job.

Safety Gate Every later part's Safety Gate box in this book points back to this one. Before you connect a single lab VM to anything else, verify three concrete conditions: (a) no Layer 3 route exists between any lab VLAN and your home network or any host holding real credentials; (b) every lab segment enforces default-deny egress, with only the specific, logged exceptions in Table 4.2; and (c) no single guest — physical or virtual — has a network interface bridged onto a lab VLAN and your home LAN at the same time. Section 6 below gives you the exact ping/traceroute/DNS checks that confirm all three. If you can't check all three right now, stop here — nothing past this part in this book assumes an unverified network, and Appendix A4 expands this into the full pre-flight checklist every later Safety Gate box cites (Appendix A4 is not yet authored — see [README.md](#); until it exists, treat Section 6's checks below as the operative pre-flight procedure).

1. The isolation model this book assumes

[CONCEPT] Regardless of which topology pattern from Part 2 you picked — single-host, multi-host, or hybrid — this book's lab network is four logical segments, each a separate VLAN with its own virtual

switch or bridge: a management segment for hypervisor and firewall administration, a lab/victim segment for the Windows and Linux endpoints under test (Parts 9 and 10) and the attack-simulation targets from Part 18, an NSM segment carrying only mirrored traffic for the Zeek/Suricata sensor (Parts 12 through 14), and a honeynet segment for the intentionally vulnerable decoy services built in Part 15. The only traffic allowed between any two segments is a short, explicit allow-list; everything else is denied by default.

The honeynet segment gets the strictest rule of the four: it may accept inbound connections on its decoy ports from the internet, but it may not initiate an outbound connection anywhere except the one logging path back to the SIEM. A honeypot's entire value is that it gets compromised — the isolation model's job is making sure that compromise has nowhere further to go. Part 15's expanded Safety Gate restates this specific control rather than assuming a reader remembers it from here, because a honeynet is the part of this book where getting it wrong has the most direct consequence.

flowchart TB

```
INTERNET["Home internet connection"]
HOME["Home/production network<br/>personal devices, real credentials"]
FW["Lab firewall/router VM<br/>built in Part 6"]
SIEM["SIEM ingest<br/>built in Part 8"]
```

```
subgraph MGMT["VLAN 10 - Management"]
    HV["Hypervisor + firewall admin interfaces"]
end
```

```
subgraph VICTIM["VLAN 20 - Lab/victim"]
    EP["Windows + Linux endpoints, Parts 9-10"]
end
```

```
subgraph NSM["VLAN 30 - NSM span"]
    SENSOR["Zeek/Suricata capture interface, no routable address"]
end
```

```
subgraph HONEY["VLAN 40 - Honeynet"]
    DECOY["Decoy services, Part 15"]
end
```

```
INTERNET -->|inbound-only, decoy ports| FW
FW -->|explicit allow only| MGMT
FW -->|explicit allow only| VICTIM
FW -->|mirrored capture, no route| NSM
FW -->|inbound-only, zero egress| HONEY
```

```
EP -->|log forward| SIEM
SENSOR -->|log forward via mgmt path| SIEM
DECOY -->|log forward only| SIEM
```

```
HOME -->|no route| MGMT
HOME -->|no route| VICTIM
HOME -->|no route| HONEY
MGMT -->|no route| HONEY
VICTIM -->|no route| HONEY
HONEY -->|no egress| INTERNET
```

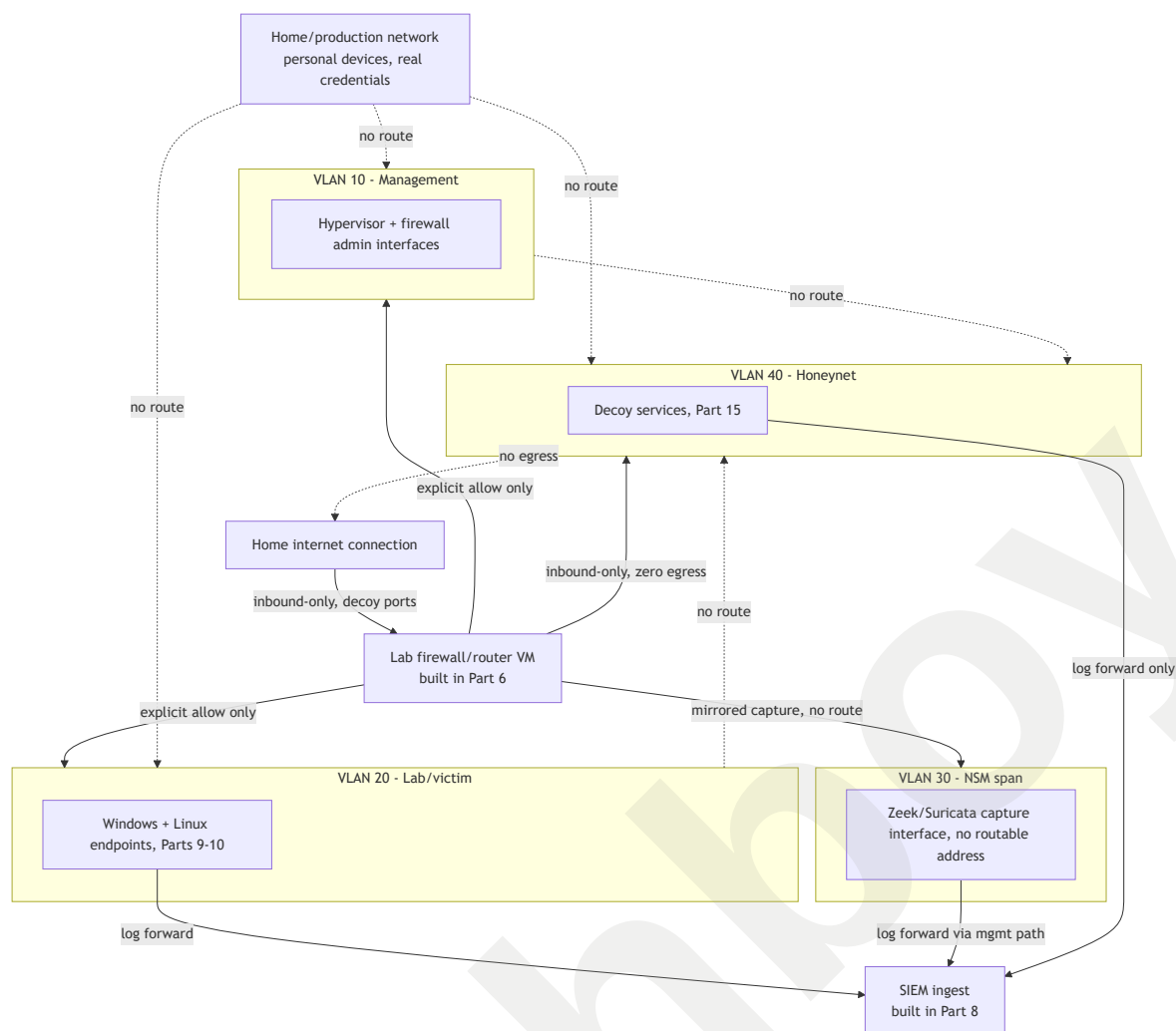


Figure 4.1 — Isolated lab VLAN topology. *CONCEPTUAL.* Illustrates the target segmentation this part designs — management, lab/victim, NSM, and honeynet VLANs, each behind a default-deny firewall boundary with explicit allow rules limited to admin access and log forwarding. This is an architecture sketch, not a capture from a running build; see Figure 6.1 (Part 6) for the same topology after a real pfSense/OPNsense build, and Figure 15.1 (Part 15) for the honeynet segment specifically once decoys are deployed on it.

2. VLAN tagging fundamentals for a home lab

[CONCEPT] 802.1Q VLAN tagging is what lets one physical switch (and, on a single-host build, one physical NIC) carry several logically separate broadcast domains at once — a frame gets tagged with a VLAN ID as it leaves a trunk port and only reaches devices configured for that same ID. Without it, a single-host lab with one spare NIC would need one physical switch port per segment, which most home hardware simply doesn't have.

[SETUP] The gap most readers hit first: a lot of consumer/SOHO switches marketed as having "VLAN support" only do port-based VLANs — one VLAN per physical port, no tagging — not true 802.1Q trunking of multiple tagged VLANs down a single uplink to a hypervisor's NIC. If your plan is one physical NIC carrying all four VLANs in Table 4.1 to an external switch, confirm the switch's datasheet specifically says "802.1Q tagged VLAN" or "trunk port," not just "VLAN." An inexpensive managed

switch with real trunking (commonly \$30–80 for an 8-port model) is a real line item in Part 3's hardware budget, not an optional nice-to-have, the moment your topology needs more segments than you have spare physical ports.

The table below lays out the VLAN plan this book's later parts build against. Treat it as a starting template, not a fixed requirement — an extra segment (for example, a dedicated VM network for Part 16's threat-intel feed poller) slots in the same way.

Table 4.1 — Home lab VLAN plan (CONCEPTUAL SAMPLE).

VLAN ID	Segment name	Purpose	Inbound allowed	Outbound allowed
10	Management	Hypervisor console, firewall admin, SIEM admin	Admin workstation only, on management ports (for example, 443, 22)	NTP and package-mirror traffic only
20	Lab/victim	Windows/Linux endpoints under test (Parts 9-10); attack-simulation targets (Part 18)	Management VLAN only, for admin SSH/WinRM	SIEM ingest ports only (Part 8)
30	NSM	Zeek/Suricata capture interface (Parts 12-14)	Mirrored traffic only — no routable inbound needed	SIEM ingest ports only, from the sensor's separate management path
40	Honeynet	Decoy services (Part 15)	Internet, decoy ports only (for example, 22, 80, 443)	Log-forwarding path to the SIEM only — nothing else

3. Isolated virtual switches inside the hypervisor

[CONCEPT] A VLAN ID alone doesn't isolate anything if the hypervisor bridges every VM to the same virtual switch regardless of tag. The layer that actually matters is the virtual switch (or "port group," or "internal network," depending on the product's vocabulary): a virtual switch with no physical NIC attached at all is exactly as isolated as pulling a cable, and a VM assigned to the wrong virtual switch is not isolated no matter what VLAN ID its guest OS thinks it's on.

[SETUP] Part 5 walks through a full hypervisor install; the isolation-specific piece of that build, ahead of anything else in this book, looks like this on Proxmox (the platform the author's own lab runs, per Part 5):

1. Create a new Linux bridge (Datacenter > the node > System > Network > Create > Linux Bridge) — for example, `vmbr1` — and leave its bridge port field empty rather than assigning a physical NIC.
2. Repeat for each segment that needs its own broadcast domain — a minimum lab build wants at least `vmbr1` (lab/victim) and `vmbr2` (honeynet) in addition to the default `vmbr0`.
3. Assign each VM's virtual network device to the bridge matching its intended segment from Table 4.1, in that VM's hardware configuration — never leave a segment-sensitive VM on the default `vmbr0`.
4. If a single physical uplink has to carry multiple tagged VLANs out to an external managed switch (common when Tier 1 or Tier 2 hardware from Part 3 has only one spare NIC), enable that

bridge's VLAN-aware option and tag each VM's virtual NIC with the correct VLAN ID, rather than relying on separate bridges alone.

ESXi's equivalent is a vSwitch with no physical uplink and a VLAN ID set per port group; VirtualBox calls the same concept an "internal network," with no host adapter attached; Hyper-V calls it a "private" virtual switch. Whatever the product, the question to ask before powering on a single VM is identical: does this virtual switch have a path to a physical NIC, and if so, what else is on that NIC?

Engineering Reality Proxmox's default `vmbr0` bridge is wired directly to the host's physical NIC out of the box — every VM you create and forget to reassign inherits a live path to whatever that NIC is plugged into, which on a single-host lab is usually your home router's own switch port. Vendor documentation on "creating an isolated network" tends to show the clean case — a bridge with no uplink at all — and rarely warns you that the default bridge is the opposite of isolated. Check every VM's network device against Table 4.1 before you power it on, not after; a VM that's already running with the wrong bridge assignment has already had however long it's been up to reach whatever `vmbr0` reaches.

4. Default-deny egress and firewall rule design

[CONCEPT] Most consumer routers ship default-allow on outbound traffic — anything inside can reach anything outside unless you add a rule blocking it. This book's segmentation model inverts that: nothing crosses a segment boundary unless a rule explicitly allows it, and the rule names a specific source, destination, and port rather than a whole segment. The inbound side of that (what can reach the honeynet from the internet) gets the obvious attention; the outbound side is where the actual risk sits, because the danger in a compromised honeypot or a compromised endpoint isn't the initial foothold — it's what that foothold can then reach.

[SAFETY] The honeynet segment's outbound rule is the tightest in the whole plan: it may reach the SIEM's log-ingest port and nothing else — not a public DNS resolver, not an NTP server, not the lab's own internal resolver from Part 7 on any port other than the one the SIEM needs. If a decoy service needs DNS resolution to function, it resolves against the lab's internal resolver over an explicitly allowed path, logged the same way every other honeynet action is logged (Part 15) — it never gets a general-purpose route to "the internet" just because that would be more convenient to configure.

Table 4.2 is the rule set Part 6 actually implements on the firewall/router VM. Read it here as the design to sanity-check before any product-specific configuration begins.

Table 4.2 — Default-deny segmentation rule set (CONCEPTUAL SAMPLE).

Source	Destination	Port/Protocol	Action
VLAN 40 (Honeynet)	Internet	any	Deny
VLAN 40 (Honeynet)	VLAN 10/20/30	any	Deny
Internet	VLAN 40 (Honeynet)	22 , 80 , 443 (decoy ports)	Allow
VLAN 40 (Honeynet)	SIEM host (VLAN 10)	5044 /tcp (log forwarding)	Allow
VLAN 10 (Management)	VLAN 20 (Lab/victim)	22 /tcp, 5985 /tcp, 5986 /tcp (admin SSH/WinRM)	Allow
VLAN 10 (Management)	VLAN 30 (NSM)	22 /tcp	Allow
VLAN 10 (Management)	VLAN 40 (Honeynet)	22 /tcp, from a single jump-host address only	Allow
VLAN 20 (Lab/victim)	VLAN 10 (Management)	any	Deny
VLAN 20 (Lab/victim)	SIEM host (VLAN 10)	1514 /tcp, 1515 /tcp	Allow
VLAN 30 (NSM)	SIEM host (VLAN 10)	5044 /tcp (log forwarding)	Allow
VLAN 30 (NSM)	Internet	any	Deny
Admin workstation	VLAN 10 (Management)	443 /tcp, 22 /tcp	Allow
any	any	any	Deny

Two rows deserve a second look before you treat this table as complete. First, the admin SSH/WinRM row exists because Table 4.1 promises the lab/victim VLAN accepts inbound admin access from the management VLAN — without an explicit allow rule for it, the terminal deny-all row below would silently block the exact access path Table 4.1 says exists, and you'd spend an afternoon debugging a "broken" WinRM connection that the rule set never actually permitted. Second, the **VLAN 30 (NSM) → SIEM host** row is not the sensor's mirror/SPAN capture interface talking — Section 1 established that interface gets no routable address at all — it's the sensor's separate management-path interface, which sits on the NSM VLAN itself rather than on VLAN 10, reaching the SIEM through this one explicit, narrowly-scoped allow rule instead of an implicit, unrouted "it's already on the management segment" pass. The **VLAN 30 (NSM) → Internet: Deny** row above stays anyway, as a belt-and-suspenders rule limiting that same interface to the one destination it actually needs.

The last row is the one that matters most, and the one most SOHO routers don't ship with: an explicit, terminal deny-all rule that catches everything the rows above it didn't name. Without it, "default-deny" is a design intention the device never actually enforces — most firewall products evaluate rules top to bottom and fall through to an implicit allow if nothing else matched, which is the opposite of what this table needs.

5. Where isolation quietly breaks

[TROUBLESHOOTING] None of the following are exotic. Every one of them is a normal, easy mistake that produces a lab that looks correctly segmented in the VLAN plan and isn't, in practice, isolated at all.

- **A VM with two NICs, one on each side.** A guest given a second virtual NIC "for convenience" — one on a lab VLAN, one bridged to the physical/home network — turns that VM into an accidental router the moment IP forwarding is enabled anywhere on it. This doesn't require the operator to deliberately enable forwarding: Windows Internet Connection Sharing and a default

`net.ipv4.ip_forward=1` left over from an unrelated tutorial both do it silently, with no warning that the box just became a bridge between two networks that were supposed to have zero route to each other.

- **A forgotten port-forward on the home router.** An old inbound port-forward pointed at a lab host's former IP address, left in place after that VM was torn down, silently exposes whatever DHCP hands that address to next — which might be an unrelated home device, or a rebuilt lab VM that never expected to be internet-reachable.
- **An unmanaged switch inserted mid-chain.** A VLAN plan that's logically correct on paper still flattens to one broadcast domain the moment a cheap unmanaged switch gets added between the trunk port and a lab device — most unmanaged switches don't understand 802.1Q tags at all and either drop tagged frames or (worse, depending on the switch) pass them through untagged to every port.
- **A DNS query that leaks off the lab.** A lab VM configured to fall back to a secondary DNS server, or an OS feature that races queries across every active network interface (Windows' multi-homed name resolution behavior is the common culprit), can end up asking your home network's own resolver to look up lab hostnames — which both defeats the isolation model and hands your home network a record of lab activity it was never supposed to see. Part 7 builds the lab's own resolver specifically so this doesn't need to happen; verify every lab VM actually uses it exclusively.
- **A "just for troubleshooting" rule that never gets removed.** The single most common way a firewall rule set drifts from Table 4.2 to something looser: a temporary any-any rule added to diagnose a connectivity problem, which fixes the symptom immediately and then sits there indefinitely because the underlying issue got "solved" and nobody circled back to remove the workaround.

Blind Spot Everything in this part addresses network-layer isolation — VLANs, virtual-switch boundaries, firewall rules between segments. None of it protects you if the hypervisor host itself is compromised (a VM-escape bug, or malware on the physical machine hosting all of these virtual switches) or if the workstation you administer the lab from is also the one you use for everyday browsing and gets compromised independently of anything in the lab. Network segmentation limits what a compromised lab guest can reach; it does nothing at all for a compromised lab operator or a compromised hypervisor host, because both of those sit above the boundary this part draws, not inside it.

6. Verifying isolation before you build anything on it

[HANDS-ON LAB] Goal: confirm, empirically, that the topology in Table 4.1 and the rule set in Table 4.2 actually behave the way they're written before any other part in this book adds a single component on top of them.

1. From a VM on the lab/victim VLAN, ping the home network's router/gateway IP address. Expect no reply.

2. From the same VM, run a traceroute (`tracert` on Windows, `traceroute` on Linux) to any known home-network host's IP address. Expect it to fail at the lab firewall's own hop — it should never resolve a hop beyond that boundary.
3. From a VM on the honeynet VLAN, attempt an outbound connection to an arbitrary internet host on port 80 — for example, `curl http://example.com` on Linux. Expect a timeout. Only the specific inbound decoy ports and the one outbound logging path in Table 4.2 should work from this segment; nothing else should.
4. From a VM on the lab/victim VLAN, resolve any DNS name and confirm the answering resolver is the lab's own internal one (Part 7), not an address outside the lab.

Validation Test Setup: *At least one VM on the lab/victim VLAN, and the firewall/router VM from Part 6 (or, if Part 6 hasn't been built yet, whatever device currently sits between the lab segment and your home network) enforcing Table 4.2's rules. **Action:** `ping <home-network-gateway-IP>` from the lab VM. **Expected result:** The request times out or returns "destination unreachable" — zero replies. A single successful reply means a route exists that shouldn't, and nothing else in this book should be built on top of this topology until that route is found and removed.*

7. Hardware and licensing implications of segmentation

[COST/RESOURCE] Segmentation done properly is a real line item, not a free upgrade to a topology you were building anyway. A managed switch with genuine 802.1Q trunking support, if your build needs one, runs roughly \$30–80 for a small home-lab-sized 8-port model — Part 3's hardware tiers should already have this in the parts list if your chosen topology pattern from Part 2 needs more physical ports than you have. A firewall/router VM (pfSense or OPNsense, built in Part 6) is a light guest by this book's standards — budget roughly 1–2 vCPUs and 1–2GB of RAM for it at idle — but it's still a permanent, always-on tenant on whatever hypervisor Part 5 builds, and it needs to be counted against Part 3's resource tiers alongside the SIEM and every endpoint, not treated as free overhead because it's "just a firewall."

If your single-host build only has one built-in NIC and needs a second or third physical uplink to reach an external managed switch, a USB Ethernet adapter is the usual fix — and the usual place this specific build quietly breaks.

Lab Note *If your firewall/router VM's extra NICs are USB Ethernet adapters, buy ones with a known-good Linux driver reputation before building anything else on top of them. A cheap USB-C NIC that drops packets under load will make every isolation check in Section 6 look like a segmentation bug when it's actually a \$10 adapter falling over. Intel- and ASIX-chipset adapters are the safe default; unbranded RTL8153 clones are the ones that cause this — and they're also, unhelpfully, the cheapest and most common ones sold under a dozen different storefront names.*

8. Making isolation a habit, not a one-time check

[SAFETY] Section 6's checks confirm the topology is correct on the day you build it. They don't stay true automatically. Every time you add a new decoy service to the honeynet, join a new endpoint to the lab/victim VLAN, or change a firewall rule for any reason — including a "just for troubleshooting" rule from Section 5's list — re-run the same checks rather than assuming the isolation you verified once is still the isolation you have now. Appendix A4 expands this into the full, step-by-step pre-flight checklist that Part 15's expanded Safety Gate and Part 18's mandatory pre-flight checklist both cite by name rather than repeating in full; this part defines the controls, Appendix A4 operationalizes them into something you actually run, checkbox by checkbox, before powering on anything intentionally vulnerable. Appendix A4 is not yet authored (see [README.md](#)) — until it is, re-running Section 6's checks above is the actual checklist.

9. What this feeds into the rest of the series

[CONCEPT] This part builds no telemetry by itself — it's architecture, not a log source. But once Part 6 turns Table 4.2 into a real firewall build, the deny-log entries that firewall generates every time something tries to cross a boundary it shouldn't are the very first telemetry this entire book produces, arriving before Part 8 has a SIEM to put them in and long before Part 9's `auditd` rules or Part 10's Sysmon config exist. Forward that firewall's deny log into the SIEM the moment Part 8 is standing, even before any endpoint is joined — a lab with no endpoints yet still has a firewall worth watching.

That telemetry also produces one of the cleanest detections a home lab can generate, and it's worth handing forward explicitly rather than leaving it as an incidental fact about this part:

- **Detection Engineering Handbook V2** teaches baselining and anomaly detection (its Parts 22–33) on telemetry that usually has to be filtered hard to separate signal from normal noise. A firewall deny event for traffic this part's rule set says should structurally never happen — honeynet-to-management, lab/victim-to-management outside the allowed admin path, anything sourced from the NSM segment heading to the internet — has no legitimate reason to exist at all. Build a detection rule keyed on exactly those source/destination pairs from Table 4.2, and you have a near-zero-false-positive alert before that volume's more general correlation logic is even needed. That's a more reliable starting detection than most signature-based rules this lab will ever produce, precisely because Table 4.2 already defines what "should never happen" means with no ambiguity.
- **SOC Playbook Handbook's** Playbook Library assumes an alert arrives already tagged with enough network context to route correctly — the same login-anomaly shape from the lab/victim VLAN and from the honeynet VLAN calls for two entirely different playbook branches, one a real investigation and one the decoy behaving exactly as designed. Tag every event this lab produces with its source VLAN at ingest, from the very first firewall log Part 8 receives, so that field exists for triage later instead of being reconstructed after the fact from IP ranges.
- **SOC Manager's Operating Handbook** treats a working, verifiably isolated lab as the substrate its Part 8 assessment design and Part 11 onboarding/ramp-up content run on top of — you can't score a repeatable practice drill consistently, or run Part 18's attack-simulation tooling against a team in training, on a network topology nobody can describe with confidence. This part's

verification habit (Section 6, and Appendix A4 in full) is what makes that substrate trustworthy enough to build a drill on, not an optional hardening step layered on afterward.

None of that cross-book handoff is content this part teaches directly — it's the reason getting this part's design right, and re-verifying it every time something changes, pays off three books later instead of just one.

Cross-references: This book — Part 1 (the ownership/isolation rule), Part 2 (the management/lab-victim/honeynet vocabulary this part builds against), Part 3 (hardware tiers this segmentation runs on), Part 5 (hypervisor install hosting these virtual switches), Part 6 (the real firewall build enforcing Table 4.2), Part 7 (the internal resolver Section 5's DNS-leak check depends on), Part 9, Part 10 (endpoints living on the lab/victim VLAN), Part 12 (NSM sensor placement on the mirror-only VLAN), Part 15 (honeynet, expanded Safety Gate), Part 18 (attack-simulation pre-flight checklist), Appendix A1 (diagram templates), Appendix A4 (the full pre-flight checklist). Detection Engineering Handbook V2 — Parts 22–33 (baselining/anomaly detection on this part's deny-log telemetry). SOC Playbook Handbook — Playbook Library (VLAN-tagged alert triage). SOC Manager's Operating Handbook — Part 8, Part 11 (assessment design and onboarding on top of a verified-isolated lab).

Section B — Core Platform Build

neshbooy

Part 5 — Choosing and Installing a Hypervisor

Why this part exists

Part 3 gave you hardware tiers and a resource budget. Part 4 gave you the isolation architecture you'll eventually build. Neither of those parts put a hypervisor on a disk. This part does exactly that: it compares the handful of virtualization platforms a home-lab builder actually chooses between, then walks through a real installation start to finish on the platform this book's own author runs.

This part stops at "the hypervisor boots, the web console loads, and the host has one network bridge with no isolation on it yet." It does not create your first guest VM (that starts in Part 6, with the firewall VM), and it does not build the segmented network Part 4 designed — a fresh hypervisor install has exactly one flat bridge and zero isolation by default, and that's fine at this stage because nothing running on it yet needs protecting from anything. What this part hands you is a working, updated Type-1 or Type-2 platform, sized correctly for the tier you picked in Part 3, ready for Part 6 to start building on top of.

Safety Gate *Nothing in this part deploys a honeypot, a deliberately vulnerable service, or attack-simulation tooling — a bare hypervisor install has nothing on it worth attacking, so it's fine for the host to sit on your home LAN during this part exactly like any other appliance. The checkable condition that matters here is a forward one: do not create a vulnerable practice VM, a decoy service, or an Atomic Red Team target directly on this host's default bridge once it's running. That guest goes on the isolated segment Part 4 designed and Part 6 builds — verified against the Appendix A4 pre-flight checklist — not on the flat network this install leaves behind. If you're already itching to spin something interesting up the moment the web UI loads, that's the one thing to hold off on.*

1. Type-1 versus Type-2, and why the distinction is worth caring about

1.1 Bare metal versus hosted

[CONCEPT] A Type-1 hypervisor runs directly on hardware, with no general-purpose host operating system underneath it — Proxmox VE, ESXi/vSphere, and Hyper-V (when the "Hyper-V role" takes over the boot process on Windows Server or a Windows client with Hyper-V enabled) are all Type-1 in this sense. A Type-2 hypervisor runs as an application inside a host OS you're also using for other things — VirtualBox on your daily-driver Windows or Linux desktop is the clearest example.

The difference isn't academic. A Type-1 host dedicates its RAM, CPU, and disk to virtualization and nothing else; a Type-2 host is splitting resources with whatever else the host OS is doing, including the browser tabs and background updates you didn't think about when you budgeted RAM in Part 3.

1.2 Why this matters for an always-on home lab

[COST/RESOURCE]

A lab that runs 24/7 — a SIEM ingesting continuously, a honeynet listening for opportunistic scans, Zeek watching a mirrored port — behaves differently under a Type-1 host than under a Type-2 one running on a machine you also use for email and video calls. Every restart of your daily-driver OS (a Windows update, a sleep/wake cycle, a crash) takes your entire lab down with it if it's Type-2. A dedicated Type-1 box just keeps running.

This doesn't make Type-2 wrong — it makes it the right choice for a specific situation: Part 3's Tier 1 (a repurposed laptop or mini-PC you don't want to dedicate, or a Windows machine you're not ready to wipe). Tier 2 and Tier 3 readers, dedicating a box specifically to this lab, should default to Type-1.

2. Matching a hypervisor to your hardware and your goal

2.1 The decision drivers

[CONCEPT]

Four questions decide this for almost every reader:

- Is this a dedicated box you can wipe, or a machine you still need for something else?
- Do you eventually want a Windows Server domain lab (Part 11), or are you staying Linux-first?
- Do you already have a paid vSphere entitlement through work, or are you starting from zero?
- Is "runs entirely on hardware you own" actually a requirement, or is a cloud VM's real billing and lack of a home-isolation story acceptable to you?

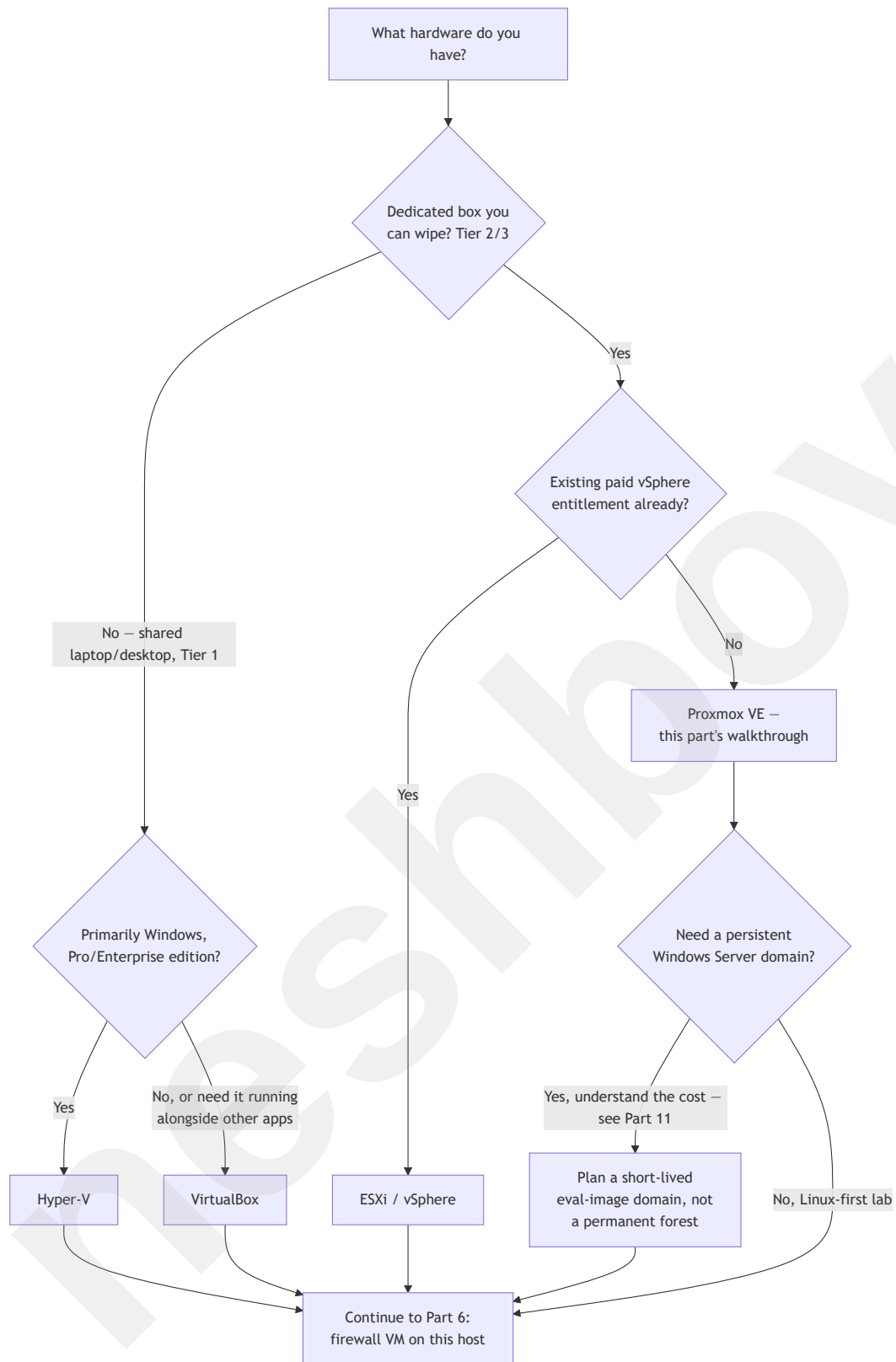
The table below maps the five realistic options against Part 3's hardware tiers and against those questions.

Hypervisor	Type	Cost	Best-fit tier (Part 3)	Windows-domain friendliness	Home-isolation story
Proxmox VE	Type-1, Linux/KVM+LXC	Free, no-subscription repo	Tier 2–3 (dedicated box)	Good — runs Windows guests fine, no native licensing help	Full — you own every bridge and firewall rule
ESXi/vSphere	Type-1	Free tier discontinued for new users after the Broadcom acquisition; otherwise licensed	Tier 2–3, mainly for readers with an existing work entitlement	Excellent — VMware and Windows Server are a long-standing pairing	Full, same as Proxmox
VirtualBox	Type-2	Free	Tier 1 (shared laptop/desktop)	Workable for a single standalone host, clumsy for more	Manual — host-only/internal networks need deliberate setup per Part 4
Hyper-V	Type-1 role on Windows	Free with Windows Pro/Enterprise/Server	Tier 1–2, if the box is already running Windows	Excellent — native Windows Server support, easiest path to Part 11's eval-image domain	Full via Hyper-V virtual switches, but fewer isolation primitives than a dedicated firewall VM
Cloud VM (AWS/Azure/GCP)	Type-1, provider-managed	Real, ongoing billing	Out of this book's main scope — see Part 22	Excellent, but licensing/cost adds up fast	None of this book's kind — isolation is the provider's VPC model, not a home network you fully control

Figure 5.1 — Hypervisor selection decision flow. *CONCEPTUAL.* Illustrates the decision path a reader follows from "what hardware do I actually have" to a specific hypervisor choice; it is a teaching sketch of the logic in §2.1, not a capture of a real decision-support tool.

flowchart TD

```
A[What hardware do you have?] --> B{Dedicated box you  
can wipe? Tier 2/3}  
B -->|No – shared laptop/desktop, Tier 1| C{Primarily Windows,  
Pro/Enterprise edition?}  
C -->|Yes| D[Hyper-V]  
C -->|No, or need it running  
alongside other apps| E[VirtualBox]  
B -->|Yes| F{Existing paid vSphere  
entitlement already?}  
F -->|Yes| G[ESXi / vSphere]  
F -->|No| H[Proxmox VE –  
this part's walkthrough]  
H --> I{Need a persistent  
Windows Server domain?}  
I -->|Yes, understand the cost – see Part 11| J[Plan a short-lived  
eval-image domain,  
not  
a permanent forest]  
I -->|No, Linux-first lab| K[Continue to Part 6:  
firewall VM on this host]  
D --> K  
E --> K  
G --> K  
J --> K
```



2.2 Resource Reality — hypervisor overhead itself

[COST/RESOURCE]

Resource Reality Proxmox VE's own footprint at idle is small — a few hundred megabytes of RAM for the management stack — but that's not the number that matters. What matters is what's left after every guest's allocation, and on real hardware that number gets tight faster than the raw specs suggest. On this book's own reference host — an Intel Core i5-4590 (4 cores, no hyperthreading) with 15GiB of RAM — `free -h` at doc time showed 6.7GiB used, 504MiB genuinely free, and 8.7GiB in buffer/cache, with 944MiB of an 8GiB swap file already in use — mild swap pressure, not yet critical, but worth watching — running fewer than a dozen lightweight LXC containers. If your Tier 2 box is at or below this spec, budget guest RAM conservatively — this isn't a "runs a little slow" ceiling, it's the point where the host starts swapping under normal load, not just peak load.

3. Why this book's walkthrough uses Proxmox VE

3.1 The rationale, and the real lab behind it

[CONCEPT] This book's hands-on installation steps target Proxmox VE specifically, for a concrete reason: it's what the author actually runs. The same host this part walks through installing is the base of the running Proxmox environment cited as a REAL LAB EXAMPLE throughout *Detection Engineering Handbook V2* — the honeynet (CT103, CT108, CT113), the Pi-hole resolver (CT100), and the vulnerability-scanner platform (CT104) all sit on this exact box. Choosing a different hypervisor changes specific steps in §4 and §6 below, not the overall shape of the book — Proxmox is this part's worked example, not a requirement to follow this book.

The hardware itself is unglamorous on purpose: an HP EliteDesk 800 G1 TWR desktop, Intel Core i5-4590 @ 3.30GHz (4 cores, no hyperthreading), 15GiB RAM, a 931.5GiB SSD as the primary disk, and a second, currently-unused 119.2GiB SSD held as spare capacity. Proxmox VE 9.2.2 on Debian GNU/Linux 13 (trixie), kernel 7.0.2-6-pve. Nothing about this specification is exotic — it's a nine-to-ten-year-old office desktop, which is precisely the point: Part 3's Tier 2 doesn't require new hardware, and this book's own real evidence is built on proof of that.

3.2 Build Autopsy — the e1000e crash nobody warns you about in the installer

Build Autopsy — the network card that took the host down four times

The plan: Install Proxmox VE on the box's single onboard NIC (an Intel I217-LM, `e1000e` driver) using every installer default, including default network offload settings, since the installer doesn't expose an offload toggle and there was no obvious reason to touch one.

Why it seemed reasonable: TCP Segmentation Offload (TSO) and Generic Segmentation Offload (GSO) are enabled by default on essentially every Linux NIC driver because they reduce CPU overhead under load — turning them off is something you'd normally do to fix a problem, not something you'd pre-emptively do on a fresh install with no symptoms yet.

How it failed: Four abrupt crashes in roughly 27 hours (confirmed via `last -x`) — Sep 8 20:51, Sep 8 22:28, Sep 9 22:21, and Sep 9 23:52 — traced through `journalctl -b -1` to the `e1000e` driver's well-documented "Hardware Unit Hang" bug on the I217/I218 NIC family under sustained load with TSO enabled. This isn't a Proxmox bug or a misconfiguration; it's a known upstream driver issue that just happens to surface once a NIC is finally pushed hard enough — which a home lab, unlike a lightly-used desktop, will eventually do.

The fix: Disable both offloads on the affected NIC and make the change survive reboots, not just the current session.

The fix targets Debian's `/etc/network/interfaces` on Proxmox VE 9.2, and applies at runtime immediately, before the persistent change takes effect on the next boot.

```
# Immediate fix – takes effect without a reboot, but does not survive one on its own
ethtool -K nic0 tso off gso off
```

Confirm the runtime change held before moving on to the persistent step: `ethtool -k nic0` should report `tcp-segmentation-offload: off` and `generic-segmentation-offload: off`. Make it persistent by adding a `post-up` line under the NIC's stanza in `/etc/network/interfaces` (substitute your own interface name for `nic0`):

```
iface nic0 inet manual
    post-up /sbin/ethtool -K nic0 tso off gso off
```

Lab Note If your host uses an Intel I217/I218-family NIC (`e1000e` driver — check with `ethtool -i <interface>`), apply this fix during initial setup, before you've moved anything critical onto the host. Waiting for the first crash to teach you this costs a corrupted VM disk or two if the crash lands mid-write; applying it pre-emptively costs one command.

4. Installing Proxmox VE step by step

4.1 Before you start

[SETUP] Confirm the following before booting the installer — skipping any of these turns a 20-minute install into a repeat trip to re-flash USB media.

Check	What to verify	Why it matters
Virtualization extensions	Intel VT-x/AMD-V enabled in BIOS/UEFI	Proxmox refuses to start most guests without it; some BIOS revisions ship this disabled by default
ISO source	Downloaded from <code>proxmox.com/downloads</code> , checksum verified	A corrupted ISO produces confusing mid-install failures, not a clean error
Install media	USB flashed with <code>dd</code> , Rufus, or Balena Etcher in "DD image" mode, not a plain file copy	A file-copied ISO won't boot on most firmware
Target disk identified	Know which physical disk (by size, not just device letter) is the intended install target	The installer will happily erase the wrong disk if you guess wrong — verify by size against what you documented in Part 3
Network plan	Static IP, gateway, and DNS server decided ahead of time	The installer asks for these during setup; improvising them mid-install is where typos happen

4.2 Running the installer

[SETUP] This procedure targets the Proxmox VE 9.2 graphical installer, booted from USB on UEFI or legacy BIOS firmware.

1. Boot from the installer USB and select **Install Proxmox VE** (not the debug or rescue options) at the boot menu.
2. Accept the EULA.
3. Select the target disk. If you have more than one disk (as this book's reference host does — a 931.5GiB primary and an unused 119.2GiB spare), double-check you've selected the one you intend by size, not by slot position.
4. Under **Options**, choose the filesystem: `ext4` on LVM (this book's reference build) is the simpler, more universally-documented choice; `ZFS` (RAID-Z variants) adds checksumming and snapshots but wants more RAM and, ideally, more than one disk to be worth its overhead — most Tier 2 single-disk builds should stay on `ext4`/LVM unless you specifically want `ZFS`'s features.
5. Set the country, time zone, and keyboard layout. Get the keyboard layout right — a mismatch here silently changes which physical key produces which character in the root password you're about to set, which surfaces later as password entries that mysteriously "don't work" at first boot.
6. Set the root password and an admin email address (used for the host's own automated notification emails — a real address you check, not a placeholder).
7. Configure the management network: hostname (FQDN, e.g. `pve.home.local`), the NIC to bind to, static IP/CIDR, gateway, and DNS server.
8. Review the summary screen and confirm. This is the last point before the target disk is wiped — verify the disk selection one more time here.
9. Let the install finish and reboot when prompted, removing the USB media first.

4.3 The three failures that eat the most time

[TROUBLESHOOTING]

- **"No bootable device" or the installer never appears.** Almost always a USB written as a plain file copy instead of a raw disk image, or UEFI Secure Boot blocking an unsigned bootloader. Re-flash with a tool that writes a raw image (Rufus in "DD" mode, `dd`, or Balena Etcher), and try disabling Secure Boot if the media still won't boot.
- **Installer completes, but guests won't start / no virtualization extensions detected.** Reboot into BIOS/UEFI setup and enable VT-x (Intel) or AMD-V (AMD) — some vendors ship it off by default, and Proxmox's own installer doesn't hard-block on this, so the failure only surfaces the first time you try to start a VM.
- **Wrong disk wiped.** No fix — this is why step 3 and step 8 above both exist as disk-selection checkpoints. If it happens, the lesson is procedural (verify by size, twice), not technical.

5. Storage layout: the default split, and the trap inside it

5.1 What the installer actually creates

[CONCEPT] On a single-disk ext4/LVM install, the Proxmox installer creates a volume group spanning the whole disk, then splits it into a modest root logical volume (mounted at `/`, holding the OS and the `local` storage pool — ISOs, templates, and every guest backup), a swap volume, and a large thin-provisioned pool (`local-lvm`) for guest disk images. The installer's default root LV size is conservative — sized for the OS, not for a lab that will accumulate backups, ISOs, and container templates for months.

5.2 Engineering Reality — where that default split actually goes wrong

Engineering Reality Proxmox's documentation frames `local` and `local-lvm` as separate concerns — OS/backups versus guest disks — and on paper that separation looks clean. In practice, on this book's own reference host, the `root` logical volume (96GiB) sits at 92% used while `local-lvm`'s thin pool (794.30GiB) sits at 2.47% used, because `local` — which lives on the root filesystem — is absorbing every `vzdump` backup, every LXC rootfs stored as a raw file rather than an LVM volume, and every downloaded ISO and container template. A reader watching only "how full is my storage pool" in the Proxmox UI can miss this entirely, because `local-lvm`'s near-empty gauge looks fine while the actual filesystem underneath `local` quietly fills up. Check `df -h /` directly, not just the storage-pool percentages in the web UI.

5.3 What to do about it at install time

[SETUP] Two changes at install time avoid this trap entirely rather than fixing it later:

1. In the installer's **Options** screen, use the advanced LVM options to increase `hdspace`/root sizing beyond the default if the disk has room — this book's reference host would have avoided its capacity warning entirely with a root LV closer to 150–200GiB rather than 96GiB, given the same disk was carrying 6+ months of backups and ISOs.

2. Plan, from day one, to move backup targets off the local disk once you add any kind of network storage (an NFS share, a second physical disk, cloud backup) — Part 20 covers retention policy in depth, but the decision to *not* let every backup default to `local` forever is cheapest to make before the disk fills, not after.

6. First boot: reaching the web UI and the minimum hardening pass

6.1 Logging in and switching repositories

[SETUP] This targets a freshly-installed Proxmox VE 9.2 host with no changes made yet beyond the installer defaults.

1. From any machine on the same network, browse to `https://<host-ip>:8006`.
2. Accept the browser's self-signed-certificate warning (expected on a fresh install — Proxmox generates its own cert, and there's no CA behind it yet).
3. Log in with username `root` and the password set during installation, realm `Linux PAM`.
4. Proxmox VE ships pointed at the enterprise repository by default, which requires a paid subscription to update from. Switch to the free no-subscription repository so `apt update` actually pulls updates without a subscription key:

```
# Disable the enterprise repo (requires a paid subscription key)
sed -i 's/^deb/#deb/' /etc/apt/sources.list.d/pve-enterprise.list

# Add the no-subscription repository (community-supported, no key required)
echo "deb http://download.proxmox.com/debian/pve trixie pve-no-subscription" \
  > /etc/apt/sources.list.d/pve-no-subscription.list

apt update && apt full-upgrade -y
```

5. Reboot if the kernel was updated (`pveversion -v` will show a kernel version mismatch against `uname -r` if a reboot is pending).

Validation Test Setup: Proxmox VE installed and rebooted per §4.2, repository switch from §6.1 applied. **Action:** From the host console (or SSH), run `pveversion` and `uptime`; from a browser on the same network, load `https://<host-ip>:8006`. **Expected result:** `pveversion` reports a version string beginning `pve-manager/9.x`; `uptime` shows the host has been up since the post-install reboot with no unexpected restarts; the web UI loads the login screen without a connection-refused or certificate-mismatch error blocking access entirely (the self-signed-cert browser warning is expected and not a failure).

6.2 What "no isolation yet" actually means at this point

[SAFETY] The host you just built has one bridge (`vbr0`), bound to your one physical NIC, carrying host management traffic and every future guest's network traffic on the same flat segment as your home LAN. That is the correct, expected state for a hypervisor with zero guests on it — there is nothing

here yet for isolation to protect. It stops being correct the moment you create a guest meant to be intentionally vulnerable, attacked, or exposed as a decoy: that guest needs the segmented, default-deny network Part 4 designs and Part 6 builds, verified against Appendix A4, before it powers on. This part's install doesn't build that segmentation — don't mistake "the hypervisor is running" for "it's safe to put a honeypot on it."

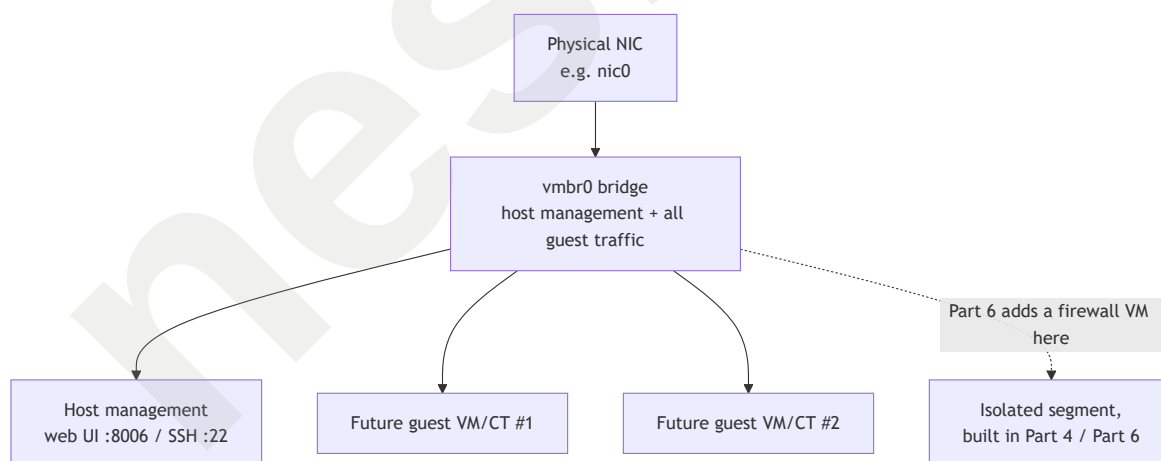
7. The networking foundation this install leaves behind

7.1 One bridge, one NIC, everything sharing it

[CONCEPT] A default single-NIC Proxmox install creates exactly one Linux bridge, `vmbr0`, with the physical NIC as its sole member. Host management (the web UI, SSH) and every guest's virtual NIC attach to the same bridge, which means — at this stage — a guest's traffic is indistinguishable, switch-side, from the host's own management traffic. This is normal for a fresh install and is exactly what Part 4 and Part 6 exist to change.

Figure 5.2 — Default single-NIC bridge topology after a fresh install. *CONCEPTUAL.* Illustrates what a stock Proxmox install looks like network-wise immediately after §6 — one bridge, one physical NIC, host management and future guests all sharing it — before Part 6 adds a firewall VM and Part 4's isolated segment exists. The general shape mirrors the author's own running host's bridge layout, but this is a teaching sketch, not an exported diagram of a specific point-in-time configuration.

```
graph TD
    NIC[Physical NIC<br/>e.g. nic0] --> VMBR0[vmbr0 bridge<br/>host management + all guest traffic]
    VMBR0 --> HostMgmt[Host management<br/>web UI :8006 / SSH :22]
    VMBR0 --> GuestA[Future guest VM/CT #1]
    VMBR0 --> GuestB[Future guest VM/CT #2]
    VMBR0 -.Part 6 adds a firewall VM here.-> Future[Isolated segment,<br/>built in Part 4 / Part 6]
```



7.2 What this install cannot tell you about isolation

[CONCEPT]

Blind Spot A hypervisor install, by itself, cannot tell you whether your physical network is actually isolated from anything — that's a property of your router, switch, and VLAN configuration, none of which the Proxmox installer touches or verifies. A clean install with no errors gives you zero assurance that a guest you create later can't reach your home LAN; it only means the hypervisor software itself is working. Treat "the install succeeded" and "isolation is verified" as two completely separate claims, because they are — Part 4's architecture and Appendix A4's checklist are what actually establish the second one.

8. Other hypervisors: what changes if you don't run Proxmox

8.1 VirtualBox

[CONCEPT] VirtualBox is the right default for Part 3's Tier 1 reader: a shared laptop or desktop you're not ready to dedicate or wipe. Installation is a normal application install on top of whatever OS you're already running. The tradeoff is networking: isolating a VirtualBox guest requires deliberately choosing Host-Only or Internal Network adapter modes per VM rather than getting a segmented network by default, and running more than two or three guests at once on a laptop's RAM budget gets uncomfortable fast — revisit Part 3's Tier 1 sizing before assuming VirtualBox alone can carry a full SIEM-plus-endpoints-plus-NSM stack.

8.2 Hyper-V

[CONCEPT] Hyper-V ships as an optional Windows feature on Windows 10/11 Pro, Enterprise, and Education, and as a role on Windows Server. Enabling it turns the Windows kernel itself into a Type-1 hypervisor host, which has one practical consequence worth knowing before you enable it: older versions of other Type-2 hypervisors (VirtualBox, VMware Workstation) couldn't run guests at all once Hyper-V was active, because both wanted exclusive control of the same CPU virtualization extensions. Current VirtualBox versions work around this by running through the Windows Hypervisor Platform when Hyper-V is enabled, but pay a real performance cost compared to running with Hyper-V off — if you want both, test which combination your specific versions tolerate before committing a build to it. Hyper-V is the most natural path to Part 11's short-lived eval-image Windows Server domain, since Windows Server is a first-party guest on its own native hypervisor.

8.3 ESXi / vSphere

[CONCEPT] ESXi is a mature, thoroughly-documented Type-1 hypervisor with decades of enterprise data-center deployment behind it, and if you already hold a vSphere entitlement through work, it's a legitimate choice for a home lab. The practical reality for a reader starting from zero: Broadcom discontinued the free ESXi hypervisor download for new users following its acquisition of VMware, so this option is now realistically available only to readers with an existing entitlement, not a "download it free tonight" option the way it was for years. This book doesn't walk through an ESXi install for that reason — most readers can't reproduce it without a license they don't have.

8.4 Cloud VMs

[CONCEPT] A cloud provider's VM (AWS EC2, Azure, GCP Compute Engine) is a real Type-1-backed virtualization platform, but it changes the book's entire premise: you're paying real, ongoing money per hour, and "isolated from my home network" is replaced by "isolated within the provider's VPC/VNet model," governed by that provider's terms of service rather than a switch in your own house. This book's safety architecture (Part 4) is built specifically around a home network you fully control; a cloud tenant is a different enough risk and cost model that it's deliberately out of this book's main scope. Part 22's closing synthesis points toward it as a next step once you've outgrown a home lab, not before.

9. What this install hands off

[CONCEPT] Even at this early stage, before a single lab-specific service exists, the host itself already produces one real telemetry source worth knowing about: its own authentication log and system journal (`journalctl`, `/var/log/auth.log`). *Detection Engineering Handbook V2*, Part 3 covers how to read Linux authentication and privilege-escalation telemetry once you're forwarding it, and the *SOC Playbook Handbook*'s Linux logging technical-reference chapters cover what to actually do when one of those events looks wrong — both apply the moment you forward this host's own logs into the SIEM Part 8 builds, not just to the guest endpoints Part 9 builds later. If you're tracking the actual hours a build like this costs — install, the offload fix, the storage remediation in §5.3 — that record is the same kind of input the *SOC Manager's Operating Handbook*'s Part 20 budget framing uses at enterprise scale, just at a single-operator size; Part 3 already draws this analogy for hardware cost, and it holds for build-time cost too.

What this part does not hand off yet is anything resembling a working lab. The hypervisor is running, updated, and reachable — that's the whole deliverable. Part 6 builds the firewall VM on top of it; Part 4's segmentation becomes real hardware and rules starting there, not here.

Cross-references: Part 3 (Hardware and Resource Budgeting — the tier this install targets), Part 4 (Network Isolation and Segmentation Architecture — the isolation this install explicitly does not yet build), Part 6 (Perimeter Firewall and Segmentation Build — the next guest created on this host), Part 8 (Building the SIEM Platform — where this host's own auth log eventually gets forwarded), Part 11 (The Windows Domain Lab Problem — relevant if §8.2's Hyper-V path leads toward a domain), Part 20 (Lab Maintenance, Patching, and Snapshot/Backup Strategy — the storage-growth trap from §5.2 in full), Appendix A2 (Hardware Tiers & Cost Worksheets), Appendix A4 (The Safety & Isolation Pre-Flight Checklist — required before any guest built on this host can be intentionally vulnerable or exposed); *Detection Engineering Handbook V2*, Part 3 (Linux authentication telemetry); *SOC Playbook Handbook*, Linux logging technical-reference chapters; *SOC Manager's Operating Handbook*, Part 20 (Building & Defending the SOC Budget, loose analogy only).

Part 6 — Perimeter Firewall and Segmentation Build

Why this part exists

[CONCEPT] Part 4 designed a lab network that cannot reach — and cannot be reached by — anything you don't own: four segments (Management, Endpoint, NSM, Honeynet), a default-deny posture between them, and a list of the specific ways isolation quietly breaks. This part is where that design stops being a diagram and becomes a running firewall VM with real interfaces, real VLAN tags, and a real rule base you can point traffic at and watch get blocked or allowed. If Part 4 is the blueprint, this is the pour-the-foundation part: nothing in Parts 7 through 21 — no SIEM, no endpoint, no honeypot, no attack simulation — goes on this network until the build in this part is in place and the isolation it claims to provide has actually been verified, not assumed.

Two things this part does not cover, on purpose. It does not configure Suricata or Snort rules, even though both run comfortably on the same firewall VM — that content, and the decision of whether to run an IDS inline or as a passive NSM sensor, belongs to Part 14. And it does not build the DNS/DHCP services that will actually hand out addresses on these VLANs beyond the bare minimum needed to prove connectivity — that's Part 7's job. What this part delivers is the box, the VLANs, and the rule base that every later part's traffic has to cross.

Safety Gate *Everything past this point assumes you're building on the hypervisor from Part 5, on hardware with no other production role in your household — this firewall VM's WAN interface will sit between your home network and every intentionally exposed or intentionally vulnerable service the rest of this book builds. Before wiring a single virtual NIC, confirm the physical switch port and virtual switch/bridge this VM's LAN-side interface attaches to are not shared with any device holding real credentials or real data, and that the Honeynet VLAN this part creates (§4) has no firewall rule, NAT rule, or static route back to your home subnet — not "blocked by default," but checked, per the Appendix A4 pre-flight procedure, after every rule-base change in this part, not just once at the end. Appendix A4 is not yet authored (see [README.md](#)); until it is, Part 4 §6 and §8 are the operative checks to re-run.*

1. What Part 4 designed and what this part actually builds

[CONCEPT] Part 4 settled on four VLANs, and this part keeps that vocabulary rather than inventing a new one:

- **Management** — the segment you administer from: the firewall's own web GUI, the hypervisor's management interface, and, once Part 8 builds it, the SIEM.
- **Endpoint** — the Windows and Linux hosts Parts 9 and 10 build, generating the Sysmon and auditd telemetry this whole book exists to produce.
- **NSM** — the sensor segment Parts 12 through 14 build out, watching traffic rather than participating as an ordinary peer on it.

- **Honeynet** — the decoy segment Part 15 populates, built specifically to be attacked and therefore held to the strictest isolation rule in the book.

Part 4 stated the rule that governs all four: default-deny between every segment, with only the specific paths a later part actually needs punched through as explicit allow rules. This part is where "default-deny" and "explicit allow" turn into an actual rule you write in a firewall GUI, in a specific order, evaluated by a specific engine — and where the gap between "I wrote a deny rule" and "the traffic is actually denied" gets checked instead of assumed.

2. Choosing a firewall platform: pfSense vs. OPNsense

[CONCEPT] Both pfSense Community Edition and OPNsense are free, FreeBSD-based, and capable of everything this part asks of a firewall: VLAN-tagged interfaces, a stateful packet filter, NAT control, and a web GUI you can drive without memorizing `iptables` syntax. Table 6.1 compares the two on the axes that actually matter for a home lab build, not on a feature checklist neither reader will exhaust.

Table 6.1 — pfSense Community Edition vs. OPNsense, home-lab-relevant differences.

Axis	pfSense CE	OPNsense
Base OS	FreeBSD, Netgate-maintained fork	FreeBSD/HardenedBSD, community fork of pfSense's original codebase
License	Apache 2.0 (CE)	BSD 2-clause
Rule evaluation	Interface rules evaluated top-down, first match; floating rules evaluated first regardless of tab	Same top-down/floating-first model, slightly different GUI grouping
Package ecosystem	Smaller; some packages moved behind Netgate's paid tiers over time	Broader plugin list retained free, including Suricata and Zenarmor
GUI update cadence	Slower, more conservative	Faster point releases, more frequent UI changes
Best fit for this book	Reader who wants the most-documented option with the largest base of third-party tutorials	Reader who wants a more actively developed plugin ecosystem, particularly if Part 14's IDS work matters to them

This part uses pfSense CE as the worked example because it has the deepest base of public documentation for a first-time build, and every step below states plainly which parts are pfSense-specific menu paths versus general firewall concepts that carry over to OPNsense with a different click path. Neither platform's install has been build-tested against this book's own hardware as of this draft — treat the exact menu paths as pfSense/OPNsense's own documented behavior at time of writing, and expect minor UI drift across versions.

Engineering Reality Both platforms' official documentation describes VLAN tagging as "just add a VLAN interface." In practice, the VLAN tag never reaches the firewall at all if the underlying virtual switch port group (or physical switch port, if this VM's NIC is passed through) isn't explicitly set to trunk/tagged mode — a port group left in "access" or "untagged" mode silently strips the tag before the firewall VM ever sees the frame, and the symptom is a VLAN interface that comes up but never receives DHCP requests or ARP traffic from anything, with no error on the firewall side at all.

3. Sizing and installing the firewall VM

[COST/RESOURCE] A firewall VM pushing home-lab traffic volumes — a handful of endpoints, a sensor, a honeypot fielding opportunistic internet scanning — is not resource-hungry compared to the SIEM in Part 8. Table 6.2 gives the floor.

Table 6.2 — Firewall VM sizing (CONCEPTUAL SAMPLE).

Resource	Minimum	Comfortable	What breaks below the minimum
vCPU	1	2	Rule-set evaluation and stateful tracking add latency under load; a single-vCPU firewall handling Suricata inline (Part 14) later saturates quickly
RAM	1GB	2GB	Below 1GB, the web GUI and package manager compete with the packet filter for memory and the GUI becomes sluggish or unresponsive during rule-base edits
Disk	8GB	20GB	Below 8GB, OS updates and firewall logs (which this book wants you keeping, not disabling) fill the disk within weeks
NICs	2 (WAN, LAN trunk)	2, virtio/paravirtualized	An emulated NIC type on some hypervisors drops VLAN tags in ways a paravirtualized NIC doesn't — check your hypervisor's specific guidance before troubleshooting a "missing" VLAN as a firewall config problem

Resource Reality None of this is where a home lab's resource budget actually goes — that's the SIEM (Part 8, 8GB alone) and any Windows domain attempt (Part 11). A 2GB, 2-vCPU firewall VM is a rounding error against Part 3's hardware tiers, which is exactly why there's no excuse to undersize the one component every other part's traffic has to cross.

[SETUP] These steps target pfSense CE 2.7.x running as a VM on the Proxmox host from Part 5; the console-level steps are nearly identical on OPNsense with different menu wording once you reach the GUI.

1. Create the VM in Proxmox with 2 vCPU, 2GB RAM, a 20GB virtual disk, and two virtio network interfaces — the first bridged to the WAN-facing virtual switch (the one with a path to your home network's internet egress), the second bridged to a dedicated internal virtual switch that will carry all four VLANs as trunked traffic.

2. Attach the pfSense CE installer ISO and boot the VM.
3. Walk through the installer's default partitioning and let it reboot into the installed system.
4. At the console menu, assign interfaces: option 1, then match the WAN and LAN NICs to the two virtio devices in the order Proxmox presents them (the console lists MAC addresses, which is the reliable way to tell them apart if the ordering isn't obvious).
5. Set the LAN interface's IPv4 address to a static address on a temporary subnet (for example `192.168.99.1/24`), and enable its DHCP server with a small temporary range — this is only to reach the web GUI for the first time and gets replaced once VLAN interfaces exist in §4.
6. From a workstation temporarily connected to that LAN segment, browse to `https://192.168.99.1` and complete the setup wizard (admin password, hostname, time zone).

Validation Test Setup: pfSense CE installed per the steps above, temporary LAN IP `192.168.99.1/24` assigned. **Action:** From a workstation on that temporary LAN segment, browse to `https://192.168.99.1` and log in with the admin password set during setup.

Expected result: The pfSense dashboard loads and shows both interfaces (WAN, LAN) as up, with WAN showing a real IP address obtained from your home network's DHCP.

4. Building the VLAN interfaces

[SETUP] With the base install reachable, replace the temporary flat LAN with the four VLANs Part 4 designed. Table 6.3 is the plan this build follows; adjust the third octet if it collides with your existing home network.

Table 6.3 — VLAN plan (CONCEPTUAL SAMPLE).

VLAN ID	Segment	Subnet	Firewall IP	DHCP range	Populated by
10	Management	10.10.10.0/24	10.10.10.1	10.10.10.100–10.10.10.199	Part 8 (SIEM), a jump host
20	Endpoint	10.10.20.0/24	10.10.20.1	10.10.20.100–10.10.20.199	Parts 9–10 (Linux, Windows hosts)
30	NSM	10.10.30.0/24	10.10.30.1	10.10.30.100–10.10.30.199	Parts 12–14 (Zeek, Suricata sensor)
40	Honeynet	10.10.40.0/24	10.10.40.1	10.10.40.100–10.10.40.199	Part 15 (decoy services)

1. Under `Interfaces > Assignments > VLANs`, add a VLAN with parent interface set to the internal (LAN-side) NIC and VLAN tag `10`; repeat for tags `20`, `30`, and `40` — four VLAN pseudo-interfaces total, all sharing the one physical/virtual NIC as trunked traffic.
2. Under `Interfaces > Assignments`, assign each new VLAN to an available OPT slot, then rename each (`OPT1` to `MGMT`, `OPT2` to `ENDPOINT`, `OPT3` to `NSM`, `OPT4` to `HONEYNET`) so the rule base in §6 reads clearly.
3. On each renamed interface's configuration page, enable the interface, set it to a static IPv4 address matching Table 6.3's firewall IP (for example `10.10.10.1/24` for `MGMT`), and save.
4. Under `Services > DHCP Server`, enable DHCP on each of the four interfaces with the range from Table 6.3, and set a distinct, short lease time (for example, 2 hours) — a home lab churns through VM rebuilds often enough that long leases just mean stale entries.

5. Retire the temporary flat LAN from §3 once all four VLANs are confirmed reachable — either repurpose that physical interface assignment into one of the four VLANs' parent, or leave it unassigned.

[TROUBLESHOOTING] If a VLAN interface shows "up" but never receives a DHCP request from a test client, the fault is almost never the firewall config — it's the virtual switch. Confirm the internal virtual switch (Proxmox's Linux bridge or an OVS bridge, depending on your Part 5 setup) is passing VLAN tags through rather than stripping them at the port level; on a Proxmox Linux bridge this usually means the VM's network device has VLAN-aware trunking enabled and no single default VLAN tag set on the port itself.

Figure 6.1 shows the topology this section produces: one firewall VM, one trunked internal NIC, four VLAN interfaces, and the deny-by-default posture §5 and §6 add on top.

```
graph TB
    WAN["Home network / ISP uplink"]
    FW["Firewall VM<br/>pfSense CE"]
    MGMT["VLAN 10 – Management<br/>10.10.10.0/24<br/>SIEM, jump host (Part 8)"]
    ENDP["VLAN 20 – Endpoint<br/>10.10.20.0/24<br/>Windows/Linux hosts (Parts 9-10)"]
    NSM["VLAN 30 – NSM<br/>10.10.30.0/24<br/>Zeek/Suricata sensor (Parts 12-14)"]
    HONEY["VLAN 40 – Honeynet<br/>10.10.40.0/24<br/>Decoy services (Part 15)"]

    WAN <--> |"NAT egress: Management only. Inbound: honeynet port-forward only"| FW
    FW --- |trunked NIC| MGMT
    FW --- |trunked NIC| ENDP
    FW --- |trunked NIC| NSM
    FW --- |trunked NIC| HONEY

    ENDP --> |"allow: SIEM ingest port"| MGMT
    NSM --> |"allow: SIEM ingest port"| MGMT
    HONEY --> |"allow: one telemetry port, one-way"| MGMT
    MGMT --> |"allow: admin ports"| ENDP
    MGMT --> |"allow: admin ports"| NSM
    MGMT --> |"allow: jump host only"| HONEY

    ENDP --x |"deny by default"| HONEY
    HONEY --x |"deny by default"| ENDP
    HONEY --x |"deny: no route home"| WAN
```

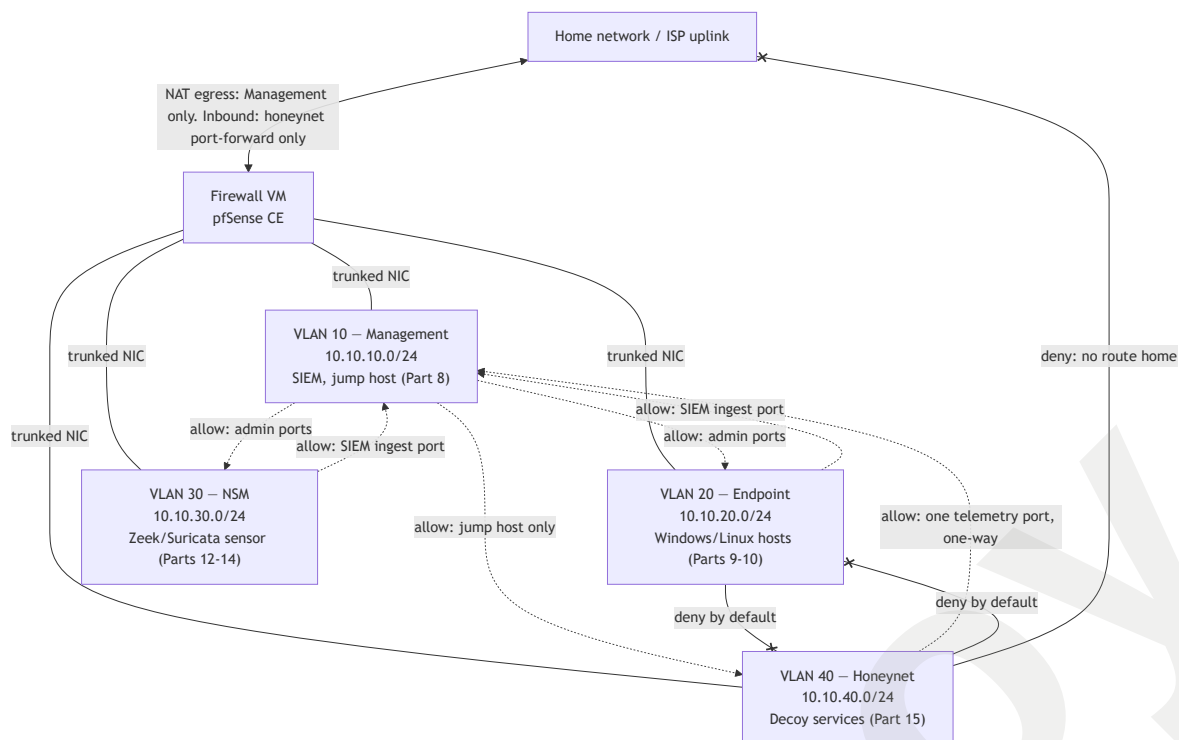


Figure 6.1 — Firewall VM and four-VLAN segmentation topology. *CONCEPTUAL.* Illustrates the target end state of this part's build: one firewall VM trunking four VLANs, with default-deny between every pair of segments and only the specific allow paths from Table 6.4 punched through. This is an architecture sketch of the intended build, not a capture from a running deployment — Parts 8, 9, 10, and 15 populate these segments with the actual services the dotted allow-arrows exist for.

5. Deny-by-default: the base rule the rest of this part depends on

[SAFETY] Both pfSense and OPNsense ship a default rule on the LAN-type interface that allows that interface's own subnet to reach anything — "LAN to any" — because most home-router use cases want an open internal network by default. A home lab practicing segmentation cannot keep that default: it's the single rule that would quietly undo every VLAN boundary this part just built.

1. On each of the four VLAN interfaces (`Firewall > Rules`, one tab per interface), delete or disable any pre-existing "allow all" rule that isn't one you added deliberately.
2. Confirm each interface tab ends with an implicit deny — pfSense and OPNsense both deny by default when no rule matches, so an interface tab with zero rules is already "deny everything in and out" for that interface; verify this is genuinely the state on all four tabs rather than assuming it.
3. On the WAN interface, leave the built-in "block bogon networks" and "block private networks" options enabled — these block obviously spoofed or misrouted traffic arriving from the internet side before it ever reaches your rule base.

With every VLAN interface at "deny everything," nothing on this network can reach anything else, including the internet. That's the correct starting state — §6 and §7 now add back exactly the paths later parts actually need, and no others.

6. Explicit allow rules: what actually needs to cross a boundary

[SETUP] Table 6.4 is the full allow-list this build needs before any later part goes live. It intentionally does not include an "any-any" row anywhere — every row names a specific source, destination, and port.

Table 6.4 — Inter-segment firewall rules (CONCEPTUAL SAMPLE).

Source	Destination	Port/protocol	Action	Feeds
Endpoint	Management	1514 /TCP, 1515 /TCP	Allow	Part 8 (SIEM agent traffic, Wazuh worked example)
NSM	Management	5044 /TCP	Allow	Part 8, Part 13 (Zeek log shipping via Beats)
Honeynet	Management	5044 /TCP (one-way, outbound from Honeynet only)	Allow	Part 8, Part 15 (decoy session telemetry)
Management	Endpoint	22 /TCP, 5985 /TCP, 5986 /TCP	Allow	Parts 9–10 (SSH, WinRM admin access)
Management	NSM	22 /TCP	Allow	Parts 12–14 (sensor admin access)
Management	Honeynet	22 /TCP, from a single jump-host address only	Allow	Part 15 (scoped admin access)
Endpoint	Honeynet	any	Deny	— endpoints never initiate contact with the decoy segment
Honeynet	Endpoint	any	Deny	— a compromised decoy must not reach anything else
Honeynet	Management	anything besides the row above	Deny	— telemetry only, not general access
Any lab VLAN	WAN (home network / internet)	any, except Management's explicit egress	Deny	— see §7 for Management's own scoped egress

Table 6.5 gives the SIEM ingest ports by platform, since Part 8 hasn't committed you to one yet — adjust the Endpoint-to-Management and NSM-to-Management rows above to match whichever platform you actually build.

Table 6.5 — Common SIEM ingest ports by platform, per each project's own documentation.

Platform	Agent/event port	Enrollment/management port	Notes
Wazuh	1514 /UDP or TCP	1515 /TCP	TCP is more reliable across a routed VLAN boundary than UDP; prefer TCP for the Endpoint-to-Management rule
Elastic (Filebeat/Winlogbeat to Logstash)	5044 /TCP	—	Also used for Zeek/Suricata log shipping if standardized on Beats
Graylog	514 /UDP or TCP (syslog), 12201 /UDP or TCP (GELF)	9000 /TCP (web UI)	Syslog UDP loses messages under packet loss; prefer GELF or syslog-TCP for anything you don't want silently dropped

1. Under `Firewall > Aliases`, create an alias for each VLAN subnet (`MGMT_NET`, `ENDPOINT_NET`, `NSM_NET`, `HONEYNET_NET`) — this makes the rules in the next step readable and means a later subnet change (unlikely, but possible) is a one-place edit instead of a rule-by-rule hunt.
2. On the Endpoint interface tab, add a rule: source `ENDPOINT_NET`, destination `MGMT_NET`, destination port matching your chosen SIEM platform from Table 6.5, action Pass.
3. Repeat on the NSM and Honeynet tabs for their respective rows in Table 6.4.
4. On the Management interface tab, add the three admin-access rules — source `MGMT_NET`, destination the specific other VLAN alias, the specific admin port, action Pass. Do not write a single "Management to any, any port" rule; each destination gets its own row.
5. Leave every other combination unwritten. The deny-by-default state from §5 already blocks everything you haven't explicitly allowed — a rule base that only ever grows explicit allow rows, with nothing else, is the entire point of this section.

Lab Note Write the alias names to match Table 6.4's column headers exactly, and keep a copy of that table next to the firewall GUI while you build the rules. A rule base built from a plan you're reading off, row by row, is the difference between finishing in 20 minutes and spending an hour second-guessing which VLAN was which.

7. Egress and the honeynet's one-way path

[SAFETY] Every VLAN above got an allow rule for the specific internal paths it needs. None of them got an allow rule for reaching the internet — and one of them, Honeynet, must never get one, because the Safety Gate at the top of this part depends on it having no route home and no route out except the single inbound port-forward Part 15 will add.

Build Autopsy — "the honeynet segment that could still dial home"

The plan: Build the four VLANs, apply deny-by-default on every interface tab per §5, and treat that as sufficient isolation for the Honeynet segment — no explicit allow rule out to the WAN, so nothing gets out.

Why it seemed reasonable: Deny-by-default on the interface rules genuinely does block Honeynet-initiated connections outbound through the packet filter itself — the interface tab has no Pass rule sending Honeynet traffic to WAN, so it looks airtight.

How it failed: pfSense and OPNsense both default `Firewall > NAT > Outbound` to Automatic mode, which generates an outbound NAT mapping for every internal interface's subnet — including a newly added VLAN — with no separate action required. NAT and the packet filter are evaluated together: a packet from the Honeynet subnet heading to WAN can still get NAT-translated and forwarded if any rule anywhere (a stray floating rule, a rule pasted from a tutorial that says "allow this network out to any") ends up matching it, and Automatic mode means that NAT translation is silently ready and waiting the moment such a rule exists — the isolation was one rule-base mistake away from failing with no warning, not architecturally incapable of failing.

The fix: Switch `Firewall > NAT > Outbound` to Manual mode, delete the automatically generated mapping for the Honeynet subnet specifically, and add one floating rule on the Honeynet interface, evaluated before anything else, that explicitly blocks Honeynet-to-WAN and Honeynet-to-any-RFC1918-destination-besides-Management traffic — a rule that fails closed even if a later, careless rule gets added beneath it.

[SETUP] Apply the fix from the Build Autopsy above before this segment carries any real traffic:

1. Under `Firewall > NAT > Outbound`, switch mode from Automatic to Manual.
2. Recreate the automatically generated outbound NAT mappings for Management, Endpoint, and NSM (these three do need normal internet egress for OS updates, time sync, and, for Management, threat-intel feed pulls in Part 16) — copy the Automatic-mode mapping's settings before switching, so you're not guessing at the translation address.
3. Do not recreate a mapping for the Honeynet subnet.
4. Under `Firewall > Rules > Floating`, add a rule above everything else: source `HONEYNET_NET`, destination "any," action Block, applied to the Honeynet interface — this is the explicit, fails-closed statement that the earlier implicit deny should have been from the start.
5. Leave a single, narrow exception ready for Part 15: an inbound port-forward on WAN to one specific Honeynet host and port, for the decoy service itself. Do not create it yet — that's Part 15's job, once there's an actual decoy service listening.

Blind Spot This part's rule base only sees traffic that actually routes through the firewall. A misconfigured hypervisor virtual switch that bridges two VLANs' virtual NICs directly at Layer 2 — a mistake in the Part 5 vSwitch setup, not this firewall's rule base — defeats every rule in this section without the firewall ever seeing a packet to block. Appendix A4's isolation checklist covers verifying the vSwitch layer separately (not yet authored — see [README.md](#)); until it exists, re-check the Part 5 vSwitch/port-group settings by hand. This part's validation in §8 only proves the firewall's own rules are correct, not that traffic is actually forced through the firewall in the first place.

8. Verifying isolation actually holds

[HANDS-ON LAB] The goal here is to prove three things with actual traffic, not read the rule base and assume it's correct: the required allow paths work, the required deny paths don't, and the Honeynet segment specifically has no way home.

1. Stand up one temporary test VM on the Endpoint VLAN and one on the Honeynet VLAN — a minimal Linux image is enough; you're not testing the OS, you're testing the network.
2. From the Endpoint test VM, attempt to ping the Management interface's firewall IP (`10.10.10.1`). Expect no reply — ICMP isn't in Table 6.4's allow list, and that's correct; a working SIEM ingest port doesn't require ICMP to succeed.
3. From the Endpoint test VM, attempt a connection to whichever SIEM ingest port you chose from Table 6.5 against a placeholder listener on the Management VLAN (a simple `nc -l` on the target port is enough for this test, before Part 8's real SIEM exists). Expect success — this is the one path this segment needs, and it should work.
4. From the Honeynet test VM, attempt to reach the Endpoint test VM on any port, and attempt to reach an external internet address (for example, a ping to a public DNS resolver). Expect both to fail outright, with the firewall's live log (`Status > System Logs > Firewall`) showing the packets hit the floating block rule from §7, not a silent timeout you can't attribute to anything.
5. From the Endpoint test VM, attempt to reach the Honeynet test VM on any port. Expect failure, confirming the deny is symmetric and not just one-directional.

Validation Test Setup: Four VLANs built per §4, deny-by-default and the Table 6.4 allow rules applied per §5–6, Honeynet floating block rule applied per §7, one temporary test VM on Endpoint and one on Honeynet. **Action:** From the Honeynet test VM, run `ping -c 3 1.1.1.1` (or your preferred external address) and `curl` against the Management VLAN's firewall IP on any port. **Expected result:** Both fail with no response, and the firewall's live log shows the traffic matching the Honeynet floating block rule by name — not a generic "no route" error from the test VM itself, which would prove nothing about whether the firewall is the thing actually stopping it.

Tear down both temporary test VMs once this checklist passes — they existed only to generate traffic to test against, and leaving them running just adds two more hosts to keep patched for no ongoing purpose.

9. Troubleshooting segmentation builds

[TROUBLESHOOTING] Four failure modes account for most of what actually goes wrong building this out, beyond the VLAN-tagging issue already covered in §4:

- **A floating rule silently overrides an interface-tab rule you were sure was correct.** Both platforms evaluate floating rules before interface-tab rules, regardless of which tab you're looking at in the GUI. If a Table 6.4 allow rule appears correctly configured but still doesn't pass traffic, check `Firewall > Rules > Floating` first — a leftover floating block rule from an earlier experiment is a common, easy-to-miss cause.
- **A rule edit doesn't take effect until existing states clear.** pfSense and OPNsense track connections as stateful sessions; tightening a rule after a permissive session is already established doesn't retroactively kill that session. After any rule-base change during this build, clear states (`Diagnostics > States > Reset States`) before re-testing, or you'll validate against a stale, already-permitted connection instead of the new rule.
- **DHCP relay or a stray second DHCP server leaks addresses across VLANs.** If a test client on one VLAN receives an address from a different VLAN's range, check for a second, forgotten DHCP server on that segment (a hypervisor's built-in DHCP feature is a common culprit) rather than assuming the firewall's own DHCP config is wrong.
- **A rule that references an alias silently stops matching after the alias is edited but not saved and applied.** Both platforms require an explicit "Apply Changes" step after alias edits, separate from saving the alias itself; a rule that used to work and now doesn't, with no other change made, is worth checking here before anything more exotic.

10. What this build feeds downstream

[CONCEPT] This part's output isn't detection content by itself — it's the plumbing three later parts, and two other NESHBOY volumes, build on directly.

- **Within this book:** Part 7 hangs internal DNS/DHCP off the VLAN structure built in §4. Part 8's SIEM sits in the Management VLAN and is the destination for every ingest-port allow rule in Table 6.4. Part 15's honeynet occupies the segment this part deliberately starved of any route home, and reuses the exact floating-block pattern from §7's Build Autopsy as its own starting posture, expanded with the port-forward Part 15 adds.
- **Detection Engineering Handbook V2:** the firewall's own deny log (`Status > System Logs > Firewall`, or once Part 8 exists, that log forwarded into the SIEM) is itself a telemetry source, not just an isolation mechanism — a logged deny of Honeynet-to-WAN traffic that shouldn't be attempting that in the first place is a compromise indicator, not routine noise. Detection Engineering Handbook V2's network-detection content (its Part 14) is where you'd build an actual detection rule on top of that log once it's flowing; this part only guarantees the log exists and means something specific when it fires.

- **SOC Playbook Handbook:** once a firewall-deny alert like the one above exists in the SIEM, it needs a response procedure — what to check first, who or what gets isolated, when to treat it as confirmed compromise of the decoy versus a benign misconfiguration. That triage logic lives in the SOC Playbook Handbook's Playbook Library, not here; this part's job ends at "the alert exists and is meaningful."
- **SOC Manager's Operating Handbook:** network segmentation of this shape — a documented VLAN plan, a default-deny rule base, an explicit allow-list instead of an implicit trust boundary — is exactly the kind of control a Field Test (SOC Manager's Operating Handbook, Part 8) would assess in a real organization. Running this build yourself is a legitimate, hands-on way to understand what that kind of assessment actually checks for, even though this book's version of it protects a home lab rather than an employer's network.

Cross-references: Part 4 (Network Isolation and Segmentation Architecture — the design this part builds), Part 5 (Choosing and Installing a Hypervisor — where this firewall VM runs), Part 7 (Internal DNS and Core Network Services), Part 8 (Building the SIEM Platform — the Management-VLAN destination for Table 6.4's ingest rules), Part 14 (Deploying Suricata for Intrusion Detection — the IDS content this part deliberately excludes), Part 15 (Honeypots and Honeynet Architecture — the segment this part starves of a route home), Appendix A3 (Build Checklists & Config Snippet Reference — pfSense/OPNsense rule-table template), Appendix A4 (The Safety & Isolation Pre-Flight Checklist); Detection Engineering Handbook V2, Part 14 (Network Detection Engineering); SOC Playbook Handbook, Playbook Library; SOC Manager's Operating Handbook, Part 8 (Field Test / assessment design).

Part 7 — Internal DNS and Core Network Services

Why this part exists

Every part after this one assumes lab hosts can resolve a hostname and get an IP address without asking anything outside the lab to do it for them. Part 8's SIEM needs endpoints to find it by name. Part 13 and Part 14's sensors need `conn.log` and alert output to show real hostnames, not bare IPs, or a reader spends the rest of the book reverse-resolving addresses by hand. Part 15's honeynet needs decoy services to look like they belong on a network, which means answering DNS like a real network segment would. None of that works if the lab's name resolution is either missing entirely or, worse, silently riding on the reader's home router's DNS forwarder — the exact dependency Part 4 built the isolation to remove.

This part builds the lab's own DNS and DHCP: a resolver that serves only the isolated lab segments from Part 4 and Part 6, forwards outbound queries through the firewall's explicit allow rule rather than the home network's default gateway, and — as a side effect of doing its job at all — produces the cheapest, earliest telemetry source in this entire book. A DNS query log exists the moment the resolver is running; it costs no extra agent, no extra parser, and no extra RAM beyond the resolver itself. Every later part that talks about "what to do with lab telemetry" assumes something like this is already emitting rows into a log file.

Safety Gate *The resolver and DHCP service built in this part must bind only to the isolated lab-segment interface(s) established in Part 4 and built out in Part 6 — never to an interface that also reaches your home network's broadcast domain. Two concrete conditions to verify before this service starts answering queries: (1) its upstream forwarder is a public recursive resolver (for example `1.1.1.1` or `9.9.9.9`) reached only through the firewall's explicit outbound-53 allow rule, never through your home router's own DNS; and (2) if you run more than one lab segment (endpoint, NSM, honeynet), the honeynet segment's resolver — covered fully in Part 15 — must not share zone data with the others, so a decoy host answering a lookup never leaks a real lab hostname. Run the Appendix A4 pre-flight checklist before pointing any lab host at this resolver.*

1. Why the lab needs its own name resolution

[CONCEPT] A lab with no internal DNS still has a working network — hosts can reach each other by IP, and outbound traffic still routes through the firewall from Part 6. What breaks is everything that assumes names: a SIEM dashboard full of raw IPv4 addresses instead of `wazuh-mgr.lab.internal`, a Sysmon `DestinationHostname` field that comes back empty, and — the failure mode that actually matters for this book's safety story — a lab host that, absent any other resolver, falls back to whatever DNS server DHCP handed it. If that DHCP server is your home router, every query a lab host makes now transits your home network's resolver, and by extension your ISP's, defeating the isolation Part 4 spent an entire part building.

The fix is not complicated: the lab segment needs its own authoritative-enough DNS server (it doesn't need to be authoritative for the whole internet, just for the handful of internal hostnames the lab cares about, forwarding everything else) and its own DHCP server handing out that resolver's address along with lab-scoped IP leases. Building that pair is this part's entire scope. Interpreting what a DNS log tells you about a specific threat — DGA-pattern queries, NXDOMAIN bursts, DNS tunneling — is Detection Engineering Handbook V2, Part 15's job, cited again in §10 below; this part stops at "the log exists and is trustworthy."

2. Choosing a resolver and DHCP stack

[CONCEPT] Four realistic options cover almost every home-lab build. The table below supports one decision: which stack to install in §4, based on how much DHCP integration and blocklist/sinkhole behavior you want without hand-writing zone files.

Option	DHCP included?	Query log format	Blocklist / sinkhole support	Resource footprint	Best fit
Pi-hole (on top of <code>dnsmasq</code> / <code>unbound</code>)	Yes, via <code>dnsmasq</code>	Plaintext <code>pihole.log</code> , plus a queryable SQLite/FTL database	Built-in gravity list, per-domain block, web UI	Light — see §4's Resource Reality	Readers who want a web UI and ad-block-style logging out of the box
Plain <code>dnsmasq</code>	Yes, native	Plaintext syslog lines (<code>query</code> , <code>forwarded</code> , <code>cached</code> , <code>reply</code>)	Manual, via <code>address=/domain/0.0.0.0</code> lines	Lightest of the four	Readers who want minimal moving parts and are comfortable editing one config file
BIND9 + ISC DHCP	Separate daemons, both mature	<code>named</code> query log (verbose, needs explicit logging channel config)	Manual, via response-policy zones (RPZ)	Heaviest — two daemons, more RAM	Readers who want a resolver that behaves like enterprise/production DNS infrastructure, at the cost of more setup
Technitium DNS Server	Yes, native	Structured (JSON-capable) query log via its own API	Built-in blocklist subscriptions	Moderate — a .NET runtime, more than <code>dnsmasq</code> alone	Readers who want an API-first resolver for scripted log pulls into Part 8's SIEM

This book uses Pi-hole (running on `dnsmasq` underneath) as the worked example for the rest of this part, for one direct reason: it is what the author's own lab actually runs, on a Linux container named CT100, which means every log excerpt from here on is a REAL LAB EXAMPLE rather than a constructed one. `dnsmasq` alone is functionally identical for everything this part builds — swap the web UI and gravity-list logic for a hand-edited `/etc/dnsmasq.conf` if you'd rather skip the extra layer.

3. Placing the resolver on the isolated topology

[SAFETY]

Where the resolver sits matters more than which software it runs. It needs an interface on the lab segment(s) it serves, a forwarder path to the internet that goes through the firewall's explicit allow rule from Part 6 (not a default route back to your home gateway), and — if your build has more than one lab segment — a decision about whether one instance serves all of them or each segment gets its own.

For a single-segment lab (Part 2's smallest topology pattern), one resolver instance is enough. For a multi-segment build with a separate honeynet (Part 15), the honeynet segment must not share this resolver's zone data — a probe against a decoy host that gets back a real internal hostname (`siem-mgr.lab.internal` instead of a decoy-appropriate name) has just told whoever's poking at it more about your real lab than a honeypot should ever reveal. Give the honeynet its own resolver instance, configured in §8 as a sinkhole-only teaching setup with no knowledge of the endpoint or management segments' real hostnames.

Figure 7.1 shows the target placement: one lab resolver serving the management, endpoint, and NSM segments, and a separate, zone-isolated resolver for the honeynet segment, with no forwarder path from either instance back through the home network.

flowchart LR

```
subgraph HOME["Home network – untouched, no route to lab"]
```

```
    HR["Home router / ISP DNS"]
```

```
end
```

```
subgraph FW["Part 6: perimeter firewall (pfSense/OPNsense)"]
```

```
    FWR["Default-deny inter-segment rules;\nexplicit allow: outbound UDP/TCP 53"]
```

```
end
```

```
subgraph LAB["Isolated lab segments (Part 4)"]
```

```
    DNS["Lab DNS/DHCP resolver\n(Pi-hole on dnsmasq)"]
```

```
    MGMT["Management VLAN hosts"]
```

```
    EP["Endpoint VLAN\n(Linux + Windows lab hosts)"]
```

```
    NSM["NSM VLAN\n(Zeek / Suricata sensor)"]
```

```
end
```

```
subgraph HN["Honeynet segment (Part 15)\nseparate resolver, no shared zone data"]
```

```
    HNDNS["Sinkhole-only resolver"]
```

```
    DECOY["Decoy services"]
```

```
end
```

```
PUB[("Public recursive resolvers\n1.1.1.1 / 9.9.9.9")]
```

```
HR -. no route .- FWR
```

```
FWR -->|explicit allow, port 53 only| PUB
```

```
FWR --> DNS
```

```
DNS --> MGMT
```

```
DNS --> EP
```

```
DNS --> NSM
```

```
FWR --> HNDNS
```

```
HNDNS --> DECOY
```

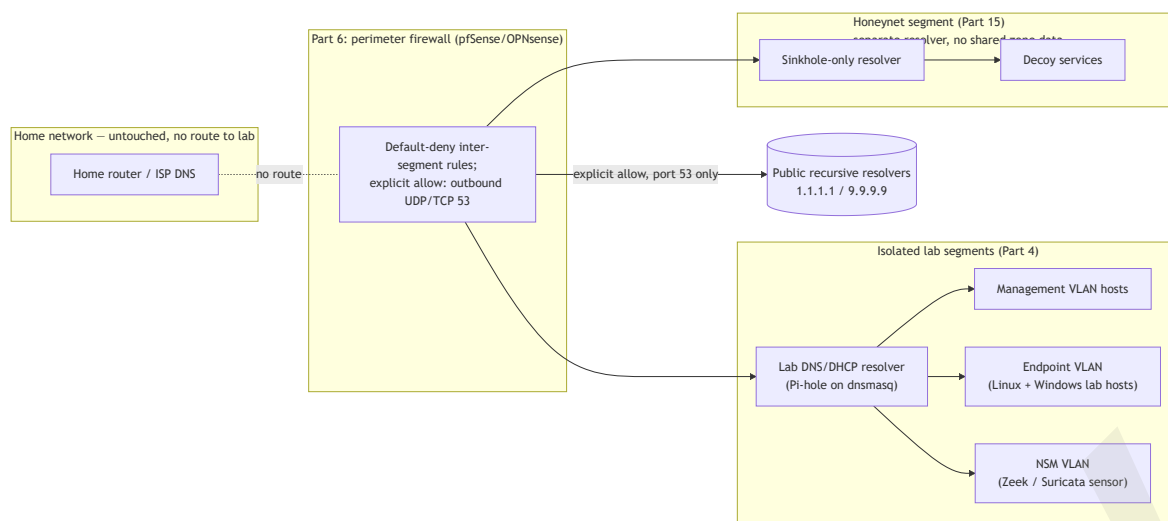


Figure 7.1 — Target lab DNS/DHCP placement across isolated segments. CONCEPTUAL. Illustrates the recommended split between a shared resolver for the management/endpoint/NSM segments and a zone-isolated resolver for the honeynet segment, with no forwarder path back through the home network. This is an architecture target for a reader's own build, not a capture from a running system — compare against the honest caveat in the Lab Note below, where the author's own real resolver does not fully match this diagram.

4. Building the lab resolver: Pi-hole/dnsmasq install and DHCP scope

[COST/RESOURCE]

Resource Reality Pi-hole's own documentation lists 1 vCPU and 512MB of RAM as sufficient for a home network's query volume, with 4GB of disk mostly consumed by the FTL query database over time, not the software itself. This is the lightest telemetry-producing component in the entire book — a rounding error next to Part 8's SIEM (8GB minimum) or Part 10's Windows endpoints. If your hardware tier from Part 3 can run anything at all, it can run this.

[SETUP]

The steps below target Pi-hole's official installer on Debian-family Linux (the author's own CT100 runs Debian in an LXC container on Proxmox; the same steps apply inside any Debian/Ubuntu VM or container on your chosen hypervisor from Part 5).

1. Create the lab resolver's VM or container with one NIC on the lab segment's interface only — verify this in your hypervisor's network settings before proceeding, not after.
2. Install Pi-hole with the official installer: `curl -sSL https://install.pi-hole.net | bash`.
3. During the interactive install, select the lab segment's interface (not any interface touching your home network) and choose upstream DNS providers that are public recursive resolvers — `1.1.1.1` and `9.9.9.9` are reasonable defaults — never your home router's IP.
4. After install, open the Pi-hole admin UI and enable DHCP under **Settings** → **DHCP**, setting the range to the lab-segment subnet from Part 4's VLAN plan — for example `10.10.20.50–10.10.20.150` — never your home network's `192.168.x.x` range. This is the supported, version-

stable way to manage DHCP on a Pi-hole install; hand-editing a `dnsmasq.d` drop-in file for DHCP is a plain-`dnsmasq`-only technique (§2), and doing both on the same box risks two DHCP configs answering the same segment (§9).

5. In the same **Settings** → **DHCP** screen, set the lab segment's local domain suffix to `lab.internal` so lab hosts resolve each other as `<hostname>.lab.internal` instead of bare IPs.
6. Restart the service: `pihole restartdns`.
7. On the firewall from Part 6, add the explicit outbound-53 allow rule from this resolver's IP to your chosen public resolvers, and confirm the default-deny rule still blocks every other path out.

The one setting worth double-checking against the Safety Gate above: Pi-hole's interface-listening mode. Set it to "listen on interface only, permit all origins" scoped to the lab segment's interface — the alternative "listen on all interfaces" setting will happily answer queries from your home network too if the container is ever dual-homed, which is precisely the bridged-NIC failure mode Part 4 warned about.

5. Validating resolution and DHCP end-to-end

[HANDS-ON LAB]

Goal: confirm a lab host gets a DHCP lease from the new resolver, resolves an internal hostname, and resolves a public one through the firewall's allowed path — all without touching your home network's DNS.

Validation Test Setup: Lab resolver installed per §4, one test VM or container on the lab segment with DHCP enabled and no static DNS override. **Action:** Power on the test host, then run `nslookup pihole.lab.internal` followed by `nslookup github.com` from that host. **Expected result:** The test host's assigned IP falls inside the DHCP range configured in §4 (checkable via `ip a` on the host or the Pi-hole web UI's "DHCP leases" page); `pihole.lab.internal` resolves to the resolver's own lab-segment IP; `github.com` resolves successfully through the forwarder, and the query appears in the resolver's own query log within seconds — confirming the whole path (DHCP, internal resolution, forwarded resolution, logging) works without a fallback to any other resolver.

If `github.com` fails to resolve but the internal name works, the firewall's outbound-53 allow rule from step 7 in §4 is the first thing to check — a resolver with no forwarder path can still answer for names it knows and correctly fail everything else, which looks like a broken resolver but is actually a working one with no route out.

6. DNS query logs as an early, cheap telemetry source

[CONCEPT]

Every `dnsmasq`-based resolver logs each query, whether it was served from cache, forwarded upstream, or blocked, plus every DHCP lease event, in a single plaintext log. No agent install, no parser configuration, no SIEM required to start looking at it — `tail -f /var/log/pihole/pihole.log` is a working telemetry feed on day one. Figure 7.2 shows a real excerpt.

```

Sep 15 08:32:46 dnsmasq[2221]: forwarded software-static.download.prss.microsoft.com to 1.1.1.1
Sep 15 08:32:46 dnsmasq[2221]: reply software-static.download.prss.microsoft.com is <CNAME>
Sep 15 08:32:47 dnsmasq[2221]: query[A] mykulprint.lan from 192.168.1.169
Sep 15 08:32:47 dnsmasq[2221]: config mykulprint.lan is NXDOMAIN
Sep 15 08:32:49 dnsmasq[2221]: query[AAAA] mobile.events.data.microsoft.com from 192.168.1.169
Sep 15 08:32:49 dnsmasq[2221]: gravity blocked mobile.events.data.microsoft.com is ::
Sep 15 08:32:49 dnsmasq[2221]: query[A] mobile.events.data.microsoft.com from 192.168.1.169
Sep 15 08:32:49 dnsmasq[2221]: gravity blocked mobile.events.data.microsoft.com is 0.0.0.0
Sep 15 08:32:50 dnsmasq-dhcp[2221]: DHCPREQUEST(eth0) 192.168.1.7 b0:19:21:83:77:bf
Sep 15 08:32:50 dnsmasq-dhcp[2221]: DHCPACK(eth0) 192.168.1.7 b0:19:21:83:77:bf Archer

```

Figure 7.2 — CT100 Pi-hole/dnsmasq query log excerpt. REAL LAB EXAMPLE. Captured from the author's own running CT100 container (Debian LXC, Pi-hole on `dnsmasq`) on 2026-09-15. Shows a `gravity blocked` line — Pi-hole's default blocklist stopping a client from reaching `mobile.events.data.microsoft.com`, a Microsoft telemetry endpoint — alongside ordinary forwarded and cached lookups, and a DHCP lease renewal from the same daemon. No secrets are present in DNS query logs of this kind; nothing has been redacted.

[HANDS-ON LAB] Read the excerpt line by line and identify four distinct event types before moving on: a `forwarded` query (resolver had no cached answer, asked upstream), a `config`-sourced NXDOMAIN (resolver authoritatively knows this name doesn't exist — useful for spotting lab hosts probing for hostnames that were never registered), a `gravity blocked` line (blocklist match, the sinkhole behavior built out fully in §8), and a `DHCPACK` (lease granted, tying an IP back to a MAC address at a specific time — the same correlation a SIEM in Part 8 will do automatically once this log is ingested).

Lab Note The author's own CT100 shown in Figure 7.2 doubles as the household's everyday ad-blocking resolver — that's a real, working setup for a single-operator home network, but it also means CT100 itself sits partly outside this book's isolation boundary: it serves the whole home LAN (192.168.1.0/24), not a segment-scoped lab VLAN. Don't copy that shortcut for your own isolated build. Stand up a second, dedicated instance bound only to the lab-segment interface from §3 — repointing your one household resolver at the lab and calling it isolated defeats the Safety Gate at the top of this part.

Blind Spot A resolver only sees queries that are actually sent to it. A client hardcoded to a public DNS-over-HTTPS endpoint — increasingly a default "secure DNS" browser setting, and a known evasion technique for malware avoiding local logging — never asks this resolver anything, and produces zero rows in `pihole.log` for traffic that is very much happening. This part's query-log telemetry is blind to any host that bypasses local resolution entirely; detecting that bypass itself (unexpected outbound 443/853 traffic that isn't matched to any prior DNS lookup) is Detection Engineering Handbook V2, Part 15 territory, not something a query log can show for a query it never received.

7. Watching the resolver host itself

[CONCEPT] The resolver is a Linux host like any other lab endpoint, and it needs the same baseline scrutiny Part 9 applies to every Linux lab host — this section is a pointer forward, not a duplicate build. `auth.log`'s `sudo` entries are a useful early example of why: they show exactly who ran what, as whom, which is precisely the kind of legitimate-administrative-activity baseline Part 9 teaches you to establish before you can recognize anomalous `sudo` use.

```
Jun 17 09:40:23 pihole sudo[4161]:      root : TTY=pts/1 ; PWD=/root ; USER=pihole ;  
COMMAND=/usr/bin/bash /opt/pihole/gravity.sh --force  
Aug 17 15:28:38 pihole sudo[143836]:    root : TTY=pts/1 ; PWD=/root ; USER=root ;  
COMMAND=/usr/bin/pihole-FTL --config
```

Figure 7.3 — CT100 `auth.log` `sudo` invocations excerpt. REAL LAB EXAMPLE. Captured from the author's own running CT100 container on 2026-09-15, spanning entries from June through August 2026. Both lines are expected administrative activity — a blocklist update (`gravity.sh --force`) run as the `pihole` service account, and a config check run as root — kept here specifically as a baseline example of normal use, not a finding.

Full `auditd` deployment, rule-set design, and forwarding this host's logs into the Part 8 SIEM belong to Part 9; the point of including it here is narrower — do not let "it's just the DNS box" become a reason to skip monitoring it. A resolver with root access to its own gravity list and DHCP scope is worth watching precisely because it's small and easy to forget.

8. Sinkholing as a teaching setup

[HANDS-ON LAB] A sinkhole is just a DNS response that lies on purpose — instead of resolving a domain to its real address, the resolver answers with `0.0.0.0` or a controlled address you own, so any client that tries to reach that domain gets stopped or redirected before it leaves the resolver. Pi-hole's gravity list already does this for ad/telemetry domains, as Figure 7.2 showed. This section builds the same mechanism for teaching purposes: deterministic, on-demand "blocked" telemetry you can trigger whenever you want it, rather than waiting for an ad network to phone home.

1. Create a dedicated list file, `/etc/pihole/lab-sinkhole.list`, containing domain names you fully control the meaning of — teaching-only names under a domain you own, or RFC 2606 reserved test names (`example.com`, `example.org`) — never real, currently-active malicious infrastructure.
2. Add the list as a custom blocklist source in the Pi-hole admin UI ("Group Management" → "Adlists"), pointed at a local file path rather than a remote URL.
3. Run `pihole -g` to force a gravity list rebuild, pulling in the new entries.
4. From a lab test host, query one of the listed domains and confirm the resolver answers with the sinkhole address rather than forwarding upstream.

Safety Gate *Never populate a teaching sinkhole list with real, currently active malicious domains for the purpose of testing whether your resolver blocks them — doing so risks a lookup racing ahead of the block (a cache miss forwarded upstream before the list reloads) and, more importantly, provides no benefit a fabricated or reserved test domain doesn't already provide just as well. This sinkhole exists to exercise the block path, not to have a supervised near-miss with real attacker infrastructure. Curated threat-intel domain feeds belong in Part 16, ingested for enrichment and correlation, not queried directly by a lab host under any circumstances.*

This deterministic block behavior becomes genuinely useful once Part 18's attack-simulation tooling exists: a scripted technique that performs a DNS lookup against a known test domain gives you a repeatable, on-demand event to validate that Part 8's SIEM actually ingests the resulting `gravity blocked` line end to end — a much cheaper validation loop than waiting for a real ad network to trigger the same code path.

9. Troubleshooting common resolver/DHCP failures

[TROUBLESHOOTING] Three failure modes account for most of the friction readers hit with this build.

- **Two DHCP servers answer the same segment.** If the lab segment's switch or virtual switch still has an uplink into a network segment where another DHCP server is active (a home router, or a hypervisor's own built-in DHCP on that vSwitch), lab hosts intermittently get leases from the wrong server. Fix: disable any competing DHCP service on that specific interface/vSwitch — check the hypervisor's virtual network settings, not just the physical router, since several hypervisors quietly run their own DHCP by default on NAT-mode virtual switches.
- **Public names resolve, but the lab's own hostnames don't.** This usually means the `lab.internal` domain suffix set in §4's **Settings** → **DHCP** screen didn't get picked up — confirm the setting saved, then restart with `pihole restartdns reload-lists`. For a plain-`dnsmasq` install (§2) instead of Pi-hole, re-check the `domain=` directive in your `dnsmasq.conf` for a typo. A resolver that only answers for the internet and not your own lab hostnames is answering half its job.
- **Nothing resolves, and the resolver's own log shows queries arriving.** The resolver can see the query (it's on the right interface) but its forwarder path is blocked — check the Part 6 firewall's outbound-53 allow rule for the resolver's specific source IP, not a broader "allow DNS" rule that might be scoped to the wrong subnet after a VLAN renumber.

10. Where this telemetry goes next

[CONCEPT] This part deliberately stops at "the log exists, it's trustworthy, and you know what each line means." Three things happen to it after that, each owned by a different volume in this series:

- **Detection Engineering Handbook V2, Part 15** builds actual detection logic on top of this telemetry once it's flowing into a SIEM — NXDOMAIN-burst patterns that suggest a DGA, query

volume anomalies, and correlating a `gravity blocked` hit against the threat-intel feeds Part 16 of this book adds later. This part produces the raw material; that part teaches you to reason about it.

- **SOC Playbook Handbook's Playbook Library** owns what to actually do the first time a real alert fires off this telemetry once Part 8's SIEM is watching it — for example, triaging a sinkhole/gravity-block hit that wasn't one of your own §8 test queries. This part doesn't teach triage; it makes sure there's something worth triaging.
- **SOC Manager's Operating Handbook, Part 8 (assessment design) and Part 11 (onboarding/ramp-up)** can use this exact telemetry source as a low-cost, deterministic practice-drill input — the sinkhole from §8 is cheap enough to reset and rerun as many times as a training exercise needs, which is exactly the repeatable-drill property that volume's Field Test framing, expanded in this book's own Part 22, is built around.

Forwarding `pihole.log` (or `dnsmasq`'s syslog output) into the SIEM built in Part 8 is that part's job, not this one's — the pipeline, parser, and index design belong there. What this part guarantees going in is that the source log itself is real, isolated, and already meaningful before a single line of it reaches a dashboard.

Cross-references: Part 4 (network isolation and segmentation architecture); Part 6 (perimeter firewall and segmentation build); Part 8 (SIEM ingest for this part's DNS/DHCP logs); Part 9 (Linux endpoint auditd/log-forwarding treatment of the resolver host itself); Part 15 (honeynet-segment resolver isolation and sinkhole architecture at scale); Part 16 (threat-intelligence feed integration, referenced in §8's Safety Gate); Part 18 (attack-simulation tooling that exercises the §8 sinkhole on demand); Appendix A4 (safety and isolation pre-flight checklist); Detection Engineering Handbook V2, Part 15 (DNS-based detection logic); SOC Playbook Handbook, Playbook Library (alert triage); SOC Manager's Operating Handbook, Parts 8 and 11 (practice-drill design and onboarding use of this telemetry).

Part 8 — Building the SIEM Platform

Why this part exists

[CONCEPT] Every part before this one built infrastructure that doesn't, by itself, produce anything you can search: an isolated network (Part 4), a hypervisor (Part 5), a firewall with segmentation rules (Part 6), and internal DNS (Part 7). This part builds the first thing in the lab whose entire job is to receive, store, and make searchable whatever the rest of the book produces. By the end of it, you'll have a self-hosted SIEM running on its own VM, one test log line proven to arrive and show up in search, and a clear map of where this part's job stops and the next three books' jobs start. It deliberately does not cover parsing quality, field normalization, or correlation logic — Wazuh, Elastic, and Graylog all ship opinions about how a raw log becomes a structured event, and getting those opinions right is Detection Engineering Handbook V2's job (Parts 5–7), not this one's. This part's finish line is "logs arrive and are searchable," full stop.

Safety Gate *The SIEM host you build in this part is the single most valuable pivot point in your entire lab — it will eventually hold credentials, agent enrollment keys, and a live view into every endpoint you point at it. Everything here assumes it lives inside the isolated segment architecture from Part 4, reachable from the endpoint VLAN only on the specific ingest ports it needs (§4) and reachable for dashboard access only from your management VLAN, never bridged to your home network or exposed to the open internet on any port. Before you install anything, confirm the VM's virtual NIC is attached to the correct isolated virtual switch from Part 5 and that the firewall rules in §4 exist before the first agent enrolls — an unauthenticated or default-credentialed SIEM dashboard reachable from the wrong segment defeats the isolation of every other part in this book at once. If you haven't run the Appendix A4 pre-flight checklist since building Part 6's firewall, run it now before continuing.*

1. Choosing a SIEM platform for a home lab

[CONCEPT] "SIEM" gets used loosely enough in vendor marketing that it's worth pinning down what this part actually means by it: one platform that ingests logs from multiple sources, stores them in a way that's indexed for fast search, and gives you a query interface and a dashboard on top. Three self-hosted options cover nearly every home-lab build you'll see discussed: Wazuh (a security-focused platform built on OpenSearch, bundling a host-agent and some detection content out of the box), the Elastic Stack (Elasticsearch, Logstash or Beats, and Kibana, assembled from separate components), and Graylog (its own server process backed by MongoDB for metadata and Elasticsearch or OpenSearch for the actual log storage). None of the three requires a paid license to run at home-lab scale, and all three are genuinely used in production SOCs — you are not settling for a toy version of anything by picking any one of them.

The table below compares the three on what actually matters at this size: how much RAM the platform wants before you've added a single endpoint, and how much hand-configuration the install leaves you

to do afterward. These are vendor-documented sizing figures, not numbers this book's own author has build-tested against a running install — the real lab this book draws REAL LAB EXAMPLE evidence from (Parts 5, 7, 9, and 15) is Linux-and-network-only and currently runs no commercial SIEM, so treat this comparison as a planning reference, not a captured result, and re-check each platform's current sizing docs before you commit hardware.

Platform	RAM at idle (single node)	Components you install	Setup friction	Best fit for this book
Wazuh	~8GB (roughly 4GB OpenSearch heap plus manager overhead)	Manager, indexer (OpenSearch), dashboard, host agent	Low — one installer script does all three at once	Readers who want detection content and host-integrity checks bundled in, without wiring separate components together themselves
Elastic Stack	~12–16GB comfortably (Elasticsearch heap, Logstash or Beats, and Kibana each budgeted separately)	Elasticsearch, Logstash and/or Beats, Kibana	Medium-high — each component is installed, versioned, and configured on its own	Readers planning to lean on Detection Engineering Handbook V2's query and correlation content later, since Elastic's query language is what most of that content assumes
Graylog	~8–12GB (Graylog server plus a MongoDB metadata store plus an Elasticsearch/OpenSearch backend)	Graylog server, MongoDB, Elasticsearch or OpenSearch	Medium — three components, but Graylog's own docs walk through wiring them together directly	Readers who want a simpler day-to-day search UI than Kibana and don't need Wazuh's bundled agent features

This book's worked walkthrough (§3) uses Wazuh, purely because its single installer script is the lowest-friction path to "logs arrive and are searchable" for a first SIEM build — every ingest, verification, and troubleshooting concept in §3–§6 applies to the other two platforms with different commands and file paths, not a different architecture.

All three platforms ship a free, self-hosted edition with no artificial cap on data volume or agent count for a home-lab's scale — the commercial tiers vendors sell on top (managed cloud hosting, enterprise support contracts, extra compliance-reporting modules) are aimed at organizations, not a single-operator lab, and nothing in this book's build assumes you'll ever need one.

What Would Change My Mind This part recommends Wazuh as the lowest-friction first SIEM for a reader with no prior preference, on the grounds of its single installer script. If a reader reports the Elastic Stack's separately-versioned components staying reliably in sync across upgrades with less manual intervention than Wazuh's bundled release cycle — not just "Elastic has more features," which isn't the claim being made — that specific maintenance-burden comparison, not the general recommendation, would need revision.

2. Sizing the host before you install anything

[COST/RESOURCE]

Skipping this section and installing straight from a vendor quick-start guide is the single most common way this build goes wrong, and it's the subject of §3's Build Autopsy — read the numbers below before you provision the VM, not after the installer fails.

Resource Reality Wazuh's own sizing guidance treats 4 vCPU and 8GB of RAM as the floor for a single all-in-one node serving up to roughly 25 agents, and the indexer component underneath it (OpenSearch) wants at least 4GB of that RAM as dedicated JVM heap before it reliably stays running under real load. Below that floor, the indexer doesn't run slowly — it fails its own bootstrap checks outright, or gets killed by the kernel's out-of-memory handler the moment the manager hands it its first real batch of agent data. If Part 3 put your build on the entry-level hardware tier (a single repurposed laptop or mini-PC), plan on giving the SIEM the whole box: it should not share a host with Windows endpoints, NSM sensors, or a honeynet segment on the same tier of hardware.

The table below maps Part 3's three hardware tiers onto what each can actually run for a SIEM alone, before adding endpoints, NSM sensors, or a honeynet on the same box.

Hardware tier (Part 3)	Total RAM available	What it can run as a SIEM	What it can't
Repurposed laptop/mini-PC	8–16GB	One SIEM platform as the box's only real workload	The SIEM plus Windows endpoints plus NSM sensors on the same host — move up a tier before combining
Dedicated home-server box	32–64GB	SIEM plus several endpoints plus Zeek/Suricata, each on a separate VM with real headroom	A persistent multi-DC AD forest layered on top without hitting swap (see Part 11)
Multi-node cluster	64GB+ across hosts	SIEM, endpoints, NSM, and a honeynet each on dedicated VMs or hosts with no resource contention	—

If you're building on the entry-level tier and know you'll eventually want Zeek (Part 13) or a Windows endpoint (Part 10) running at the same time, size the SIEM VM at 8GB minimum now rather than resizing it under a running install later — shrinking a live OpenSearch heap after the fact is far more disruptive than starting with headroom.

3. Installing a self-hosted SIEM: a Wazuh walkthrough

3.1 Preparing the VM

[SETUP]

Before running any installer, the VM itself needs to exist in the right place on the network Part 4 designed and Part 6 built.

1. In your hypervisor (Part 5), create a new VM with at least 4 vCPU and 8GB RAM, per §2's floor.
2. Attach its virtual NIC to the management or monitoring VLAN from Part 4's segmentation plan — not the endpoint VLAN, and never the honeynet segment.

3. Install Ubuntu Server 22.04 LTS (or your distribution of choice; the steps below target Ubuntu specifically) with a static IP reserved on the DNS server built in Part 7.
4. Confirm the VM can reach the internet outbound for package downloads.
5. Confirm — using the Part 6 firewall's logs, not just an assumption — that nothing outside the lab segment can reach it yet.

3.2 Running the all-in-one installer

[SETUP] The commands below target Wazuh 4.7's all-in-one installer script running on Ubuntu Server 22.04 LTS; if you're installing a newer 4.x release, the script name is the same but the download path in Wazuh's own docs will have moved.

```
curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh
sudo bash ./wazuh-install.sh -a
```

The `-a` flag tells the script to install the manager, indexer, and dashboard together on this one node — the "all-in-one" pattern this book uses throughout. The script takes several minutes and prints an admin username and a generated password for the dashboard at the very end; copy that password immediately, since it isn't shown again and the terminal scrollbar is your only other copy. To confirm the install actually succeeded rather than silently failing partway through, check that all three services are active before doing anything else:

```
sudo systemctl status wazuh-manager wazuh-indexer wazuh-dashboard
```

All three should report `active (running)`. If `wazuh-indexer` instead shows `failed` or is stuck `activating`, stop here and go to §6 before continuing — the dashboard will often still load and let you log in even when the indexer underneath it never came up, which is exactly the failure mode in the Build Autopsy below.

Build Autopsy — the all-in-one node that never finishes booting

The plan: Take Part 3's mid-tier hardware budget — an 8GB repurposed mini-PC — and run Wazuh's all-in-one installer directly on it, on the assumption that the official script handles all the sizing decisions for you.

Why it seemed reasonable: The installer's own defaults don't warn you it needs more than what you gave it, and 8GB sounds like plenty of room for a lab with two or three endpoints.

How it failed: OpenSearch — the indexer underneath Wazuh's dashboard — refuses to start below its default heap allocation and a kernel setting (`vm.max_map_count`) most quick-start guides don't mention. On an 8GB VM where the host OS and a couple of other lab services are already using part of that memory, the indexer either fails its own bootstrap checks outright or gets OOM-killed the moment the manager hands it its first real batch of agent data. The dashboard still loads, you can still log in, and it shows nothing — because the component actually holding the logs never came up.

The fix: Treat §2's Resource Reality numbers as a floor, not a suggestion — give the SIEM VM a dedicated 8GB with the host OS and every other lab service running on separate hardware or a separate VM entirely. On genuinely constrained hardware, install the manager and indexer as two separate VMs instead of one all-in-one node, so a memory-hungry indexer restart doesn't take the whole platform down with it.

3.3 Closing the front door before anything connects

[SAFETY] The generated admin password from §3.2 is the only thing standing between "isolated lab SIEM" and "open door with a search bar." Log into the dashboard at `https://<siem-ip>` immediately after install, using the printed credentials, and confirm the certificate warning you get is the installer's self-signed cert (expected at this stage — Part 8 doesn't cover replacing it with one from your own internal CA) and not a sign you've reached the wrong host. Do this before wiring in a single forwarder in §4; an endpoint enrolling against a SIEM you haven't logged into yet is an endpoint enrolling against a dashboard still sitting on installer defaults.

4. Building the first ingest pipeline

4.1 Picking a forwarder

[CONCEPT] A SIEM only ever sees what something else sends it. For Wazuh, that's almost always the Wazuh agent — a small process installed on each endpoint that ships logs, file-integrity events, and (on Windows) Sysmon/Event Log data back to the manager over its own protocol on port `1514/tcp`, with enrollment on port `1515/tcp`. Elastic-stack builds typically use Filebeat instead, shipping to Logstash or directly to Elasticsearch on port `5044/tcp`. Graylog accepts both syslog and GELF inputs depending on the source. This part builds and proves the pipeline itself, using the SIEM host as its own test source;

Part 9 and Part 10 cover installing the Wazuh agent (or Filebeat) on the actual Linux and Windows endpoints once they exist.

4.2 Opening exactly the ports this needs

[SAFETY] Before wiring in a single forwarder, the firewall built in Part 6 needs explicit allow rules for exactly these ports — everything else stays denied by default, per Part 4's segmentation model.

Source	Destination	Port/protocol	Action
Endpoint VLAN	SIEM host	1514/tcp (Wazuh agent)	Allow
Endpoint VLAN	SIEM host	1515/tcp (Wazuh agent enrollment)	Allow
Endpoint VLAN	SIEM host	5044/tcp (Filebeat, if used instead of or alongside the Wazuh agent)	Allow
Management VLAN	SIEM host	443/tcp (dashboard HTTPS)	Allow
Any other source	SIEM host	any	Deny (default)

Add these as explicit rules in the pfSense/OPNsense interface built in Part 6 rather than a catch-all "allow endpoint VLAN to SIEM" rule — a broad rule here quietly reopens exactly the blast-radius problem Part 4 exists to prevent, since it would let a compromised endpoint reach anything else the SIEM host happens to expose, not just the ingest ports it actually needs.

4.3 The ingest architecture

[CONCEPT] The diagram below shows where the pieces from §4.1 and §4.2 sit relative to the VLAN boundary and firewall rules from Parts 4 and 6 — the shape this part is building toward, before any endpoint actually exists to populate it.

```
flowchart LR
    subgraph EP["Endpoint VLAN (Part 4)"]
        WinEP["Windows endpoint<br/>Sysmon + Wazuh agent (Part 10)"]
        LinEP["Linux endpoint<br/>auditd + Wazuh agent (Part 9)"]
    end
    subgraph MG["Management / Monitoring VLAN (Part 4)"]
        SIEM["SIEM host<br/>manager + indexer + dashboard"]
        Analyst["Analyst workstation"]
    end
    FW["Firewall - default-deny,<br/>explicit allow rules only (Part 6)"]
    WinEP -- "1514/tcp agent traffic" --> FW
    LinEP -- "1514/tcp agent traffic" --> FW
    FW -- "allowed ingest ports only" --> SIEM
    Analyst -- "443/tcp dashboard" --> FW
    FW -- "dashboard access only" --> SIEM
```

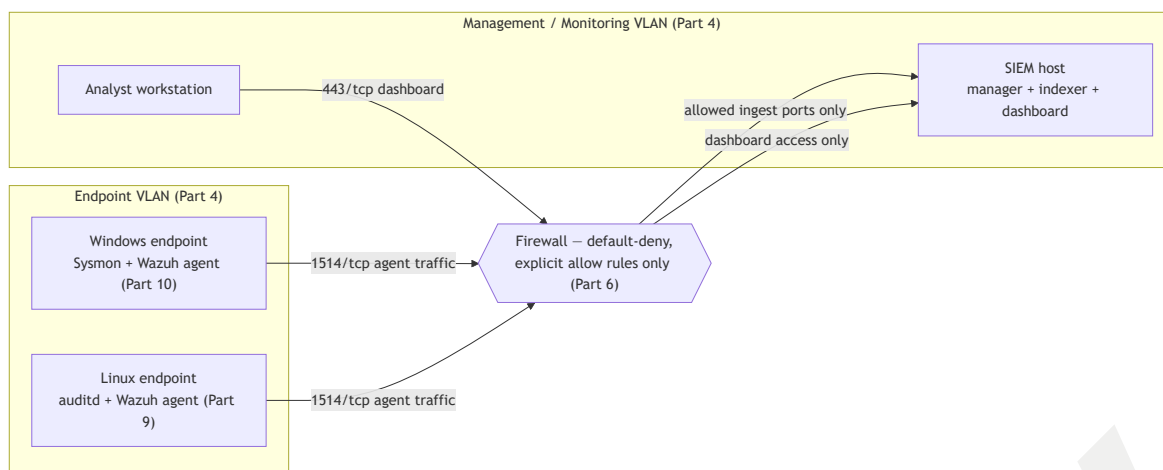


Figure 8.1 — SIEM ingest pipeline inside the isolated lab segment. *CONCEPTUAL.* Illustrates where endpoint forwarders, the SIEM host, and analyst dashboard access sit relative to the Part 4 VLAN boundary and the Part 6 firewall's allow rules. This is an architecture sketch, not a capture from a running build — no endpoints exist yet at this point in the book, and the author's own real lab currently runs no commercial SIEM/EDR product, so no REAL LAB EXAMPLE screenshot of this specific pipeline exists to cite here or elsewhere in this part.

5. Verifying logs arrive and are searchable

[HANDS-ON LAB]

Goal: prove the ingest pipeline moves a log line from the SIEM host itself into the searchable index, without depending on any endpoint build from later parts. Run this before moving on to Part 9 or Part 10 — if it fails, adding real endpoints on top only adds more sources that also won't show up, which is a much harder problem to debug than one missing test line.

1. On the SIEM host, generate a local syslog test line with a tag you can search for later.
2. Open the dashboard.
3. Search for that tag.
4. Confirm the entry's timestamp matches when you actually ran the command, not some other time.

```
logger -t labtest "part08 ingest check $(date +%s)"
```

Validation Test Setup: SIEM installed per §3, dashboard reachable, no endpoints forwarding yet. **Action:** `logger -t labtest "part08 ingest check $(date +%s)"` on the SIEM host itself.

Expected result: A new entry tagged `labtest` appears in the SIEM's search view within seconds, with a timestamp matching when you ran the command. If nothing appears after a minute, the pipeline itself is broken — go to §6 before wiring in any real endpoint.

6. Troubleshooting the installs that go wrong

[TROUBLESHOOTING]

Two failure modes account for nearly every broken first install, beyond the undersized-hardware failure already covered in §3's Build Autopsy.

The indexer bootstraps but the manager can't reach it. Wazuh's manager and indexer communicate over a local port that a host firewall (`ufw`, if it's enabled on the VM itself, separate from the Part 6 network firewall) can block without any obvious error in the dashboard. Check `sudo systemctl status wazuh-indexer` for `active (running)` first, then check the manager's own log at `/var/ossec/logs/ossec.log` for connection-refused errors before assuming the ingest pipeline in §4 is the problem.

Agent enrollment fails with a certificate or key mismatch. This shows up once you reach Part 9 or Part 10 and try to enroll a real endpoint: the agent reports it can't validate the manager's certificate, or enrollment succeeds but no events ever arrive. This is almost always a clock-skew problem, not a certificate problem — if the endpoint's clock has drifted more than a few minutes from the SIEM host's, enrollment and encrypted agent traffic both fail in ways that look like a certificate issue in the log output. Confirm both hosts are pointed at the same NTP source before troubleshooting anything else.

The dashboard suddenly stops accepting new data after weeks of running fine. OpenSearch and Elasticsearch both flip an index to read-only once the underlying disk crosses a "flood-stage" watermark — 95% full by default — as a safety measure against writing into a completely full filesystem. The dashboard keeps loading and old data stays searchable, but nothing new lands, with no obvious error unless you check the indexer's own log for a `flood_stage_watermark` message. Running `df -h` on the SIEM VM's data volume is the first check, well before assuming the ingest pipeline itself broke; Part 21 covers the retention policy that keeps you from hitting this watermark in the first place.

Blind Spot A log line landing in the index and showing up in search proves the pipeline moves bytes from source to storage. It proves nothing about whether those bytes were parsed into the right fields, whether a timestamp reflects when the event actually happened versus when it was ingested, or whether the same field name means the same thing across two different log sources. This part's finish line — logs arrive and are searchable — is the floor Detection Engineering Handbook V2, Parts 5–7 build on, not a substitute for that work.

7. What this part stops at, and where the telemetry goes next

[CONCEPT]

A working, searchable SIEM with one verified test log line is infrastructure, not a detection capability — the gap between the two is deliberate, and closing it is explicitly the other volumes' job rather than something duplicated here.

- **Detection Engineering Handbook V2, Parts 5–7** cover parsing quality, field normalization, and time-handling in depth — the difference between a log that's merely searchable and one that's actually structured well enough to build a detection on. Read these before you trust any dashboard panel built on this part's pipeline.
- **Detection Engineering Handbook V2, Parts 22–33** cover the correlation, baselining, and threat-intel scoring logic that turns raw searchable events into an actual alert. Nothing in this part fires an alert on anything — it only makes the underlying data visible.

- **Detection Engineering Handbook V2, Parts 34–36** cover threat-hunting methodology once telemetry exists to hunt in — directly applicable the moment Part 9 and Part 10's endpoints start forwarding real data through the pipeline this part built.
- **SOC Playbook Handbook's Playbook Library** covers what a fired alert actually means and how to triage it — relevant once Detection Engineering Handbook V2's rule logic is layered on top of this part's ingest pipeline and something actually fires.
- **SOC Manager's Operating Handbook, Part 8 (assessment design) and Part 11 (onboarding/ramp-up)** cover structuring a repeatable practice drill against a team — relevant once this lab has enough of the rest of the book built (endpoints, NSM, honeynet) to generate the kind of telemetry a drill needs.
- **This book's own Part 17** builds the dashboards worth looking at on top of everything this part and Parts 9–16 feed into the SIEM. **Part 19** closes the loop fully, pairing a simulated action with a specific detection and a specific playbook.

Lab Note *Don't point every endpoint at the SIEM the day you finish this part's install. Wire in one test source, confirm it with \$5's Validation Test, and only then move on to Part 9 and Part 10's real endpoint builds — debugging "why isn't anything arriving" against five endpoints at once is a much worse afternoon than debugging it against one.*

Cross-references: Part 3 (hardware tiers and resource budgeting), Part 4 (network isolation and segmentation architecture), Part 5 (hypervisor installation), Part 6 (firewall and segmentation build), Part 7 (internal DNS), Part 9 (Linux endpoints and auditd), Part 10 (Windows endpoints and Sysmon), Part 17 (dashboards and visualization), Part 19 (purple-team drills), Appendix A4 (safety and isolation pre-flight checklist); Detection Engineering Handbook V2 Parts 5–7, 22–33, and 34–36; SOC Playbook Handbook's Playbook Library; SOC Manager's Operating Handbook Parts 8 and 11.

Section C — Endpoint Telemetry

neshboy

Part 9 — Linux Endpoints and auditd Deployment

Why this part exists

Every other part in Section C and D of this book assumes something is generating logs worth searching. This part is where that stops being an assumption. You'll stand up one or more Linux lab endpoints, turn on the telemetry Linux gives you for free (journald, syslog), then layer `auditd` on top for the telemetry Linux does not give you for free — who touched `/etc/shadow`, what ran `execve` and with what arguments, whether a file that shouldn't change just changed. This is also, honestly, the part of this book with the deepest real-world backing: the author's own running lab is Linux-only, and several of the figures below are pulled directly from that lab's actual logs rather than constructed for the page. Where that isn't true — and it isn't true everywhere in this part — the text says so plainly rather than letting the reader assume otherwise.

Safety Gate *Everything below assumes the Linux endpoint(s) you build here live on the lab/victim network segment designed in Part 4 and firewalled in Part 6 — not bridged, NATed, or dual-homed onto your home LAN, and not reachable from the open internet. Before you provision a single container or VM for this part, confirm two things: the endpoint's only route to anything is through the perimeter firewall built in Part 6, and the only destination that firewall permits outbound from this segment is the SIEM ingest port on the management VLAN (Part 8). If you're going to run the identity-watch exercises in §8 by editing `/etc/passwd` or `/etc/shadow` as a test, do it on a disposable lab endpoint you're prepared to rebuild — not a box that also holds a real credential you use elsewhere. Run the Appendix A4 pre-flight checklist if this is the first endpoint you're standing up on this segment.*

1. What this part builds, and what it deliberately skips

[CONCEPT] This part covers three things, in order: provisioning a Linux endpoint sized for the job, confirming journald/syslog already produce useful baseline telemetry with zero extra software, then installing and tuning `auditd` to close the specific gap journald leaves — file-integrity-style watches and process-execution auditing. It does not cover how to read what any of this produces once it lands in a SIEM; that's Detection Engineering Handbook V2's job (cited by part number in §9 below), and this part stops the moment the telemetry is confirmed to arrive intact and searchable.

It also does not cover Windows endpoints (Part 10) or why this book puts Linux first (the short version: the author's real lab is Linux-only, so this part gets to lean on real evidence immediately instead of constructing it — see `BOOK-INDEX.md` provenance item 6 if you want the full reasoning).

2. Choosing and sizing Linux lab endpoints

[CONCEPT] A lab endpoint's job in this book is narrow: run some real services, get used a little, and produce telemetry a SIEM can ingest and a reader can practice against. It does not need to be a beefy

box. What it needs is a stable identity (a hostname and IP that don't drift every rebuild), enough headroom that `auditd` and a log forwarder don't starve whatever service you're also running on it, and — this is the part that actually catches people — the right virtualization choice for what you're asking `auditd` to do. More on that in §4.

2.1 Container or VM

[COST/RESOURCE] Table 9.1 sizes three common endpoint roles against the two virtualization choices this book's readers are likely using per Part 5 (Proxmox LXC containers or KVM/VirtualBox VMs). The right column is the one most readers skip past and shouldn't — it's the reason §4 exists as its own section.

Table 9.1 — Linux lab endpoint sizing by role. The table below supports the decision of which platform (container or VM) and how many resources to commit to each endpoint role before provisioning anything.

Endpoint role	Platform	Min RAM	Min disk	Runs <code>auditd</code> cleanly?
General-purpose service host (DNS, web app, file share)	LXC container	512MB	4GB	No — see §4
General-purpose service host (DNS, web app, file share)	KVM/VirtualBox VM	1GB	8GB	Yes
Dedicated <code>auditd</code> -instrumented endpoint (this part's main exercise)	KVM/VirtualBox VM	1GB	10GB	Yes
Log/journal forwarder relay (optional, if not folding into the SIEM host)	LXC container	512MB	4GB	Not needed — no rules run here

[SETUP] This chapter targets Debian 12 (bookworm) and Ubuntu 22.04/24.04 LTS as the endpoint OS — both ship a current audit userspace (`auditd` package roughly version 3.0–3.1) and current `systemd`/`journald`. If you're on a different distribution, the package names in §4 will differ (RHEL/Rocky use the same `auditd` package name but a different default rule-loading path); the rule syntax itself in §5 is portable across all of them since it's a kernel-level interface, not a distribution-specific one.

3. Baseline telemetry before you install anything extra: journald and rsyslog

[CONCEPT] Before `auditd` enters the picture, a default Linux install is not silent. `journald` already captures every `sudo` invocation, every `systemd` unit state transition, and every SSH authentication attempt, success or failure, with no configuration at all. This matters for two reasons: it's real telemetry you can forward into the Part 8 SIEM today, and it's the honest baseline against which "why do I even need `auditd`" gets answered in §5 — `journald` tells you *that* someone ran a command as another user; it does not tell you *what that command touched* unless the command's own output says so.

The three figures below aren't constructed examples. They're excerpts from the author's own running lab — a Pi-hole DNS container (CT100) and a vulnerability-scanner platform container (CT104), both Debian-based LXC containers on the Proxmox host this book's Part 5 describes — captured directly from `/var/log/auth.log` and `journalctl` with no modification beyond trimming line counts for the page.

```

Jul 07 11:54:58 pihole sudo[33546]:      root : TTY=pts/1 ; PWD=/root ; USER=root ; COMMAND=/usr/bin/ss
-tulpn
Jul 07 11:54:58 pihole sudo[33546]: pam_unix(sudo:session): session opened for user root(uid=0) by
(uid=0)
Aug 17 15:28:38 pihole sudo[143836]:      root : TTY=pts/1 ; PWD=/root ; USER=root ;
COMMAND=/usr/bin/pihole-FTL --config

```

Figure 9.1 — Baseline `sudo` invocation telemetry, no extra tooling installed. REAL LAB EXAMPLE.

Captured from `/var/log/auth.log` on the author's CT100 Pi-hole container via `grep -i sudo`. Shows the invoking user, target user, and exact command for two legitimate administrative `sudo` calls — this is the shape of the baseline you'd build anomaly detection against once this telemetry reaches Detection Engineering Handbook V2's baselining content (its Part 31).

```

Sep 09 19:26:06 vulnscan systemd[1]: vulnscan-web.service: Deactivated successfully.
Sep 09 19:26:06 vulnscan systemd[1]: Stopped vulnscan-web.service - Vulnerability Scanner Dashboard
(web).
Sep 09 19:26:07 vulnscan systemd[1]: Starting vulnscan-web.service - Vulnerability Scanner Dashboard
(web)...
Sep 09 19:26:07 vulnscan systemd[1]: Started vulnscan-web.service - Vulnerability Scanner Dashboard
(web).

```

Figure 9.2 — `systemd` unit state transitions via `journalctl`. REAL LAB EXAMPLE. Captured from the author's CT104 vulnerability-scanner container, showing the canonical Stopping → Deactivated → Stopped → Starting → Started sequence for a real application unit (`vulnscan-web.service`). No `auditd`, no agent, no configuration — this is what `journalctl -u <unit>` gives you the moment the unit exists.

admin	ssh:notty	<redacted-internal-host>	Mon Sep 14 01:17 - 01:17	(00:00)
root	ssh:notty	<redacted-internal-host>	Mon Sep 14 01:06 - 01:06	(00:00)
root	ssh:notty	<redacted-internal-host>	Mon Sep 14 01:05 - 01:05	(00:00)

Figure 9.3 — Failed SSH authentication burst via `lastb`. REAL LAB EXAMPLE, source address redacted for this book. Dozens of failed `admin`/`root` SSH attempts in rapid succession against the author's CT104 container, sourced from a single host elsewhere on the author's home network (address withheld here; the original capture is cited unredacted in Detection Engineering Handbook V2's evidence manifest, where the exposure was already assessed as low-risk since it's an internal address). `lastb` never exposes the attempted password, so no credential redaction was needed — only the source address is trimmed here as an extra precaution for this book. The pattern itself — dozens of attempts per minute against a small set of usernames — is exactly the shape a brute-force detection rule keys on, real or simulated.

4. Installing `auditd`

[SETUP] `auditd` gives you the layer `journald` doesn't: kernel-level watches on specific files (did anything write to `/etc/shadow`) and kernel-level watches on specific syscalls (did anything call `execve`, and with what arguments). The commands below target Debian 12/Ubuntu 22.04's `auditd` package.

```
sudo apt update
sudo apt install auditd audispd-plugins
sudo systemctl enable --now auditd
sudo systemctl status auditd
```

Confirm the service is active and check that rules are loading with `auditctl -l` — an empty list is expected until §5 adds rules, but the command itself should return cleanly, not an error.

Engineering Reality The Linux kernel's audit subsystem is not namespace-aware — it's a single, global facility owned by the host kernel, not something an unprivileged container gets its own copy of. Run the commands above inside an unprivileged LXC container (the same kind CT100 and CT104 actually are) and `auditctl -l` typically fails outright, or the daemon starts but silently produces no records, because the container was never granted `CAP_AUDIT_CONTROL` against the host's audit netlink socket. This is exactly why Table 9.1 marks the dedicated `auditd` endpoint row "VM," not "container": if you want the rule-set content in §5 to actually work, build this specific endpoint as a KVM/VirtualBox VM per Part 5, not as an LXC container, even though every other endpoint role in this book is happy as a container.

Validation Test Setup: `auditd` installed and running per the commands above, on a VM-based endpoint (not an LXC container). **Action:** `sudo auditctl -w /etc/hostname -p r -k lab_validation` followed by `cat /etc/hostname`, then `sudo ausearch -k lab_validation`. **Expected result:** `ausearch` returns at least one `type=SYSCALL` record with `key="lab_validation"` and a `comm="cat"` field — proof the kernel audit hooks are live end to end on this host before you invest in the full rule set in §5.

5. Writing an auditd rule set

[CONCEPT] `auditd` rules live in `/etc/audit/rules.d/*.rules` and compile into the running rule set via `augenrules --load`. Every rule you write here is one of two shapes: a **watch** (`-w <path> -p <perms> -k <key>`, tripped when anything accesses that path with those permissions), or a **syscall rule** (`-a always,exit -F arch=b64 -S <syscall> -k <key>`, tripped whenever that syscall runs, regardless of path). Every rule should carry a key — the `-k` flag — so `ausearch -k <key>` can pull exactly the records you care about instead of grepping the whole log.

5.1 Watching identity and access-control files

[SETUP] The three watches below are the minimum baseline for identity telemetry — anything editing account, credential, or privilege-escalation configuration on this endpoint.

```
# /etc/audit/rules.d/identity.rules
-w /etc/passwd -p wa -k identity
-w /etc/shadow -p wa -k identity
-w /etc/group -p wa -k identity
-w /etc/sudoers -p wa -k sudoers
-w /etc/sudoers.d/ -p wa -k sudoers
-w /etc/ssh/sshd_config -p wa -k sshd_config
```

Load the rules and confirm they're active:

```
sudo augenrules --load
sudo auditctl -l | grep -E "identity|sudoers|sshd_config"
```

You should see six active watch rules echoed back, each carrying the key you assigned — this is the check to run before moving to §5.2, not after something breaks.

5.2 Watching process execution

[SETUP] The `identity` key (`-k identity`, from §5.1) tells you a file changed; it does not tell you what process changed it or what else that process did in the same breath. A syscall rule on `execve` closes that gap by logging every process a shell or service actually launches.

```
# /etc/audit/rules.d/execution.rules
-a always,exit -F arch=b64 -S execve -k exec
-a always,exit -F arch=b32 -S execve -k exec
```

Both architecture lines matter on a 64-bit host: a 32-bit binary invoking `execve` generates a `b32` syscall entry, and a rule scoped only to `arch=b64` misses it silently — no error, just an absent record. Load and confirm the same way as §5.1, then generate one event to check the shape of the output.

```
sudo augenrules --load
whoami
sudo ausearch -k exec -ts recent | tail -n 20
```

A single `whoami` invocation produces a matched set of records sharing one `msg=audit(<epoch>:<serial>)` identifier: a `type=SYSCALL` line with `syscall=59` (the `execve` syscall number on x86-64) and `key="exec"`, paired with a `type=EXECVE` line showing the literal argument vector (`a0="whoami"`), and a `type=PATH`/`type=CWD` pair recording the binary's path and the working directory it ran from. `ausearch` groups all of these under the shared serial number automatically — you don't need to correlate them by hand.

5.3 A baseline key-tagging plan

[SETUP] Table 9.2 lays out the full baseline this part recommends before you add anything workload-specific — use it as the starting rule file, not the finished one.

Table 9.2 — Baseline auditd key-tagging plan. The table below supports deciding which watches belong in a first rule set versus which are workload-specific additions to layer on later.

Key	Rule type	Watched target	Purpose
identity	Watch, -p wa	/etc/passwd, /etc/shadow, /etc/group	Detect account/credential file tampering
sudoers	Watch, -p wa	/etc/sudoers, /etc/sudoers.d/	Detect privilege-escalation config changes
sshd_config	Watch, -p wa	/etc/ssh/sshd_config	Detect SSH access-policy changes
exec	Syscall, execve	All process execution	Baseline and hunt on process-launch activity
lab_validation	Watch, -p r	/etc/hostname (Validation Test only)	Confirm the audit pipeline is alive — remove after §4's test

Lab Note Snapshot the VM after §4's install and before you load a single rule from §5. A typo in a syscall rule (a wrong architecture flag, a malformed `-F` filter) is a five-minute rollback from that snapshot instead of chasing why `augenrules --load` silently accepted a rule that never matches anything.

5.4 Locking the rule set

[SETUP] `-e 2` in a rules file makes the running rule set immutable until the next reboot — no process, including root, can add, remove, or flush a rule without rebooting first. This is a real integrity control (it's what stops an attacker with root from just turning off the watch that would catch them), and it's also a real inconvenience while you're still iterating.

```
# append to the end of the last rules.d file loaded, e.g. 99-finalize.rules
-e 2
```

Build Autopsy — the audit.log that filled a lab disk in an afternoon

The plan: Broaden the §5.1–5.2 rule set to "watch everything" for maximum visibility — add `-p rwx` watches on `/etc`, `/var`, and `/home` wholesale, on top of the `execve` syscall rule, reasoning that more coverage is strictly better than less.

Why it seemed reasonable: The `auditctl` syntax makes broad watches just as easy to write as narrow ones, and nothing in the tool itself warns you before you load a rule that will fire constantly.

How it failed: A `-p rwx` watch on a high-traffic directory tree matches not just writes but reads and execute-permission checks on every file underneath it — package managers, log rotation, and the `execve` rule from §5.2 all generate reads through `/etc` and `/var` continuously. On a 10GB lab VM, unfiltered audit output at that volume fills `/var/log/audit` in hours, and `auditd.conf`'s default `disk_full_action = SUSPEND` stops auditing entirely the moment the partition fills — the exact opposite of the visibility the broad rule set was trying to buy.

The fix: Scope watches to the specific files that actually matter (Table 9.2), not directory trees; scope the `execve` rule's output by filtering in the SIEM rather than the kernel if volume is still too high; and set `space_left_action = email` (or a script action) in `/etc/audit/auditd.conf` at some threshold below 100% so you get warned before `disk_full_action` ever triggers, not after.

6. Forwarding audit and journal telemetry into the SIEM

[CONCEPT] Before the config, it's worth seeing the whole path an event takes — from a kernel-level audit rule or a journald entry, through local forwarding, across the segmentation boundary, to the SIEM.

flowchart LR

```
subgraph EP["Linux lab endpoint (this part)"]
    AUD["auditd rules\nidentity, sudoers, sshd_config, exec"] --> LOG["LOG\n--> /var/log/audit/audit.log"]
    LOG --> PLUGIN["PLUGIN\n--> audisp-syslog plugin"]
    JRNL["journald\nsudo, SSH auth, systemd units"] --> FWDJ["FWDJ\n--> ForwardToSyslog=yes"]
    PLUGIN --> RSYS["RSYS\n--> local rsyslog"]
    FWDJ --> RSYS
end
RSYS -->|"omfwd, TCP, mgmt VLAN only"| FW["FW\n--> Perimeter firewall\n(Part 6)"]
FW -->|"allowed: SIEM syslog port"| SIEM["SIEM ingest\n(Part 8)"]
FW -->|"default-deny — no route"| HOME["HOME\n--> Reader's home network\n(must stay unreachable)"]
```

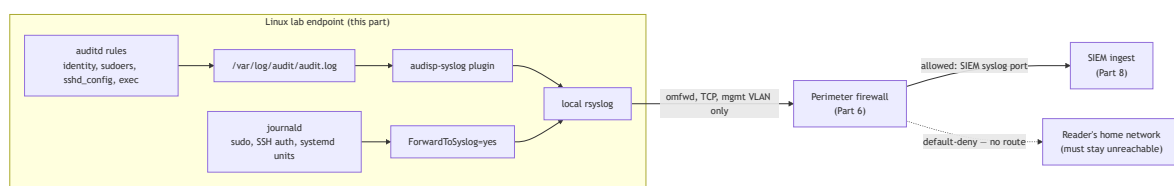


Figure 9.4 — Linux endpoint telemetry pipeline and isolation boundary. *CONCEPTUAL.* Illustrates how `auditd` and `journald` telemetry produced on a Part 9 lab endpoint reaches the Part 8 SIEM through a local `rsyslog` forwarder, and how the Part 4/6 segmentation confines that forwarder's only permitted egress to the SIEM's syslog port while denying any route back to the reader's home network. This is an architecture sketch of the intended build, not a capture from a running deployment.

[SETUP] `auditd`'s `audisp-syslog` plugin is the most portable forwarding path across whichever SIEM you built in Part 8 — Wazuh, the Elastic stack, or Graylog all accept syslog input, so this section targets that plugin rather than a SIEM-specific agent. Enable it in `/etc/audit/plugins.d/syslog.conf`:

```
# /etc/audit/plugins.d/syslog.conf
active = yes
direction = out
path = /sbin/audisp-syslog
type = always
args = LOG_LOCAL6
format = string
```

Restart `auditd` to pick up the plugin change, then point local `rsyslog` at the SIEM's syslog listener for both the audit facility and the general journal:

```
# /etc/rsyslog.d/60-forward-to-siem.conf
local6.*      @@<siem-mgmt-vlan-ip>:<siem-syslog-port>
auth,authpriv.*  @@<siem-mgmt-vlan-ip>:<siem-syslog-port>
```

The `@@` prefix forces TCP rather than UDP — worth keeping for audit data specifically, since a dropped UDP datagram of a `SYSCALL / EXECVE / PATH` triplet loses a correlated event silently, with nothing in the local log to flag the gap. Also set `ForwardToSyslog=yes` in `/etc/systemd/journald.conf` so the `sudo` and `systemd` unit telemetry from §3 rides the same forwarding path.

```
sudo systemctl restart auditd rsyslog
sudo systemctl restart systemd-journald
```

Validation Test Setup: Forwarding configured per the two files above, SIEM ingest confirmed reachable per Part 8. **Action:** `sudo ausearch -k identity -ts recent` locally to confirm a record exists, then search the SIEM for the same event (by `key=identity` or the equivalent field your SIEM's syslog parser assigns it) within roughly one minute of generating it. **Expected result:** The identical `SYSCALL / PATH` pair appears in the SIEM's search index, not just the local `audit.Log` — if it's on the endpoint but not in the SIEM, the break is in the forwarding path (§6), not the rule set (§5).

Resource Reality A lab endpoint with the Table 9.2 baseline rule set generates roughly a few hundred kilobytes to low single-digit megabytes of raw `audit.Log` per day of light, human-driven use — the `execve` rule is the dominant contributor by volume, since every shell command and every script's internal process spawns counts. Budget at least 5GB of disk headroom beyond the OS itself specifically for `/var/log/audit` on any endpoint running the `exec` key, and set `max_log_file_action = ROTATE` with a bounded `num_logs` in `auditd.conf` — without rotation, this is a slow version of the Build Autopsy failure above, not a different one.

7. Tuning for a lab-sized host: backlog and rate limits

[TROUBLESHOOTING] Two `auditd` behaviors surprise readers who configured everything in §5–6 correctly and still see gaps. First: the kernel audit backlog (tunable via `auditctl -b <limit>`) is a fixed-size buffer between the kernel generating a record and `auditd` consuming it — a burst of syscalls matching your rules (a `find /` crossing a watched path, a build script spawning hundreds of processes against the `exec` key) can exceed the default backlog faster than `auditd` drains it, and the kernel's default failure mode is to silently drop the overflow rather than block anything. If your event counts in the SIEM look thinner than the activity you generated, check `dmesg` for `audit: backlog limit exceeded` before assuming a rule is wrong. Second: `auditd.conf`'s `rate_limit` setting (0 by default, meaning unlimited) exists specifically to cap events-per-second if a rule turns out noisier than expected in practice — set it once you've measured a baseline rate under normal use, not before, or you'll rate-limit away the exact activity you're trying to capture.

8. Hands-on lab: watching identity-file access end to end

[HANDS-ON LAB] Goal: confirm, on your own lab endpoint, that a real identity-file modification produces a correlated, forwarded, searchable event — not just that the pipeline is theoretically wired correctly.

1. On the VM-based endpoint from §2–§4, run `sudo useradd -m labtestuser` to create a disposable test account.
2. Immediately run `sudo ausearch -k identity -ts recent` locally and confirm a `type=SYSCALL` record appears with `key="identity"` and a `type=PATH` record naming `/etc/passwd`.
3. Search the Part 8 SIEM for the same event within about one minute.
4. Clean up: `sudo userdel -r labtestuser`, and confirm a second, distinct `identity`-keyed event appears for the deletion.

Expected result: two distinct, correlated identity events — one for the account creation, one for the deletion — both visible in the SIEM with the process (`useradd`/`userdel`) and target file intact. If step 2 succeeds locally but step 3 never shows the event, the forwarding path from §6 is where to look, not the rule set.

Blind Spot `auditd` and `journald` both tell you what happened on this host's kernel and init system. Neither sees anything that happens inside another container or VM sharing the same hypervisor unless that guest runs its own instrumentation — and per the Engineering Reality in §4, an LXC container can't run `auditd` at all. A reader who instruments only the VM-based endpoint from this part and assumes the LXC-based service containers built elsewhere in this book (Part 7's DNS container, for instance) are equally visible is wrong; those containers' visibility is whatever `journald` and application logs give you, full stop, until Part 20's maintenance content or a future revision addresses host-level auditing of the hypervisor itself.

What Would Change My Mind This part recommends VMs over LXC containers specifically for `auditd`-instrumented endpoints, on the grounds that the Linux audit subsystem isn't namespace-aware. Kernel namespace-aware audit filtering has been an active, if slow-moving, area of upstream discussion; if a stable kernel and audit-userspace release ships genuine per-namespace audit rule support that Debian/Ubuntu backport into a supported release, this part's platform recommendation in Table 9.1 would need revision — not the underlying principle that you need to know which layer your rules actually run at.

9. Where this telemetry goes next

[CONCEPT] This part's job ends the moment `identity`, `sudoers`, `sshd_config`, and `exec`-keyed events, plus the baseline `journald`/`syslog` telemetry from §3, are confirmed arriving intact in the Part 8 SIEM. Three things a reader should do with that telemetry next, each owned by a different NESHBOY volume:

- **Read the fields correctly.** Detection Engineering Handbook V2, Part 3 (Telemetry Engineering I: Host and Identity Sources) is where `SYSCALL / EXECVE / PATH` field semantics, `journald` field normalization, and this exact `auditd`-key vocabulary get turned into parsed, queryable schema — this part produces the raw material that chapter assumes exists.
- **Build detection logic on top of it.** Baselining "normal" `sudo` and `exec` activity for this specific endpoint, and writing correlation logic that flags a deviation, is Detection Engineering Handbook V2, Part 31's job, not this part's — §5.1's `identity` key existing is the prerequisite, not the detection.
- **Know what to do when it fires.** Once a detection built on this telemetry fires for real, SOC Playbook Handbook's Linux logging technical-reference chapters own triage procedure for exactly this event shape — what to check first on a host that just produced an unexpected `identity`-keyed event, and how to scope whether it's isolated or part of something larger.

Part 19 (Generating Practice Telemetry and Purple-Team Drills) later in this book closes the loop the other direction — pairing a scoped, reversible attack-simulation action against this exact endpoint with the specific detection and playbook it should trigger, so the telemetry this part builds doesn't just sit in a SIEM unused.

Cross-references: Part 4 (network isolation this part's Safety Gate depends on), Part 5 (hypervisor choice underlying Table 9.1's container/VM split), Part 6 (perimeter firewall enforcing the egress-only-to-SIEM rule), Part 8 (SIEM ingest this part forwards into), Part 10 and Part 11 (Windows endpoint content and the Windows/AD honesty gap), Part 19 (purple-team drills that consume this part's telemetry), Appendix A3 (auditd baseline rule file, reusable starting point for §5), Appendix A4 (pre-flight isolation checklist); Detection Engineering Handbook V2, Part 3 and Part 31 (deh:part03, deh:part31); SOC Playbook Handbook's Linux logging technical-reference chapters.

neshbooy

Part 10 — Windows Endpoints and Sysmon Deployment

Why this part exists

Part 9 built Linux endpoints because that's what the author's real lab actually runs. Most public detection content, most EDR vendor documentation, and most of Detection Engineering Handbook V2's own Windows-focused material (its Parts 8–9) assume Sysmon and Windows Event Log telemetry exists somewhere. A SOC home lab that never produces that telemetry can't practice against a large share of what the rest of this library teaches. This part closes that gap the honest way: one or more standalone, non-domain-joined Windows virtual machines, Sysmon deployed with a real configuration instead of the near-useless default, and both Sysmon and Windows Event Log forwarded into the SIEM Part 8 built.

"Standalone" is the operative word. This part does not build an Active Directory domain, a domain controller, or any GPO-driven rollout — Part 11 explains exactly why, using the author's own abandoned attempt at that build as the case study. What you get here is real, useful, and honestly scoped: process-creation, network-connection, and file-creation telemetry from a single Windows host, plus the Windows security log's own authentication and privilege events in their local (non-domain) form. What you don't get is domain authentication, Kerberos ticketing, or lateral-movement telemetry that depends on a trust boundary — those need a domain, and a domain is Part 11's problem to scope, not this part's problem to fake.

Safety Gate *The Windows VM (or VMs) this part builds must live entirely inside the isolated lab/victim network segment designed in Part 4 — the same segment, not a look-alike copy of it on a different vSwitch. Before installing anything, confirm the VM's virtual NIC is attached to that segment's VLAN and not to a default or bridged network the hypervisor may have offered during VM creation; a single mis-attached NIC gives this Windows host a direct path to your home network, which defeats every isolation control built in Parts 4 and 6. Verify the attachment in the hypervisor's own NIC settings, not just by checking the IP address Windows reports — a host can show a lab-segment IP and still be bridged if the underlying vSwitch port group is wrong.*

1. Scope: what this build is, and what it deliberately isn't

[CONCEPT] A standalone Windows endpoint is a Windows VM that boots, runs, and authenticates entirely on its own — a local administrator account, no domain controller to check in with, no Group Policy pushed from anywhere. That's a real limitation compared to almost every production Windows environment a reader will eventually work in, and it's worth naming plainly rather than glossing over: this build teaches Sysmon and Windows Event Log fundamentals, not domain-authentication detection content. Detection Engineering Handbook V2's Parts 8–9 cover how to read the telemetry this part produces; nothing in those parts requires a domain to be meaningful, since process creation, network connections, and file writes happen identically whether or not a host is domain-joined.

Build Autopsy — the abandoned Windows/AD forensics lab

The plan: Stand up a persistent two-DC Active Directory forest with several domain-joined Windows endpoints, generating realistic domain-authentication and Kerberos telemetry for detection practice.

Why it seemed reasonable: A domain lab looked like the obvious next step after a standalone Windows host, and free evaluation-licensed Server ISOs made the software cost zero.

How it failed: Domain controller promotion, a GPO-driven Sysmon and audit-policy rollout, and keeping two rotating evaluation licenses alive turned into ongoing maintenance the rest of the lab (the honeynet, the vulnerability scanner) never demanded. The build stalled at a half-configured single DC and was never finished.

The fix: No fix was completed, and this part doesn't pretend otherwise. This book teaches the standalone host you're reading about right now, and documents the domain build's real cost honestly in Part 11 instead of walking through a lab that doesn't exist.

2. Sizing the Windows guest: licensing options and resource cost

[COST/RESOURCE] A Windows VM costs more than any single Linux endpoint from Part 9, in every dimension that matters for a home lab: disk, RAM, and licensing complexity. The table below compares the realistic options for a lab guest OS; it supports the decision of which image to actually download before spinning up a VM.

Option	License cost	Real lifespan	Typical idle RAM	Notes
Windows 11 Enterprise Evaluation	Free (Microsoft Evaluation Center)	90 days, extendable with <code>slmgr /rearm</code> (commonly three resets before reinstall is required)	2–3GB	Closest to a real managed endpoint; plan for a periodic rearm or reinstall cycle
Windows Server 2022/2025 Evaluation, used as a workstation	Free (Microsoft Evaluation Center)	180 days, also rearm-extendable	2GB	Server-role defaults and Event Log noise differ from a real workstation — skip this unless you specifically want server telemetry
Retail Windows 10/11 (a license you already hold, not reused from an active machine)	Cost of the license, often \$0 if idle	Indefinite	2–3GB	No expiry to manage, but don't reuse a key that's activating a production machine
Windows on ARM or other non-x86 builds	Varies	Varies	Varies	Avoid — Sysmon and guest-driver compatibility on non-x86 hosts is inconsistent enough to cost more lab time than it saves

Resource Reality Budget 4GB of RAM and 80GB of thin-provisioned disk as the practical floor for one Windows 11 Enterprise Evaluation VM doing real Sysmon-plus-agent work, not the 4GB/64GB Microsoft lists as the bare installer minimum — the installer minimum assumes an idle desktop, not a host also running a forwarding agent and generating test telemetry on demand. On a hypervisor host already running the SIEM from Part 8 and a couple of Linux endpoints from Part 9, this is usually the single biggest RAM line item in the whole lab; check the tier budget from Part 3 before adding a second Windows host.

What Would Change My Mind This part recommends evaluation media over retail Windows for a first Windows lab endpoint, on the grounds that eval images can be freely rearmed for a fixed lab lifecycle without spending a license slot. If you already hold a retail Windows 10 or 11 license sitting completely idle — not activating any other machine — that changes the calculus, and retail is the simpler choice for you specifically: no rearm cycle to track, no eventual reinstall.

3. Provisioning the standalone Windows VM

3.1 Choosing and downloading an eval image

[SETUP] These steps target the Windows 11 Enterprise Evaluation ISO from Microsoft's Evaluation Center, current as of this writing.

1. Go to the Microsoft Evaluation Center and select the Windows 11 Enterprise evaluation download.
2. Choose the 64-bit ISO — Sysmon and every forwarding agent this part uses assumes 64-bit.
3. Verify the downloaded ISO's checksum against the value Microsoft's download page lists, before mounting it anywhere.
4. Copy the ISO to wherever your hypervisor (Part 5) expects installation media — a Proxmox ISO storage volume, a Hyper-V media folder, or equivalent.

Confirm the download worked by checking the ISO's file size against the value Microsoft's download page lists; a truncated download is the single most common cause of an installer that hangs partway through Windows Setup.

3.2 Building the VM inside the lab segment

[SAFETY] The choice that matters most in this step: the VM's virtual NIC must be attached to the lab/victim VLAN from Part 4, not any default network the VM-creation wizard offers.

[SETUP] Create the VM per your hypervisor's normal workflow from Part 5, with these lab-specific choices: at least 4GB of RAM (8GB if the host has headroom) and a 60–80GB thin-provisioned virtual disk. Boot from the ISO and run through Windows Setup normally.

Figure 10.1 sketches where this VM sits relative to the firewall, internal DNS, and SIEM built in earlier parts, so the network attachment step above has a concrete picture behind it rather than an abstract instruction.

```

flowchart LR
    subgraph Home["Home network (untouched)"]
        HomeRouter["Home router / ISP uplink"]
    end

    subgraph FW["Lab firewall/router – Part 6"]
        Rules["Deny-by-default inter-VLAN rules"]
    end

    subgraph LabVLAN["Lab/victim VLAN – Part 4"]
        WinHost["Standalone Windows 11 VM\nSysmon + forwarding agent\nno domain join"]
    end

    subgraph CoreVLAN["Core services VLAN – Part 4"]
        DNS["Internal DNS/DHCP – Part 7"]
        SIEM["SIEM ingest – Part 8"]
    end

    WinHost -->|"Sysmon + WinEvtLog, ports 1514/1515"| SIEM
    WinHost -->|"DNS queries"| DNS
    LabVLAN -->|"explicit allow rules only"| CoreVLAN
    FW -->|"default-deny, no route"| Home
    LabVLAN --- FW
    CoreVLAN --- FW

```

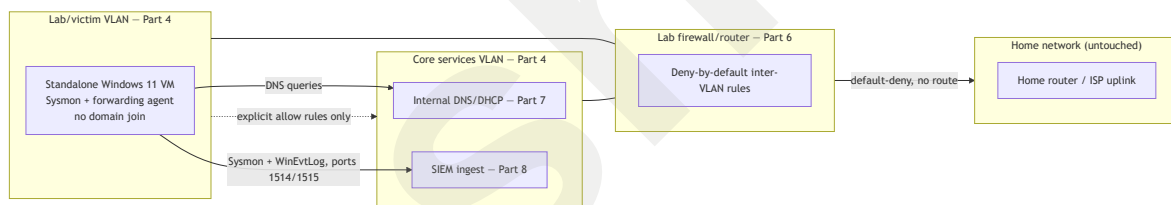


Figure 10.1 — Standalone Windows endpoint inside the lab network segment. CONCEPTUAL. Illustrates where the Part 10 Windows host sits relative to the firewall, internal DNS, and SIEM built in Parts 4, 6, 7, and 8. This is an architecture sketch showing the intended topology, not a capture from a running build — see Figure 6.2 (Part 6) for the same segmentation model after a real firewall build.

Engineering Reality Hypervisor documentation implies attaching a NIC to a VLAN is the whole story; in practice, a VM using a paravirtualized NIC driver (VirtIO on Proxmox/KVM, or the Hyper-V synthetic adapter without integration services installed) boots with no working network at all until the matching guest driver package is installed inside Windows. If the VM shows no IP address after setup completes, install the hypervisor's guest-tools package first — don't assume the VLAN attachment is wrong before ruling this out.

3.3 First boot and local-account setup

[SETUP] During Windows Setup's out-of-box experience, choose "Domain join instead" (or the equivalent option to skip a Microsoft account) and create a local administrator account. Do this even though a fresh VM has no domain to join yet — the option matters because skipping it correctly is what keeps this host standalone by default rather than by later remediation. Set a strong local password; this VM sits on an isolated segment, but "isolated" is not the same as "fine with a weak password," since anything else that later joins the same segment (Part 15's honeynet explicitly does not belong here, but a future exercise host might) inherits whatever habits this VM sets.

Lab Note *Take a hypervisor snapshot immediately after this first boot, before installing Sysmon or any forwarding agent. Every misconfigured Sysmon rule or bad agent enrollment in the rest of this part is then a two-minute snapshot revert instead of a 20-minute reinstall — take it now, before you're tempted to skip it because the install "went fine."*

4. Baseline hygiene: time, DNS, and update egress

[SAFETY] Point the Windows VM's DNS at the internal resolver built in Part 7 rather than a public resolver — this keeps the endpoint from depending on anything outside the lab segment to resolve names, and keeps lab DNS queries visible in the query log Part 7 already set up.

[SETUP] Set Windows Time to sync against a source reachable inside the lab (Part 7's host, if it also serves NTP) or through the one controlled egress path described below; skip this and forwarded events risk arriving with a timestamp meaningfully off from the SIEM's own clock.

[SAFETY] Windows Update needs some form of outbound HTTPS access to Microsoft's update endpoints to patch a long-lived lab host, and "some form of outbound access" is exactly the kind of blanket statement Part 4 exists to prevent. Add one explicit, narrow allow rule at the lab firewall (Part 6) for outbound HTTPS from this VM's specific IP, and leave everything else on default-deny egress. Don't widen that rule to "allow all outbound" for convenience — a single narrow allow rule is easy to audit later; a broad one quietly becomes the path something else uses.

5. Installing and configuring Sysmon

5.1 Installing the Sysmon service

[CONCEPT] Sysmon (System Monitor) is a free Microsoft Sysinternals tool that installs as a Windows service and driver, logging process creation, network connections, file creation, and several dozen other event types to a dedicated Windows Event Log channel. A default Sysmon install with no configuration file logs almost nothing useful — process creation with a thin field set and no filtering logic, which is why the next step matters more than the install itself.

[SETUP] This targets Sysmon v15.x (Sysinternals, current as of this writing) on Windows 11. Run the following from an elevated PowerShell prompt after copying the Sysmon binary and a config file to the endpoint:

```
.\Sysmon64.exe -accepteula -i C:\ProgramData\sysmon\sysmon-config.xml
```

Confirm it worked by running `Get-Service sysmon64` and checking for a `Running` status, then opening Event Viewer to `Applications and Services Logs > Microsoft-Windows-Sysmon/Operational` and confirming new entries appear as you use the machine.

5.2 Applying a real configuration baseline

[SETUP] Don't write a Sysmon configuration from scratch for a first build. Adapt one of the maintained public baselines — SwiftOnSecurity's `sysmon-config` and Olaf Hartong's `sysmon-modular` are the two most widely used starting points — and trim it to what your lab's hardware can log without flooding disk. Appendix A3 hosts this book's own trimmed baseline, built from those community sources, for exactly this purpose rather than repeating a full config inline in every part that needs one. Appendix A3 is not yet authored (see `README.md`); until it is, adapt one of the two community baselines above directly.

CONCEPTUAL SAMPLE — an illustrative excerpt only, not a complete or build-tested configuration; adapt a maintained public baseline instead of typing one from scratch.

```
<Sysmon schemaversion="4.90">
  <EventFiltering>
    <ProcessCreate onmatch="exclude">
      <!-- excluded - this fires on every service-hosted process and adds volume
           with little detection value on a single-host lab -->
      <Image condition="end with">svchost.exe</Image>
    </ProcessCreate>
    <NetworkConnect onmatch="include">
      <DestinationPort condition="is">3389</DestinationPort>
    </NetworkConnect>
  </EventFiltering>
</Sysmon>
```

After applying any config change, confirm it took effect by running `Sysmon64.exe -c` with no other arguments, which prints the currently loaded configuration back to the console.

6. Forwarding Windows Event Log and Sysmon telemetry to the SIEM

[SETUP] Part 8 left the choice of SIEM platform open between Wazuh, the Elastic stack, and Graylog. The forwarding mechanism differs by choice:

- **Wazuh** — install the Windows agent MSI and point it at the manager's lab IP during a silent install.
- **Elastic stack** — install Winlogbeat, configured to ship both the standard Windows Event Log channels and the `Microsoft-Windows-Sysmon/Operational` channel.
- **Graylog** — use the Graylog Sidecar with a Winlogbeat or NXLog backend, which Graylog's own documentation configures similarly to the Elastic case above.

The Wazuh path is shown here as the worked example, since it needs the fewest moving parts for a first build. Run this from an elevated PowerShell prompt on the Windows endpoint, targeting a Wazuh 4.x manager already reachable on the lab segment:

```
msiexec.exe /i wazuh-agent.msi /q WAZUH_MANAGER="10.10.10.5" WAZUH_AGENT_GROUP="windows-endpoints"
```

The agent connects to the manager on port 1514 for ongoing event traffic and port 1515 for one-time enrollment; both need an explicit allow rule at the lab firewall if Part 6's rule set doesn't already have one for SIEM ingest. Confirm the agent enrolled by checking the Wazuh manager's dashboard for this host in an **Active** state, or by running `Get-Service -Name WazuhSvc` locally and confirming it's running.

7. Validating the pipeline end to end

[HANDS-ON LAB] Generate one known event on the endpoint and confirm it arrives at the SIEM before treating the pipeline as working.

Validation Test Setup: Sysmon installed with the Appendix A3 baseline (not yet authored — see [README.md](#); substitute an adapted SwiftOnSecurity/ `sysmon-modular` config per §5.2 until it exists), the Wazuh agent (or Winlogbeat/Sidecar equivalent) enrolled and running, both confirmed per the previous two sections. **Action:** `powershell -Command "Start-Process cmd.exe -ArgumentList '/c whoami'"` on the Windows endpoint. **Expected result:** A Sysmon Event ID 1 (Process Create) entry appears in the SIEM within seconds, showing `ParentImage` = the PowerShell process and `Image` pointing to `cmd.exe`, followed immediately by a second Event ID 1 entry for `whoami.exe` with `cmd.exe` as its parent.

If nothing arrives within a minute or two, work through the troubleshooting section below before assuming the exercise itself is broken.

8. Troubleshooting the most common forwarding failures

[TROUBLESHOOTING] Three failure modes account for most "nothing showed up" reports on a first Windows endpoint build:

- **Sysmon logs locally but nothing forwards.** Check the forwarding agent's own service status first (`Get-Service WazuhSvc` or the Winlogbeat equivalent) — Sysmon working correctly says nothing about whether the agent is running at all.
- **The agent shows connected but events don't appear in search.** Check clock skew with `w32tm /query /status`. A Windows host whose clock has drifted more than a few minutes from the SIEM's own time can have its events silently misordered or filtered by time-range queries that assume roughly synchronized clocks.
- **Enrollment fails outright, agent never reaches Active.** Confirm the firewall rule from Part 6 actually allows ports 1514 and 1515 from this VM's specific IP — a rule scoped to the wrong

subnet after a VLAN renumbering is the most common cause, and it fails silently with no error on the Windows side beyond a generic connection timeout in the agent's own log.

9. Local hardening that still matters inside an isolated segment

[SAFETY] Isolation upstream doesn't excuse a wide-open host downstream — anything else that ever shares this segment (a future exercise VM, an Atomic Red Team runner in Part 18) inherits whatever attack surface this host leaves open.

[SETUP] On the standalone Windows endpoint:

1. Confirm the built-in Guest account is disabled (it is by default on modern Windows, but verify with `net user guest`).
2. Disable SMBv1 if it's present: `Disable-WindowsOptionalFeature -Online -FeatureName SMB1Protocol`.
3. Leave Remote Desktop disabled unless a specific exercise needs it, and if you do enable it temporarily, restrict it to the management VLAN in the Part 6 firewall rules, not the whole lab segment.
4. Turn on PowerShell Script Block Logging via the `EnableScriptBlockLogging` registry value under `HKLM:\Software\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging`, so Event ID 4104 entries are available once Detection Engineering Handbook V2's PowerShell-focused content needs them.

10. What a standalone host can't teach you

[CONCEPT] Naming the gap here matters more than papering over it.

Blind Spot A standalone Windows host has no domain to authenticate against, so it can never generate Kerberos ticket-granting events (Event IDs 4768 and 4769), domain-scoped logon events with a real domain in the `TargetDomainName` field, or any telemetry that depends on a trust boundary between hosts — because there is no trust boundary here, only one machine. Group Policy–driven configuration rollout, DCSync-style detection content, and lateral-movement telemetry between domain-joined peers are all out of reach on this build specifically. Part 11 names the real cost of closing that gap rather than pretending this part already closed it.

11. From telemetry to detection, response, and practice: where this build feeds the rest of the library

[CONCEPT] This part's job stops at "Sysmon and Windows Event Log events are searchable in the SIEM." What a reader does with them next is explicitly other volumes' territory, and the point of building this pipeline at all is to feed them:

- **Detection Engineering Handbook V2, Parts 8–9** cover how to actually read Sysmon Event ID 1 (Process Create), Sysmon Event ID 3 (Network Connect), and Sysmon Event ID 11 (FileCreate), plus core Windows security events like Event ID 4624 (An account was successfully logged on) and 4625 (An account failed to log on), against this exact telemetry shape. Parts 22–33 of that same book cover building correlation logic, baselining, and threat-intel enrichment on top of it — this part produces the raw material those parts assume already exists.
- **SOC Playbook Handbook's Playbook Library** owns what to actually do once an alert fires from this telemetry — this part builds the pipe, not the response procedure at the other end of it.
- **SOC Manager's Operating Handbook, Part 8** (assessment design) and **Part 11** (onboarding/ramp-up) use a working endpoint-telemetry pipeline like this one as the substrate for a structured, repeatable practice drill for a team, rather than a solo exercise.
- **Within this book**, Part 18's Atomic Red Team-style attack simulation and Part 19's purple-team drills both assume an endpoint that already forwards clean Sysmon telemetry — this part is a direct prerequisite for generating known-cause events to validate detections against, not a parallel, unrelated exercise.

12. Build checklist

[SETUP] The table below recaps this part's steps as a pre-flight checklist — run through it before calling the build done, especially before handing this host off to an exercise in Part 18 or 19.

Item	Why it matters	Verify by
VM attached only to the lab/victim VLAN (Part 4)	A bridged or mis-tagged NIC defeats every isolation control built so far	Hypervisor's NIC/vSwitch setting, not just the VM's reported IP
No domain join, local admin account only	Domain-authentication telemetry is Part 11's scope, not this part's	<code>systeminfo</code> reports <code>Workgroup</code> , not a domain name
Sysmon installed with a real config, not the default	A default, unconfigured Sysmon logs almost nothing filterable	<code>Get-Service sysmon64</code> shows <code>Running</code> ; <code>Sysmon64.exe -c</code> prints the intended config
Time synced to an in-lab or explicitly allowed source	Clock skew beyond a few minutes causes the SIEM to misorder or drop events	<code>w32tm /query /status</code> shows a small, recent offset
Forwarding agent installed and enrolled with the Part 8 SIEM	Local-only logging teaches you nothing at the SIEM layer	New events visible in SIEM search within seconds of generating one
Egress restricted to DNS plus one allowed update path	An unrestricted Windows VM is the isolation failure this book exists to prevent	Part 6's firewall rule table shows explicit allow rules only, default-deny below them
Clean-install snapshot taken before any config change	A broken config is a snapshot revert, not a reinstall	Snapshot exists, dated before the first Sysmon/agent change

Cross-references: Part 4 (network isolation this part's Safety Gate depends on), Part 6 (firewall rules gating SIEM ingest and update egress), Part 7 (internal DNS/time this endpoint uses), Part 8 (the SIEM this telemetry forwards to), Part 9 (the Linux-endpoint parallel build), Part 11 (the Windows domain-lab honesty chapter this part's scope defers to), Part 18–19 (attack-simulation and purple-team drills that consume this telemetry), Appendix A3 (Sysmon config-as-code baseline), Appendix A4 (safety pre-flight checklist referenced by Parts 15/18/19); Detection Engineering Handbook V2 Parts 8–9 and 22–33; SOC Playbook Handbook's Playbook Library; SOC Manager's Operating Handbook Parts 8 and 11.

Part 11 — The Windows Domain Lab Problem: Scope, Cost, and an Honest Failure

Why this part exists

Part 10 built one or more standalone Windows hosts — Sysmon deployed, Windows Event Log forwarding into the SIEM, no domain anywhere in sight. That scope decision wasn't an oversight. A full Active Directory forest is the natural next step a reader reaches for after a standalone host works, and this part exists to tell you honestly what that next step actually costs before you spend a weekend finding out yourself.

The author tried it. It didn't finish. This part is that build's autopsy, not a walkthrough dressed up to look like one. `BOOK-INDEX.md` and the author's own `LAB-EVIDENCE-MANIFEST.md` are explicit that the real, running home lab is Linux-and-network only — no domain controller, no GPO, no Kerberos telemetry captured anywhere in it. Every Windows/AD claim below is either `CONCEPTUAL` (a diagram of what was planned) or `OFFICIAL REFERENCE` (Microsoft's own published requirements and licensing terms) — never `REAL LAB EXAMPLE` or `CONTROLLED LAB EXAMPLE`, because no domain build here was ever run to completion. Part 9's auditd deployment gets to cite real captured evidence from CT100 and CT104; this part doesn't get that luxury, and it says so instead of faking it.

What you get instead: a specific breakdown of why a domain lab is a materially different undertaking than a standalone host, a Build Autopsy of the attempt that stalled, and a realistic scoped-down path that doesn't require you to become an unpaid part-time AD administrator just to generate Kerberos telemetry once.

Safety Gate Everything in this part's scoped-down alternative (§8) assumes the same isolated lab segment built in Part 4, verified against the Appendix A4 pre-flight checklist. A domain controller opens listening surface a standalone host doesn't have — LDAP on ports 389 and 636, Kerberos on port 88, SMB on port 445, and DNS on port 53 if it's also the domain's DNS server — and every one of those must be reachable only from inside the lab segment, never from your home network or the open internet. If you build even a short-lived eval-image domain controller, run the Appendix A4 checklist before you join the first endpoint to it, not just before its first boot. A domain controller with default GPO password policy and no LAPS is a realistic Kerberoasting target on purpose (Part 18 uses it that way) — which means an exposed one is a realistic Kerberoasting target for someone who isn't you.

1. What "a Windows domain lab" actually requires

[CONCEPT] A standalone Windows host, as built in Part 10, is one VM: install the OS, install Sysmon, point its Windows Event Forwarding or a Wazuh/Elastic agent at the SIEM, done. A domain adds an entirely different category of moving parts on top of that, not just more of the same:

- **A domain controller (DC)** running Active Directory Domain Services (AD DS), which is also usually your domain's DNS server — a second service with its own failure modes layered onto the first.
- **A second DC**, if you want the forest to survive one box rebooting for patching without the whole domain going dark — which is the realistic baseline for anything you'd call "resilient," not a nice-to-have.
- **Organizational Units (OUs) and Group Policy Objects (GPOs)** to push configuration — including the Sysmon and audit-policy baseline this book cares about — to every domain-joined endpoint centrally, instead of touching each host by hand.
- **Domain-joined endpoints**, which now authenticate via Kerberos instead of local logon, producing an entirely different telemetry shape (ticket requests, not just local logon events) that Detection Engineering Handbook V2's Part 13 is the one that teaches you to read.
- **A licensing/evaluation-image lifecycle** for every Windows Server instance involved, since a hobbyist lab isn't buying retail Windows Server licenses for boxes that exist to get torn down and rebuilt.

A single forest also carries its own internal versioning concept — the forest and domain functional level, which gates which AD DS features are available and which older domain-controller OS versions could theoretically still join. A single-DC lab never has to think about functional-level mismatches, because there's nothing else in the forest to disagree with; a two-DC forest built from two different eval ISO builds can, in principle, hit a mismatch the moment the second DC tries to join, which is one more pre-flight check a standalone host never asks of you.

None of these five items exists in isolation — DC promotion assumes working DNS, GPO rollout assumes DC replication is healthy, and the eval-license clock is running on every server in the forest independently. That's the shape of the problem: not one hard step, but five moving parts whose failure modes compound.

2. Build Autopsy: what was attempted and how it stalled

[CONCEPT] The plan going in looked reasonable on paper. Here's what was actually tried, and where it stopped.

Build Autopsy — the abandoned Windows/AD forensics lab

The plan: Stand up a persistent two-DC Active Directory forest with several domain-joined Windows endpoints, generating realistic AD authentication and Kerberos telemetry for detection practice.

Why it seemed reasonable: A domain lab is the natural next step after standalone Windows hosts, and eval-licensed Server ISOs make the software cost zero. The author's other lab components — a Proxmox hypervisor, a honeynet, several monitored Linux containers — were all running comfortably on the same hardware tier, so a domain forest looked like an incremental addition rather than a new category of work.

How it failed: DC promotion, GPO-driven Sysmon/audit-policy rollout, and keeping two eval licenses alive on a rotating basis turned out to demand a level of ongoing maintenance the author's other running lab components didn't need. GPO changes took a replication cycle to show up or silently didn't apply because of OU-linking or WMI-filter mistakes that gave no error message anywhere — just an endpoint that quietly kept its old audit policy. Diagnosing that ate hours that a Linux `auditd` rule edit (Part 9) never demanded, because that config takes effect the moment the file is reloaded and says so in the log if it doesn't. The build stalled at a half-configured single DC — DNS integration working, GPO rollout to endpoints not — and was never finished.

The fix: No fix was completed. This book teaches standalone, non-domain-joined Windows hosts (Part 10) and documents the domain build's real cost honestly here instead of presenting a walkthrough of a lab that doesn't exist.

Lab Note If you're tempted to try this yourself: snapshot the VM immediately after DC promotion succeeds and before you touch a single GPO. GPO troubleshooting is where this build burns hours, and a bad GPO link or WMI filter can leave a domain in a state that's faster to revert from a snapshot than to diagnose by reading replication logs at 11pm.

3. The five-way cost stack: domain lab vs. standalone hosts

[COST/RESOURCE] The table below compares what Part 10's standalone-host build actually costs against what each additional domain component adds, so the decision in §8 has real numbers behind it instead of a vague "it's harder."

Cost dimension	Standalone host (Part 10)	Domain lab addition
Setup time, first pass	1–2 hours (install OS, deploy Sysmon config)	4–8 hours minimum for DC promotion, DNS integration, and a first GPO that actually applies
Ongoing maintenance	Near zero — patch occasionally, config doesn't drift on its own	Recurring — eval rearm cycles, GPO drift diagnosis, replication health checks on every reboot
Licensing lifecycle	None (client eval or existing license, no rearm clock)	Windows Server eval ISO's 180-day clock per DC, renewable a limited number of times (\$4)
Failure visibility	High — Sysmon either loads its config or logs why it didn't	Low — a GPO that fails to apply typically produces no error, just silently stale policy
Resource floor	2–4GB RAM, 1–2 vCPU per host	Adds 2GB RAM minimum per DC, plus DNS service overhead, before any endpoint joins

[CONCEPT] The row that matters most for a single-operator hobbyist lab is failure visibility, not raw resource cost. An `auditd` rule that's wrong tells you immediately — it either loads or `auditctl -l` shows nothing changed. A GPO that's wrong tells you nothing; the endpoint just keeps behaving like the policy was never pushed, and you find out only by manually checking the setting on the endpoint itself. That asymmetry is most of why the Build Autopsy in §2 burned the hours it did.

4. Licensing and the evaluation-image lifecycle

[CONCEPT] Windows Server evaluation ISOs are free to download directly from Microsoft and legitimately usable in a lab, but they carry a 180-day evaluation timer per installed instance. Microsoft's own documentation ([OFFICIAL REFERENCE](#) — see [REFERENCES.md](#) entry for the Windows Server evaluation licensing terms) states the timer can be extended with `slmgr /rearm`, up to a small fixed number of times, after which the installation either shuts down periodically or requires conversion to a licensed retail/volume edition to keep running.

[TROUBLESHOOTING] The specific trap: `slmgr /rearm` resets the evaluation clock, but it does not reset a domain controller's own internal state — AD DS doesn't care about the Windows license timer, but a DC that shuts itself into a notification/reduced-functionality state mid-cycle because a rearm was missed can leave the rest of the forest unable to authenticate against it until it's manually recovered. On a two-DC forest, that means tracking two independent rearm clocks on a recurring calendar reminder, not a one-time setup step — exactly the "rotating basis" maintenance burden named in the Build Autopsy above.

Engineering Reality Microsoft's evaluation-license documentation describes the rearm process as a simple maintenance command; in practice, a rearm you forget to run before the 180-day window closes on a domain controller (as opposed to a standalone eval host) doesn't just stop that one machine — it can put every domain-joined endpoint's authentication into a degraded state until the DC is recovered, since Kerberos ticket issuance depends on the DC being fully functional. A standalone Windows host missing its rearm window just stops working on its own; a DC missing its rearm window takes dependents down with it.

[TROUBLESHOOTING] The obvious workaround — snapshot the DC before the eval clock runs out, then roll back to "buy more time" — is specifically the wrong move on a domain controller, and Microsoft's own AD DS documentation warns against it directly ([OFFICIAL REFERENCE](#)). Restoring a DC from a snapshot taken before other domain controllers or objects changed can trigger a USN rollback: the restored DC's replication counter no longer matches what its replication partners believe already happened, and AD DS defends against the resulting inconsistency by disabling that DC's outbound replication rather than silently corrupting the directory. On a lab forest, the practical symptom is a DC that looks like it booted fine but stops replicating anything, with the fix (reformat and re-promote, or use a hypervisor's AD DS-aware snapshot integration if it has one) costing more time than the rearm you were trying to avoid. Snapshot endpoints freely; treat DC snapshots as a one-way "before I do something destructive" safety net, not a routine undo button.

5. GPO-driven rollout: why central policy multiplies maintenance

[CONCEPT] Part 10 deploys Sysmon by copying a config file to each host and running one installer command per machine — tedious at scale, but each step either visibly succeeds or visibly fails on the spot. A domain's Group Policy model exists specifically to avoid that per-host repetition: link a GPO with the Sysmon config and audit-policy settings to the OU containing your endpoints, and every joined machine picks it up on its next policy refresh cycle without you touching it directly.

The problem is exactly the failure-visibility gap already named in §3. GPO application depends on a chain of conditions — the GPO must be linked to the right OU, not blocked by an inheritance-blocking setting further down the tree, not excluded by a WMI filter or security filtering on the GPO itself, and the endpoint must actually complete a policy refresh cycle (`gpupdate /force` if you don't want to wait for the default interval). Get any one of those wrong and the observable symptom is identical to a correctly-applied GPO that just hasn't propagated yet: nothing happens, no error, no log entry pointing at the actual cause. Diagnosing which of those four or five possible causes is the real one is where the Build Autopsy's hours went, and it's a genuinely different skill than debugging a local config file.

Blind Spot A standalone Windows host (Part 10), however well-configured, cannot produce domain-authentication telemetry — no Kerberos ticket requests (Event IDs 4768 and 4769), no cross-host lateral-movement signal that depends on shared domain credentials, no GPO-driven policy-compliance drift to detect. If your goal is specifically to see and interpret that class of telemetry, no amount of additional standalone-host tuning gets you there; you need at least the scoped-down domain exercise in §8, because the gap is architectural, not a configuration depth problem.

6. Resource Reality: the floor for a persistent multi-DC forest

[COST/RESOURCE]

Resource Reality Microsoft's published minimum for a Windows Server installation is 2GB of RAM and 2 vCPUs per instance ([OFFICIAL REFERENCE](#), Windows Server system requirements) — treat that as a floor for "boots," not "usable." A two-DC forest alone commits 4–6GB of RAM and 4 vCPUs before a single endpoint joins, once you budget realistic headroom above the bare minimum for AD DS and DNS running together on each box. Add two domain-joined Windows 10/11 endpoints at Part 3's own 2–4GB-each tier and the persistent forest's baseline footprint lands at 10–14GB of RAM and 8+ vCPUs sitting idle, before this book's SIEM (Part 8, 8GB alone) or NSM sensors (Parts 13–14) get a share of the same host. On the mid-tier hardware budget from Part 3, that's the point where a domain forest stops being "one more VM" and starts being the majority of the box's committed memory — for a component whose telemetry payoff, measured in distinct detection content it unlocks over the standalone-host baseline, is narrower than its resource share suggests.

7. Topology: what was planned versus where the build actually stopped

[CONCEPT] The diagram below is a planning sketch, not a captured topology — no part of the two-DC forest in this diagram was ever fully built. It exists to show precisely where the Build Autopsy's build stopped relative to what a finished forest would have looked like, so the gap is visible rather than described only in prose.

flowchart TD

```
A["Deploy Windows Server eval ISO – DC1"] --> B["Promote DC1 to first domain controller"]
B --> C["Configure DC1 as domain DNS; verify SRV records"]
C --> D["Create OUs; link first Sysmon/audit-policy GPO"]
D -. -> |"build stalled here – GPO would not reliably apply"| X["Build abandoned: half-configured single DC"]
D --> E["Join Windows endpoints to domain"]
E --> F["Confirm GPO-pushed Sysmon config on each endpoint"]
F --> G["Deploy DC2; verify AD replication"]
G --> H["Forward Kerberos/AD auth telemetry to SIEM"]

classDef reached fill:#2b3a55,stroke:#7fa8d9,color:#fff
classDef unreached fill:#3a3a3a,stroke:#888,color:#bbb,stroke-dasharray: 4 3
classDef stopped fill:#611,stroke:#c33,color:#fff,stroke-width:2px
class A,B,C,D reached
class E,F,G,H unreached
class X stopped
```

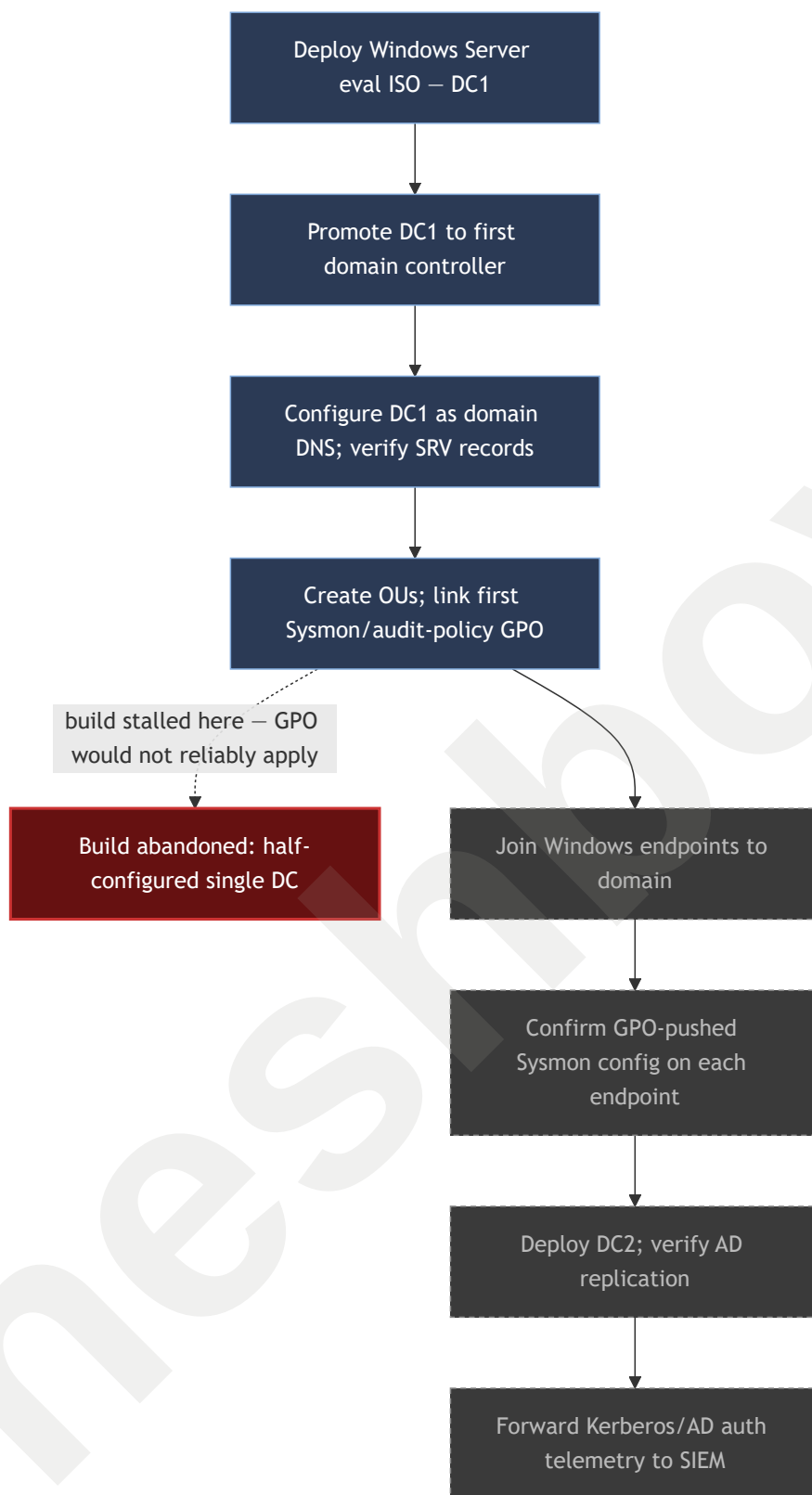


Figure 11.1 — Planned two-DC domain build versus where it actually stopped. *CONCEPTUAL.*

Illustrates the sequence a persistent multi-DC forest build requires (DC promotion through Kerberos telemetry reaching the SIEM) against the point the author's own attempt actually reached before stalling. Solid-filled nodes were completed; dashed nodes (endpoint join, GPO confirmation, second DC, telemetry forwarding) were never reached. No captured screenshot accompanies this diagram — see

§9.1 of [STYLE-GUIDE.md](#): an AD-domain screenshot for this part is a documented non-goal, not a pending capture.

8. The realistic scoped-down alternative

[CONCEPT] A persistent multi-DC forest is not the only way to get domain-authentication telemetry, and for a single-operator hobbyist lab it's usually the wrong default given §3's maintenance asymmetry. Two narrower paths get most of the teaching value at a fraction of the ongoing cost.

[SETUP] **Option A — stay with standalone hosts (Part 10's default).** If your goal is Sysmon and Windows Event Log telemetry, process-creation chains, and endpoint-side detection content, a domain buys you nothing extra for that specific goal. Part 10 already covers it, and it's the right stopping point for most readers.

[SETUP] **Option B — a short-lived, single-exercise eval-image domain.** The maintenance burden in §3 and §4 only accrues over time a domain lab stays running; a domain that exists for a weekend and gets torn down afterward never hits its first rearm deadline and never accumulates GPO drift, because there's no "later" for either problem to show up in. The exercise itself is four steps:

1. Build exactly one DC and one or two domain-joined endpoints from a fresh eval ISO.
2. Snapshot the whole set once DC promotion and the domain join both succeed.
3. Run one bounded exercise against it — a GPO-driven Sysmon rollout as a learning exercise in its own right, or a Kerberoasting simulation paired with Part 18's attack-simulation content — and capture the telemetry you wanted.
4. Either revert to the snapshot for the next exercise or delete the VMs outright.

[SAFETY] Whichever option you pick, a domain controller's additional listening surface (LDAP, Kerberos, SMB, DNS — restated from this part's Safety Gate) must stay inside the Part 4 isolated segment for the entire time it exists, including the short-lived Option B build — "it's only up for a weekend" is not an isolation control.

The table below compares all three paths side by side for the decision this section is actually asking you to make.

Path	Setup effort	Ongoing maintenance	Licensing risk	Telemetry realism	Recommended for
Standalone hosts only (Part 10)	Low — 1–2 hours per host	Near zero	None	No domain-auth telemetry at all	Most readers; sufficient for Sysmon/endpoint detection content
Short-lived eval-image domain (Option B)	Moderate — 4–8 hours per exercise cycle	Low — torn down before rearm/GPO-drift windows matter	Low — never lives long enough to hit the rearm clock	Full Kerberos/AD-auth telemetry for the exercise's duration	Readers who specifically need Kerberos/GPO telemetry for a bounded drill
Persistent multi-DC forest	High — see §2's Build Autopsy	High — recurring rearm cycles, GPO drift diagnosis	Moderate — two independent rearm clocks to track indefinitely	Full, continuous domain-auth telemetry	Readers with a specific ongoing need that justifies the maintenance cost — not the default

What Would Change My Mind This part recommends against a persistent multi-DC AD forest for a single-operator hobbyist lab on the grounds of ongoing maintenance cost. If a reader reports running one comfortably on commodity hardware for six-plus months with under an hour a month of upkeep, that specific claim — not the general "AD labs are hard" framing — would need revision, and this part would need a documented update rather than silent removal of the caveat.

9. What a finished forest would have unlocked

[CONCEPT] It's worth being precise about what the abandoned build in §2 would have bought over Option B's short-lived domain, since "more realistic telemetry" is otherwise a vague claim. A persistent forest's main advantage is duration: some domain-attack techniques are more convincing, or only observable at all, against a domain that has accumulated real history — service accounts with stale Kerberos tickets, group memberships that drifted over months, replication metadata old enough to look organic rather than freshly seeded. T1558 (Steal or Forge Kerberos Tickets), the technique Option B's Kerberoasting exercise (paired with Part 18) targets, works identically well against a one-weekend domain, because it only needs a service account with a Kerberos service principal name — it doesn't care how old the forest is. T1003.006 (OS Credential Dumping: DCSync), by contrast, is more convincing to practice against a forest with real replication history and more than one DC actually exchanging updates, which Option B's single-DC build never has and the abandoned two-DC forest was specifically trying to reach.

That's the honest trade: a persistent multi-DC forest buys you a small number of techniques that specifically depend on multi-DC replication or long-lived directory state, at the ongoing maintenance cost documented in §3 and §4. For the much larger set of domain-authentication and Kerberos-ticket telemetry that Detection Engineering Handbook V2's Part 13 teaches you to read, Option B's short-lived build produces functionally identical log entries.

10. Where this telemetry goes: handing off to the other three books

[CONCEPT] Whichever path from §8 you take, once domain-authentication telemetry exists in your SIEM, this book's job is done — reading it, hunting in it, and responding to it belongs to the other three NESHBOY volumes, not here:

- **Detection Engineering Handbook V2, Part 13** owns interpreting the telemetry itself — what a Kerberos ticket-granting-service request (Event ID 4769) with an unusual encryption type means, what a DCSync-pattern replication request looks like in the logs, and the detection logic built on top of it. Part 11 is barred from re-explaining that content (per [BOOK-INDEX.md](#) provenance item 8) precisely so it has one owner instead of two slightly different versions.
- **Detection Engineering Handbook V2, Parts 8–9** cover the Windows Event ID and Sysmon interpretation fundamentals that apply whether or not the host is domain-joined — read those first if Part 10 was your only stop.

- **SOC Playbook Handbook's Playbook Library** owns what to actually do when a domain-authentication alert fires — triage steps for a suspicious ticket-request pattern or an unexpected privileged-group change, once Detection Engineering Handbook V2's logic has flagged it.
- **SOC Manager's Operating Handbook, Part 8 (assessment design)** is where a short-lived Option B exercise (§8) becomes a repeatable team drill rather than a one-off personal experiment — the same bounded-domain-plus-teardown pattern scales to "run this exercise with two analysts and compare their Kerberoasting-detection triage time."
- **SOC Manager's Operating Handbook, Part 20 (Building & Defending the SOC Budget)** is worth reading alongside §3's cost table if you're making the case, inside a real organization, for why a lab environment's maintenance cost is a legitimate budget line and not just "spin up another VM" — the asymmetry this part documents at hobbyist scale is the same asymmetry that shows up in enterprise AD-lab budgeting, just with more zeros.

Part 11 itself produces no telemetry of its own — it's the honesty layer sitting between "here's what a standalone host teaches you" (Part 10) and "here's what a domain adds and what it actually costs to get there" (this part), so that whichever path a reader takes out of §8, they took it with real numbers instead of a vendor quick-start's implied "it's basically the same amount of work."

Cross-references: Part 3 (hardware/resource tiers cited in §6), Part 4 and Appendix A4 (isolation controls required by this part's Safety Gate), Part 9 (contrast case — real, build-tested Linux telemetry with no domain-equivalent maintenance burden), Part 10 (the standalone-host baseline this part explains why it stopped short of extending), Part 18 (attack-simulation pairing for the short-lived eval-domain exercise in §8), [deh:part8](#), [deh:part9](#), [deh:part13](#) (Windows/Sysmon and Kerberos/AD detection-logic content this part explicitly defers), [soc-playbook:playbook-library](#) (triage procedures for domain-authentication alerts), [manager:part08](#), [manager:part20](#) (drill design and budget framing for a domain exercise run inside a real organization).

Section D — Network Security Monitoring

neshbooy

Part 12 — Packet Capture and Traffic Visibility

Fundamentals

Why this part exists

Everything in Section D of this book — Zeek in Part 13, Suricata in Part 14 — assumes packets are already arriving at a place those tools can read them. That assumption is the part most home-lab builders skip past, and it's the reason a first Zeek or Suricata install so often "runs" and produces nothing: the tool isn't broken, it's watching an interface that never sees the traffic in question. A virtual switch, like a physical switched network, doesn't broadcast every frame to every port the way an old Ethernet hub did. If you want a sensor to see traffic between two other VMs, you have to deliberately build a path for a copy of that traffic to reach it. That's the entire subject of this part: mirror and tap mechanics, promiscuous-mode configuration per hypervisor, and where inside the segmentation model from Part 4 a sensor actually belongs.

This part is deliberately conceptual and reference-based rather than a screenshot-driven walkthrough of the author's own running lab. The author's real, running Proxmox environment — cited as REAL LAB EXAMPLE evidence throughout this book and Detection Engineering Handbook V2 — produces host-based telemetry (auditd records, DNS query logs, honeynet session logs) today, not a standing packet-capture sensor. The mechanics below are accurate, documented behavior for each hypervisor named, not a claim that this exact build has been run end to end and captured as evidence. Part 13 and Part 14 are where that gap should close, once a reader (or a future edition) builds the sensor this part describes and runs Zeek or Suricata against it.

Safety Gate *Everything in this part assumes the segmentation built in Part 4 is already verified end to end via the Appendix A4 pre-flight checklist before you touch any mirror or promiscuous-mode setting. Two failure modes are specific to packet capture and worth naming outright, not folded into a general caution. First, a mirror source scoped too broadly — mirroring a trunk or uplink port instead of only the lab VLAN(s) defined in Part 4 — pulls real home-network packets into a lab capture file that later gets forwarded to a lab SIEM, screenshotted, or shared; confirm exactly which port group or bridge a mirror session reads from before you start it, not after you notice unfamiliar addresses in the output. Second, a sensor VM built with a management vNIC and a promiscuous mirror-destination vNIC becomes an unintended bridge between segments Part 4 built to be separate if IP forwarding is ever enabled on it. The mirror-destination adapter gets no default route, no forwarding capability, and no reason to ever originate traffic — it only listens.*

1. Why a virtual switch doesn't just show you the traffic

[CONCEPT] A hub-based Ethernet segment sent every frame to every port, which is exactly why early intrusion-detection systems could plug into any port and see everything. A switch — physical or virtual — deliberately doesn't do that. It learns which MAC address lives behind which port and forwards

unicast traffic only to that port, both to reduce collision domains and, as a side effect, to make casual eavesdropping harder. Every hypervisor's virtual switch (Proxmox's Linux bridge, VMware's vSwitch, VirtualBox's internal/host-only networking, Hyper-V's virtual switch) behaves the same way by default: a VM's vNIC receives only the frames actually addressed to it, plus broadcast and multicast traffic.

That default is correct and desirable for a production network — it's also exactly why installing Zeek on some VM you happen to have spare capacity on and pointing it at that VM's own vNIC produces almost nothing interesting. The VM sees its own traffic. It doesn't see the conversation between your Linux endpoint from Part 9 and your Windows endpoint from Part 10 unless one of them is a party to it. Seeing that conversation requires either putting the sensor in the actual path the traffic already travels (inline) or arranging for a copy of it to be delivered somewhere else (tap or mirror). Section 2 covers which of those two approaches fits a home lab, and Section 3 covers the specific setting, per hypervisor, that makes it happen.

2. Capture topologies: tap, mirror, and inline placement

[CONCEPT] Three architectural patterns cover essentially every NSM sensor placement, in a home lab or otherwise.

- **Inline** places the sensor directly in the traffic's path — it's a mandatory hop, not a bystander. A firewall running Suricata in IPS mode is inline: traffic must pass through it to reach its destination at all. Inline gives you the option to block, not just observe, at the cost that a crashed or overloaded sensor can take the network path down with it.
- **Tap** is a physical or virtual device that passively copies traffic without being a hop in the path — traditionally a hardware device sitting between two other devices on a cable, splitting the signal. Virtual environments rarely have a true tap device; what they have instead is —
- **Mirror (SPAN)** — a switch feature that copies frames seen on one or more source ports to a separate destination port, without that destination port being part of the traffic's actual path. This is the pattern this part builds toward for a home lab: lower operational risk than inline (a sensor failure doesn't take down connectivity), and directly supported, in some form, by every hypervisor covered in Section 3.

The distinction that matters for a lab builder: inline sensors sit *in* the segmentation Part 4 already built (most naturally, on the firewall/router VM from Part 6, since traffic between segments already passes through it). Mirror-fed sensors sit *beside* it, on a dedicated receive-only path. Figure 12.1 sketches the mirror pattern this part uses for the rest of the book.

```

flowchart LR
    subgraph MGMT["Management VLAN (Part 4)"]
        HV["Hypervisor mgmt interface"]
    end
    subgraph EP["Endpoint VLAN (Parts 9-10)"]
        LX["Linux endpoint VM"]
        WIN["Windows endpoint VM"]
    end
    subgraph HNSEG["Honeynet VLAN (Part 15)"]
        DECOY["Decoy service VM"]
    end
    RTR["Firewall / router VM (Part 6)"]
    subgraph NSMSEG["NSM collection segment – receive-only"]
        SENSOR["Sensor VM: tcpdump now, Zeek/Suricata in Parts 13-14"]
    end

    LX --- RTR
    WIN --- RTR
    DECOY --- RTR
    HV -. mgmt traffic only, never mirrored .-> RTR
    RTR -- mirrored copy of inter-segment traffic --> SENSOR

```

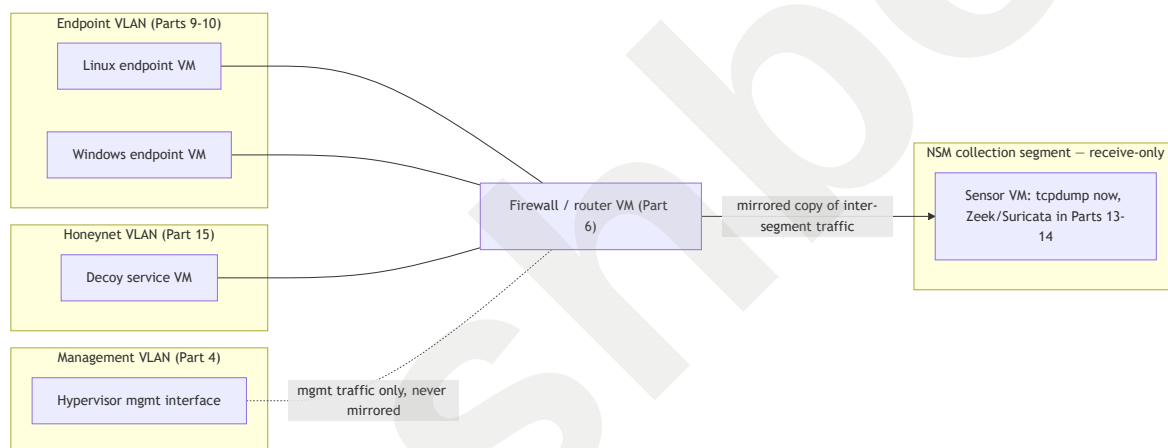


Figure 12.1 — Mirror-fed sensor placement inside the Part 4 segmentation model. *CONCEPTUAL.*

Illustrates where a mirror-fed sensor sits relative to the management, endpoint, and honeynet segments from Part 4: traffic between segments already passes through the Part 6 firewall/router VM, which is the natural mirror source, and the sensor receives a one-way copy on a dedicated collection segment with no route back into any other segment. This is an architecture sketch, not a capture from a running build. Diagram ID [FIG-12-01](#).

The table below compares the three patterns directly, since "which one do I build" is the decision this section supports.

Pattern	Sees traffic that...	Can block traffic?	Failure mode if sensor dies	Typical home-lab fit
Inline	Physically transits the sensor	Yes	Path down until sensor recovers or is bypassed	Suricata in IPS mode on the Part 6 firewall VM itself
Tap (hardware)	Is split off a physical cable	No	None — passive splitter, no dependency	Rare in an all-virtual lab; relevant if bridging to a physical switch
Mirror / SPAN (virtual)	Is copied by the vSwitch/bridge to a destination port	No	None — mirror destination going down doesn't affect the mirrored traffic's real path	Default choice for this book; built in Section 3

3. Promiscuous mode and mirror configuration per hypervisor

[SETUP] Each hypervisor exposes mirroring or promiscuous-mode capture through a different mechanism, and none of them use identical terms for the same idea. The table below is the quick-reference; the subsections that follow give the actual steps for the hypervisor this book's Part 5 builds (Proxmox) and enough detail on the other three to translate the same pattern if you're on one of them instead.

Hypervisor	Mechanism	Where it's configured	Preserves VLAN tags in the capture?	Common gotcha
Proxmox VE (Linux bridge)	<code>tcpdump</code> bound directly to the bridge device, or a <code>tc</code> mirrored/ingress redirect	Hypervisor host shell (root)	Yes, if captured above the tagging point	Requires host shell access — there is no GUI toggle for this in the Proxmox web UI
Proxmox VE (Open vSwitch)	Native OVS mirror port (<code>ovs-vsctl</code>)	Hypervisor host shell (root)	Yes	Only available if the bridge was built as an OVS bridge, not the default Linux bridge
VMware ESXi / vSphere	Distributed-switch mirror session, or port-group security policy set to Promiscuous Mode: Accept	vSphere Client (dvSwitch) or per-port-group security policy (standard switch)	Yes for dvSwitch mirror sessions; standard-switch promiscuous mode only sees that port group	Promiscuous mode must be accepted at both the vSwitch and the port-group level — setting only one does nothing
VMware Workstation	Adapter set to "Bridged" isn't relevant here; promiscuous support depends on host OS driver, inconsistent	Virtual network editor, per-adapter	Inconsistent	Workstation's promiscuous support is the weakest of the four — treat Workstation as a poor fit for this part's exercises and prefer ESXi or Proxmox if NSM work is the priority
Oracle VirtualBox	Promiscuous Mode Policy per adapter: Deny / Allow VMs / Allow All	VM Settings → Network → Adapter → Advanced	Yes, for internal/host-only networks	Default policy is Deny — a sensor's adapter must be set to at least Allow VMs before it sees sibling-VM traffic
Hyper-V	Explicit port mirroring (source/destination), not a promiscuous checkbox	PowerShell (<code>Set-VMNetworkAdapter</code>) or Hyper-V Manager advanced features	Yes	Hyper-V has no single "promiscuous" switch — you must explicitly mark source adapters and a destination adapter, or nothing mirrors

3.1 Proxmox VE: capturing on the Linux bridge

[SETUP] This targets Proxmox VE 8.x running the default Linux-bridge networking model built in Part 5 — not Open vSwitch, which is covered separately below. A Linux bridge (`vmbr1`, for example, backing an endpoint VLAN) exposes itself as a real interface on the hypervisor host. When a packet-capture tool binds to that bridge device in promiscuous mode, the kernel's bridging code passes it a copy of frames actually forwarded across the bridge — not just broadcast and multicast, but the unicast traffic between the VMs attached to it. This means the simplest sensor on Proxmox isn't a separate VM at all; it's `tcpdump` run directly on the hypervisor host, against the bridge interface:

```
tcpdump -i vmbr1 -w /var/tmp/lab-capture.pcap
```

Confirm this is actually working, before trusting it for anything, by checking that the interface's packet counters are climbing while lab VMs generate traffic: `ip -s link show vmb1` should show an increasing RX/TX count between two checks a few seconds apart.

Running the capture on the host itself is the lowest-friction option, but it means the hypervisor's own management plane has visibility into lab traffic — acceptable for a single-operator home lab, but worth knowing if you'd rather isolate that role. The alternative is a dedicated sensor VM fed by a `tc` mirrored redirect on the bridge, which copies frames to a second interface a VM can read without granting that VM host-level access. This targets Proxmox VE 8.x with the `iproute2` `tc` tooling already present in the default install:

```
tc qdisc add dev vmb1 ingress
tc filter add dev vmb1 parent ffff: protocol all u32 match u32 0 0 \
  action mirrored egress mirror dev <mirror-destination-iface>
```

Confirm it worked using the same interface-counter check on `<mirror-destination-iface>`, or the Validation Test in Section 6.

3.2 Proxmox VE with Open vSwitch: native mirror ports

[SETUP] If the Part 5 build used Open vSwitch bridges instead of the Linux bridge default (a deliberate choice, not the out-of-the-box Proxmox behavior), OVS supports mirror ports natively. This targets Open vSwitch as packaged for Proxmox VE 8.x:

```
ovs-vsctl -- --id=@p get port <mirror-destination-port> \
  -- --id=@m create mirror name=lab-mirror select-all=true output-port=@p \
  -- set bridge vmb1 mirrors=@m
```

Confirm it worked the same way as the Linux-bridge case: rising counters on the mirror destination while lab traffic runs, or the Section 6 Validation Test.

3.3 VMware, VirtualBox, and Hyper-V: the same pattern, different knobs

[SETUP] On VMware ESXi, promiscuous mode is a security-policy setting that must be accepted at both the distributed or standard switch level and the port-group level — checking only one leaves the port group still blocking promiscuous frames. A distributed-switch mirror session (vSphere Client → Networking → Port Mirroring) is the closer analog to Proxmox's `tc`/OVS mirror and is the better choice if the vSphere license tier supports it, since it doesn't require setting an entire port group promiscuous. On VirtualBox, the relevant setting is the adapter's Promiscuous Mode Policy (VM Settings → Network → Adapter → Advanced), which defaults to Deny and must be raised to at least Allow VMs for a sensor VM's adapter to see traffic from sibling VMs on the same internal or host-only network. On Hyper-V, there's no promiscuous checkbox at all — mirroring is explicit, configured per virtual-adapter as source or destination. This targets Hyper-V on Windows Server 2019 or later, or Windows 11 Pro/Enterprise with Hyper-V enabled, run from an elevated PowerShell session on the Hyper-V host:

```
Set-VMNetworkAdapter -VMName "lab-endpoint-01" -PortMirroring Source
Set-VMNetworkAdapter -VMName "lab-sensor" -PortMirroring Destination
```

Confirm it worked by generating traffic to or from `lab-endpoint-01` and checking that it appears in a capture tool running inside `lab-sensor`.

Engineering Reality Vendor documentation for promiscuous mode and port mirroring describes it as seeing "all traffic on the segment." In practice, a virtual switch delivers a copy of traffic to a promiscuous or mirror-destination port only for frames that actually reach that specific port group or bridge. Two VMs on separate port groups, connected only through the Part 6 firewall/router VM, generate traffic that never touches the endpoint-VLAN port group at all until the router has already forwarded it. Put the capture point on the interface actually doing the routing, or on a mirror session fed from there — not on whichever port group happens to be easiest to reach from wherever you already have a spare VM.

4. Sizing the capture/sensor host

[COST/RESOURCE] A capture host's resource profile depends almost entirely on what you keep, not what you see. Reading packets off the wire costs relatively little CPU at home-lab traffic volumes; writing every byte of every packet to disk indefinitely is what actually runs out of room.

Capture mode	CPU	RAM	Disk growth	Realistic retention on a home lab
Short debug capture (<code>tcpdump -w</code> , manual start/stop)	Under 5% of one core at lab traffic volumes	512MB	Fast — full payload, no rotation	Minutes to hours, then delete
Continuous full-packet capture with rotation (<code>-C</code> / <code>-W</code> flags)	Under 10% of one core	1GB	10-20GB per day once a honeynet segment is drawing scanner traffic	A few days at most before disk pressure forces deletion
Metadata/alert output only (Zeek <code>conn.log</code> / <code>ssl.log</code> , Suricata <code>eve.json</code> — Parts 13-14)	1-2 CPU cores recommended once rule matching is active	2-4GB	A small fraction of full-packet capture for the same traffic	Weeks to months, subject to Part 21's retention guidance

Resource Reality A sensor VM doing full-payload `tcpdump -w` capture at even modest home-lab traffic volumes can fill 20GB of disk in under a day once a honeynet segment starts drawing opportunistic scanner traffic — full-packet retention is a short-lived debugging tool in this book, not a standing default. Part 13 and Part 14 exist specifically because Zeek's and Suricata's metadata and alert output cost a small fraction of that disk footprint for the same traffic. Treat raw pcap capture from this part as the thing you turn on to prove the sensor works (Section 6), then turn back off.

5. Sensor placement inside the Part 4 segmentation model

[SAFETY] A mirror-fed sensor should live on its own segment — call it the NSM collection segment — that exists for exactly one purpose: receiving mirrored traffic and nothing else. It should have no route to the management segment, no route to the endpoint segment, and no route to the honeynet segment; the mirror mechanism itself is what delivers traffic to it, not routing. This matters because a sensor is, almost by definition, running software (Zeek, Suricata, or even just `tcpdump` with a permissive filter) that parses untrusted, attacker-influenced input — a honeynet decoy in Part 15 exists specifically to attract that input. A parsing bug in an NSM tool processing a malicious packet is a real, if uncommon, attack surface; a sensor with no route anywhere confines the blast radius of that bug to itself.

The natural mirror source, in the topology this book builds, is the Part 6 firewall/router VM — every inter-segment conversation already passes through it, so mirroring its internal interfaces captures endpoint-to-endpoint, endpoint-to-honeynet, and endpoint-to-internet traffic in one place, rather than needing a separate mirror per segment. This is also why Section 1's point about switches not broadcasting matters operationally: mirroring an endpoint VM's own port group only shows that VM's own traffic, never the honeynet's.

Blind Spot *A packet capture sees frames, not intent. Once a lab endpoint speaks TLS — which is most of what a normal Windows or Linux host does by default — this part's capture point shows connection metadata (addresses, ports, certificate fields, the TLS `SNI` value) but not the decrypted payload. That's enough to feed Part 13's Zeek `ssl.Log` and the connection-level detection content Detection Engineering Handbook V2 builds on top of it; it is not enough to "watch a credential cross the wire in cleartext" as a teaching moment unless an exercise deliberately uses an unencrypted protocol for that specific purpose. Don't design a lab exercise around visibility this capture point structurally cannot provide.*

6. Validating the capture point

[HANDS-ON LAB] Before pointing Zeek or Suricata at a sensor in Part 13 or Part 14, confirm the sensor actually receives traffic at all — a tool with nothing to read produces empty logs that look identical to "nothing happened" and "the sensor is misconfigured," and telling those apart later is far more annoying than checking now.

1. Bring up the Linux endpoint VM from Part 9 and a second lab VM to serve as a traffic target (any lab VM with a listening service, or just something to ping).
2. Start a capture on the sensor, filtered narrowly so the result is unambiguous: `tcpdump -i <mirror-iface> -n icmp`.
3. From the Linux endpoint, run `ping -c 4 <target-VM-IP>`.
4. Watch the sensor's terminal for output while the ping runs, not after.

CONCEPTUAL SAMPLE — illustrative `tcpdump` output, not a captured screenshot

```
14:02:11.442019 IP 10.20.1.11 > 10.20.1.15: ICMP echo request, id 5321, seq 1
14:02:11.442390 IP 10.20.1.15 > 10.20.1.11: ICMP echo reply, id 5321, seq 1
14:02:12.443011 IP 10.20.1.11 > 10.20.1.15: ICMP echo request, id 5321, seq 2
14:02:12.443295 IP 10.20.1.15 > 10.20.1.11: ICMP echo reply, id 5321, seq 2
```

Validation Test Setup: Sensor VM's mirror-destination adapter attached per Section 3, `tcpdump` installed, the Part 6 firewall/router VM's relevant interface mirrored per whichever hypervisor subsection applies. **Action:** `ping -c 4 <target-VM-IP>` from the Part 9 Linux endpoint, while running `tcpdump -i <mirror-iface> -n icmp` on the sensor. **Expected result:** Four ICMP echo request/reply pairs appear in the sensor's `tcpdump` output within the same second each is sent. If nothing appears, the mirror isn't attached to the source you think it is — see Section 8's first troubleshooting entry before assuming the hypervisor setting itself is wrong.

Lab Note If you're not sure a mirror is attached to the right point, generate one very identifiable packet — a ping to an address nothing else in the lab ever contacts — and `grep` for it in the capture. It's a five-second sanity check that beats staring at a wall of unrelated broadcast traffic trying to convince yourself the mirror probably works.

7. From packets to detections: what this feeds across the series

[CONCEPT] This part's only output is a place where traffic can be read; it deliberately stops there. What happens next fans out across the rest of this book and into the other three NESHBOY volumes, and it's worth being explicit about the handoff at each stage so you know which book owns which decision once you have this pipeline running:

- **Part 13 (Zeek) and Part 14 (Suricata)** attach directly to the capture point this part builds — neither tool has anything to analyze without it. Both parts are written to reuse the exact same mirror-fed sensor rather than each building their own.
- **Detection Engineering Handbook V2, Parts 14-15** own writing detection logic against the `conn.log` / `ssl.log` / `dns.log` records and Suricata alerts this pipeline produces — this book teaches you to build the pipe, not how to reason about what's suspicious in a given `conn.log` entry.
- **SOC Playbook Handbook's Playbook Library** owns what an analyst does once one of those detections actually fires as an alert — this book's job ends at "the alert exists and reached a SIEM," not at "here's how to triage it."
- **SOC Manager's Operating Handbook, Part 8 (assessment design)** becomes buildable against real telemetry, not a hypothetical, once this pipeline exists — a manager designing a repeatable practice assessment needs a lab that actually produces network telemetry to assess against, and this part is where that requirement gets satisfied.
- **This book's own Part 19 (purple-team drills)** pairs a specific simulated action against this pipeline with a specific detection and a specific playbook, closing the loop the three bullets

above describe individually.

8. Troubleshooting common capture failures

8.1 Mirror configured, sensor sees nothing

[TROUBLESHOOTING] The most common cause is a mirror source scoped to the wrong bridge, port group, or virtual adapter — not a broken mirror mechanism. Confirm the source is correct before doubting the feature: check `ip -s link show <bridge>` on Proxmox, or the equivalent interface-counter view on the other hypervisors, and verify the counters climb while lab traffic runs. If the source interface itself shows no traffic, the problem is upstream of the mirror entirely — likely the wrong bridge or port group was mirrored, not the mirror setting itself.

8.2 Sensor sees only broadcast/ARP, no unicast

[TROUBLESHOOTING] This is the signature of a promiscuous-mode setting that wasn't actually accepted at every layer that checks it. On ESXi, this means the vSwitch-level and port-group-level security policies disagree — both must explicitly accept promiscuous mode. On VirtualBox, this means the adapter's Promiscuous Mode Policy is still at its Deny default. Re-check the exact setting named in Section 3's table for your hypervisor rather than assuming "promiscuous mode is on somewhere" is sufficient.

8.3 Capture disk fills faster than expected

[TROUBLESHOOTING] This is a sizing miss, not a bug — Section 4's table exists because full-payload capture with no rotation genuinely does consume disk this fast once a honeynet segment starts drawing opportunistic scanner traffic. Add `-C <size>` and `-W <count>` to any standing `tcpdump` capture to rotate files instead of writing one unbounded file, or — the better long-term fix — stop relying on raw pcap retention past the point where Part 13 or Part 14's metadata/alert output is running, per Part 21's storage guidance.

Cross-references: Part 4 (segmentation architecture this part's mirror points depend on); Part 5 (the Proxmox hypervisor build this part's Section 3 configuration targets); Part 6 (the firewall/router VM this part treats as the natural mirror source); Part 9 and Part 10 (the endpoints generating the traffic captured here); Part 13 (deploying Zeek against this capture point); Part 14 (deploying Suricata against the same point); Part 21 (storage growth and retention once continuous capture is running); Appendix A3 (config snippet reference); Appendix A4 (the safety and isolation pre-flight checklist referenced in this part's Safety Gate); Detection Engineering Handbook V2, Parts 14-15 (writing detection logic against the resulting Zeek/Suricata output); SOC Playbook Handbook's Playbook Library (triaging alerts that logic produces); SOC Manager's Operating Handbook, Part 8 (assessment design against this telemetry pipeline).

Part 13 — Deploying Zeek for Network Security Monitoring

Why this part exists

Part 12 gave you a place to see traffic: a mirrored interface, in promiscuous mode, sitting inside the isolated segment from Part 4. Seeing packets and understanding them are different problems. A raw packet capture is unreadable at scale — Zeek's job is to sit on that mirrored interface and turn every connection into a structured, searchable log line before it ever reaches your SIEM. This part installs Zeek against the Part 12 capture point, walks through the log families that matter on day one (`conn.log`, `dns.log`, `http.log`, `ssl.log`, `notice.log`, `weird.log`), and forwards those logs into the Part 8 SIEM. It stops at the log arriving and being searchable. What a specific `conn.log` entry actually means — is this connection a port scan, a beacon, a DNS tunnel — is Detection Engineering Handbook V2's job (its Parts 14–15), not this book's.

Safety Gate *Everything in this part assumes the mirrored interface you're pointing Zeek at is the one built and verified in Part 12, sitting on the isolated lab segment from Part 4 — not a mirror of your home network's real traffic, and not an interface with any route back out to the internet beyond the explicit, allow-listed path to your Part 8 SIEM. Zeek is a passive listener; it never sends a packet to anything it's watching. But the sensor host it runs on is a full Linux box on your lab network, and a full Linux box with no isolation boundary is just one more thing an attacker in that segment can pivot through. Before installing Zeek, confirm the Appendix A4 pre-flight checklist passes for the segment the sensor lives on — specifically that the sensor's only outbound path is to the SIEM's ingest port, nothing else.*

1. What Zeek actually does — and what it deliberately doesn't

[CONCEPT] Zeek is a network security monitor (NSM), not an intrusion detection system (IDS). The distinction matters mechanically, not just as vocabulary: an IDS like Suricata (Part 14) inspects traffic against a rule set and raises an alert when something matches — it screams. Zeek inspects the same traffic and writes a structured record of every connection whether or not anything looks wrong — it narrates. Point Zeek at an hour of ordinary lab traffic and you get thousands of log lines describing DNS lookups, TCP handshakes, and HTTP requests that were all completely benign. That's the intended output. The value shows up later, when you need to answer "what else did this host talk to in the ten minutes before the alert fired" — a question an IDS alone can't answer, because an IDS only logged the thing that matched a rule.

Zeek ships with a small built-in notion of "notable" behavior of its own — the `notice.log` family, covering things like a certificate that fails validation (`SSL::Invalid_Server_Cert`) or a protocol violation Zeek's own parsers flag as unexpected. Treat these as Zeek raising its hand, not as an alert queue: `notice.log` entries are worth watching, but they're a small fraction of what Zeek produces and were never meant to replace a dedicated IDS.

1.1 Where Zeek and Suricata split the work

The table below supports one decision: whether you need Zeek, Suricata, or both on the same capture point, and what each one is actually good for once it's running.

Question	Zeek	Suricata
Alerts on a known-bad signature	No — has no rule-matching engine of its own	Yes — this is its core function
Logs every connection, matched or not	Yes — this is its core function	Only via <code>eve.json</code> <code>flow/</code> <code>stats</code> event types, less rich
Extracts application-layer metadata (<code>http.log</code> URIs, <code>ssl.log</code> SNI/JA3, <code>dns.log</code> queries)	Yes, in depth, per protocol	Partial, via <code>http/</code> <code>tls/</code> <code>dns</code> <code>eve.json</code> event types
Good starting point for "what else did this host do"	Yes — this is exactly what it's for	Weaker — built for matching, not narrating
Needs a maintained rule feed to stay useful	No	Yes (Part 14 covers ET Open and similar)
Typical home-lab pairing	Runs alongside Suricata on the same tap	Runs alongside Zeek on the same tap

Most production NSM deployments run both on the same tap for exactly this reason — Suricata tells you something matched, Zeek tells you everything else that connection did. §7's Build Autopsy covers why this book's own author didn't do that pairing on the one live segment where it would have mattered most.

flowchart LR

```
subgraph LAB["Isolated lab segment (Part 4)"]
    SW["Mirror/SPAN port<br/>(Part 12 capture point)"]
    SENSOR["Zeek sensor host<br/>promiscuous capture NIC"]
    SW -->|"mirrored copy of all segment traffic"| SENSOR
end
SENSOR -->|"conn.log, dns.log, http.log,<br/>ssl.log, notice.log, weird.log"| SHIP["Log shipper<br/>(Filebeat, or Wazuh agent localfile)"]
SHIP -->|"forwarded events, management path only"| SIEM["SIEM ingest<br/>(Part 8)"]
SENSOR -. "blocked — no route out<br/>except to the shipper/SIEM (Part 4/6)" .-> OUT["Home network / internet"]
```

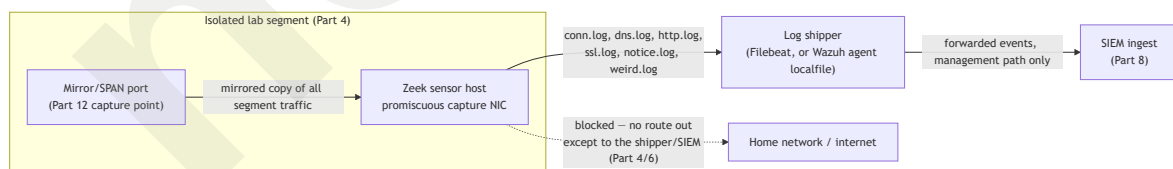


Figure 13.1 — Zeek sensor placement and log flow into the lab SIEM. CONCEPTUAL. Illustrates where a Zeek sensor sits relative to the Part 12 mirror port and the Part 8 SIEM ingest pipeline inside the isolated lab segment from Part 4. This is an architecture sketch, not a capture from a running build — no Zeek instance currently runs in the author's own lab (see §7).

2. Sizing a Zeek sensor for a home lab

[COST/RESOURCE]

Zeek's CPU and RAM cost scales with connection rate and how many analysis scripts you load, not with link speed on its own — a quiet 1Gbps lab segment with a handful of endpoints costs Zeek far less than a busy 100Mbps segment with a chatty honeynet on it. The table below gives three tiers matched to Part 3's hardware tiers, sized for a standalone `zeekctl` deployment (one process doing capture, analysis, and logging — no manager/proxy/worker split, which only starts paying off well above home-lab traffic volumes).

Tier	Matches (Part 3)	Segment traffic	vCPU	RAM	Disk for logs
Light	Repurposed laptop/mini-PC	1–3 quiet endpoints, no honeynet on the same tap	1	1GB	1–2GB/week, plaintext JSON logs
Standard	Dedicated home-server box	4–10 endpoints, or a honeynet segment with moderate scanner traffic	2	2GB	5–10GB/week
Heavy	Multi-node cluster	A busy honeynet segment (see §7) or sustained internal traffic generation (Part 19)	4	4GB	20GB+/week — plan retention per Part 21

Resource Reality Zeek's own manual describes RAM use as "modest" for typical deployments; on this book's home-lab hardware, "modest" still means a standalone instance watching a honeynet-scale connection rate (hundreds of new connections per minute from background internet scanners, not a quiet home segment) will sit at 1.5–2GB resident and spike higher during log rotation. On a 2GB VM with nothing else running, that's fine. On a 2GB VM that's also running a log shipper, a cron-driven enrichment job, or anything else from this book's later parts, expect the OOM killer to take Zeek down first — it's usually the largest single process on the box. Give the sensor its own VM rather than co-locating it with the SIEM or an endpoint.

3. Installing Zeek at the Part 12 capture point

[SETUP]

The steps below target Zeek 6.0/6.1 installed from the official `security:zeek` package repository on Ubuntu Server 22.04 LTS, deployed as a single standalone node with `zeekctl` — the right starting configuration for every tier in §2's table. If you're on a different distribution, the package repo has equivalent instructions for Debian, RHEL/CentOS, and Fedora; the `zeekctl` steps after installation are identical regardless of package source.

3.1 Adding the package repository and installing Zeek

1. Import the repository's signing key and add the repo:

```
curl -fsSL https://download.opensuse.org/repositories/security:/zeek/xUbuntu_22.04/Release.key \
| gpg --dearmor | sudo tee /usr/share/keyrings/zeek-archive-keyring.gpg >/dev/null
echo "deb [signed-by=/usr/share/keyrings/zeek-archive-keyring.gpg] \
https://download.opensuse.org/repositories/security:/zeek/xUbuntu_22.04/ /" \
| sudo tee /etc/apt/sources.list.d/security:zeek.list
```

2. Refresh package metadata and install:

```
sudo apt update
sudo apt install -y zeek
```

3. Zeek installs to `/opt/zeek` rather than the usual `/usr` tree. Add its `bin` directory to your path so `zeekctl` is reachable without a full path every time:

```
echo 'export PATH=/opt/zeek/bin:$PATH' | sudo tee /etc/profile.d/zeek.sh
source /etc/profile.d/zeek.sh
```

Confirm the install worked before touching any configuration: `zeek --version` should print `6.0` or `6.1`. If the command isn't found, the `PATH` export above either didn't run or was applied in a shell you're no longer in — open a fresh terminal and retry before assuming the package install itself failed.

3.2 Pointing Zeek at the mirror interface

Zeek's node configuration lives at `/opt/zeek/etc/node.cfg`. For a standalone deployment, edit the `interface` line to match the exact interface name of the mirrored NIC from Part 12 — never the sensor's management interface, or Zeek will happily monitor its own SSH session instead of the traffic you actually built this for.

```
[zeek]
type=standalone
host=localhost
interface=ens19
```

Run `ip link show` on the sensor first if you're not certain of the mirror interface's name — guessing wrong here is the single most common reason a fresh Zeek install produces empty logs (see §6).

Before deploying, load the JSON logging policy so every log family writes newline-delimited JSON instead of Zeek's default tab-separated format — JSON is what every log shipper in §5 expects, and switching formats later means re-pointing every downstream parser. Add one line to

`/opt/zeek/share/zeek/site/local.zeek:`

```
# JSON output – every downstream shipper in this book (Filebeat, Wazuh's
# localfile reader) expects newline-delimited JSON, not Zeek's default TSV.
@load policy/tuning/json-logs.zeek
```

Deploy the configuration — this both validates the config and starts the standalone node:

```
sudo zeekctl deploy
```

`zeekctl deploy` runs a syntax check before starting anything; if `node.cfg` or `local.zeek` has a typo, it reports the error and leaves the previous state untouched rather than starting a half-broken sensor.

Validation Test Setup: Zeek deployed per §3.1–3.2, mirror interface confirmed via `ip link show` before editing `node.cfg`. **Action:** `sudo zeekctl status`, then generate a small amount of traffic on the monitored segment (any endpoint browsing to an internal or external site) and run `tail -f /opt/zeek/Logs/current/conn.log`. **Expected result:** `zeekctl status` reports the `zeek` node as `running`. Within a few seconds of the test traffic, new JSON lines appear in `conn.log` with populated `id.orig_h`, `id.resp_h`, and `duration` fields — if the file exists but stays empty while you generate traffic, the mirror isn't actually reaching this interface; go back to Part 12's verification steps before troubleshooting Zeek itself.

4. The log families worth knowing on day one

[CONCEPT] Zeek writes dozens of log files once enough protocols are exercised, but a home lab's first month of real use lives almost entirely in six of them. Every one of these logs shares a `uid` field for the connection it describes — the single most useful fact about Zeek's log design, and the reason §9's hands-on exercise is built around tracing one `uid` across multiple log files.

Log file	What it records	A field worth knowing first
<code>conn.log</code>	Every TCP/UDP/ICMP connection, matched or not	<code>conn.log</code> 's <code>duration</code> field — connection length in seconds; long-lived low-traffic connections are the shape a beacon leaves behind
<code>dns.log</code>	Every DNS query and response Zeek observes	<code>dns.log</code> 's <code>query</code> field — the queried name, before any resolution happens
<code>http.log</code>	Every unencrypted HTTP request/response pair	<code>http.log</code> 's <code>uri</code> field — the requested path, paired with <code>method</code>
<code>ssl.log</code>	TLS/SSL handshake metadata (not decrypted payload)	<code>ssl.log</code> 's <code>server_name</code> field — the SNI hostname the client requested, when present
<code>notice.log</code>	Zeek's own built-in "something worth a look" events	The <code>note</code> field holds the <code>Notice::Type</code> value, e.g. <code>`SSL::Invalid_Server_Cert`</code>
<code>weird.log</code>	Protocol behavior Zeek's parsers flag as malformed or unexpected, not necessarily malicious	The <code>name</code> field holds Zeek's internal weird-name string for the anomaly — a plain string, not a formal enum type the way <code>notice.log</code> 's <code>note</code> field is — e.g. <code>`dns_unmatched_reply`</code>

Two details worth knowing before you rely on `ssl.log`: JA3/JA3S TLS client-fingerprint hashes are not populated by default and are not part of the base Zeek install at all — they come from a third-party package (`zeek/salesforce/ja3`), installed with `zkg install zeek/salesforce/ja3` and then loaded in `local.zeek` with `@load Salesforce/JA3`, a different mechanism from §3.2's bundled `policy/tuning/json-logs.zeek`. And `server_name` is only populated when the client sends SNI in the clear; a client using encrypted SNI (increasingly common on TLS 1.3) will leave that field empty, which is a real, specific limit on what this log can tell you — not a misconfiguration to chase.

5. Forwarding Zeek logs into the SIEM

[SETUP] With JSON logging active (§3.2), Zeek writes each log family as its own file under `/opt/zeek/logs/current/`, rotated hourly into dated subdirectories. Getting those files into the Part 8 SIEM is the same log-shipping problem Parts 9 and 10 already solved for auditd and Sysmon — point a shipper at the files, forward the events.

If Part 8's SIEM choice was Wazuh, the shipping path runs through the Wazuh agent's own `<localfile>` log reader rather than a separate Filebeat install on the sensor. Wazuh's `<location>` tag does support a basic `*` wildcard, so `/opt/zeek/logs/current/*.log` will technically match every family — but for a small, fixed set of exactly-named files like these five, one stanza per file is more explicit and easier to reason about than trusting the agent's periodic wildcard rescan to pick up exactly the files you expect, no more and no fewer. Each log family gets its own stanza in `/var/ossec/etc/ossec.conf` on the sensor:

```
<ossec_config>
  <localfile>
    <log_format>json</log_format>
    <location>/opt/zeek/logs/current/conn.log</location>
  </localfile>
  <localfile>
    <log_format>json</log_format>
    <location>/opt/zeek/logs/current/dns.log</location>
  </localfile>
  <localfile>
    <log_format>json</log_format>
    <location>/opt/zeek/logs/current/http.log</location>
  </localfile>
  <localfile>
    <log_format>json</log_format>
    <location>/opt/zeek/logs/current/ssl.log</location>
  </localfile>
  <localfile>
    <log_format>json</log_format>
    <location>/opt/zeek/logs/current/notice.log</location>
  </localfile>
</ossec_config>
```

Restart the agent after editing this file (`sudo systemctl restart wazuh-agent`) — `ossec.conf` is only read at agent start, not watched live. If Part 8's SIEM choice was the Elastic stack or Graylog instead, the equivalent is Filebeat's own Zeek module (`filebeat modules enable zeek`, then set `var.paths` in `/etc/filebeat/modules.d/zeek.yml` to the same `/opt/zeek/logs/current/*.log` glob Filebeat's `paths:` syntax does support) or a Graylog Beats/GELF input pointed at the same files — the underlying files and fields are identical regardless of which shipper reads them.

Validation Test Setup: Zeek forwarding configured per this section, targeting the Part 8 SIEM.

Action: Repeat the traffic generation from §3's Validation Test, then search the SIEM for events from the sensor host within the last few minutes. **Expected result:** Zeek-sourced events appear in the SIEM with the same field names as the raw JSON (`id.orig_h`, `duration`, `query`, `uri`, etc., depending on which log family matched your test traffic) and a timestamp within a few seconds of the local `conn.log` entry. A large forwarding delay (minutes, not seconds) usually means the shipper is buffering on a bulk-send interval rather than failing outright — check the shipper's own logs before assuming Zeek itself is the problem.

6. Troubleshooting the first week

[TROUBLESHOOTING]

- `zeekctl status` shows the node as **crashed immediately after deploy**. Almost always a permissions problem, not a config problem: Zeek needs `CAP_NET_RAW` and `CAP_NET_ADMIN` to open the interface in promiscuous mode, and a standalone package install doesn't grant these automatically on every distribution. Check `journalctl -u zeek` (or `/opt/zeek/logs/current/stderr.log`) for a permission-denied message on the capture interface; if present, run `sudo setcap cap_net_raw,cap_net_admin=eip /opt/zeek/bin/zeek` and redeploy.
- **Logs exist but stay nearly empty despite real traffic on the segment.** This is a Part 12 problem wearing a Zeek costume, not a Zeek problem. Confirm the mirror interface is actually receiving traffic independent of Zeek entirely — `ip -s link show ens19` (substituting your interface) and watch the RX byte counter climb while you generate test traffic. If it doesn't climb, go back to the hypervisor's mirror/SPAN configuration; Zeek can't analyze packets that never arrive at its NIC.
- **Edited `local.zeek` but the change (JSON logs, JA3) never took effect.** Editing the file alone doesn't reload Zeek's running configuration — you have to redeploy. Run `sudo zeekctl check` first to catch a syntax error before it takes down the running node, then `sudo zeekctl deploy` to apply and restart.
- **Sensor's disk fills up faster than expected.** JSON-format logs are larger than Zeek's default tab-separated format (the same information, plus JSON's field-name overhead repeated on every line), and rotated logs are not compressed by default. Budget disk per §2's table, and see Part 21 for a realistic retention policy before you're choosing between deleting logs and running out of space mid-capture.

7. Build Autopsy — pairing Zeek with Suricata on the real honeynet gateway

Build Autopsy — "run Zeek alongside Suricata on the honeynet edge gateway"

The plan: Add a Zeek instance next to the already-running Suricata deployment on the author's real honeynet edge gateway, watching the same mirrored honeynet interface, so an analyst reviewing a Suricata alert could pivot straight into Zeek's connection-level narration of the surrounding session — the standard NSM pairing described in §1.1.

Why it seemed reasonable: The gateway already had a working capture point and a working Suricata deployment (Part 14 covers that build); adding Zeek to the same tap looked like it should be a small marginal cost for a meaningful gain in context around every alert.

How it failed: The gateway shares its physical Proxmox host with the rest of the author's real lab — Pi-hole, a Tor relay, a vulnerability scanner, and the honeynet's own sacrificial decoy VMs all run on the same box, which was already committed at roughly 22 vCPUs against 4 physical cores and running close to its full 15GB of RAM, with swap already in active use. §2's Resource Reality box describes what Zeek costs on hardware with headroom to spare; this host didn't have that headroom, and the honeynet segment specifically sees a higher, spikier connection rate than a quiet home segment because it's a deliberate target for internet background scanning. Adding a second capture process to an already-oversubscribed shared host risked degrading the gateway's existing, working Suricata pipeline rather than adding a clean second signal next to it.

The fix: No fix was completed on that gateway — it's disclosed here as a real, resource-driven decision, not silently omitted. Suricata alone remains that segment's live telemetry source. This book's Zeek walkthrough (§3–§5) targets a purpose-built sensor VM sized per §2's tiers, not a shared production box already carrying other services — a reader following this part on dedicated hardware isn't inheriting the exact resource conflict the author hit on a box that was never sized for a second NSM tool in the first place.

8. Blind Spot: what Zeek cannot see

Blind Spot Zeek analyzes what crosses the wire it's mirrored from, in whatever form it arrives in — it does not decrypt anything. `ssl.Log` gives you handshake metadata (SNI when present, negotiated cipher, certificate details, a JA3 fingerprint if that policy script is loaded) but never the contents of the encrypted session itself, which today is most of a home lab's real traffic. A client using encrypted SNI defeats even the `server_name` field. Traffic tunneled through a VPN that terminates outside the mirrored segment shows up as one encrypted flow to the VPN endpoint, with everything inside it invisible — Zeek narrates the tunnel's existence, not what travels through it. None of this is a configuration mistake to fix; it's the actual limit of what a passive network sensor can ever see, and it's the reason this book still deploys endpoint telemetry (Parts 9–10) rather than treating NSM as a substitute for it.

9. Hands-on lab: tracing one session across three log files

[HANDS-ON LAB]

Goal: confirm you can follow a single connection across `conn.log`, `http.log`, and `ssl.log` using the shared `uid` field from §4, rather than just seeing that log files exist. This exercise generates ordinary outbound web traffic to two public, benign test endpoints — nothing here targets a system you don't own or don't have explicit permission to test.

One prerequisite before step 1: Part 4's Table 4.2 leaves the lab/victim VLAN's outbound access at SIEM-ingest-ports-only by default, so a lab endpoint's `curl` to a real internet host will time out, not just run slowly, until you add one explicit rule for it. Add a single, scoped, temporary allow rule on the Part 6 firewall — lab/victim VLAN to the two test hosts' addresses, ports 80/443 only — for the duration of this exercise, then remove it once you're done. Don't generalize that into a standing "lab/victim VLAN to internet, any port" rule to save yourself the trouble next time; that's exactly the kind of convenience-driven rule Part 4 §4 built the terminal deny-all to prevent.

1. From a lab endpoint on the monitored segment (a Part 9 or Part 10 build), make a plain HTTP request to a benign, purpose-built test site: `curl -A "zeek-lab-test/1.0" http://neverssl.com`.
2. On the Zeek sensor, find the matching entry: `grep '"id.resp_p":80' /opt/zeek/logs/current/conn.log | tail -1`, and note its `uid` value.
3. Search `http.log` for the same `uid`: `grep '<uid-value>' /opt/zeek/logs/current/http.log`.
4. From the same endpoint, make an HTTPS request to a site you expect to support TLS: `curl -A "zeek-lab-test/1.0" https://example.com`.
5. Repeat steps 2–3 against `ssl.log` for this second connection's `uid`.

Expected result: step 3 returns an `http.log` line whose `uid` matches the `conn.log` entry from step 2, with `method = GET` and `user_agent` containing `zeek-lab-test/1.0` — proof the two logs describe the same session, not just similar-looking traffic. Step 5 returns an `ssl.log` line with a populated `server_name` field. If step 3 or 5 returns nothing, re-check that JSON logging (§3.2) is active — a stray tab-separated log from before that change won't match a JSON-formatted `grep` pattern.

Lab Note Keep a second terminal open with `tail -f` on whichever log file you're testing against before you generate the traffic, rather than generating traffic first and grepping after. Watching the line appear in real time is the fastest way to notice a timestamp mismatch, a missing field, or a `uid` that doesn't look like the one you expected — problems that are much harder to spot scrolling back through a file after the fact.

10. From Zeek logs to detections, playbooks, and drills

[CONCEPT]

Everything above stops at "the log exists and is searchable in the SIEM." That's a deliberate scope boundary, not an oversight — turning a `conn.log` field into an actual detection is a different skill, owned by a different volume in this series.

- **Detection logic:** Detection Engineering Handbook V2's Parts 14–15 build directly on the log families this part produces — a vertical or horizontal port-scan detection, a beaconing detection

scoring the coefficient of variation of a connection's inter-arrival intervals, and lateral-movement detections over SMB/RDP session data all assume a connection-summary source that looks exactly like `conn.log`. If you build nothing else from this part, `conn.log` flowing into the SIEM is the prerequisite those chapters are written against.

- **Alert response:** When a detection built from this telemetry fires — or a `notice.log` entry like ``SSL::Invalid_Server_Cert`` shows up on its own — SOC Playbook Handbook's Playbook Library owns what an analyst does next: triage steps, escalation thresholds, and closure criteria. This part's job ends at the alert existing in a searchable form; that book's job starts at the alert firing.
- **Practice drills:** This book's own Part 19 (Generating Practice Telemetry and Purple-Team Drills) pairs a simulated action against a lab endpoint with a specific Detection Engineering Handbook V2 detection and a specific SOC Playbook Handbook playbook, closing the loop from raw Zeek output back to a repeatable exercise. SOC Manager's Operating Handbook's Part 8 (assessment design) and Part 11 (onboarding/ramp-up) frame how to run that same drill as a structured, repeatable program activity rather than a one-off test.
- **Visualization:** Part 17 builds dashboards on top of everything ingested through Parts 8–16, including this part's Zeek logs — connection volume over time, top talkers, and `notice.log` counts are a natural first panel set once this part's forwarding pipeline is live.

None of that downstream work is possible if the pipeline built in §3–§5 isn't actually running and forwarding. Validate it with the tests in this part before assuming the later ones will have anything to read.

Cross-references: Part 4 (network isolation this sensor's Safety Gate depends on), Part 8 (SIEM ingest destination), Part 9/Part 10 (endpoints generating the traffic Zeek observes), Part 12 (the mirrored capture point this part attaches to), Part 14 (Suricata — the complementary alerting layer on the same tap), Part 17 (dashboards built on this part's forwarded logs), Part 19 (pairing this telemetry with a purple-team drill), Part 21 (log retention once volume grows), Appendix A3 (Zeek config snippet baseline), Appendix A4 (isolation pre-flight checklist referenced in this part's Safety Gate); Detection Engineering Handbook V2, Parts 14–15 (turning `conn.log`/`dns.log` fields into scan, beaconing, and lateral-movement detections); SOC Playbook Handbook's Playbook Library (triaging a fired Zeek-sourced alert); SOC Manager's Operating Handbook, Part 8 and Part 11 (structuring a repeatable drill around this telemetry).

Part 14 — Deploying Suricata for Intrusion Detection

Why this part exists

Part 13 put a sensor on the wire and taught Zeek to describe every connection that crosses it — who talked to whom, over what protocol, for how long. This part puts a second piece of software on that same wire and teaches it to do something different: compare every packet against a database of thousands of known-bad patterns and raise an alert the instant one matches. Zeek answers "what happened here?" comprehensively, after the fact. Suricata answers "does this match something bad?" narrowly, in real time. A lab running one without the other is missing half of what a real network security monitoring (NSM) stack looks like — which is exactly why Parts 13 and 14 are built to sit on the same capture point and get read side by side, rather than as two disconnected installs.

This part does not teach detection logic, correlation tuning, or full MITRE ATT&CK mapping — that depth belongs to *Detection Engineering Handbook V2*'s network detection engineering content, cited throughout below. It does not cover running Suricata inline as an intrusion prevention system (IPS) that can drop live traffic — §9 explains why that's a deliberate non-goal for a single-operator home lab. What it does cover, start to finish: installing Suricata against the capture point Part 12 built, pointing it at a real rule source, confirming it actually alerts on something, tuning the noisiest categories out, and getting the result into the Part 8 SIEM where the rest of this book's telemetry already lives.

Safety Gate *Suricata in this part runs in passive IDS mode against a mirrored copy of traffic from the isolated lab segment built in Part 4 and tapped in Part 12 — it never sits inline on a path that carries anything you'd be upset to lose if a rule misfires or the sensor crashes. Two things must be true before you install anything below: first, the mirror/SPAN port feeding the sensor's capture interface carries traffic only from the lab's endpoint, honeynet, and simulation segments, never from your home network's management VLAN; second, the sensor's own management interface (the one you SSH into, and the one `suricata-update` uses to fetch rules) is the only interface with a route to the internet, and the capture interface itself has no IP address at all. Confirm both against the Appendix A4 pre-flight checklist before you bring the capture interface up. Nothing in this part instructs you to point Suricata, or anything upstream of it, at a network you don't own — the traffic it inspects is traffic you generated yourself, in Parts 15, 18, and 19.*

1. Suricata's job next to Zeek: signatures vs. stories

[CONCEPT] Zeek and Suricata both read the same packets and both produce structured logs, which is why it's tempting to think of them as competitors. They're not. Zeek's `conn.log`, `dns.log`, `http.log`, and friends (Part 13) are a comprehensive, protocol-aware transcript — every session gets a record whether or not anything about it looks suspicious. Suricata's default output, a single file called `eve.json`, is signature-driven — a session generates a record in it only when something in the packet stream matched a rule someone wrote to catch a specific known pattern: a particular exploit's byte sequence, a

malware family's command-and-control beacon shape, a scanner's telltale User-Agent string. Zeek tells you everything that happened. Suricata tells you, immediately, when something that happened matches something already known to be bad.

[CONCEPT] That difference has a direct consequence for what each tool can and can't catch. Suricata will never alert on a technique with no rule written for it — a genuinely novel attack, or one that simply hasn't made it into the ET Open ruleset yet, passes through silently. Zeek has no such gap, because it isn't trying to recognize badness at all; it just describes. The two tools are complementary for exactly this reason: Zeek gives you the haystack to search later, and Suricata gives you an immediate, narrow flag on the needles someone else has already catalogued. Table 14.1 puts the split side by side.

The table below compares Zeek's and Suricata's roles directly, to make clear why this book builds both instead of picking one.

Property	Zeek (Part 13)	Suricata (this part)
Detection model	Descriptive — logs every session, no badness judgment	Signature-based — alerts only on a rule match
Primary output	Multiple protocol-specific logs (<code>conn.log</code> , <code>dns.log</code> , <code>http.log</code> , <code>ssl.log</code>)	Single <code>eve.json</code> event stream, typed by <code>event_type</code>
Catches unknown/novel activity	Yes, as a describable session — a human or a hunt query still has to notice it	No — a technique with no matching rule produces no alert
Catches known attack patterns instantly	Only if you write correlation logic on top of its logs	Yes, out of the box, from the ET Open ruleset
Resource profile at lab scale	Moderate CPU, log volume scales with session count	CPU scales with both traffic volume and the number of loaded rules
Where its depth lives	Detection Engineering Handbook V2, Parts 14–15	Detection Engineering Handbook V2, Part 14

2. Where Suricata sits: reusing Part 12's capture point

[CONCEPT] Suricata needs the same thing Zeek needs: a copy of raw packets from somewhere that sees lab traffic, delivered to an interface in promiscuous mode with no IP address of its own. If you built the mirror/SPAN configuration in Part 12 and pointed Zeek at it in Part 13, Suricata reuses the identical interface — there is no separate tap to build. Both tools can run on the same sensor VM, reading the same interface independently in `af-packet` mode; neither one consumes packets the other needs, since the NIC driver hands each listening process its own copy.

Build Autopsy — "one less VM, one shared bottleneck"

The plan: *Save a VM by installing Suricata directly on the router/firewall appliance (Part 6's pfSense or OPNsense box) using its built-in Suricata package, instead of standing up a separate sensor.*

Why it seemed reasonable: *The firewall already sees every packet crossing segment boundaries, the package is a checkbox install, and running one fewer VM matters on the smaller hardware tiers from Part 3.*

How it failed: *Rule matching is CPU-intensive, and so is stateful firewall rule evaluation on the same box. Under a traffic burst — a Part 18 attack-simulation run, or several honeynet decoys getting scanned at once — the two workloads compete for the same cores. The observed failure isn't a clean crash; it's rising firewall latency and silently dropped Suricata capture threads at the same time, so the lab's segmentation enforcement and its intrusion detection degrade together, exactly when the traffic generating the load is the traffic you most want both watching correctly.*

The fix: *Keep the firewall doing firewall work and give Suricata its own sensor VM on a mirrored port, identical to how Part 13 places Zeek. It costs the VM Part 3 budgeted for, but it means a traffic spike that stresses one tool doesn't silently starve the other.*

[SETUP] If the sensor VM is already running Zeek from Part 13, no new capture-interface work is needed — skip to §3. If this is the first NSM tool going on this VM, revisit Part 12's promiscuous-mode and virtual-switch mirroring steps for your specific hypervisor before continuing; Suricata will start and appear to run correctly even if the capture interface is receiving nothing, which is a much more confusing failure to debug than a clean install error.

```

flowchart LR
    subgraph LABNET["Isolated lab segment (Part 4)"]
        EP["Lab endpoints\n(Parts 9-10)"]
        HP["Honeynet decoys\n(Part 15)"]
        SIM["Attack-simulation\ntraffic (Part 18)"]
    end
    end
    SW["Lab switch\nmirror/SPAN port\n(Part 12)"]
    subgraph SENSOR["NSM sensor VM"]
        ZEEK["Zeek\n(Part 13)"]
        SURI["Suricata\n(this part)"]
    end
    end
    SIEM["SIEM ingest\n(Part 8)"]
    DASH["Dashboards\n(Part 17)"]

    EP --> SW
    HP --> SW
    SIM --> SW
    SW -- "mirrored copy, af_packet" --> ZEEK
    SW -- "mirrored copy, af_packet" --> SURI
    ZEEK -- "conn/dns/http/ssl logs" --> SIEM
    SURI -- "eve.json alerts" --> SIEM
    SIEM --> DASH

```

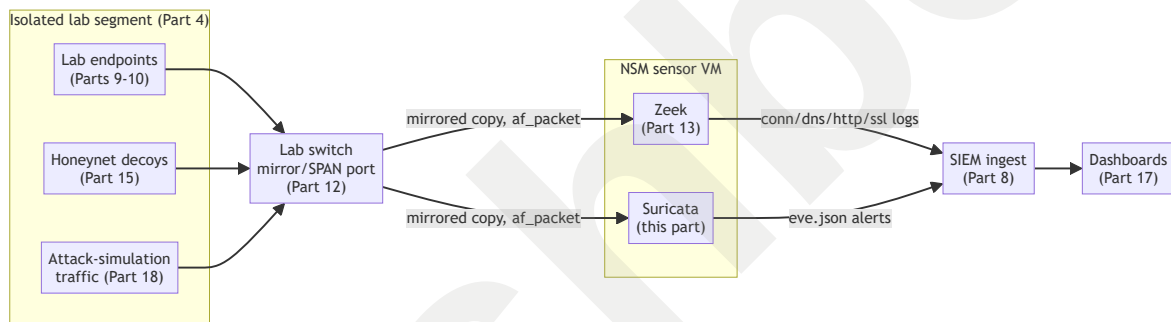


Figure 14.1 — Suricata's place in the NSM capture pipeline. *CONCEPTUAL*. Illustrates how Suricata and Zeek read independent copies of the same mirrored traffic from the Part 12 capture point and forward their respective outputs into the Part 8 SIEM. This is an architecture sketch, not a capture from a running sensor — it shows the intended topology this part and Part 13 build toward, not a screenshot of either tool's own interface. Diagram ID [FIG-14-01](#).

3. Installing Suricata

[SETUP] These steps target Ubuntu Server 22.04 LTS and Suricata's 7.x series via the OISF-maintained stable PPA — the commands don't change across 7.x point releases, but check `apt-cache policy suricata` for whatever is current in that repository when you actually build this, rather than assuming the version pinned above is still the newest one.

```

sudo add-apt-repository ppa:oisf/suricata-stable
sudo apt update
sudo apt install -y suricata
suricata -V

```

Confirm the install worked by checking that `suricata -V` prints a version string starting with `7.` and that `systemctl status suricata` shows the service loaded (it may not be `active` yet — that's expected before the capture interface is configured in §4).

Lab Note Before you ever restart the live service after editing `suricata.yaml`, run `sudo suricata -T -c /etc/suricata/suricata.yaml -v`. It parses the config and rule set without touching the running sensor, and it turns "I have a typo somewhere in 40 lines of YAML" into a specific line number in about two seconds — faster than restarting the service and then digging through `journalctl` to find out why it didn't come back up.

4. Configuring the capture interface and HOME_NET

[SETUP] Suricata's main config file, `/etc/suricata/suricata.yaml`, needs two changes before it's useful in this lab: an `af-packet` stanza pointing at the mirrored interface, and a `HOME_NET` variable describing the lab's own address space so rules that check traffic direction (inbound versus outbound) evaluate correctly.

```
# CONCEPTUAL SAMPLE – invented interface name and lab CIDR for illustration;
# substitute your sensor's actual mirrored interface and Part 4 segment addressing.
vars:
  address-groups:
    HOME_NET: "[10.20.30.0/24]"
    EXTERNAL_NET: "!$HOME_NET"

af-packet:
  - interface: ens19
    cluster-id: 99
    cluster-type: cluster_flow
    defrag: yes
    use-mmap: yes
    ring-size: 2048
```

[COST/RESOURCE] `cluster-type: cluster_flow` matters more than it looks like it should: it tells Suricata to hash packets to worker threads by flow, so a single TCP session's packets always land on the same thread instead of getting split across cores out of order. Leaving this at its default on a multi-core sensor produces reassembly errors under real load that look like a broken NIC, not a config setting.

Resource Reality Suricata's CPU cost scales with both traffic volume and the number of loaded rules, not just one or the other. A quiet lab segment pushing a few megabits a second against the full ET Open ruleset (upward of 30,000 signatures) idles comfortably on 2 vCPUs. The same 2 vCPUs fall behind — visible as a climbing `capture.kernel_drops` counter in `stats.log` — the moment a Part 18 attack-simulation burst or a honeynet scan spike pushes real throughput up while every one of those signatures is still being checked against every packet. Budget 4 vCPUs for a sensor VM that runs both Zeek and Suricata together on anything above the smallest hardware tier in Part 3, and watch `stats.log` after your first real traffic burst before assuming the sizing is fine.

Enable and start the service now that the capture interface and address groups are set, and check the log for capture errors rather than assuming a clean systemd status means packets are actually arriving.

```
sudo systemctl enable --now suricata
sudo tail -n 30 /var/log/suricata/suricata.log
```

A healthy start shows a line reporting the `af-packet` interface came up and a rule-loading summary with a nonzero signature count; an interface name typo produces a clear "no such device" error in the same log within the first few lines.

5. Rule sources and suricata-update

[SETUP] A fresh Suricata install ships with almost no rules loaded — the detection value comes from `suricata-update`, a separate tool that fetches, merges, and installs rule sets from one or more sources. The default source, Emerging Threats Open (`et/open`), is enabled automatically and is the right starting point for a home lab.

```
sudo suricata-update
sudo suricata-update list-enabled-sources
sudo systemctl restart suricata
```

Confirm the update actually pulled rules by checking that `list-enabled-sources` prints at least `et/open`, and that the restarted service's startup log (as in §4) reports a much larger signature count than the near-zero count from the first install.

The table below compares the rule sources `suricata-update list-sources` can enable, to help decide which ones actually belong in a single-operator lab instead of turning on everything available.

Rule source	Cost	Update cadence	Best lab fit
<code>et/open</code> (Emerging Threats Open)	Free	Daily	Default baseline — broad coverage, the source every worked example in this part assumes
<code>et/pro</code> (Proofpoint ET Pro)	Paid subscription	Daily, ahead of Open	Skip for a hobbyist build unless you specifically want to compare Open's coverage against Pro's
<code>oisf/trafficid</code>	Free	Tied to Suricata releases	Application/protocol identification rather than attack detection — pairs well with Zeek's protocol logs from Part 13
<code>sslbl/ssl-fp-blacklist</code> (abuse.ch)	Free	Periodic	Flags known-malicious TLS certificate fingerprints; small rule count, low CPU cost to add
<code>local.rules</code> (self-authored)	Free (your time)	On demand	Lab-specific test markers and Part 19 purple-team drill signatures

[SETUP] Custom rules belong in `/etc/suricata/rules/local.rules`, referenced from `suricata.yaml`'s `default-rule-path` and rule-files list. This one confirms the file is actually being loaded without depending on any ET Open signature firing first.

```
# CONCEPTUAL SAMPLE — a lab-only test rule, not from ET Open, confirming local.rules loads
alert http any any -> $HOME_NET any (msg:"LOCAL TEST - benign lab marker string in URI"; content:"soc-home-lab-marker"; http.uri; sid:9000001; rev:1; classtype:not-suspicious;)
```

Requesting a URL containing the literal string `soc-home-lab-marker` from any lab host should produce a `sid:9000001` alert in `eve.json` within seconds of a `sudo systemctl restart suricata` — if it doesn't, `local.rules` isn't on the loaded rule-files list and the reference in `suricata.yaml` needs a second look.

6. Verifying detection: your first Suricata alert

[HANDS-ON LAB] A custom test rule proves the pipeline works but proves nothing about a real signature. The next check uses an actual ET Open rule and a domain that exists specifically to trip it — `testmyids.com`, whose response body is engineered to match `sid:2100498`, "GPL ATTACK_RESPONSE id check returned root," one of the most widely deployed test signatures in any IDS ruleset.

Validation Test Setup: Suricata running in `af-packet` mode with the ET Open ruleset updated within the last 24 hours (§5), and `eve.json` alert output confirmed via the `local.rules` check in §5. **Action:** From any lab endpoint on the mirrored segment: `curl http://testmyids.com/`
Expected result: A new record in `/var/log/suricata/eve.json` with `event_type: "alert"`, `alert.signature_id: 2100498`, and `alert.signature` containing "GPL ATTACK_RESPONSE id check returned root," with `src_ip` matching the lab endpoint that ran the request.

If nothing appears within a few seconds, check three things in order: that the endpoint's traffic actually crosses the mirrored port (a routing or VLAN-tagging error upstream means Suricata never sees the packets at all), that `stats.log` doesn't show a rising `capture.kernel_drops` counter (an overloaded sensor

silently drops packets before they reach rule matching), and that `et/open` is genuinely in the enabled-sources list from §5.

7. Forwarding EVE JSON alerts into the SIEM

[SETUP] `eve.json` is a newline-delimited JSON file, which every SIEM option from Part 8 can parse directly — the exact ingest mechanism differs by platform, so this section shows the Wazuh path as a concrete worked example rather than three parallel installs. If you built Part 8 on the Elastic stack or Graylog instead, the equivalent step is pointing that platform's log-shipper (Filebeat's Suricata module, or a Graylog JSON input) at the same file path.

```
<!-- On the Wazuh agent running on the Suricata sensor VM, added inside ossec.conf -->
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>
```

Restart the Wazuh agent (`sudo systemctl restart wazuh-agent`) after this change, then re-run the §6 Validation Test — the same `testmyids.com` alert should now show up as a searchable event in the Part 8 SIEM within a few seconds of appearing in `eve.json`, confirming the forwarding path end to end rather than just the local file.

Engineering Reality Suricata's own documentation describes `eve.json` as a single unified event stream; in practice, on a lab sensor running the full ET Open ruleset against even modest traffic, that file fills with `event_type: "http"` and `event_type: "dns"` metadata records for every single session in addition to actual `event_type: "alert"` records — because those log types are enabled by default alongside alerting. Forward the whole file into the SIEM without a filter and the alert signal drowns in protocol metadata Zeek is already logging more completely in Part 13. Either disable the non-alert `eve-log` types you don't need in `suricata.yaml`'s `outputs` section, or filter on `event_type: "alert"` at ingest — decide which before your SIEM's disk fills with a duplicate of Zeek's job.

8. Tuning noise: disabling irrelevant rule categories

[TROUBLESHOOTING] The full ET Open ruleset is written for every network it might ever run on, which means most of it targets traffic your lab will never generate — enterprise VPN protocols, gaming platforms, peer-to-peer file-sharing signatures. Loading all of it costs CPU cycles for zero detection value and, worse, some categories (particularly `policy.rules`) are tuned to alert on behavior that's completely normal and simply looks unusual to a signature written for a corporate network's baseline. `suricata-update` reads a `disable.conf` file to drop whole categories before rules are compiled.

```
# disable.conf – categories excluded because this lab generates no real
# production email, P2P, or gaming traffic for them to meaningfully cover
group:policy.rules
group:p2p.rules
group:games.rules
```

Place this at `/etc/suricata/disable.conf`, re-run `sudo suricata-update`, and confirm the loaded signature count in the post-restart log (§4) dropped — that's the check that the exclusion actually took effect, since a typo in the group name silently matches nothing and leaves every rule loaded.

[TROUBLESHOOTING] A second, more targeted noise source is a single rule that fires correctly but isn't useful for your lab's specific traffic pattern — rather than disabling a whole category over one bad rule, `threshold.config` can rate-limit or suppress by exact `sid`. This is the difference between "this category doesn't apply here" (`disable.conf`) and "this specific signature is correct but too chatty" (`threshold.config`) — conflating the two is the most common Suricata tuning mistake in a home lab, since it's tempting to reach for the blunter tool first.

9. IDS vs. IPS mode, and why this book stays passive

[SAFETY] Suricata can run in two fundamentally different modes: IDS (intrusion detection), passive, reading a copy of traffic and only ever alerting; and IPS (intrusion prevention), inline, sitting directly on the traffic path with the power to drop packets that match a rule. Every step in this part builds IDS mode — the mirrored-port capture interface has no way to drop anything even if you wanted it to, since it never sees the original packets, only a copy. This is a deliberate scope decision, not an oversight: IPS mode requires Suricata to sit inline on the actual forwarding path (typically replacing or extending Part 6's firewall placement), and a misconfigured or overloaded inline sensor doesn't just fail to alert — it drops legitimate traffic, which on a home lab with no on-call team to notice and roll it back can turn a detection exercise into a self-inflicted outage with no attacker involved at all.

Blind Spot A signature-based sensor only ever alerts on what someone already wrote a rule for. A technique with no matching signature — a genuinely novel exploit, a slow-and-quiet technique deliberately shaped to avoid known patterns, or simply something the ET Open maintainers haven't gotten to yet — crosses the mirrored port and generates nothing in `eve.json` at all. Pair every exercise built on this part's alerts with Part 13's Zeek logs, which describe the same traffic regardless of whether any signature matched it; an attack-simulation run in Part 18 that produces zero Suricata alerts is not evidence nothing happened, only evidence nothing matched a rule Suricata was loaded with.

10. Where this telemetry goes next

[CONCEPT] Getting a `sid:2100498` alert into the SIEM is this part's entire job — what an analyst, a detection engineer, or a lab operator running a drill does with it belongs to the other three volumes in this library, and each one picks it up at a different point.

- *Detection Engineering Handbook V2*'s network detection engineering content (its Part 14) is where the raw alerts this part produces get correlated, suppressed for known-benign patterns, and mapped to MITRE ATT&CK techniques where a mapping genuinely applies — for example, a TLS-on-a-plaintext-port protocol-confusion alert Suricata can generate maps to T1595.002 (Active Scanning: Vulnerability Scanning) in that book's own worked example. This part only gets the signal flowing; deciding which alerts are worth a standing detection rule is that book's job, not this one's.
- *SIGNAL TO ACTION: The Complete SOC Playbook Handbook*'s Playbook Library owns the analyst-facing triage steps once an alert fires for real — what to check first, what to escalate, and when a `testmyids.com`-style validation alert should simply be closed as a confirmed-benign test versus investigated. This part builds the pipe the alert travels through; it doesn't tell you what to click when the alert actually lands at 2 a.m.
- The *SOC Manager's Operating Handbook*'s assessment-design content (its Part 8) is where the combination of this part's Suricata alerts, Part 15's honeynet captures, and Part 18's attack-simulation runs becomes the graded evidence behind a repeatable practice drill — did the alert fire, did it get triaged correctly, how long did that take. This part produces the raw signal a drill measures; designing the drill itself is that book's job.

What Would Change My Mind This part recommends `et/open` as the default and only rule source for a hobbyist single-operator lab, on the grounds that `et/pro`'s faster update cadence isn't worth the subscription cost at this scale. If a reader's lab specifically needs same-day coverage of an emerging CVE for a purple-team drill tied to a real disclosure timeline — a scenario Part 19 could plausibly build — that specific gap, not a general "paid rules are better," would be the concrete reason to revisit `et/pro`.

Cross-references: Part 4 (network isolation this part's Safety Gate depends on), Part 6 (why Suricata doesn't share a host with the firewall — §2's Build Autopsy), Part 8 (SIEM ingest target for `eve.json`), Part 12 (capture-point and mirror-port prerequisites), Part 13 (Zeek — the paired NSM tool this part is built to sit beside), Part 15 (honeynet traffic this sensor will alert on), Part 18 (attack-simulation traffic this sensor will alert on), Part 19 (purple-team drills consuming this part's alerts), Appendix A3 (Suricata config snippet reference), Appendix A4 (safety pre-flight checklist); *Detection Engineering Handbook V2* Part 14 (network detection engineering — alert tuning and MITRE mapping); *SOC Playbook Handbook* Playbook Library (alert triage procedure); *SOC Manager's Operating Handbook* Part 8 (assessment/drill design using this telemetry).

Section E — Deception and External Signal

neshbooy

Part 15 — Honeypots and Honeynet Architecture

Why this part exists

Every part before this one built something that has to be reached on purpose — a SIEM you query, an endpoint you log into, a sensor you configure. A honeypot is the first thing in this book built to be found by accident, by anyone, and to say nothing when it's touched except "log this." That single design goal — attract unsolicited contact and record it faithfully — is also why this part carries the strictest isolation requirement in the entire book. A decoy service is deliberately unpatched, deliberately exposed, and deliberately left open on specific ports for the whole internet to find. Every other part in this book defends something; this one is the one part that's supposed to lose, on purpose, in a place where losing costs nothing.

This part builds a honeynet: more than one decoy service, plus the plumbing that turns isolated hits on separate decoys into a single correlated attacker session with MITRE ATT&CK tags attached. It cites the author's own running honeynet — three Linux containers, CT103, CT108, and CT113, referenced throughout *The Detection Engineering Handbook V2* — as a REAL LAB EXAMPLE throughout, because this is one of the few parts in this book where the evidence is genuinely a capture from a live system, not a construction. What this part does not do is teach you to interpret what the honeynet catches. Deciding whether a correlated session is a real threat worth escalating, scoring it by risk, or hunting through its history for a missed pattern is Detection Engineering Handbook V2 territory, cited by part number in §12. This part's job stops at "the decoys exist, they're isolated correctly, and the sessions they produce are trustworthy, correlated, and queryable."

Safety Gate Everything in this part assumes the honeynet segment built in Part 4 has zero route back to your home network, your management network, or any host holding real credentials, and that its only path to the internet is inbound-only, restricted to the exact ports the decoy services in §4 and §5 listen on — never the real SSH port 22, never a management port of any kind. Before powering on a single decoy, run the Appendix A4 pre-flight checklist in full (Appendix A4 is not yet authored — see [README.md](#); until it exists, Part 4 §6's isolation checks and Part 6 §8's firewall validation are the operative pre-flight procedure), and run it again now even if you already ran it while building Part 4's segmentation — a honeypot is built to be attacked, and if isolation fails here the blast radius is whatever the honeynet's VLAN or virtual switch can physically reach, not whatever the firewall rule intended it to reach. Three conditions specific to this part, on top of Appendix A4's general checklist: (1) the honeynet VLAN/vSwitch has no L2 adjacency to the management or lab/victim networks — a shared bridge with a firewall rule stacked on top is not the same thing as a genuinely separate segment, and §3's Build Autopsy below is exactly this mistake, caught in the author's own real lab; (2) the collector/correlation backend covered in §6 is reachable only from the management network, never exposed on the same interface the decoys listen on; and (3) no decoy service's hostname, banner, or credential set uses anything that exists anywhere else in your life — a decoy that leaks a real naming convention has told an attacker more about your real network than a honeypot should ever reveal.

1. What a honeypot is for, and what it isn't

[CONCEPT] A honeypot is a service with no legitimate users. Anything that talks to it is, by definition, either a misconfigured scanner, an automated bot, or an actual attacker — there's no "normal traffic" baseline to build here because there's no normal use case at all. That single property is what makes honeypot telemetry unusually clean compared to everything else in this book: a Sysmon or auditd event needs a baseline of legitimate activity subtracted out before it's interesting, but every single connection a decoy service receives is worth at least a glance.

Honeypots split into two rough interaction levels, and the split determines what a decoy can teach you:

- **Low/medium-interaction** decoys emulate a service's surface behavior — they'll accept a connection, present a banner, log a credential attempt — without running the real underlying software. Cowrie (SSH/Telnet) is the canonical example: it looks enough like an SSH server to collect banners and credential-stuffing attempts, but there's no real shell behind it for an attacker to actually compromise.
- **High-interaction** decoys run real, deliberately unpatched software — an old phpMyAdmin, an outdated Apache — with a real (if sacrificial) filesystem behind it. These catch more sophisticated interaction, including actual exploit attempts against a real vulnerable code path, at the cost of running genuinely exploitable software that must be treated as fully compromised the moment anything beyond routine probing touches it.

A honeypot does not stop, slow down, or deceive a determined attacker who already knows what they're targeting — it exists to catch *unsolicited* contact from scanners and opportunistic bots that don't

know or care what they've found. That's a narrower promise than the word "honeypot" sometimes implies in security marketing, and it's worth stating precisely now, because §11 comes back to exactly this limit once the rest of this part has built something.

2. Honeynet architecture: from one decoy to a correlated segment

[CONCEPT] A single decoy service tells you that something touched it. A honeynet — several decoys plus the plumbing to correlate what they see — tells you that the *same source* touched several of them, in what order, and whether the pattern looks like reconnaissance, an exploit attempt, or something that kept going afterward. That correlation is what turns raw connection logs into the kind of evidence Detection Engineering Handbook V2 can actually reason about.

The author's own honeynet splits this across three roles, and this part's build follows the same split rather than inventing a different one:

- **Decoy services** — the SSH and HTTP listeners built in §4 and §5, whose only job is accepting connections and logging everything about them.
- **Edge/gateway** — a host sitting at the honeynet segment's boundary, running the segment's own baseline logging (service state, connection attempts against non-decoy ports) independently of the decoys themselves. The author's CT108 ("honeynet-edge") fills this role.
- **Collector/correlation backend** — the component that ingests raw events from every decoy and the edge host, stores them, and runs the correlation logic in §6 that turns isolated hits into attacker sessions. The author's CT103 runs this role, backed by a SQLite database (`honeypot.db`) with `ssh_events`, `http_events`, and `network_events` tables feeding an `attack_sessions` table.

flowchart TB

```
INTERNET(("Internet"))
```

```
subgraph HOME["Home network / management / lab-victim networks"]
```

```
  MGMT["Management VLAN\n(SIEM, admin consoles)"]
```

```
end
```

```
subgraph HN["Honeynet segment – Part 4 isolation, inbound-only"]
```

```
  EDGE["Edge/gateway host\n(baseline service logging)"]
```

```
  SSHD["SSH decoy\nTCP 2222 (Cowrie-style)"]
```

```
  HTTPD["HTTP decoy\nTCP 80 (§5)"]
```

```
  COLLECT["Collector/correlation backend\nssh_events / http_events / network_events\n-> attack_sessions"]
```

```
  EDGE --> COLLECT
```

```
  SSHD --> COLLECT
```

```
  HTTPD --> COLLECT
```

```
end
```

```
INTERNET -->|inbound-only,\nde decoy ports only| SSHD
```

```
INTERNET -->|inbound-only,\nde decoy ports only| HTTPD
```

```
COLLECT -->|management-facing interface only\nsee Safety Gate| MGMT
```

```
MGMT -. no route .-> HN
```

```
HOME -. no route .-> HN
```

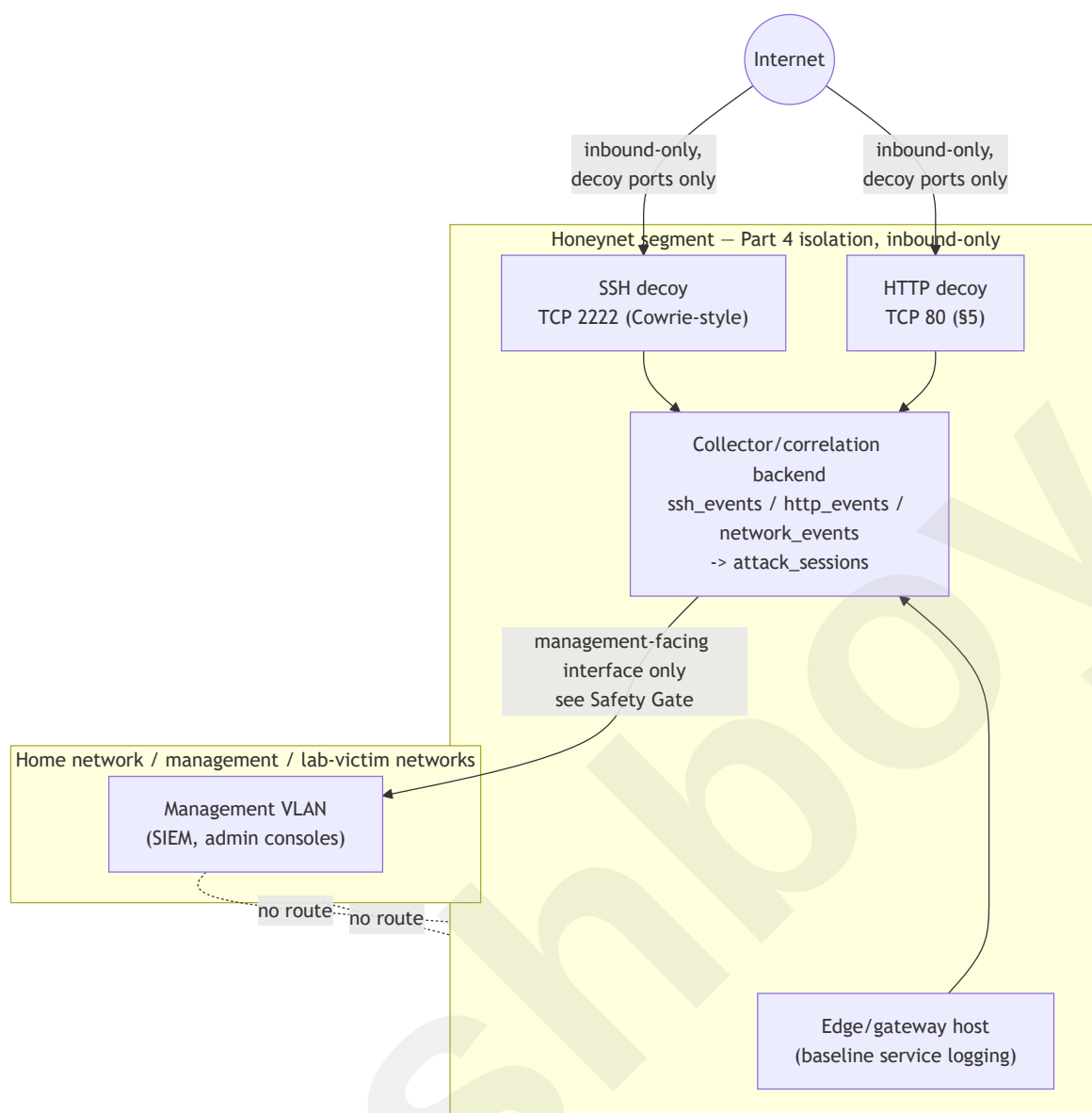


Figure 15.1 — Honeynet segment architecture: decoys, edge host, and correlation backend.

CONCEPTUAL. Illustrates the three-role split this part builds toward — decoy services and an edge/gateway host feeding a collector/correlation backend, which is reachable from the management network but never the reverse — modeled on the role split in the author's own running honeynet (CT103, CT108, CT113). This is an architecture target, not a capture from a running build; Figure 15.5 shows the correlation backend's actual output once built.

The table below compares four realistic honeypot software choices for the decoys themselves, to support choosing what to install in \$4 and \$5.

Table 15.1 — Honeypot software options compared. The table below supports one decision: which decoy software to install in \$4/\$5, based on how much realism you want against how much exposure you're willing to run.

Tool	Interaction level	Protocols covered	Resource footprint	Best fit for
Cowrie	Low/medium	SSH, Telnet	Light — a single Python process, well under 512MB RAM	Readers wanting credential-stuffing/banner-grab telemetry with no real shell exposure
Custom HTTP decoy (nginx/Flask logging every request)	Low	HTTP/HTTPS	Lightest — a web server plus request logging	Readers wanting request-path and user-agent telemetry (recon, exploit-path probing) with no real application behind it
Dionaea	Low	SMB, FTP, HTTP, MSSQL, and other malware-targeted protocols	Light to moderate	Readers specifically interested in malware-dropper capture across several legacy protocols at once
Deliberately unpatched real service (old phpMyAdmin, old Apache)	High	Whatever the real software speaks	Moderate — it's genuinely running the software	Readers who understand the sacrificial-host commitment in §5's Safety Gate and want real exploit-attempt telemetry, not simulated banners
T-Pot (bundled multi-honey-pot + Suricata + ELK)	Mixed (bundles several of the above)	Dozens of protocols simultaneously	Heaviest — a multi-container stack, budget against Part 3's higher tiers	Readers who want broad protocol coverage out of one install rather than building each decoy by hand

This book's worked examples in §4 and §5 build a Cowrie-style SSH decoy and a custom HTTP decoy — the two lightest, most instructive options — and treat the deliberately-unpatched high-interaction pattern (§5's sidebar) as an explicit, higher-risk variant rather than the default.

3. Placing the honeynet on the isolated topology

[SAFETY] Where the honeynet sits matters more than which software runs on it. It needs its own VLAN or virtual switch, established in Part 4, with three properties that are each individually necessary and none of which is sufficient on its own: an inbound-only path from the internet restricted to the exact decoy ports in §4/§5, zero route to the management or lab/victim networks in either direction, and a collector backend (§6) that pulls or receives events without exposing itself on the decoy-facing interface.

The table below extends Appendix A4's general pre-flight checklist with five checks specific to a honeynet build — run these in addition to, not instead of, the full appendix procedure. Appendix A4 is not yet authored (see [README.md](#)); until it is, treat Part 4 §6's isolation checks as that general procedure.

Table 15.2 — Honeynet-specific pre-flight checklist. Run every row below before a decoy's first exposure to the internet, and again after adding any new decoy or changing the segment's firewall rules.

Check	What to verify	Why it matters
VLAN/vSwitch adjacency	The honeynet's VLAN or vSwitch has no bridged interface, trunk port, or shared vSwitch with the management or lab/victim segments	A firewall rule sitting on top of a shared L2 segment is not isolation — see the Build Autopsy below
Inbound port scope	The firewall's inbound-allow rule matches the decoy's listening port exactly (2222, 80), not a range or "any"	An over-broad inbound rule can expose a management port on the same host by accident
Collector exposure	The collector/correlation backend (§6) does not listen on the decoy-facing interface at all	A compromised decoy with a route to the collector's listening port could tamper with the evidence it's supposed to produce
Naming/credential hygiene	No decoy hostname, domain suffix, or credential matches anything used anywhere else in your life or your real lab segments	A leaked real naming convention tells an attacker more about your actual network than the decoy should ever reveal
Scanner/tooling scope exclusion	Any vulnerability scanner or asset-discovery tool elsewhere in the lab has the honeynet's IP range explicitly excluded from its scan targets	Internal scanning traffic reaching the honeynet defeats the "unsolicited contact only" premise this whole part depends on — see the Build Autopsy below

Build Autopsy — the honeynet edge that wasn't as isolated as intended

The plan: The author's own lab runs a vulnerability-scanner platform (CT104, "vulnscan") whose job is to sweep every container on the Proxmox host for real, findable vulnerabilities — SSH configuration weaknesses, outdated services, weak ciphers. Its scan scope was configured broadly, on the theory that excluding any host from vulnerability coverage just creates a blind spot in the scanner's own reporting.

Why it seemed reasonable: A vulnerability scanner is supposed to reach everything it's scanning for, and the honeynet's decoy hosts are real Linux containers running real (if sacrificial) services — scanning them for real misconfigurations looked like thoroughness, not a mistake.

How it failed: Giving the scanner a route to the honeynet segment means the honeynet segment has a route back — not to the internet, but to another internal lab host outside the isolation boundary this part's Safety Gate requires. CT108's own `sshd` journal shows exactly this: legacy SSH protocol and cipher-suite negotiation failures from 192.168.1.96 (CT104), which is indistinguishable in the raw log from a real attacker's protocol-fingerprinting scan — identifiable as internal traffic only because the analyst reviewing it already recognizes CT104's IP by heart. That's not the firewall-rule-only mistake Part 4 warns about in the abstract; it's a scanner explicitly configured with the honeynet's address range in scope, deliberately opening the exact path this book's isolation model exists to prevent.

The fix: Exclude every honeynet-segment IP range from any vulnerability scanner's target scope, permanently, and treat "the scanner already reaches everything else, so scanning the honeynet too is just thorough" as the specific rationalization to catch yourself making. As of this writing, the author's own lab has not fully implemented this exclusion — the Figure 15.2 excerpt below is included specifically because it documents a live, unresolved instance of this gap, not a fixed one. Build your own honeynet's scanner exclusion rule before this part's decoys go live, rather than discovering the gap the way this lab did: by reading your own edge host's journal months later.

```
Sep 10 16:21:03 honeynet-edge sshd[539]: Unable to negotiate with 192.168.1.96 port 43036: no matching
key exchange method found. Their offer: diffie-hellman-group1-sha1 [preauth]
Sep 10 16:21:03 honeynet-edge sshd[541]: Unable to negotiate with 192.168.1.96 port 43058: no matching
host key type found. Their offer: ssh-rsa [preauth]
Sep 10 16:21:03 honeynet-edge sshd[540]: error: Protocol major versions differ: 2 vs. 1
Sep 10 16:21:04 honeynet-edge sshd[550]: Connection closed by 192.168.1.96 port 43082 [preauth]
```

Figure 15.2 — CT108 honeynet-edge sshd journal showing internal scanner traffic from CT104. REAL LAB EXAMPLE. Captured from the author's own running CT108 container (`journalctl -u sshd`) on 2026-09-15; log lines span 2026-09-10. Source IP 192.168.1.96 is CT104's vulnerability-scanner platform, not an external attacker — the evidence backing the Build Autopsy above.

4. Building the SSH decoy

Resource Reality Cowrie's own documentation lists a single Python process as the entire runtime — 1 vCPU and 256–512MB of RAM is enough to run it comfortably, with disk use dominated by however much JSON event logging you retain, not the software itself. This is one of the lightest components in this book to run; the isolation work in §3 costs more planning effort than this decoy costs hardware.

[SETUP] The steps below target Cowrie's official installation method on Debian-family Linux, matching the honeynet segment's isolated container/VM.

1. Create the decoy's VM or container with its only network interface on the honeynet VLAN from §3 — verify this in your hypervisor's network settings before proceeding, exactly as Part 7 warned for the DNS resolver.
2. Install Cowrie's dependencies and clone the official repository per its current install guide.
3. Create and activate Cowrie's Python virtual environment.
4. Configure `cowrie.cfg` to listen on port `2222` for SSH — never port 22, which this book reserves exclusively for real administrative access on the management network, never a decoy.
5. Set a fake hostname and filesystem fingerprint in Cowrie's configuration that matches neither your real lab's naming convention nor any hostname used elsewhere in this book's builds, per the Safety Gate's naming-hygiene condition.
6. Start Cowrie as a service (`bin/cowrie start`) and confirm its JSON event log (`var/log/cowrie/cowrie.json`) begins writing.
7. On the firewall from Part 6, add the inbound-allow rule for TCP `2222` to this decoy's IP only, and confirm the segment's default-deny rule still blocks every other inbound path.

Validation Test Setup: Cowrie installed and listening on port 2222 per the steps above, on the isolated honeynet VLAN. **Action:** From a host outside the honeynet segment but still under your control (never from the open internet during initial testing), run `ssh -p 2222 test@<decoy-ip>` and attempt any password. **Expected result:** The connection is accepted, a fake shell prompt or credential rejection appears, and a corresponding JSON entry appears in `cowrie.json` within seconds, recording the source IP, attempted username, and password. If nothing appears in the log, recheck the firewall rule from step 7 before assuming Cowrie itself is broken — a decoy that never receives a test connection can't log one.

id	ts	src_ip	dst_port	username	password
client_version					
-----	-----	-----	-----	-----	-----

10117	2026-09-10T09:08:30.443603Z	107.174.80.154		root	-----fuck-----
10104	2026-09-10T09:07:30.815869Z	176.53.159.196		support	support
10092	2026-09-10T07:47:47.021592Z	186.242.162.94		admin	admin
10036	2026-09-10T02:26:30.572762Z	47.93.39.183		root	KgW41vmgMK

Figure 15.3 — Real SSH honeypot credential-attempt excerpt. REAL LAB EXAMPLE. Captured from the author's own CT103 honeynet backend (`ssh_events` table) on 2026-09-15; underlying events span 2026-09-08 through 2026-09-10. Source `176.53.159.196` repeated the `support / support` pair across multiple days — a sustained, low-and-slow default-credential sweep against this single decoy. These are attacker-supplied fake credentials, not real secrets belonging to this lab, so nothing here required redaction.

5. Building the HTTP decoy

[SETUP] A minimal HTTP decoy needs only a web server that logs every request in detail and returns a plausible-looking response — it does not need a real application behind it to be useful, because most of what hits an internet-facing web decoy is recon and exploit-path fingerprinting, not a sophisticated multi-step attack.

1. Create the decoy's VM or container on the honeynet VLAN, same isolation requirement as §4.
2. Install nginx or Apache.
3. Configure it to serve a minimal, generic landing page — nothing that names the real software stack this book's other parts actually run.
4. Enable detailed access logging including the request method, full path, and user-agent string (Apache's default `combined` log format captures all three).
5. Bind the listener to port `80` only.
6. Add the matching inbound-allow rule on the Part 6 firewall, scoped to this decoy's IP.
7. Restart the web service and confirm the access log begins writing on any test request.

Safety Gate If you choose the high-interaction variant from Table 15.1 — a real, deliberately unpatched application like an old phpMyAdmin — treat that host as fully compromised the moment it's internet-facing, not as a normal lab endpoint that happens to be old. Never reuse credentials, SSH keys, or TLS certificates from any other host in your lab on this decoy. Patch the underlying OS and any component other than the intentionally vulnerable application itself (Part 20 covers this split in full) — patching the deliberately vulnerable component defeats the entire point of building it.

id	ts	src_ip	method	path	user_agent
694	2026-09-09T21:36:41.187764+00:00	223.123.43.135	POST	/GponForm/diag_Form	Hello, World
693	2026-09-09T21:28:40.194169+00:00	89.21.67.141	GET	/sse	Mozilla/5.0 (compatible; Infrawatch/1.0; +https://infrawat.ch/)
692	2026-09-09T21:28:39.410032+00:00	89.21.67.165	GET	/mcp	Mozilla/5.0 (compatible; Infrawatch/1.0; +https://infrawat.ch/)
686	2026-09-09T20:40:51.502493+00:00	85.217.149.47	GET	/favicon.ico	Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0 Safari/537.36 ModatScanner/1.2 (+https://modat.io/)

Figure 15.4 — Real HTTP decoy exploit-probe excerpt. REAL LAB EXAMPLE. Captured from the author's own CT103 honeynet backend (`http_events` table) on 2026-09-15; underlying events span 2026-09-09. Row 694 is a POST to `/GponForm/diag_Form` — the well-known path for the GPON router authentication-bypass RCE (CVE-2018-10561/CVE-2018-10562), still mass-scanned for today, mapping to T1190 (Exploit Public-Facing Application). Rows 692–693 are from an internet-wide scanner ("Infrawatch/1.0") specifically probing for exposed MCP (Model Context Protocol) endpoints — a reminder that decoy telemetry reflects whatever the internet is currently scanning for, not a fixed, predictable set of probes.

6. Session correlation: turning raw hits into attacker sessions

[CONCEPT] A raw `ssh_events` row and a raw `http_events` row from the same source IP, twelve seconds apart, are two isolated facts until something joins them. The author's own collector backend does this with a `correlation.py` process that reads all three raw tables (`ssh_events`, `http_events`, `network_events`), groups activity by `source_ip` within a session window, and writes the result into an `attack_sessions` table carrying a severity/status pipeline — `recon` → `probe` → `exploit_attempt` → `possible_success` → `confirmed_postexploit` — plus a JSON array of MITRE ATT&CK technique IDs per session.

session_id	source_ip	status	mitre_techniques_json
notes			
-----	-----	-----	-----

d48b795e47c04c6fbb456b17ddd219b9	16.5.0.236	possible_success	["T1595", "T1046", "T1083", "T1190"]
exploit attempt, then continued activity from same source			
ba0ab1f38e11455f83e48298d7b45c1e	45.156.128.45	exploit_attempt	["T1595", "T1046", "T1190"]
exploit-shaped web request observed (traversal/CVE-path-like)			
be97051a26de4c4a87631942fa7fd5ed	64.62.156.192	possible_success	["T1595", "T1046", "T1083", "T1190"]
exploit attempt, then continued activity from same source			

Figure 15.5 — Honeynet correlated attack session, MITRE-tagged. REAL LAB EXAMPLE. Output of the honeynet platform's own session-correlation logic, captured from the author's running CT103 honeynet backend, showing `possible_success`-status sessions tagged with T1595 (Active Scanning), T1046 (Network Service Discovery), T1083 (File and Directory Discovery), and T1190 (Exploit Public-Facing Application). Captured 2026-09-15; source IPs and internal decoy addresses shown are real but represent no risk to disclose, since the targets are sacrificial decoy hosts by design. A `possible_success` status means the platform observed an exploit-shaped request followed by continued activity from the same source — flagged for manual analyst review, not confirmed compromise.

Building that exact pipeline from scratch is more than this part's scope justifies — §7 below builds a smaller, teaching-scale version of the same pattern instead, so a reader understands the mechanism without needing to reproduce a production-grade correlation engine on day one.

7. A hands-on lab: a lightweight correlation script

[HANDS-ON LAB] Goal: write a small script that flags any source IP hitting both the SSH decoy and the HTTP decoy within a 15-minute window as a multi-service probe, tags it with two MITRE technique IDs,

and appends the result to a plain log file — a scaled-down teaching version of the pattern §6 describes at production scale, not a reproduction of the author's own `correlation.py`.

CONCEPTUAL SAMPLE — illustrative teaching script, reading from two plaintext log files with a fixed column layout rather than the author's own SQLite schema; adapt the parsing logic to whatever log format your §4/§5 decoys actually produce.

```
import csv
from datetime import datetime, timedelta

WINDOW = timedelta(minutes=15)

def load_events(path, ts_col, ip_col):
    with open(path, newline="") as f:
        for row in csv.DictReader(f):
            yield datetime.fromisoformat(row[ts_col]), row[ip_col]

ssh_hits = list(load_events("ssh_decoy_log.csv", "timestamp", "src_ip"))
http_hits = list(load_events("http_decoy_log.csv", "timestamp", "src_ip"))

flagged = set()
for ssh_ts, ssh_ip in ssh_hits:
    for http_ts, http_ip in http_hits:
        if ssh_ip == http_ip and abs(ssh_ts - http_ts) <= WINDOW:
            flagged.add(ssh_ip)

with open("multi_service_sessions.log", "a") as out:
    for ip in flagged:
        # T1595 Active Scanning, T1046 Network Service Discovery
        out.write(f"{datetime.utcnow().isoformat()} source_ip={ip} "
                  f"mitre=[\"T1595\\\", \"T1046\\\"] note=multi-service probe\n")
```

Validation Test Setup: Two CSV exports (or hand-built sample files) of decoy hits, `ssh_decoy_log.csv` and `http_decoy_log.csv`, each with `timestamp` and `src_ip` columns, containing one shared IP within 15 minutes on both. **Action:** Run `python correlate.py` (the script above) against the sample files. **Expected result:** `multi_service_sessions.log` gains one new line naming the shared source IP and tagging it `T1595 / T1046` — confirming the join logic actually matches across both decoys' timestamps and IP fields, not just within one file.

8. Forwarding honeynet telemetry into the SIEM

[SETUP] This part stops at "correlated sessions exist and are queryable on the collector backend" — building the ingest pipeline that pulls Cowrie's JSON log, the HTTP decoy's access log, and the `attack_sessions` table into Part 8's SIEM is that part's job, not this one's. Two practical notes worth knowing before you get there: Cowrie's JSON log lines are already newline-delimited JSON, which most log shippers (Filebeat, Wazuh's agent) can tail natively; a web server's access log needs the same Apache/nginx parsing treatment any other web log in this book gets. Export the `attack_sessions` table

on a schedule (a cron job running a `sqlite3 -json` query) rather than trying to tail a database file directly — SIEM log shippers expect a stream of lines, not a live database connection.

9. Troubleshooting common honeynet build failures

[TROUBLESHOOTING] Three failure modes account for most of the friction readers hit standing up a first honeynet.

- **No inbound traffic ever arrives.** Most home internet connections sit behind CGNAT (carrier-grade NAT) or a consumer router with no public IP directly reachable — if your ISP doesn't hand you a routable public address, a decoy on your home connection may simply never be reachable from the open internet no matter how the firewall rule is configured. Confirm your WAN IP is actually public (not a `100.64.0.0/10` CGNAT address) before assuming the decoy itself is broken.
- **The port-forward points at the wrong internal IP after a reboot.** If the decoy's VM or container doesn't have a static IP or a DHCP reservation, a reboot can hand it a different address than the one the firewall's port-forward rule targets, silently breaking inbound access with no error anywhere. Fix: reserve a static lease for every decoy host, not just a convenient default.
- **The inbound rule is broader than the decoy's actual port.** A rule written as "allow inbound to this host" instead of "allow inbound to this host, port 2222 only" exposes every other listening service on that decoy — including, potentially, a management or SSH-for-administration port you didn't intend to expose. Re-check the rule's port scope specifically, not just its source/destination.

10. Keeping a deliberately vulnerable honeynet safely maintained

[SAFETY] A honeynet is the one part of this lab where "patch everything" is the wrong default instruction. The collector backend, the edge host's own OS, and any web server or SSH daemon *hosting* a decoy need normal patching discipline — an unpatched collector is just a second, unintended vulnerable target. The decoy's intentionally vulnerable component itself — Cowrie's simulated banner, or a deliberately old phpMyAdmin version chosen for \$5's high-interaction variant — must not be patched, or it stops being the thing this part built it to be. Part 20 covers this exact split (what to patch, what to deliberately leave alone) for the whole lab, not just the honeynet, and is the part to read in full before this segment has been running unattended for more than a few weeks.

Lab Note *Snapshot every decoy host immediately after its initial build, before its first exposure to the internet — the same discipline Part 5 recommends for every other lab VM. A honeypot that gets genuinely compromised in a way you didn't expect (not just "logged a credential attempt," but an actual foothold on a high-interaction decoy) is exactly the moment you want a clean snapshot to revert to, and exactly the moment you're least likely to have taken one if you didn't do it up front.*

11. What a honeynet can't teach you

[CONCEPT] A honeypot only ever sees traffic addressed to it.

Blind Spot An attacker who compromises a real, production-shaped host elsewhere on your lab network and pivots internally without ever touching a decoy IP generates nothing in the honeynet's logs at all. This part's honeynet teaches you to build a magnet for opportunistic internet-wide scanning and credential-stuffing — genuinely useful, real telemetry — but it is not a comprehensive internal intrusion-detection layer, and nothing in this part should be read as substituting for the endpoint and NSM telemetry Parts 9–14 build elsewhere in your lab.

What Would Change My Mind §6 and §7 both correlate sessions by `source_ip` alone. That key is stable for a single actor operating from one address for the length of one campaign, but it breaks in two directions: a NAT gateway or shared-hosting IP can put several unrelated scanners behind one address (over-merging them into one false session), and a rotating-infrastructure attacker defeats it entirely by never reusing an address twice (under-merging what should be one tracked campaign into several unrelated ones). If a reader's own honeynet shows either failure mode clearly enough to be a repeatable pattern — not a one-off — that's grounds to widen this part's correlation key (to an ASN or CIDR range, for specific infrastructure known to behave that way) rather than treating exact-IP correlation as sufficient by default. Detection Engineering Handbook V2, Part 30 documents this exact limitation against the author's own honeynet data, and Part 35's HUNT-30-02 is a worked hunt built specifically to test for it.

12. Where this telemetry goes next

[CONCEPT] This part deliberately stops at "the sessions exist, they're correlated, and they're MITRE-tagged." Three things happen to that output next, each owned by a different volume in this series:

- **Detection Engineering Handbook V2, Part 30 (Correlation Engineering)** covers the entity-resolution problem §11's What Would Change My Mind names directly, using this same honeynet's `attack_sessions` table as its own worked example — read that part's §3.2 for the full treatment of what a `source_ip`-keyed join can and can't be trusted to merge correctly.
- **Detection Engineering Handbook V2, Part 35 (Hunt Types)** builds IOC-based and TTP-based hunts directly against this part's decoy telemetry — a reused default-credential string across the SSH decoy, and scoping the full extent of a `possible_success` session flagged in §6. This part produces the raw sessions; that part teaches you to hunt through them.
- **Detection Engineering Handbook V2, Parts 32–33 (Threat Intelligence, Risk-Based Detection)** are where a correlated session's source IPs get enriched against the threat-intel feeds this book's own Part 16 adds next, and scored by confidence and severity rather than read as a flat status field.
- **SOC Playbook Handbook's Playbook Library** owns the triage procedure for the one scenario this part's isolation is built to prevent from ever mattering for real: if a host on your actual lab/victim network — not a decoy — ever shows the same attack pattern this honeynet is

designed to catch safely, that's a real detection to investigate, not a decoy behaving as designed, and the playbook branch for that is entirely different from "log it and move on."

- **SOC Manager's Operating Handbook, Part 8 (assessment design)** can use a correlated honeynet session as a low-cost, already-labeled scenario input for a practice drill — the MITRE tags in Figure 15.5 are exactly the kind of ground truth a Field Test needs to score a trainee's analysis against.

Cross-references: Part 3 (hardware tiers, Table 15.1's resource comparisons); Part 4 (network isolation and segmentation architecture, the isolation this part's Safety Gate depends on); Part 6 (perimeter firewall build, the inbound-allow rules in §4/§5); Part 8 (SIEM ingest for the telemetry this part produces, §8); Part 9 (Linux endpoint monitoring, applied to the collector/edge hosts themselves per §10); Part 16 (threat-intelligence feed integration, enriching the source IPs this part's sessions surface); Part 18 (safe attack simulation, contrasted with this part's unsolicited, non-simulated attacker traffic); Part 20 (patching discipline that must not touch a decoy's intentional vulnerability, §10); Appendix A4 (safety and isolation pre-flight checklist, extended by Table 15.2). Detection Engineering Handbook V2 — Part 30 (correlation engineering and entity resolution), Part 35 (hunt types against honeynet telemetry), Parts 32–33 (threat intelligence and risk-based detection). SOC Playbook Handbook — Playbook Library. SOC Manager's Operating Handbook — Part 8 (assessment design).

Part 16 — Threat Intelligence Feed Integration

Why this part exists

Every telemetry source built so far in this book only knows what happened on your own network. Part 7's DNS log knows a lab host queried a domain. Part 13 and Part 14's sensors know a connection happened and, for Suricata, whether it matched a locally-loaded signature. Part 15's honeynet knows something unsolicited touched a decoy. None of them know whether the domain, IP, or file hash involved has ever been seen anywhere else doing something bad — that context comes from outside the lab, and pulling it in is this part's entire job.

This part builds the plumbing: fetching indicator data from free, open threat-intelligence feeds — AlienVault OTX (now operated under the LevelBlue brand following AT&T's 2024 divestiture of its cybersecurity unit, though the OTX name and API have carried over unchanged), abuse.ch's feeds, and open MISP instances — normalizing what comes back into something a SIEM can match against, and loading it in. It stops there deliberately. Deciding how much weight a matched indicator should carry, scoring it against other signals, and deciding when a match is worth an analyst's time are Detection Engineering Handbook V2, Part 32's job, cited again in §10. This part's finish line looks like Part 8's: the feed data arrives, it's normalized, and it's matched against something real in the SIEM — full stop, no scoring logic attached.

Safety Gate *Everything in this part assumes the feed collector — whether a small script or a self-hosted MISP instance — runs on the management VLAN from Part 4, behind a Part 6 firewall rule scoped to the exact feed-provider API endpoints in §2, not a blanket "allow HTTPS outbound" rule. Two concrete conditions to verify before pulling a single feed: (1) the firewall's egress rule lists specific destination hostnames or their resolved ranges for OTX, abuse.ch, and any MISP feed source you add, with everything else still denied by default; and (2) no indicator pulled from any feed is ever used as a live target — no lab host resolves, connects to, downloads from, or otherwise visits a domain, IP, or URL taken from a threat-intel feed, for testing or any other reason. Part 7 §8 already drew this line for its own sinkhole teaching setup; this part is the reason that line exists. Re-run the Appendix A4 pre-flight checklist if you haven't since Part 15's honeynet build, since the feed collector sits on the same management segment that talks to both the SIEM and the honeynet's correlation logic.*

1. What "feed integration" means at this scale

[CONCEPT] A threat-intelligence feed, at the scope this part covers, is a stream of indicators — domains, IPs, URLs, file hashes, and occasionally full attack-pattern descriptions — published by someone else who observed them doing something malicious. Integrating one means three things, in order: getting the data (§2–§5), keeping it somewhere a SIEM can query fast (§5, §8), and matching it against telemetry your lab already produces (§7). None of that is the same as deciding whether a match matters. A domain flagged by an eight-month-old OTX Pulse that's since been repurposed as a shared

CDN edge is a very different finding than a URL abuse.ch listed an hour ago, and telling those two apart is a scoring judgment — Detection Engineering Handbook V2, Part 32's content, not this part's.

It's also worth separating this part from Part 15's honeynet up front, since both involve "indicators" in some sense. Part 15 *produces* indicators — a decoy service gets probed, and the source IP and behavior of that probe becomes something worth recording. This part *consumes* indicators someone else produced, to enrich telemetry your lab already has. The two meet in §7, where a honeynet capture gets checked against a feed-derived list, but they're opposite ends of the same pipe.

2. Choosing feed sources

[CONCEPT] Three source types cover nearly every free/open option a home-lab reader will actually reach for. The table below supports one decision: which source (or combination) to wire into §5's worked build, based on how much you're willing to register for and how narrowly scoped you want the resulting indicator volume to be.

Feed	Auth required	Format	Update cadence	Indicator types	Best fit
AlienVault/LevelBlue OTX	Free account and API key	JSON via REST API (<code>/api/v1/pulses/subscribed</code>)	Continuous — community-submitted "Pulses" arrive whenever a contributor publishes one	Domains, IPs, URLs, file hashes; quality varies widely by contributor	Readers who want the broadest indicator variety and are willing to filter noisy community submissions
abuse.ch URLhaus	Free Auth-Key (abuse.ch began requiring this for programmatic access in 2023, after high-volume unauthenticated scraping degraded the service)	CSV or JSON bulk download, plus a queryable API	Near-real-time; the full dump refreshes every few minutes	Malicious URLs, with associated payload hashes where known	Readers who want a narrowly-scoped, well-curated feed with a low false-positive rate
abuse.ch ThreatFox	Free Auth-Key	JSON via REST API	Continuous	IOCs tagged with a specific malware family and, where known, a MITRE ATT&CK technique — a T1071 (Application Layer Protocol) tag is typical for C2-traffic indicators	Readers who want indicators that arrive pre-tagged with the malware family and technique context Part 15 and Part 19's purple-team drills can cite directly
Open MISP feed (for example, CIRCL's default feed and warninglist bundle)	None for most public feeds; an account only if joining a private sharing community	MISP's own JSON event format, pulled by a MISP instance's built-in feed-sync feature	Varies by publishing community, typically daily	Whatever the publishing community shares — IPs, domains, hashes, and full attack-	Readers already running a MISP instance (§8) who want a pre-formatted feed with

Feed	Auth required	Format	Update cadence	Indicator types	Best fit
				pattern context	no custom parsing

Registering for API access on any of these is free, but read the current terms of service before automating anything against them — abuse.ch's terms are written for personal and research use at lab scale, not for redistributing the feed commercially, and OTX's rate limits change without much notice. None of this book's build assumes a paid tier of anything in this table.

Lab Note Store every API key from this section in its own file with `chmod 600` permissions, never inline in a script and never in a file that also lives in a public repository. A leaked OTX or abuse.ch key doesn't expose anything about your lab — it just gets revoked once the provider notices misuse from wherever it leaked to — but re-issuing keys and re-auditing every script that referenced the old one is a genuinely annoying afternoon you can skip entirely by keeping keys out of version control from day one.

3. Where the feed collector sits on the isolated topology

[SAFETY] The collector — the script or MISP instance actually reaching out to §2's sources — has one job that matters for this book's safety story: it needs an outbound path to specific internet endpoints, and nothing else on the lab network should share that path. Place it on the management VLAN from Part 4, alongside the Part 8 SIEM it feeds, never on the endpoint VLAN and never anywhere near the honeynet segment. The honeynet's own decoy hosts have no legitimate reason to reach out to a threat-intel provider, and a decoy that somehow could would be a decoy with an unexplained egress path — precisely the kind of finding Part 4's isolation model exists to prevent, not produce.

The firewall table below adds to Part 6's rule set. Add these as named, specific rules rather than widening an existing "management VLAN outbound" rule — a rule this narrow is what makes it possible to notice, later, if the collector host ever tries to reach somewhere it shouldn't.

Source	Destination	Port/protocol	Action
Feed collector (management VLAN)	otx.alienvault.com	443/tcp	Allow
Feed collector (management VLAN)	urlhaus.abuse.ch, threatfox.abuse.ch	443/tcp	Allow
Feed collector (management VLAN)	Your chosen MISP feed publisher's hostname	443/tcp	Allow
Feed collector (management VLAN)	Any other destination	any	Deny (default)
Any other source	Feed collector host	any	Deny (default)

Figure 16.1 shows the resulting shape: feed providers reachable only through a scoped egress rule, a collector that never receives inbound connections from anywhere on the lab network, and an enrichment step that happens entirely inside the SIEM, against telemetry the lab already collected.

flowchart LR

```

subgraph EXT["Public feed providers – internet"]
    OTX["AlienVault/LevelBlue OTX\n(Pulses API)"]
    ABUSE["abuse.ch\n(URLhaus / ThreatFox)"]
    MISPFEED["Open MISP feed\n(e.g. CIRCL)"]
end

subgraph FW["Part 6 firewall – egress scoped to\nexact feed-provider hostnames, 443/tcp only"]
    FWR["Explicit allow: collector to feed endpoints\nDefault-deny: everything else"]
end

subgraph MGMT["Management VLAN (Part 4)"]
    COLLECT["Feed collector\n(script or MISP instance)"]
    LIST["Normalized indicator list\n(Wazuh CDB list / lookup index)"]
end

subgraph TELEM["Internal telemetry – passive match only"]
    DNS["Part 7: DNS query log"]
    NSM["Parts 13-14: Zeek + Suricata logs"]
    HN["Part 15: Honeynet session logs"]
end

subgraph SIEM["Part 8: SIEM"]
    MATCH["Enrichment/correlation rule"]
    DASH["Part 17: dashboard"]
end

OTX --> FWR
ABUSE --> FWR
MISPFEED --> FWR
FWR --> COLLECT
COLLECT --> LIST
LIST --> MATCH
DNS --> MATCH
NSM --> MATCH
HN --> MATCH
MATCH --> DASH

```

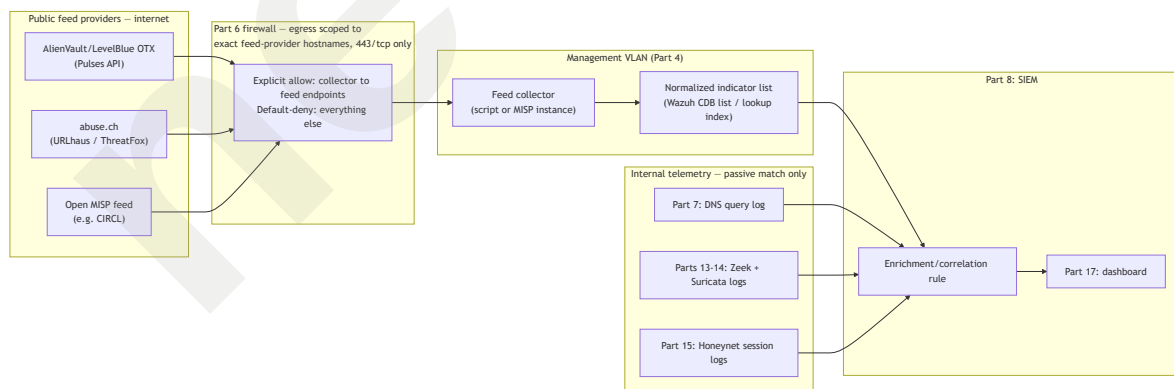


Figure 16.1 — Threat-intel feed collector placement and enrichment path. *CONCEPTUAL*. Illustrates where a feed collector sits on the management VLAN behind a scoped firewall egress rule, and how the resulting indicator list is matched inside the SIEM against internal telemetry from Parts 7, 13, 14, and 15 — never by having a lab host connect to a fetched indicator directly. No REAL LAB EXAMPLE capture of

this specific integration exists: the author's real running lab (cited elsewhere in Parts 5, 7, 9, and 15) has not run a live feed-to-SIEM enrichment pipeline end to end, so this figure is an architecture target, not a capture from a build that happened.

4. Resource cost: a lightweight script versus a full MISP instance

[COST/RESOURCE]

Resource Reality A scheduled script that pulls two or three feeds and writes a flat indicator list costs almost nothing — well under 512MB of RAM, running comfortably as a cron job on hardware that's already doing something else, and megabytes of disk for a rolling list. A self-hosted MISP instance is a different order of cost entirely: MISP's own installation documentation lists 4GB of RAM as a floor and 8GB as comfortable once its optional modules (enrichment, correlation, and taxonomy lookups) are enabled, on top of a MySQL/MariaDB database, Redis, and PHP-FPM all running as separate services. These are the vendor's own numbers, not figures this book's author has build-tested — the real lab this book draws REAL LAB EXAMPLE evidence from has never run MISP. Below MISP's floor, expect the same failure shape Part 8's Build Autopsy described for an undersized SIEM node: services that start but silently fall over the first time real event volume hits them, not a gentle slowdown.

Approach	RAM	Disk	Setup friction	Best fit
Lightweight script + flat-file/CDB list	Under 512MB	Megabytes to low tens of megabytes for a rolling list	Low — one scheduled script, one SIEM list reload	Readers on the entry-level hardware tier (Part 3), or anyone who only wants matching, not sharing
Self-hosted MISP instance	4GB minimum, 8GB comfortable with modules enabled (vendor-documented)	Several GB and growing — MISP retains full event/attribute history, not a rolling window	High — PHP, a MySQL/MariaDB database, Redis, and optionally Elasticsearch for the module set	Readers who want to publish or share indicators with other MISP communities, not just consume a feed

This part's worked build (§5) uses the lightweight script, for the same reason Part 8 picked Wazuh's single installer: it's the lowest-friction path to a working enrichment pipeline for a first build. §8 covers what changes if you'd rather run MISP.

5. Building a lightweight feed pipeline

5.1 Getting API keys without exposing them

[SETUP]

Register for a free OTX account at otx.alienvault.com and copy the API key from your account profile page. Register for a free abuse.ch Auth-Key at auth.abuse.ch — the form asks what you intend to use it for; "personal home-lab telemetry enrichment, non-commercial" is an accurate answer

and the kind of use the free tier is meant for. Save both keys as environment variables loaded from a restricted-permission file rather than pasted into any script:

```
# /opt/lab-intel/.env — chmod 600, never committed to any repository
export OTX_API_KEY="your-otx-key-here"
export URLHAUS_AUTH_KEY="your-abusech-authkey-here"
```

Source this file (`source /opt/lab-intel/.env`) before running anything in §5.2, and confirm the permissions actually stuck with `ls -l /opt/lab-intel/.env` — it should show `-rw-----`.

5.2 Fetching the feeds

[SETUP] The commands below target abuse.ch's URLhaus bulk-download endpoint and OTX's Pulses API as of this part's `last_validated` date; both providers have changed endpoint paths before without much notice, so check each provider's current API documentation if a request below returns an authentication error rather than data.

```
mkdir -p /opt/lab-intel/raw

# URLhaus recent malicious URLs, CSV, requires the free Auth-Key from §5.1
curl -s -H "Auth-Key: $URLHAUS_AUTH_KEY" \
  https://urlhaus.abuse.ch/downloads/csv_recent/ \
  -o /opt/lab-intel/raw/urlhaus_recent.csv

# OTX Pulses you're subscribed to, JSON
curl -s -H "X-OTX-API-KEY: $OTX_API_KEY" \
  https://otx.alienvault.com/api/v1/pulses/subscribed \
  -o /opt/lab-intel/raw/otx_pulses.json
```

Confirm each request actually returned data rather than an error page before moving on: `wc -l /opt/lab-intel/raw/urlhaus_recent.csv` should report more than the CSV's header line alone, and `python3 -m json.tool /opt/lab-intel/raw/otx_pulses.json > /dev/null` should exit with no error if the OTX response parsed as valid JSON.

5.3 Normalizing into one list

[SETUP] A SIEM lookup list wants one flat format, not two providers' native schemas. The script below is a teaching-scope normalizer, not a production-hardened parser — it assumes both source files from §5.2 exist and skips malformed lines rather than failing the whole run, which is the right tradeoff for a lab feed but not for anything handling data you'd be paid to get exactly right.

```
# CONCEPTUAL SAMPLE – teaching-scope normalizer, not hardened for malformed
# upstream data beyond the two skip-on-error cases shown.
import csv
import json

output_rows = []

with open("/opt/lab-intel/raw/urlhaus_recent.csv") as f:
    reader = csv.reader(row for row in f if not row.startswith("#"))
    for row in reader:
        try:
            output_rows.append(("url", row[2], "urlhaus"))
        except IndexError:
            continue

with open("/opt/lab-intel/raw/otx_pulses.json") as f:
    pulses = json.load(f)
    for pulse in pulses.get("results", []):
        for indicator in pulse.get("indicators", []):
            output_rows.append((indicator["type"].lower(), indicator["indicator"], "otx"))

with open("/opt/lab-intel/lab-threat-intel.list", "w") as f:
    for kind, value, source in output_rows:
        f.write(f"{value}:{kind}:{source}\n")
```

Run it with `python3 normalize_feeds.py` and check the result with `wc -l /opt/lab-intel/lab-threat-intel.list` — a healthy first run against a couple of subscribed OTX Pulses and a fresh URLhaus pull typically lands somewhere in the low thousands of lines, not zero and not empty aside from a header.

5.4 Loading the list into the SIEM

[SETUP] Wazuh's CDB list format takes exactly the `key:value` shape §5.3 already produced, minus the source-attribution column. Split IP-type indicators into their own list, since that's the field Wazuh's built-in `address_match_key` lookup matches against.

```
grep ':ip:' /opt/lab-intel/lab-threat-intel.list | cut -d: -f1,2 \
| awk -F: '{print $1":lab-intel"}' > /var/ossec/etc/lists/lab-threat-intel-ips
systemctl restart wazuh-manager
```

Add a rule referencing that list to `/var/ossec/etc/rules/local_rules.xml`:

```
<!-- /var/ossec/etc/rules/local_rules.xml -->
<group name="threat_intel">
  <rule id="100200" level="10">
    <list field="srcip" lookup="address_match_key">etc/lists/lab-threat-intel-ips</list>
    <description>Source IP matches a lab threat-intel indicator list entry</description>
  </rule>
</group>
```

This rule deliberately carries no `if_sid/if_group` parent. Wazuh evaluates a parentless rule against every decoded event, which is what lets one list-lookup rule match a `srcip` field regardless of whether it came

from a Zeek `conn.log` entry, a Suricata `eve.json` alert, or a honeynet session log (§7) — gating it behind an unrelated base rule would silently narrow it to only that rule's own log source. Don't borrow a base rule ID from another `local_rules.xml` example you've seen elsewhere without checking what it actually matches first: OSSEC's own rule 530, for instance, fires only on internal `ossec: output: command-check` text and has nothing to do with network telemetry — a threat-intel rule gated behind it would load without error and simply never fire.

Restart the manager (`systemctl restart wazuh-manager`) to pick up both the new list and the rule, then confirm both loaded without error: `grep -i "lab-threat-intel\|100200" /var/ossec/logs/ossec.log` should show the list and rule ID referenced with no parse error nearby.

6. Validating the pipeline with a synthetic indicator

[HANDS-ON LAB] Goal: prove the list-and-rule pipeline from §5.4 actually fires on a match, without ever pointing a lab host at a real, currently-active malicious indicator to test it — the exact prohibition in this part's Safety Gate.

Validation Test Setup: The lightweight feed pipeline from §5 loaded into the Part 8 SIEM, Part 7's DNS query log already reachable for a domain-based check as well. **Action:** Add the line `example.com:lab-intel` to a second Wazuh list, `etc/lists/lab-threat-intel-domains`, reference it from a second rule in `local_rules.xml` matching a decoded query-domain field (the exact field name depends on how Part 8's ingest pipeline maps DNS log entries — Part 8 stops at "logs arrive and are searchable," and mapping that specific field is Detection Engineering Handbook V2, Parts 5–7's parsing work, so treat the field name below as illustrative). Then, from a lab endpoint, run `nslookup example.com` — a real, harmless domain reserved under RFC 2606 specifically for documentation and testing, and safe to actually query, unlike anything pulled from a live feed. **Expected result:** An alert tagged with the `threat_intel` rule group appears in the Wazuh dashboard, matching the queried domain against the `lab-threat-intel-domains` list entry, in the same time window Part 7 §5's own DNS validation test used.

```
<!-- CONCEPTUAL SAMPLE — field name illustrative; depends on your DNS log's decoder mapping -->
<group name="threat_intel,">
  <rule id="100201" level="10">
    <list field="query" lookup="match_key">etc/lists/lab-threat-intel-domains</list>
    <description>Queried domain matches a lab threat-intel indicator list entry</description>
  </rule>
</group>
```

If nothing fires, check the list and rule load first (§5.4's `ossec.log` grep), then confirm the DNS log entry for `example.com` actually reached the SIEM at all before assuming the list-matching logic itself is broken — an ingest failure upstream of the rule looks identical to a rule that never matches.

7. Matching internal telemetry against the feed, passively

[CONCEPT] The word "passively" in this section's title is doing real work. Once §5's list is loaded, matching happens entirely inside the SIEM, against fields already present in telemetry the lab produced on its own: a `srcip` in a Zeek `conn.log` or Suricata `eve.json` alert (Parts 13–14), a queried domain in the DNS log (Part 7), or a source IP that touched a decoy in the honeynet's session log (Part 15). At no point does anything in this pipeline cause a lab host to originate traffic toward an indicator. The SIEM is comparing values it already has against a list it already loaded — the list itself never gets dereferenced as an address to connect to.

This matters most for the honeynet. A source IP that shows up in a `possible_success`-status correlated session in Part 15's honeynet logs and *also* matches a feed-derived indicator is a genuinely useful correlation — it tells you the same host that poked at your decoy has been reported doing something bad somewhere else, too. Building that specific correlation query, and deciding how much confidence to put in a match against a feed known to include noisy community submissions, is where this part's plumbing hands off directly to Detection Engineering Handbook V2, Part 32.

Engineering Reality *Feed provider documentation tends to describe indicators as "verified" or "curated"; in practice, expect real staleness and real noise regardless of source. OTX Pulses are community-submitted with no consistent review process, so indicator quality inside a single Pulse can range from a genuinely fresh C2 domain to a shared hosting IP that stopped being malicious months before the Pulse was published. abuse.ch's feeds are more narrowly curated and tend to have a lower false-positive rate, but even URLhaus retains delisted entries for a retention window before pruning them, so a match against an hours-old pull can still point at infrastructure that's already been cleaned up. Don't treat a match as confirmation of anything by itself — treat it as a reason to look at the underlying session or connection more closely, which is exactly the judgment call Part 32 formalizes.*

Blind Spot *A feed only ever contains what someone else already observed, reported, and published. A genuinely novel piece of infrastructure — a freshly registered C2 domain nobody has submitted anywhere yet, or a fast-flux setup that rotates IPs faster than any feed refreshes — produces zero matches in this part's pipeline, not because the pipeline is broken, but because there was nothing to match against yet. This part's enrichment step tells you when something you already logged has a known-bad reputation elsewhere; it says nothing about traffic that's simply ahead of every feed it's checked against.*

8. The MISP alternative

[SETUP] If you'd rather run a central indicator-sharing platform than a script — for example, because you want to publish indicators from your own honeynet (Part 15) back out to a MISP community, not just consume one — MISP is the standard open-source option. Installing it is out of this part's worked-example scope given §4's resource cost, but the shape of the decision is straightforward:

1. Provision a VM meeting §4's 4GB-minimum floor, on the management VLAN, behind the same scoped egress rule from §3.
2. Install MISP using the project's own Docker-based or Ansible-based installer (both are documented on MISP's official GitHub project and change often enough that this part doesn't reproduce specific commands here).
3. Add one or more of §2's open feeds under MISP's own "Feeds" configuration page rather than writing a custom fetch script — MISP's feed-sync feature already speaks its native JSON event format.
4. Export MISP's indicator set as a flat list on a schedule and load it into the SIEM the same way §5.4 loads the lightweight script's output — MISP doesn't replace the SIEM-side matching step, it replaces the fetch-and-normalize step.

A MISP instance is worth the resource cost the moment you want to do anything with indicators beyond consuming them for local matching — sharing your own honeynet findings with a community, tracking an indicator's full provenance, or running MISP's built-in correlation across events. For a reader who only wants §7's matching behavior, the lightweight script remains the lower-cost path with no missing capability.

9. Troubleshooting common feed-integration failures

[TROUBLESHOOTING] Four failure modes account for most of the friction readers hit with this build.

- **API requests suddenly return authentication errors after weeks of working.** Both OTX and abuse.ch rotate or expire keys under some circumstances, and abuse.ch in particular has tightened its Auth-Key requirements before with limited advance notice. Check the provider's current API status page before assuming your script broke — this is a provider-side change often enough that it's worth ruling out first.
- **The normalized list is empty or near-empty after a run that clearly downloaded data.** This is almost always a format-drift problem: a provider adds or reorders a CSV column, or nests JSON one level deeper than the last time the script ran. Check the raw file from §5.2 by hand (`head -5` on the CSV, `python3 -m json.tool | head -40` on the JSON) before assuming the SIEM-side list load is the problem.
- **The SIEM shows zero matches even against a feed with thousands of entries.** Confirm the list file itself actually reloaded — Wazuh caches CDB lists in a compiled form and needs a manager restart (§5.4) to pick up a file that changed on disk, not just a rule reload.
- **A flood of low-value matches appears the first time a large feed loads.** A feed with tens of thousands of IP entries can include ranges that overlap shared infrastructure — CDN edge nodes, cloud provider ranges reused by many tenants — that generate frequent, low-signal matches. This is a scoring problem, not a plumbing problem; note it and move on to Detection Engineering Handbook V2, Part 32 rather than trying to solve it by filtering the feed itself down to nothing.

10. Where this enrichment data goes next

[CONCEPT] This part deliberately stops at "the indicator list exists, it's loaded, and a match against internal telemetry fires a low-level alert." Three things happen to that alert next, each owned by a different volume in this series:

- **Detection Engineering Handbook V2, Part 32** owns the actual scoring judgment — how much weight a match against a specific feed source carries, how staleness and provider reputation factor in, and how to combine a threat-intel match with other signals (a honeynet correlation, a Suricata alert) into something worth an analyst's attention rather than noise. This part's `threat_intel`-tagged rule from §5.4 is the raw input that judgment operates on.
- **SOC Playbook Handbook's Playbook Library** owns what to actually do the first time a real threat-intel match fires against something that isn't one of this part's own synthetic tests — triaging a match, deciding whether it warrants isolating a lab endpoint, and documenting the finding. This part doesn't teach triage; it makes sure there's a well-formed, well-sourced alert to triage in the first place.
- **SOC Manager's Operating Handbook, Part 8 (assessment design)** can use a deliberately-seeded feed match — a synthetic indicator like §6's, or a real match generated by Part 18's attack-simulation tooling reaching out to a domain a feed happens to also flag — as a repeatable, low-cost input for a practice drill measuring how quickly a trainee correctly triages a threat-intel-sourced alert versus a signature-based one.

Part 17 builds dashboard panels on top of this part's matches alongside everything else Parts 7–16 feed into the SIEM. Part 19 closes the loop fully, pairing a Part 18 simulated action against a specific detection this part's enrichment contributes to and a specific playbook from the SOC Playbook Handbook.

Lab Note *Don't wire every feed from §2 in on day one. Start with one abuse.ch feed, confirm §6's validation test end to end, and only then add OTX or a MISP feed on top — a pipeline that's already noisy from three sources at once is a much harder thing to debug than one that's quiet and well-understood first.*

Cross-references: Part 4 (network isolation and segmentation architecture); Part 6 (perimeter firewall and segmentation build, extended with §3's egress rules); Part 7 (internal DNS query log, referenced in §6's validation test and §3's telemetry sources); Part 8 (SIEM ingest and CDB list mechanics this part builds on); Parts 13–14 (Zeek and Suricata logs matched in §7); Part 15 (honeynet session correlation matched in §7); Part 17 (dashboards built on this part's matches); Part 18 (attack-simulation tooling referenced in §10); Part 19 (purple-team drills closing the loop); Appendix A4 (safety and isolation pre-flight checklist); Detection Engineering Handbook V2, Part 32 (threat-intel scoring judgment); SOC Playbook Handbook's Playbook Library (alert triage); SOC Manager's Operating Handbook, Part 8 (assessment design).

Section F — Practice: Simulation, Dashboards & Log Generation

neshboy

Part 17 — Dashboards and Visualization Build

Why this part exists

[CONCEPT] Parts 8 through 16 built a working pipeline: a SIEM that can search what arrives (Part 8), Linux and Windows endpoints producing real telemetry (Parts 9–10), Zeek and Suricata watching the wire (Parts 13–14), a honeynet generating unsolicited attacker traffic (Part 15), and a threat-intel enrichment feed (Part 16). Every one of those parts ends the same way — a Validation Test proving one thing arrived and is searchable. None of them answer a much more ordinary question you'll ask yourself constantly once the lab is running: is everything still alive, right now, without typing eight separate queries to check? That's this part's whole job. It builds a small number of dashboard panels that answer "is the pipeline healthy" and "does anything here look unusual" at a glance, on top of the ingest built in every part before it.

This part is not a detection-logic chapter. A panel that plots Suricata alert counts over time tells you alert volume changed; it does not tell you whether that change matters, which alert deserves triage first, or how to tune the rule that's firing too often — that judgment belongs to *Detection Engineering Handbook V2*, cited throughout below. It is also not a NOC-wall chapter. A single-operator home lab produces a few gigabytes of log data a day at most, not the sustained multi-terabyte firehose a real SOC's 55-inch wall display is built to summarize, and a dashboard copied from an enterprise reference architecture will mostly show empty panels and misleading "top 10" lists built for a cardinality your lab will never generate. What follows: choosing where dashboards live, sizing that layer, installing Grafana as a second visualization surface on top of the Part 8 SIEM's own dashboards, and building a specific, small panel set that validates every source built in Parts 8–16 instead of a decorative one.

Safety Gate *The dashboard layer built in this part is a second read-only window into every source this book has built so far — every endpoint, every honeynet session, every threat-intel match — which makes it a second thing that must stay inside the isolated segment architecture from Part 4, not a "just a viewer, it's fine" exception to it. Two concrete conditions must hold before you install anything below. First, whatever host runs Grafana (or the OpenSearch Dashboards already bundled with the Part 8 Wazuh install) binds its web interface only to an address reachable from the management VLAN, never to an interface with a route to your home network or the open internet. Second, if you ever turn on Grafana's anonymous/public-viewer mode for a "wall display" style screen (§3.3's Build Autopsy covers exactly this temptation), the display device itself must be physically on the isolated lab segment — never bridged onto your home network to make the screen easier to glance at from the living room. Confirm both against the Appendix A4 pre-flight checklist before exposing a dashboard URL to anything, including yourself on a laptop that also has a home Wi-Fi connection active.*

1. What a home-lab dashboard is actually for

[CONCEPT] A real SOC's dashboard wall exists to compress a scale of data no single person could read line by line into a shape a shift of analysts can scan in seconds. Your lab has the opposite problem: with two or three endpoints, one NSM sensor, and a handful of honeynet decoys, the total event volume is small enough to read raw in the SIEM's search bar most days. So the dashboards in this part exist for three narrower, more honest reasons.

First, a fast health check. Instead of running six separate searches every morning to confirm the Linux endpoint's auditd feed, the Windows endpoint's Sysmon feed, Zeek, Suricata, the honeynet, and the threat-intel enrichment job are all still forwarding, one dashboard with six panels answers the same question in the time it takes to load a browser tab.

Second, a shared view during an exercise. Part 19's purple-team drills pair a simulated action with an expected detection; a dashboard panel that visibly moves the moment the simulated action runs is a far better shared reference during a drill than asking everyone involved to run their own query at the right second.

Third, an anomaly trigger, not an anomaly explanation. A panel showing Suricata alert volume triple overnight tells you to go look — it doesn't tell you what to do once you're looking, which is where *Detection Engineering Handbook V2* and the *SOC Playbook Handbook* pick up the thread (§7 below spells out exactly where).

2. Choosing a visualization layer

2.1 What you already have from Part 8

[CONCEPT] If Part 8's SIEM build used Wazuh, you already have a dashboard layer installed and running: OpenSearch Dashboards, bundled directly into the Wazuh install and reachable at the same HTTPS address as the rest of the platform. If you built on the Elastic Stack instead, Kibana is the equivalent bundled layer. Both are capable, full-featured visualization tools in their own right, tied to the same OpenSearch or Elasticsearch cluster the SIEM already writes to — for a reader who wants exactly one dashboard tool and no second install, everything in §4's panel list below can be built directly in whichever of these two you already have running, and you can skip §3 entirely.

2.2 Why add Grafana as a second layer

[CONCEPT] The reason to install a second tool on top of a perfectly functional bundled one is a single, specific gap: OpenSearch Dashboards and Kibana are both scoped to the one search cluster they ship with. Grafana's actual differentiator for this book isn't prettier panels — it's a single pane that can query more than one backend at once, which matters the moment your lab has more than one place data actually lives (the Part 8 SIEM's index, plus whatever Part 16's threat-intel job or Part 7's Pi-hole query logs store separately, or host-level metrics you might add in a later build). The table below compares the three options directly so you can decide whether that gap is worth a second install for your specific build, rather than defaulting to "more tools is more thorough."

Property	OpenSearch Dashboards (bundled, Wazuh)	Kibana (bundled, Elastic Stack)	Grafana (standalone, this part)
Extra install required	No — already running from Part 8	No — already running from Part 8	Yes — separate package, separate VM or container recommended
Query scope	The one OpenSearch cluster it ships with	The one Elasticsearch cluster it ships with	Any configured data source, including more than one at once
Resource cost beyond Part 8	None — shares the SIEM host's existing footprint	None — shares the SIEM host's existing footprint	Small but real — see §2.3
Best fit for this book	Readers who built Wazuh and want zero extra moving parts	Readers who built Elastic and want zero extra moving parts	Readers who want one dashboard pulling from the SIEM and other sources side by side

If your lab is small and you have no plan to add a second data source, staying on the bundled tool is the right call, not a compromise — the rest of this part still applies to it, since §4's panel list is about what to build, not which tool builds it. §3 walks through Grafana specifically for readers who do want the second layer.

2.3 Sizing the dashboard layer

[COST/RESOURCE] Grafana itself is a lightweight process compared to anything else in this book — it holds no log data of its own and simply queries whatever backend you point it at, so its own footprint is closer to a small web application than a database.

Resource Reality *Grafana's own sizing guidance treats roughly 512MB of RAM and a single vCPU as sufficient for a single-user or small-team instance with a modest dashboard count, and that number barely changes as you add panels, since almost all the real work (the actual search) happens on the OpenSearch or Elasticsearch cluster it queries, not on Grafana itself. What Grafana does not fix is an undersized SIEM underneath it: a heavy, wide-date-range panel query sent to an OpenSearch indexer that's already at the floor from Part 8's §2 sizing guidance will time out or visibly slow the whole SIEM, and the failure will look like "Grafana is slow" when the actual bottleneck is the indexer it's querying. Give Grafana its own small VM or a lightweight container rather than squeezing it onto the SIEM host itself — not because it needs the isolation for its own sake, but so a runaway dashboard query doesn't compete for the same CPU the indexer needs to keep ingesting.*

The table below maps Part 3's three hardware tiers onto what the dashboard layer can carry without starving anything else already running on that tier.

Hardware tier (Part 3)	Grafana placement	What it can carry
Repurposed laptop/mini-PC	Small LXC container or lightweight VM, separate from the SIEM host	A handful of dashboards querying one data source; avoid wide date-range panels (30+ days) on this tier
Dedicated home-server box	Dedicated small VM, 1 vCPU / 1GB RAM	The full \$4 panel set, querying the SIEM plus one or two additional sources
Multi-node cluster	Dedicated VM with headroom to spare	Multiple dashboards, multiple data sources, and a "wall display" viewer profile from §3.3 without resource concerns

3. Installing Grafana and connecting it to your lab's data

3.1 Installing the package

[SETUP] These commands target Grafana OSS's official APT repository on Ubuntu Server 22.04 LTS, the same distribution used for the Part 8 SIEM host — Grafana's own install docs cover other distributions and container images if your lab standardizes on something else.

```
sudo apt-get install -y apt-transport-https software-properties-common wget
sudo mkdir -p /etc/apt/keyrings/
wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo tee /etc/apt/keyrings/grafana.gpg > /dev/null
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main" | sudo tee -a /etc/apt/sources.list.d/grafana.list
sudo apt-get update
sudo apt-get install -y grafana
sudo systemctl enable --now grafana-server
```

Confirm the install worked before touching configuration by checking that the service is actually up and listening where you expect:

```
sudo systemctl status grafana-server
ss -tlnp | grep 3000
```

`systemctl status` should report `active (running)`, and `ss` should show a process bound to `3000/tcp` — Grafana's default port. If nothing is listening, check `journalctl -u grafana-server` for the specific startup error before assuming the network path is the problem.

3.2 Locking down the default install before connecting anything

[SAFETY] Grafana ships with a default `admin / admin` login and forces a password change on the very first web login — do that first login, and change the password, before you attach a single data source. Log in at `http://<grafana-host>:3000` from a management-VLAN workstation only, per this part's Safety Gate; if that address is reachable from anywhere else at this point, stop and revisit Part 4's segmentation and Part 6's firewall rules before continuing, since an unauthenticated-by-default dashboard tool reachable from the wrong segment is exactly the same class of problem Part 8 called out for the SIEM itself.

3.3 Adding a read-only data source, not an admin one

[SETUP] Grafana needs a plugin to query OpenSearch (the storage layer under Wazuh's indexer) or Elasticsearch directly; install it and restart the service before adding the connection.

```
sudo grafana-cli plugins install grafana-opensearch-datasource
sudo systemctl restart grafana-server
```

Confirm the plugin actually registered before wiring up a connection: `grafana-cli plugins ls` should list `grafana-opensearch-datasource`, and "OpenSearch" should now appear as a data source type under Grafana's Connections → Data sources → Add data source screen. If it doesn't appear, the restart didn't pick up the new plugin — re-run it before continuing.

The credential that connection uses matters more than the plugin itself. **[SAFETY]** Do not point Grafana's data source at the same admin account Part 8's SIEM install generated for you — that account can create users, change indices, and delete data, none of which a dashboard viewer needs. Wazuh's own OpenSearch security configuration (documented in its own admin guide, not repeated step by step here since it's a Part 8 SIEM-administration task rather than a dashboard-building one) supports creating a scoped role limited to read-only queries against the alert indices; create that role and a dedicated service account for it before wiring Grafana in.

```
# CONCEPTUAL SAMPLE – invented hostname, index pattern, and credential placeholder;
# substitute your own SIEM host address and the read-only service account you created.
apiVersion: 1
datasources:
  - name: Wazuh OpenSearch (read-only)
    type: grafana-opensearch-datasource
    access: proxy
    url: https://siem.lab.internal:9200
    jsonData:
      database: "wazuh-alerts-*"
      flavor: opensearch
      version: "2.11.0"
      tlsSkipVerify: false
    basicAuth: true
    basicAuthUser: "grafana-viewer"
    secureJsonData:
      basicAuthPassword: "<read-only-service-account-password>"
```

Place a file like the sample above under `/etc/grafana/provisioning/datasources/` and restart `grafana-server`; confirm the connection under Grafana's own Data Sources screen with its built-in "Test" button before building any panel on top of it — a failed test at this stage saves you from debugging an empty panel later and wondering whether the panel query or the connection itself is the problem.

Build Autopsy — the anonymous wall dashboard that almost bridged two networks

The plan: Turn on Grafana's anonymous-viewer mode so a spare monitor could display the honeynet session count and Suricata alert volume around the clock without anyone logging in, the way a real SOC's wall display works.

Why it seemed reasonable: Anonymous viewing is a documented, one-line Grafana config option, and a screen you never have to log into is genuinely more useful for a glance-on-the-way-past display than one behind a credential prompt.

How it failed: The spare monitor's driving mini-PC had no easy way to reach the isolated management VLAN from where it physically sat, and the fastest fix looked like plugging it into the nearest home-network switch port and adding a port-forward or a second NIC bridge so the browser could reach Grafana's address. That "fastest fix" is precisely the failure Part 4 exists to prevent — it would have put a device with a route into the home network directly on the same physical segment the dashboard was displaying honeynet and SIEM data from, defeating the isolation of every other part in this book for the sake of a convenience display.

The fix: No bridge was built. The display device was instead moved onto a dedicated switch port already trunked into the lab's isolated VLAN (the same physical wiring pattern Part 4 and Part 6 already establish for every other lab host), and anonymous viewing stayed on — but only for a device that was already inside the isolation boundary, not as a justification for extending the boundary to reach a device that wasn't.

4. Designing panels that fit a home lab's actual data volume

4.1 The panel checklist

[CONCEPT] The table below is the panel set this part actually recommends building first — six panels, each one validating a specific source from Parts 8–16 rather than decorating a dashboard with metrics nobody will check. Build these before adding anything more elaborate; a dashboard with six panels you actually look at beats one with 40 you don't.

Panel	Data source (Part)	Index/query target	What it validates
Log volume by source, last 24 hours	SIEM (Part 8)	<code>wazuh-alerts-*</code> grouped by <code>agent.name</code>	Every endpoint and sensor is still forwarding, not just the SIEM itself
Endpoint last-seen / heartbeat	Linux & Windows endpoints (Parts 9–10)	Agent check-in timestamp per <code>agent.id</code>	An endpoint hasn't silently stopped forwarding without an obvious error
Zeek session volume by protocol	Zeek (Part 13)	<code>conn.log</code> events grouped by <code>service</code>	The NSM sensor's capture interface is still receiving mirrored traffic
Suricata alert count by severity	Suricata (Part 14)	<code>eve.json</code> events where <code>event_type</code> is <code>alert</code> , grouped by <code>alert.severity</code>	Rule tuning from Part 14 §8 is holding — a sudden spike means something changed, not necessarily something bad
Honeynet session count by decoy	Honeynet (Part 15)	Session-correlation output grouped by decoy host	Every deployed decoy is still receiving unsolicited contact, not silently offline
Threat-intel indicator match count	Threat-intel enrichment (Part 16)	Enrichment job output where a match occurred	The enrichment pipeline is actually running, not just configured

4.2 Panels that don't work at lab scale

[CONCEPT] A pattern copied directly from an enterprise SOC dashboard reference — a "top 10 source IPs" table, a world map heat layer, a rolling 30-day trend line — assumes a cardinality and volume this lab doesn't generate, and building it anyway produces a panel that's technically correct and practically useless.

Engineering Reality A "top 10 source IPs" panel documented as a standard SOC dashboard pattern assumes hundreds or thousands of distinct source addresses to rank meaningfully. Point that same panel design at a home lab's honeynet, and on most days you'll see three or four decoy-facing addresses total, sorted in an order that changes every time one of them gets scanned twice instead of once — the panel isn't wrong, it's just answering a question your data is too small to make interesting. Panels that hold up at lab scale are the ones in §4.1's table: counts over time and simple groupings, not rankings that need volume to be meaningful.

5. Hands-on lab: building the lab health dashboard

[HANDS-ON LAB] Goal: build the first panel from §4.1's table — log volume by source over the last 24 hours — and confirm it actually moves when new data arrives, rather than trusting that a correctly-configured data source automatically means a correctly-configured panel.

The diagram below shows where this panel's query actually travels: from each Part 8–16 source, through the firewall's default-deny rules, into the SIEM's index, and out to Grafana for display — the same VLAN boundary every earlier part in this book has been building toward.

flowchart LR

```
subgraph EP["Endpoint & sensor sources (Parts 9-16)"]
    Sysmon["Windows Sysmon (Part 10)"]
    Auditd["Linux auditd (Part 9)"]
    Zeek["Zeek conn/dns/http (Part 13)"]
    Suri["Suricata eve.json (Part 14)"]
    Honey["Honeynet sessions (Part 15)"]
    TI["Threat-intel enrichment (Part 16)"]
end

subgraph MG["Management / monitoring VLAN (Part 4)"]
    SIEM["SIEM index (Part 8)"]
    Grafana["Grafana dashboard layer (this part)"]
    Analyst["Analyst workstation"]
end

FW["Firewall - default-deny, \nexplicit allow rules only (Part 6)"]

Sysmon --> FW
Auditd --> FW
Zeek --> FW
Suri --> FW
Honey --> FW
TI --> FW

FW -- "allowed ingest ports only" --> SIEM
SIEM -- "read-only query, scoped\nservice account (§3.3)" --> Grafana
Analyst -- "3000/tcp dashboard view" --> FW
FW -- "dashboard access only" --> Grafana
```

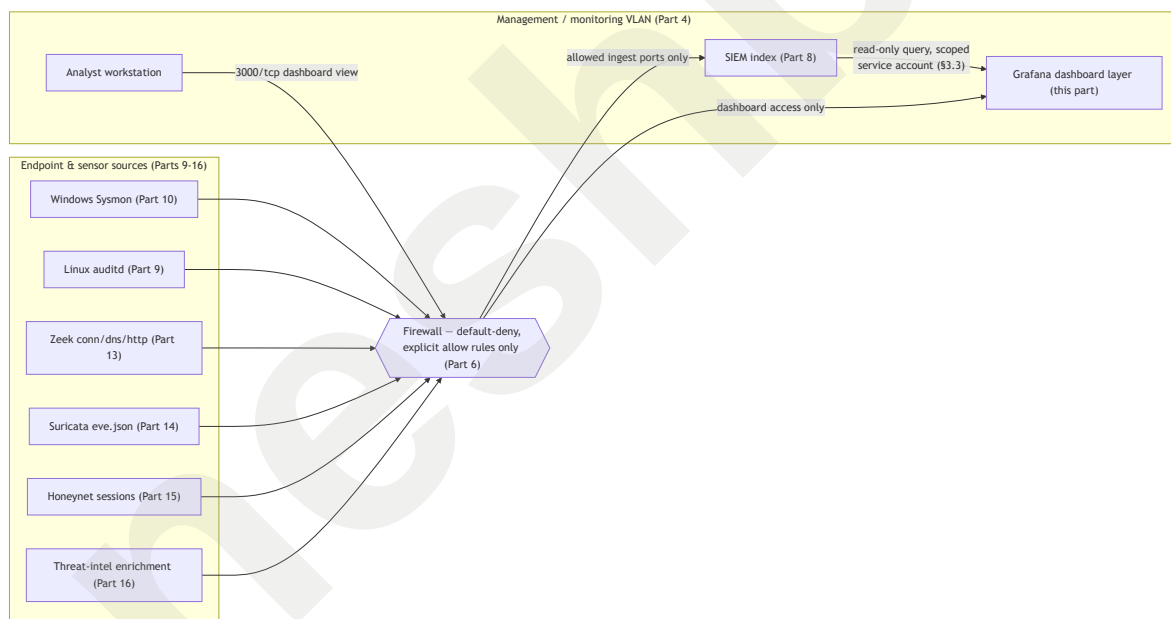


Figure 17.1 — Dashboard query path across the ingest pipeline built in Parts 8–16. *CONCEPTUAL.* Illustrates how a single Grafana panel's query travels from the SIEM's index, itself fed by every endpoint and sensor built earlier in the book, back out to an analyst workstation, all without crossing outside the Part 4 VLAN boundary or the Part 6 firewall's explicit allow rules. This is an architecture sketch, not a capture from a running dashboard — no commercial SIEM or standalone Grafana instance is currently running in the author's own real lab, so no REAL LAB EXAMPLE screenshot of this specific panel exists to cite here. Diagram ID [FIG-17-01](#).

Steps:

1. In Grafana, create a new dashboard and add a new panel using the read-only data source connected in §3.3.
2. Set the query to count documents in `wazuh-alerts-*` over the last 24 hours, grouped by `agent.name`.
3. Set the visualization type to a time-series or bar chart — either shows the same information clearly at this data volume.
4. Save the dashboard with a name that says what it's for, such as "Lab health — last 24h," not a default "New dashboard" title you'll have five duplicates of within a month.
5. On any lab endpoint, generate one new test event and confirm the panel's count for that endpoint's `agent.name` increases within the SIEM's normal ingest delay.

Validation Test Setup: Grafana installed and connected per §3, with the §5 panel built against `wazuh-alerts-*` grouped by `agent.name`. **Action:** On the Part 8 SIEM host itself, run `logger -t Labtest "part17 dashboard check $(date +%s)"` — the same test-event pattern Part 8 §5 used to confirm ingest. **Expected result:** The panel's count for that host's `agent.name` increases by one within the SIEM's normal ingest delay, without needing to refresh the browser tab manually if the dashboard's auto-refresh interval is set. If the count doesn't move, the problem is almost always the query's index pattern or time-field mapping, not the underlying data — check §6 before assuming the SIEM itself stopped ingesting.

Lab Note Set every panel's time-field mapping explicitly to the field your SIEM actually uses for event time (Wazuh's OpenSearch indices typically use `timestamp`), rather than trusting Grafana's auto-detected default. A dashboard that silently falls back to Grafana's own ingest-time field instead of the event's real timestamp will still render a chart — it just won't be the chart you think it is, and nothing about the panel looks broken to tell you that.

6. Troubleshooting dashboards that show nothing, or lie

[TROUBLESHOOTING] Three failure modes account for nearly every broken or misleading first dashboard.

A panel shows "No data" even though the SIEM search bar finds the same events. This is almost always an index pattern mismatch, not a missing-data problem — Wazuh's indices are date-stamped (`wazuh-alerts-4.x-2026.09.15`, for example), and a data source or panel query pointed at an exact index name instead of a wildcard pattern (`wazuh-alerts-4.x-*`) stops matching the moment the date rolls over. Confirm the data source's configured index pattern actually uses a wildcard before assuming the connection itself is broken.

A panel's numbers don't match what a direct SIEM search returns for the same time window. Check the time zone Grafana is applying to the panel against the time zone the SIEM stores timestamps

in — a dashboard defaulting to the browser's local time zone while the underlying index stores UTC produces a panel that's silently off by however many hours separate the two, most visibly right around midnight, when events appear to land on the wrong day entirely.

A "healthy-looking" dashboard with every panel populated stops updating entirely, and nobody notices for days. A panel with data in it from an hour ago looks identical to a panel with data in it from a week ago at a glance, especially on a bar chart with no visible timestamp axis label. Set every panel's auto-refresh interval explicitly rather than leaving it on a manual-refresh default, and treat a dashboard that hasn't visibly changed across a refresh cycle as a reason to check the underlying pipeline directly, not evidence the lab has simply been quiet.

7. Where this part stops, and what to do with the dashboards next

[CONCEPT] A dashboard that visibly moves is proof the pipeline built in Parts 8–16 is alive. It is not, by itself, a decision about whether anything it shows is a problem — closing that gap is deliberately split across this library's other three volumes, and each one has a specific, concrete next action once a panel here changes.

- **When a panel's alert-volume count spikes**, open the specific query behind it directly in the SIEM and hand that raw event set to *Detection Engineering Handbook V2*'s correlation and baselining content (its Parts 22–33) — the dashboard's job was to make you look; deciding whether the spike is a tuning problem, a real detection, or expected noise from a Part 18 or Part 19 exercise is that book's job, not a call to make from the panel alone.
- **When a panel shows a fired Suricata or honeynet alert you don't immediately recognize**, pull the matching procedure from the *SOC Playbook Handbook*'s Playbook Library rather than triaging it ad hoc from the dashboard — the panel tells you something happened; the playbook tells you what to check first, what to escalate, and when a test alert should simply be closed as confirmed-benign.
- **When you run a Part 19 purple-team drill**, use the §4.1 panel set — specifically the alert-count and honeynet-session panels — as the shared, timestamped evidence of whether the simulated action produced the expected detection within the expected window. That observation is exactly the graded input the *SOC Manager's Operating Handbook*'s assessment-design content (its Part 8) and onboarding content (its Part 11) are built to structure into a repeatable drill, rather than an informal "did it seem to work."
- **This book's own Part 19** is where the loop closes fully — pairing a specific simulated action with a specific detection this dashboard would show firing, and a specific playbook response to it. **Part 22** carries the same panels forward into the book's closing synthesis of what to actually do with a finished lab.

Blind Spot *A dashboard panel can only ever show what the SIEM already indexed correctly — it inherits every parsing and normalization gap Part 8 explicitly left to Detection Engineering Handbook V2's Parts 5–7, and it cannot distinguish "nothing happened" from "the pipeline silently stopped." A flat, empty Suricata alert-count panel means either a quiet lab or a broken sensor, and the panel alone cannot tell you which — that's exactly why §4.1's panel set includes a log-volume-by-source and endpoint-heartbeat panel next to the alert panels, so a broken pipeline shows up as a flat health panel even on a day with zero real alerts.*

What Would Change My Mind *This part recommends a deliberately small, six-panel dashboard over a larger enterprise-style layout, on the grounds that a home lab's data volume makes most enterprise panel patterns (rankings, heat maps, wide rolling trends) uninformative at this scale. If a reader's lab grows past a handful of endpoints and sensors into something closer to a small production network — enough distinct hosts and traffic that a "top 10" ranking or a geographic heat map actually varies meaningfully day to day — that specific volume threshold, not the general "keep it small" framing, would be the concrete reason to revisit this recommendation.*

Cross-references: Part 3 (hardware tiers), Part 4 (network isolation this part's Safety Gate depends on), Part 6 (firewall rules gating dashboard access), Part 7 (Pi-hole/DNS query logs as an additional Grafana data source), Part 8 (SIEM platform and its bundled OpenSearch Dashboards/Kibana), Part 9 (Linux endpoints/auditd feeding the endpoint-heartbeat panel), Part 10 (Windows endpoints/Sysmon feeding the same panel), Part 13 (Zeek session-volume panel source), Part 14 (Suricata alert-count panel source), Part 15 (honeynet session-count panel source), Part 16 (threat-intel indicator-match panel source), Part 18 (attack-simulation traffic these panels will visibly react to), Part 19 (purple-team drills using this part's panels as shared evidence), Part 22 (closing synthesis), Appendix A4 (safety and isolation pre-flight checklist); *Detection Engineering Handbook V2* Parts 5–7 (parsing/normalization this part's panels depend on) and Parts 22–33 (correlation/baselining once a panel flags something); *SOC Playbook Handbook's* Playbook Library (triage procedure once a panel-flagged alert needs action); *SOC Manager's Operating Handbook* Part 8 (assessment design) and Part 11 (onboarding/ramp-up), both consuming this part's dashboards as drill evidence.

Part 18 — Safe Attack Simulation with Atomic Red Team—Style Tooling

Why this part exists

Every earlier build part in this book gets you telemetry the honest way: install a real endpoint, install a real SIEM, wait for real activity to produce real logs. That works, but it's slow, and it never tells you whether a specific detection actually fires for the reason you think it does — you'd have to wait for a real attacker, or lie to yourself about what "normal use" happened to trigger. This part closes that gap. Atomic Red Team, and tools built the same way, give you a library of small, single-technique test cases, each mapped to a specific MITRE ATT&CK ID, each reversible by design. Run one against an endpoint you own, on a segment that can't reach anything else, and you get exactly the telemetry a specific technique should produce, on demand, with a known cause — the single most useful thing a home lab can manufacture for validating the Sysmon and `auditd` work from Parts 9 and 10.

This is also, bluntly, the part of this book most likely to get skimmed out of order. A reader who has already skipped ahead to "the fun part" may not have Part 1's ownership rule or Part 4's isolation model fresh in mind, so this part restates both in full rather than assuming they carried over. If you're reading this part first, or reading it again after a break, read the Safety Gate below before anything else — it is not a formality here.

[CONCEPT] What this part does not teach: offensive technique development, exploit writing, EDR/AV evasion, or real-world red-teaming methodology. Atomic Red Team-style tooling runs known, published, single-technique tests against systems you already own — it is a validation tool for defenders, not a red-team curriculum. If you came here looking for how to build a novel attack, this is the wrong book; if you came here looking for how to prove your Sysmon config actually catches something specific, keep reading.

Safety Gate — mandatory pre-flight checklist, restated in full for this part

This is the highest-risk-of-misuse part in the book, so the one rule that governs everything in it gets repeated here rather than assumed read: you attack and defend only systems you own, on a network segment that cannot reach — and cannot be reached by — anything you don't own. Atomic Red Team's own project documentation states its tests are meant to run in a lab environment, not against production or third-party systems; this book holds you to exactly that line, with no exceptions for "it's just a discovery command" or "it's just my own laptop for a second." Before you install anything in §4, and before every single session after that, confirm each item below against the full procedure in Appendix A4 (Appendix A4 is not yet authored — see [README.md](#); until it exists, Part 4 §6's isolation checks are the operative procedure to confirm against) — don't rely on memory of having checked it last time.

- 1. Target endpoints stay on the isolated segment.** *The Windows host from Part 10 and/or the Linux host from Part 9 that you're about to run tests against sit on the lab/victim segment built in Part 4, with the same zero-route-to-home guarantee every other part in this book depends on — not bridged, not dual-homed, not reachable from your everyday devices.*
- 2. The operator control host is also confined to that segment.** *Whatever machine runs `Invoke-AtomicTest` — a dedicated lab VM, not your daily-driver laptop — reaches the targets only over WinRM or SSH sessions that stay inside the lab segment. Opening a session from a device that's also logged into anything you care about defeats the isolation the moment that device gets compromised back.*
- 3. Egress from the segment is deny-all by default.** *The one exception is a narrow, explicit allowlist for the small number of atomics whose prerequisite step downloads a real file from the internet (§5) — and that allowlist gets reviewed before every session, not set once at Part 6 and forgotten about here.*
- 4. Every target has a fresh snapshot taken immediately before the session starts.** *Some atomics are destructive by design (§5); a snapshot from minutes ago, not a hope that the test's cleanup command works, is the actual undo button.*
- 5. Nothing here ever points at a hostname, IP, or account you don't personally own and control** — *not a coworker's machine, not a cloud tenant that isn't yours, not "just a quick scan" of anything outside the segment above.*

If any one of these isn't true right now, stop and fix it before reading further. This is the one callout in this book that never shrinks to a cross-reference.

1. What Atomic Red Team actually is

[CONCEPT] Atomic Red Team is an open-source project maintained by Red Canary ([redcanaryco/atomic-red-team](#) on GitHub): a library of small YAML-defined test cases, each called an "atomic," each mapped to one specific MITRE ATT&CK technique or sub-technique ID. A single atomic declares which platforms it supports (`windows`, `linux`, `macos`), an executor (`powershell`, `command_prompt`,

`sh`, `bash`, or `manual`), the literal command or commands to run, an optional prerequisite the test needs before it can run (a file, a tool, a registry key), and — this is the part that makes the whole model safe to use in a lab — an optional `cleanup_command` that reverses the test's effect. The companion execution framework, `Invoke-AtomicRedTeam`, wraps that YAML into three verbs a PowerShell console understands: check and install prerequisites, run the test, and clean up afterward.

[CONCEPT] "Safe" here means something specific, not a general reassurance. Each atomic is small and single-technique by design — it does one thing mapped to one ATT&CK ID, not a chained multi-stage campaign toward a real objective. Most atomics are reversible, with the reversal command shipped alongside the test itself rather than left for you to figure out. And every atomic is published, reviewed, community-maintained content — you are never writing or discovering a novel exploit to run this content, only executing something already fully documented. That narrow definition of "safe" is also its limit: safety here describes the test's blast radius on a target you own, not a guarantee about what happens if you point any of this at a system you don't own. Nothing changes that second part — see the Safety Gate above.

Table 18.1 lays out the YAML fields worth reading before you run any atomic, since skipping straight to the command line is exactly how a reader misses the one field — `cleanup_command` — that tells you whether the test undoes itself.

Table 18.1 — Atomic test YAML fields worth reading before you run one. The table below supports deciding whether a specific atomic is safe to run as-is or needs extra caution before you type the command.

Field	What it tells you	Why check it first
<code>attack_technique</code>	The MITRE ATT&CK ID this test maps to	Confirms what your detection is actually supposed to catch before you run anything
<code>supported_platforms</code>	<code>windows</code> , <code>linux</code> , <code>macos</code> , or a subset	Some technique IDs only have a test for one OS — don't assume Windows/Linux parity
<code>executor.name</code>	<code>powershell</code> , <code>command_prompt</code> , <code>sh</code> , <code>bash</code> , or <code>manual</code>	<code>manual</code> means there's no automated way to run or clean it up — you do both steps by hand
<code>dependencies</code> / <code>get_prereq_command</code>	What the test needs staged before it runs (a file, a tool, a config value)	Some prerequisites download a real file from the internet — this is where egress control (\$5) matters
<code>cleanup_command</code>	The command that reverses the test's effect	Empty or absent means nothing automated undoes this test — snapshot before you run it

2. Evidence and honesty: what's build-tested here and what isn't

[CONCEPT] In keeping with this book's own honesty standard (`STYLE-GUIDE.md` §9, and the same discipline Part 11 applies to the Windows domain gap), this part says plainly what it can and can't back with a real capture. The author's actual running lab — the Proxmox host, the honeynet, the monitored Linux containers cited as `REAL LAB EXAMPLE` evidence throughout this book and *Detection Engineering Handbook V2* — does not currently run Atomic Red Team or an equivalent framework. Every command, table, and diagram in this part is either `CONCEPTUAL` (the diagrams, the risk-tier framing) or `OFFICIAL`

REFERENCE (Red Canary's own documented install and invocation syntax, reproduced because it's the actual documented interface, not because the author ran it against a lab host and captured the result). Nothing below is labeled **REAL LAB EXAMPLE** or **CONTROLLED LAB EXAMPLE**, and nothing should be — that would be exactly the fabricated-evidence failure mode this book's review checklist is built to catch.

That gap matters less than it might for a Windows/AD domain, because unlike Part 11's abandoned build, nothing here was attempted and stalled — the project this part teaches is a well-documented, widely deployed open-source tool with a stable, publicly verifiable command surface, not a half-finished infrastructure build with unknown failure modes. The honest caveat is narrower and more specific: the *specific interaction* between this book's Sysmon baseline (Part 10), **auditd** rule set (Part 9), and a given atomic's exact telemetry shape has not been personally verified end to end in this lab. Treat every Validation Test in this part as a description of what the tool's documentation and MITRE's own technique write-up say should happen, and confirm it against your own build before trusting it as ground truth.

3. Why the isolation model matters here specifically

[SAFETY] Every other part in this book isolates a *component* — a SIEM, a honeypot, an endpoint. This part is different: it's the one place in the book where the isolated segment has to contain a tool whose entire purpose is to deliberately trigger the exact behaviors your detections, your NSM sensors, and (if you built Part 15) your honeynet are all watching for, on demand, repeatedly. Figure 18.1 shows where the operator control host and its targets sit relative to everything else this book has built.

```
flowchart LR
    subgraph SIMSEG["Attack-simulation segment (Part 4, dedicated VLAN)"]
        OP["Operator control host\nInvoke-AtomicRedTeam"]
        WINEP["Windows target endpoint\n(Part 10, Sysmon)"]
        LINEP["Linux target endpoint\n(Part 9, auditd)"]
        OP -->|"WinRM/SSH, in-segment only"| WINEP
        OP -->|"WinRM/SSH, in-segment only"| LINEP
    end
    end
    MIR["Mirror/SPAN port (Part 12)"]
    NSM["Zeek + Suricata (Parts 13-14)"]
    SIEM["SIEM ingest (Part 8)"]
    DASH["Dashboards (Part 17)"]
    FW["Perimeter firewall (Part 6)"]

    WINEP -->|"Sysmon events"| SIEM
    LINEP -->|"auditd/journald events"| SIEM
    SIMSEG -->|"mirrored copy only"| MIR --> NSM --> SIEM
    SIEM --> DASH
    SIMSEG -->|"default-deny egress;\nnarrow allowlist for\nGetPrereqs downloads only"| FW
    FW -->|"no route – must stay unreachable"| HOME["Reader's home network"]
    FW -->|"no route – must stay unreachable"| OTHER["Any system you don't own"]
```

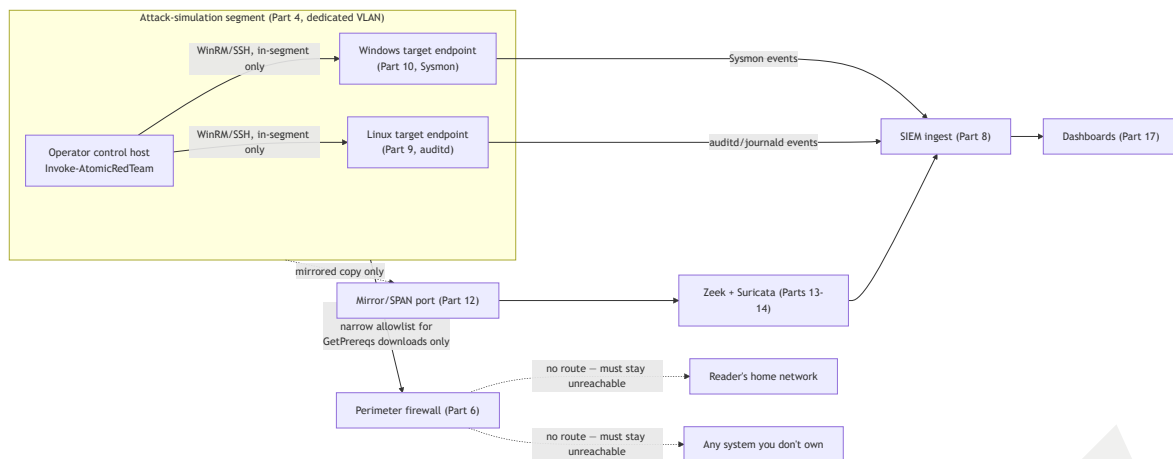


Figure 18.1 — Attack-simulation segment topology and isolation boundary. *CONCEPTUAL.*

Illustrates the intended placement of an operator control host and its target endpoints inside the Part 4 lab segment, the telemetry paths those targets already feed (Sysmon/auditd into the SIEM, mirrored traffic into Zeek/Suricata), and the two things the perimeter firewall from Part 6 must never route to. This is an architecture sketch of the intended build, not a capture from a running deployment — see §2. Diagram ID [FIG-18-01](#).

[SAFETY] The detail worth pulling out of that diagram explicitly: the operator control host is not exempt from segmentation just because it's "your" tool running "your" commands. It is a machine with credentials that can reach every target endpoint in the segment, which makes it exactly as sensitive as a domain controller or a SIEM manager — if it were ever reachable from outside the segment, compromising it would hand an attacker the same WinRM/SSH access you use for legitimate testing. Confine it the same way Part 4 confines everything else, and never reuse it as a general-purpose lab jump box for unrelated tasks.

4. Installing the Atomic Red Team execution framework

[SETUP] These steps target the `Invoke-AtomicRedTeam` PowerShell module and PowerShell 7.4.x, following Red Canary's own published installation instructions ([OFFICIAL REFERENCE](#)). PowerShell 7 (`pwsh`) is cross-platform, which is why the same module and the same commands below work against a Windows target's console, a Linux target's `pwsh` shell, or an operator control host running either OS — you are not maintaining two separate toolchains for two separate platforms.

1. On the operator control host (not a target endpoint), install PowerShell 7. Windows ships only Windows PowerShell 5.1 by default — PowerShell 7 (`pwsh`) needs a separate install even on a current Windows build, via `winget install --id Microsoft.PowerShell --source winget` or the MSI package from the [PowerShell/PowerShell](#) GitHub releases page. On Debian/Ubuntu, Microsoft's own repository provides a `powershell` package instead.
2. On the Windows target endpoint from Part 10, confirm PowerShell remoting is enabled so the control host can reach it over WinRM — `Enable-PSRemoting -Force`, run once as an administrator; it's off by default on a standalone Windows install.
3. On the Linux target endpoint from Part 9, install PowerShell 7 and register it as an SSH subsystem so the control host can reach it over PowerShell-over-SSH remoting — add a `Subsystem powershell`

`/usr/bin/pwsh -sshs -NoLogo` line to `sshd_config` and restart `sshd`, per Red Canary's own remote-execution documentation.

4. From a `pwsh` session on the operator control host, run the project's install script:

```
IEX (IWR 'https://raw.githubusercontent.com/redcanaryco/invoke-atomicredteam/master/install-atomicredteam.ps1' -UseBasicParsing)
Install-AtomicRedTeam -getAtomics
```

5. Confirm the module loaded and the atomics repository is present:

```
Get-Module -Name invoke-atomicredteam -ListAvailable
Invoke-AtomicTest T1082 -ShowDetailsBrief
```

`Invoke-AtomicTest T1082 -ShowDetailsBrief` should list one or more numbered tests for System Information Discovery without running any of them — this is a read-only listing command, safe to run on the operator control host itself while you're still setting up, before a single test executes against a target.

Validation Test Setup: `Invoke-AtomicRedTeam` installed per the steps above, on an operator control host inside the Part 4 lab segment (§3). **Action:** `Invoke-AtomicTest T1082 -ShowDetailsBrief` **Expected result:** A list of one or more numbered atomic tests for T1082 (System Information Discovery), each showing its name, supported platform(s), and executor type — no target endpoint is touched by this command, since `-ShowDetailsBrief` only reads the local atomics repository.

5. Choosing which atomics are safe to run first: risk tiers

[SAFETY] Not every atomic carries the same weight, and treating them all as interchangeable is the single most common way a reader gets an unpleasant surprise from otherwise well-documented tooling. Table 18.2 sorts the technique library into four risk tiers by what the test actually does to the target, not by how alarming its ATT&CK name sounds.

Table 18.2 — Atomic test risk tiers to check before you run one blind. The table below supports deciding what pre-run precaution, if any, a given atomic needs beyond the baseline Safety Gate checklist.

Risk tier	Example technique	What makes it this tier	Pre-run check
Read-only / discovery	T1082 (System Information Discovery)	Only queries local system state; no persistence artifact, no network egress	None beyond the baseline checklist — safe as a first test
Requires cleanup	T1547.001 (Boot or Logon Autostart Execution: Registry Run Keys)	Leaves a persistence artifact on the target until its <code>cleanup_command</code> runs	Always run the <code>-Cleanup</code> step immediately after the test; verify the artifact is actually gone before moving on
Requires network egress	T1105 (Ingress Tool Transfer)	The test's prerequisite step downloads a real file from an external URL to prove file-transfer detection	Confirm the allowlisted egress path from the Safety Gate covers exactly that destination, or stage the file locally and run offline
Destructive	T1490 (Inhibit System Recovery)	Deletes shadow copies or backup catalogs on the target — not reversible on that host without a snapshot	Never run without a fresh snapshot taken immediately beforehand; treat the target as fully disposable for this session

[TROUBLESHOOTING] The `cleanup_command` field is not a guarantee, only a documented attempt. Community-reported issues against the `atomic-red-team` repository include atomics whose cleanup step assumes a default file path, a specific registry hive location, or an execution context (elevated versus standard user) that doesn't match every environment — when that assumption doesn't hold, the cleanup command can run without error and still leave the artifact in place. Confirm the artifact is actually gone with your own check (a registry query, a file listing, a service status) rather than trusting a clean exit code from `-Cleanup` alone, especially for anything in the "requires cleanup" or "destructive" tiers above.

6. Hands-on lab: running your first atomic test end to end

[HANDS-ON LAB] Goal: run the same read-only atomic — T1082 (System Information Discovery) — against both a Windows target (Part 10) and a Linux target (Part 9), and confirm each produces the telemetry the technique should produce in the SIEM you built in Part 8. T1082 sits in TA0007 (Discovery), the ATT&CK tactic covering an adversary's attempts to learn about the system and network they're on — a near-universal early step, and one of the least destructive things to validate first.

1. On the operator control host, open a session to the Windows target endpoint over WinRM:
`$winSession = New-PSSession -ComputerName <windows-target> -Credential (Get-Credential)`. Confirm the session opens successfully from strictly inside the lab segment (§3).
2. Open a second session to the Linux target endpoint over PowerShell-over-SSH remoting:
`$linuxSession = New-PSSession -HostName <linux-target> -UserName <lab-user>`.
3. Run `Invoke-AtomicTest T1082 -ShowDetailsBrief -Session $winSession` targeting the Windows endpoint. Check that at least one listed test's executor is `command_prompt` or `powershell`.
4. Run the test itself against the Windows target: `Invoke-AtomicTest T1082 -Session $winSession`.
5. Within a minute or two, search the SIEM for a Sysmon Event ID 1 (Process Create) entry on that host, `Image` matching the discovery command the atomic ran (typically `systeminfo.exe` or a PowerShell `Get-ComputerInfo` invocation), with `ParentImage` pointing to the WinRM host process that launched it.

6. Run the platform-appropriate T1082 test against the Linux target: `Invoke-AtomicTest T1082 -Session $linuxSession` (typically a `uname -a` or `/etc/os-release` read via the `sh/bash` executor).
7. Check `ausearch -k exec -ts recent` on the Linux target for a `type=EXECVE` record whose argument vector matches the command the atomic ran, tagged with the `exec` key — the same syscall-auditing rule Part 9 built to catch every process launch on that endpoint.
8. Confirm both events also appear in the Part 8 SIEM, not just in the local Sysmon/`auditd` output. If a step 5 or step 7 event exists locally but never reaches the SIEM, the break is in the forwarding path built in Parts 9–10, not in the atomic test itself.

Validation Test Setup: *T1082 confirmed listed via `-ShowDetailsBrief` (\$4) against both a Windows target (Sysmon deployed per Part 10) and a Linux target (`auditd`'s `exec` key active per Part 9), both forwarding to the Part 8 SIEM. **Action:** `Invoke-AtomicTest T1082 -Session $winSession` and `Invoke-AtomicTest T1082 -Session $linuxSession`. **Expected result:** A Sysmon Event ID 1 (Process Create) entry for the Windows target and a `type=EXECVE` record tagged `key="exec"` for the Linux target, both visible in the SIEM within roughly a minute, both with a command line matching exactly what the atomic's YAML says it runs — this is the whole point of the exercise: known cause, known effect, confirmed in your own telemetry rather than assumed from the tool's description.*

Engineering Reality *`Invoke-AtomicTest`'s documentation describes running a test as executing the technique; in practice, the cmdlet considers a test "successful" the moment the command exits without throwing a PowerShell error — it does not independently verify that the technique's intended effect actually happened on the target. A discovery command that runs but returns empty output (a permissions issue, a missing binary, a shell that doesn't support the syntax the atomic assumes) still reports as executed with no red flag anywhere in the invoker's own output. This is exactly why step 5 and step 7 above check the SIEM and `ausearch` output directly instead of trusting the console output that the test "ran" — the atomic framework tells you a command was issued, not that the technique's evidence landed where your detection expects it.*

7. Mapping atomic runs back to the detections you already built

[CONCEPT] Running an atomic and seeing an event in the SIEM is not the same thing as having validated a detection. §6's Sysmon Event ID 1 and `auditd` `exec`-keyed records are raw telemetry — proof the pipeline works, not proof any rule fired on it. Turning that raw telemetry into an actual validated detection is explicitly *Detection Engineering Handbook V2*'s job, and it owns two specific, named pieces of this:

- **Detection Testing** (its Part 37) is where "fires correctly, fires for the right reason, doesn't fire on normal activity" gets defined rigorously — an atomic run is one concrete input to that process, not a substitute for it. Running T1082 once and seeing an event is not the same as confirming a

discovery-activity detection rule actually matches that event and doesn't also match every legitimate `systeminfo` call your help-desk scripts make.

- **Detection Coverage** (its Part 41) is the ATT&CK coverage model that explicitly rejects "detection exists on paper" as equivalent to "detection is tested" or "detection was recently validated" — an atomic run against your own lab endpoint is precisely the mechanism that moves a technique from "detection" to "tested" on that coverage model, and re-running the same atomic periodically is what keeps it at "recently validated" instead of quietly drifting back down as your rule set, parser, or SIEM version changes underneath it.

For the telemetry-interpretation layer underneath both of those — what a specific Sysmon field or `auditd` key actually means — *Detection Engineering Handbook V2's* Sysmon Detection Engineering content (its Part 9) and its Telemetry Engineering I content (its Part 3, which owns `auditd`) are the parts to read next, not this one; this part's job stops at "the known-cause event landed in the SIEM, here's exactly what caused it."

8. Sizing the simulation segment

[COST/RESOURCE] The operator control host is a new role this book hasn't sized yet — it isn't a target endpoint like Parts 9–10's hosts, and it isn't a service host like Part 7's DNS container. Table 18.3 sizes it alongside the target roles it drives, reusing Parts 9 and 10's own numbers for the targets rather than re-deriving them.

Table 18.3 — Attack-simulation segment sizing by role. The table below supports deciding how much additional hardware this part's build adds on top of what Parts 9–10 already committed.

Role	Platform	Min RAM	Min disk	Notes
Operator control host	Windows or Linux VM, PowerShell 7 (<code>pwsh</code>)	2GB	10GB	Runs <code>Invoke-AtomicRedTeam</code> ; never reused as a general-purpose lab jump box (§3)
Windows target endpoint	Reused from Part 10	2–4GB	40GB+	Snapshot immediately before every session, not just the first one
Linux target endpoint	Reused from Part 9	1–2GB	10GB+	<code>auditd</code> 's <code>exec</code> key (Part 9 §5.2) must already be active before §6's lab has anything to check

Resource Reality *The operator control host itself is light — 2GB of RAM comfortably runs PowerShell 7 and the atomics repository, since the host does almost no local work beyond issuing remote commands. The real cost this part adds isn't the control host; it's the snapshot storage the Safety Gate's checklist item 4 requires. A fresh snapshot before every session, on every target endpoint, adds up fast if you're running several atomics a week — budget disk headroom for snapshot churn the same way Part 20's maintenance content already tells you to, or the snapshot discipline this part depends on for safety becomes the first thing skipped when disk space gets tight.*

9. Cleanup discipline and the limits of automation

[TROUBLESHOOTING] §5 already named the specific failure mode — a `cleanup_command` that exits cleanly without actually reversing the artifact. The practical habit that catches this: after every atomic in the "requires cleanup" or "destructive" tier, check the artifact directly rather than trusting the invoker's exit status. For a registry-run-key atomic, that means actually querying the key, not just running `-Cleanup` and moving on. For a scheduled-task or cron-based persistence atomic, that means listing scheduled tasks or crontab entries afterward, not assuming the cleanup step's silence means success.

Lab Note *Snapshot every target immediately before a session, the same discipline Part 9's `auditd` work and Part 20's maintenance content both already teach — and for this part specifically, snapshot before the session, not after you've decided a given atomic "looks safe enough to skip it." The atomics in the read-only tier (Table 18.2) are the only ones where skipping the snapshot is even defensible, and even then, a 5-second snapshot is cheap insurance against being wrong about which tier a given test actually belongs to.*

[SAFETY] If a cleanup step fails and you can't manually reverse the artifact with confidence, the correct move is reverting the target from the pre-session snapshot, not continuing to the next atomic on a target you're no longer sure is in a known state. A lab endpoint with an unverified leftover persistence mechanism from a previous session is exactly the kind of drift that makes a later exercise's results unreliable — you won't know if a detection fired because of the atomic you just ran or a stale artifact from three sessions ago.

10. What this tooling doesn't teach you

Blind Spot *A single atomic validates one technique, in isolation, with no adversary decision-making behind it — it tells you whether your Sysmon config or `auditd` rule set produces the right event when a known command runs, and nothing about how a real intrusion actually unfolds. Real attackers chain techniques toward an objective, adapt when a step fails, and actively try to avoid detection; nothing in this part teaches EDR evasion, obfuscation, or the judgment calls a real adversary makes under uncertainty, and it shouldn't — that's a different skill than the one this book is building, and BOOK-INDEX.md is explicit that this part stops at generating known-cause telemetry, not simulating a real intrusion end to end. Part 19's purple-team drills get closer to that by pairing a sequence of atomics with a specific hunt, but even that stays a scripted, known sequence — treat every green checkmark from this part's exercises as "this one technique in isolation is covered," not "this lab would catch a real attacker."*

What Would Change My Mind This part recommends the published `atomic-red-team` community library over hand-writing your own test scripts, on the grounds that the community repository is broader, already ATT&CK-mapped, and ships cleanup commands you'd otherwise have to write yourself. If a reader's specific detection depends on a technique variant the published atomics don't cover — a particular LOLBin argument pattern, a specific encoding scheme — that specific gap, not a general "write your own instead," would be the concrete reason to author a custom atomic YAML file rather than defaulting to the library wholesale.

11. Where this telemetry goes next

[CONCEPT] This part's job ends the moment a known atomic run produces the telemetry it should, forwarded and searchable in the Part 8 SIEM (§6). What a reader does with that telemetry next belongs to the other three NESHBOY volumes, each picking it up at a different point:

- **Detection Engineering Handbook V2, Part 37 (Detection Testing)** and **Part 41 (Detection Coverage)** own turning a single atomic run into an actual validated, tracked detection-coverage entry — the distinction §7 already draws between "an event landed in the SIEM" and "a detection was tested and is currently marked recently validated."
- **Detection Engineering Handbook V2, Part 9 (Sysmon Detection Engineering)** and **Part 3 (Telemetry Engineering I, which owns `auditd`)** own reading the specific fields this part's exercises produce — what a given Sysmon field or `EXECVE` argument vector actually means beyond "an event fired."
- **SOC Playbook Handbook's Playbook Library** owns what an analyst does when a detection this part helped validate fires for real later — separating a genuine incident from a known validation exercise is a triage skill that library covers, this part doesn't.
- **SOC Manager's Operating Handbook, Part 8 (assessment design)** and **Part 11 (onboarding and ramp-up)** are where a bounded, scoped atomic run stops being a solo exercise and becomes a repeatable team drill or a new analyst's supervised first triage of a known-cause alert — this part supplies the safe, reversible action; that book supplies the program structure around running it with other people.

Part 19 (Generating Practice Telemetry and Purple-Team Drills), later in this book, is the direct continuation: it takes this part's single-atomic validation pattern and sequences several atomics into a scenario paired with a specific detection from *Detection Engineering Handbook V2* and a specific playbook from *SOC Playbook Handbook*, closing the loop this part deliberately leaves open.

Cross-references: Part 1 (the book's ownership/isolation rule, restated in full above rather than assumed read), Part 4 and Appendix A4 (the isolation controls this part's Safety Gate depends on), Part 6 (perimeter firewall enforcing the egress-only allowlist in §3 and §5), Part 8 (SIEM ingest target for both endpoints' telemetry), Part 9 (Linux/`auditd` target endpoint and the `exec` key used in §6), Part 10 (Windows/Sysmon target endpoint used in §6), Part 15 (honeynet decoys — never a valid target for this part's tooling, since they exist to catch inbound traffic, not receive outbound simulation), Part 19 (purple-team drills that sequence this part's single-atomic pattern into a scenario), Part 20 (snapshot

discipline this part's Safety Gate and §9 both depend on), Appendix A3 (build checklist/config snippet reference); *Detection Engineering Handbook V2* Part 3 (Telemetry Engineering I — `auditd` field semantics), Part 9 (Sysmon Detection Engineering), Part 37 (Detection Testing), Part 41 (Detection Coverage); *SOC Playbook Handbook* Playbook Library (triage procedure distinguishing a real incident from a known validation event); *SOC Manager's Operating Handbook* Part 8 (assessment design) and Part 11 (onboarding/ramp-up simulation-based training).

neshbooy

Part 19 — Generating Practice Telemetry and Purple-Team Drills

Why this part exists

Part 18 taught you to run one scoped Atomic Red Team technique against a lab endpoint and confirm it left a trace. That's the right first move, but it teaches a narrower skill than the rest of this book is building toward. A real SOC analyst's week doesn't look like one dramatic technique against total silence — it looks like finding a real signal inside a constant low hum of ordinary activity, and then working a chain of related events, not a single isolated action. This part adds three things a one-shot Atomic Red Team run doesn't give you: a background noise generator, so your telemetry has a realistic baseline to stand out against; a scenario-replay approach that chains several techniques into one narrative attack path instead of a single atomic test; and a purple-team drill format that explicitly pairs a simulated action with a named detection from *Detection Engineering Handbook V2* and a named response procedure from *SOC Playbook Handbook*. This is the part where everything built across the first eighteen parts stops being infrastructure and starts being a practice loop you actually run, on a schedule, with a record of what happened.

Safety Gate *Everything in this part assumes the same isolated lab/victim segment from Part 4 that Part 18's attack-simulation content depends on — nothing here is scoped to run against a system you don't own, and that includes the noise generator in §2. A one-shot Atomic Red Team test is easy to reason about because it runs once, in front of you. A cron-scheduled noise generator or a scenario-replay script runs unattended, repeatedly, whether or not you're watching — which means a stale IP address, a copy-pasted hostname, or a scenario script someone hands you with a real target baked in becomes a standing, scheduled attack tool instead of a one-time mistake. Before you put anything from this part on a cron schedule, confirm every target it touches resolves to a host on the isolated segment built in Part 4 and firewalled in Part 6, run the Appendix A4 pre-flight checklist if it's been more than a few weeks since you last verified isolation on this segment (Appendix A4 is not yet authored — see [README.md](#); until it exists, re-run Part 4 §6's isolation checks instead), and never let a noise or scenario script reach outside that segment — not even for something as apparently harmless as a real DNS lookup against the open internet.*

1. What changes in this part, and what's still out of scope

[CONCEPT] Three things distinguish this part from Part 18, and none of them is "a different tool." First, background noise generation runs continuously, at low volume, in the background, rather than as a single deliberate action you watch happen. Second, scenario replay chains multiple techniques — reconnaissance, then execution, then persistence, then discovery — into one narrative attack path, instead of testing one technique in isolation. Third, the purple-team drill format this part builds explicitly names which detection and which playbook a given simulated action is supposed to exercise,

before you run it, so the exercise has a pass/fail condition instead of just "something happened, and I looked at the logs afterward."

This part still doesn't teach offensive technique development, real-world red-teaming, or how to write detection logic — those stay exactly where Part 18 and *Detection Engineering Handbook V2* left them. It also doesn't teach staffing a purple-team program; that's the *SOC Manager's Operating Handbook's* job, cited by part number in §7. What follows assumes you've already completed Part 18's attack-simulation build and have at least one working Atomic Red Team-style tool pointed at a lab endpoint you own.

2. Background noise generation

2.1 Why realistic baseline noise matters

[CONCEPT] A detection that only ever gets tested against a perfectly quiet SIEM index has never actually been tested. Real production telemetry has a constant background rate of `sudo` invocations, DNS lookups that resolve to nothing, SSH logins, and scheduled jobs — and a threshold-based detection (more than a stated number of failed logins in a window, more than a stated NXDOMAIN rate per host) is only as good as its understanding of what "normal" looks like against that background. A lab that goes from silence straight to one dramatic Part 18 technique never forces you to answer the harder question: does this detection also fire on the ordinary noise a real endpoint generates every day, or does it only work because your lab was empty when you tested it? *Detection Engineering Handbook V2's* baselining content (its Part 31) is written assuming a telemetry stream with some real variance in it — this section is where that variance comes from if your lab doesn't otherwise generate any.

2.2 Sizing and scheduling a noise generator

[SAFETY] A noise generator earns its name from what it does, not what it's aimed at — every action it takes has to land on a host you own, inside the segment from Part 4. The version built below never leaves that segment: no external DNS lookups, no requests to real internet hosts, nothing that would still function correctly if a stale IP address slipped into the script. It runs entirely against the Part 9/10 endpoints, and optionally the Part 7 internal DNS resolver, that you already control.

[COST/RESOURCE] Table 19.1 sizes the noise patterns this section builds against the telemetry they exercise and what part of the book already ingests them — nothing here needs new forwarding configuration, since it all rides pipelines Parts 7, 9, and 13 already built.

Table 19.1 — Background noise patterns and the telemetry they exercise. The table below supports deciding which noise patterns are worth scripting for your specific detection-tuning goals, rather than scripting all of them by default.

Noise pattern	Telemetry produced	Ingested via	Why it matters
<code>sudo -l</code> privilege check	<code>exec</code> -keyed auditd events (the <code>sudo</code> process launch itself — <code>sudo -l</code> only reads <code>/etc/sudoers</code> , so Part 9's <code>-p wa identity/sudoers</code> watches never fire on it)	Part 9 forwarding pipeline	Baseline for what normal <code>sudo</code> activity looks like on this host before a real anomaly-detection rule can flag a deviation
Mixed valid/NXDOMAIN DNS queries	<code>dns.log</code> query/response pairs, or Pi-hole query log entries	Part 13 Zeek forwarding, or Part 7 Pi-hole	Baseline NXDOMAIN rate — a DNS-tunneling or domain-generation-algorithm detection needs a normal rate to compare against
Internal SSH login, disposable account	Auth-success events, <code>exec</code> -keyed process launches	Part 9 forwarding pipeline	Baseline authentication volume for a brute-force or impossible-travel detection
Internal HTTP GET to a lab-hosted service	<code>http.log</code> request records	Part 13 Zeek forwarding	Baseline request rate/URI shape for a web-scanning detection
Scheduled task creation and removal	<code>exec</code> -keyed process events	Part 9 forwarding pipeline	Baseline scheduled-task activity — the same shape T1053.003 (Scheduled Task/Job: Cron) persistence would leave, covered further in §3

Resource Reality Even a light noise generator running every few minutes on one endpoint adds up over a month — a handful of auditd `exec` records, a couple of DNS queries, and one SSH session per invocation is a few hundred extra kilobytes of raw log per day, not a meaningful load on its own. The real cost shows up if you script noise generation on every endpoint in the lab simultaneously and never stagger the schedule: a burst of identical cron jobs firing in the same second across five endpoints produces a synchronized spike that itself looks like coordinated activity in a SIEM dashboard. Stagger cron minutes per host, and budget the extra volume against Part 21's retention plan rather than assuming it's negligible just because each individual run is small.

2.3 A worked noise-generator script

[SETUP] The script below targets a Part 9 Linux endpoint and reuses commands already covered in that part — nothing here requires new software, only a cron entry and a disposable, non-privileged test account. Create `labnoise` the same way Part 9's §8 exercise creates `labtestuser`, authorize an SSH key from `labnoise` to itself on the target endpoint (`BatchMode=yes` fails closed instead of prompting for a password when cron has no terminal to prompt on), then save the script below as `/usr/local/sbin/lab-noise.sh` and mark it executable with `chmod +x`.

```
#!/bin/bash
# CONCEPTUAL SAMPLE – illustrative cron-driven noise generator using invented
# lab hostnames and addresses; substitute your own Part 9/10 endpoint names
# and confirm every target below resolves to a host you own on the isolated
# lab segment before scheduling this.
# /usr/local/sbin/lab-noise.sh

# Baseline privilege-use noise (feeds Part 9's exec key – this crontab entry
# already runs as labnoise, and sudo -l only reads /etc/sudoers, so it never
# trips the identity/sudoers watches, which are write-only, -p wa)
sudo -l >/dev/null 2>&1

# Baseline internal DNS noise: one real internal name, one deliberately
# nonexistent name, so the resolver's own logs show a realistic mix.
# 10.10.20.0/24 is the Endpoint VLAN from Part 6's Table 6.3; substitute
# your own resolver and endpoint addresses from that table.
dig +short db01.lab.internal @10.10.20.10 >/dev/null
dig +short "doesnotexist-$RANDOM.lab.internal" @10.10.20.10 >/dev/null

# Baseline internal auth noise: a low-volume SSH login against a second
# owned lab endpoint, using a disposable, non-privileged test account
ssh -o BatchMode=yes -o ConnectTimeout=3 labnoise@10.10.20.101 'whoami' >/dev/null 2>&1

# Baseline internal HTTP noise against an internally hosted lab service
curl -s -o /dev/null http://10.10.20.102/
```

Add it to cron with a staggered minute offset per endpoint, rather than a round number every host would share:

```
# /etc/cron.d/lab-noise – staggered at :07 past every 15 minutes on this
# specific host; use a different offset on every other endpoint running
# this script, per the Resource Reality note above.
7,22,37,52 * * * * labnoise /usr/local/sbin/lab-noise.sh
```

Validation Test Setup: The script and cron entry above deployed on a Part 9 endpoint, with the auditd and Zeek forwarding pipelines from Parts 9 and 13 already confirmed working. **Action:** Wait for one scheduled run, then search the Part 8 SIEM for events from this host in the few minutes around the scheduled time. **Expected result:** A low-volume mix of `exec`-keyed auditd events (the `sudo`, `dig`, `ssh`, and `curl` process launches themselves), one DNS query pair (one resolved, one NXDOMAIN), one SSH auth-success event, and one `http.Log` request record — all low-severity, none of it triggering any detection you've built so far. If a detection does fire against this baseline noise, that detection's threshold is tuned too tight for real endpoint behavior, and this is exactly the gap §7 sends back to Detection Engineering Handbook V2 to fix.

Lab Note Keep the noise generator running for at least a full week before you layer a scripted drill on top of it. A detection tuned against zero seconds of baseline noise will look like it's working the first time you test it and then generate false positives the first ordinary Tuesday it runs against real background activity — running the noise generator first is what surfaces that problem before a drill wastes your time chasing it.

3. Scenario replay: chaining techniques into an attack path

3.1 From atomic tests to a kill-chain-shaped sequence

[CONCEPT] A single Atomic Red Team test answers "does my telemetry pipeline capture this one technique." A scenario replay answers a different, harder question: "does my detection stack tell a coherent story across a sequence of related actions, the way a real intrusion actually unfolds." Table 19.2 lays out a worked five-stage scenario built entirely from techniques Part 18 already covers running one at a time — the only thing new here is the sequencing and the delay between stages, which matters because a real attacker doesn't execute five techniques in the same second.

Table 19.2 — A worked five-stage scenario-replay sequence. The table below supports planning a scenario that exercises multiple detections in sequence rather than testing techniques one at a time with no narrative connecting them.

Stage	Tactic	Technique (Part 18 test)	Runs against
1	TA0043 (Reconnaissance)	T1595 (Active Scanning) — a scoped port scan against an owned endpoint	Part 9/10 endpoint
2	TA0001 (Initial Access)	T1190 (Exploit Public-Facing Application) — a scripted exploit attempt against a deliberately vulnerable owned service	Part 9/10 endpoint
3	TA0002 (Execution)	T1059 (Command and Scripting Interpreter)	Part 9/10 endpoint
4	TA0003 (Persistence)	T1053.003 (Scheduled Task/Job: Cron)	Part 9 endpoint
5	TA0007 (Discovery)	T1083 (File and Directory Discovery)	Part 9/10 endpoint

[SAFETY] Every stage above still has to run against a host you own on the isolated Part 4 segment — chaining techniques together doesn't relax the ownership rule, it just means a mistake in one stage's target now propagates into four more stages before you notice it. Check the target host for every stage before the first run, not just the first stage.

3.2 Marking ground truth so the SIEM knows what you did, and when

[SETUP] A scenario replay is only useful for measuring detection performance if you can prove exactly when each stage ran, independent of whatever the detection stack itself claims to have seen. The simplest way to do that on a Linux endpoint is a `logger` call immediately before and after each stage,

riding the same journald/rsyslog forwarding path Part 9 already built — no new configuration is needed, since these lines carry the same tag Part 9's forwarding rules already ship to the SIEM.

```
# CONCEPTUAL SAMPLE – ground-truth markers bracketing one scenario stage;
# substitute your own scenario identifier and technique ID per stage.
logger -t purple_drill "scenario=2026-09-15-scenario01 stage=1 technique=T1595 status=start"
# ... the Part 18 Atomic Red Team test for this stage runs here ...
logger -t purple_drill "scenario=2026-09-15-scenario01 stage=1 technique=T1595 status=end"
```

Confirm the markers arrive in the SIEM before running the full five-stage sequence — search for `purple_drill` shortly after running one test marker by hand, and confirm both the `start` and `end` lines appear with a timestamp close to when you ran them. Once that's confirmed, the same pattern brackets every stage in Table 19.2, giving you a searchable, timestamped record of exactly what ran and when, independent of any detection firing or not firing.

Engineering Reality *It's tempting to skip the marker step and just trust the timestamp on whatever telemetry the technique itself generates — an auditd `exec` record, a Suricata alert. In practice, forwarding delay (a shipper's bulk-send interval, a few seconds of clock skew between the endpoint and the SIEM) means the telemetry's own timestamp and your intended action time can drift by several seconds, which is exactly the margin that matters when you're trying to measure how fast a detection actually fired relative to when the technique ran. The `Logger` marker isn't redundant with the technique's own telemetry — it's the independent clock you compare the technique's telemetry against.*

4. The purple-team drill format

4.1 Pairing a simulated action with a named detection and playbook

[CONCEPT] A purple-team drill, in this book's sense, is a scenario stage (or a single Part 18 technique) run with three things decided before you press go: which detection is supposed to fire, which playbook an analyst is supposed to follow if it does, and what "the drill passed" actually means. Figure 19.1 lays out that sequence as a repeatable lifecycle — the same eight steps apply whether the drill is one Part 18 technique or a full Table 19.2 scenario.

flowchart TD

```
A["Pick a technique\n(MITRE ATT&CK ID, Part 18 test)"] --> B["Map to a named detection\n(Detection Engineering Handbook V2)"]
B --> C["Map to a named playbook\n(SOC Playbook Handbook)"]
C --> D["Inject a ground-truth marker\n(logger, per Section 3.2)"]
D --> E["Execute the technique\n(isolated lab segment only)"]
E --> F["Confirm telemetry arrived\n(Part 8 SIEM search)"]
F --> G{"Did the mapped\ndetection fire?"}
G -->|"Yes"| H["Run the mapped playbook\nas a live triage rehearsal"]
G -->|"No"| I["Log a detection gap\n(Detection Engineering Handbook V2)"]
H --> J["Record result in the drill log\n(Section 6)"]
I --> J
J --> K["Feed results into a program\n(SOC Manager's Operating Handbook)"]
```

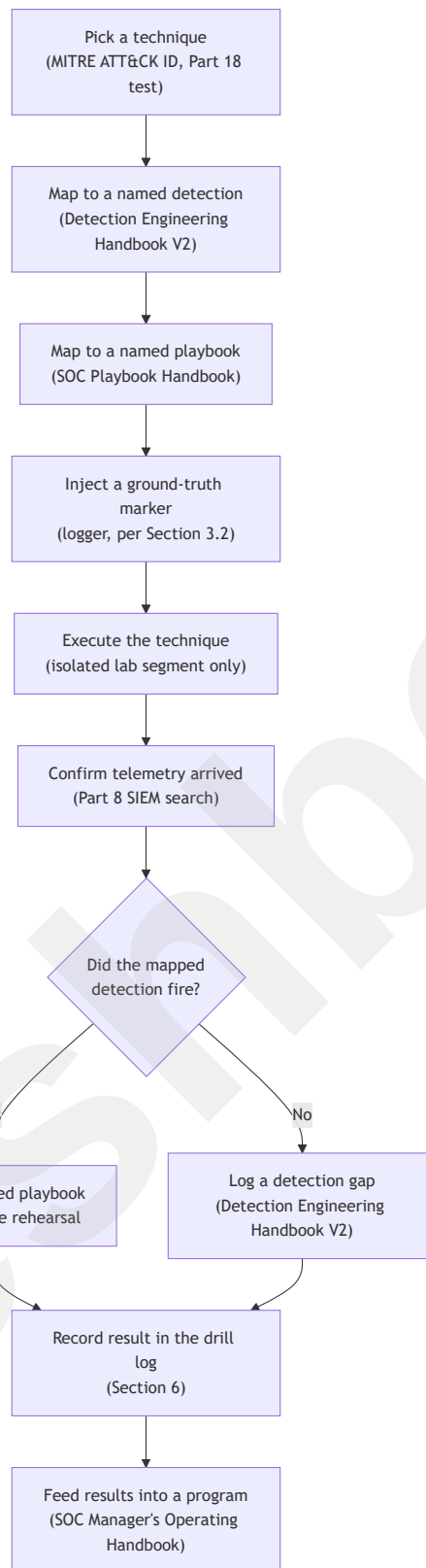


Figure 19.1 — Purple-team drill lifecycle. *CONCEPTUAL.* Illustrates the repeatable sequence this part's drill format follows, from picking a technique through recording a result and feeding it into a program-level assessment cycle. This is a process diagram of the intended workflow, not a capture from a specific tool's own interface — every step maps to a section in this part or a named part in another NESHBOY volume. Diagram ID **FIG-19-01**. Table 19.3 works through three concrete drills that follow this

same sequence, spanning a scripted action, a scripted persistence mechanism, and one drill this book doesn't have to script at all.

Table 19.3 — Three worked purple-team drills, mapped end to end. The table below supports designing a drill with a stated pass/fail condition, rather than running a technique and deciding afterward whether anything relevant happened.

Simulated action (MITRE ID)	Telemetry source	Detection Engineering Handbook V2 reference	SOC Playbook Handbook reference
T1110.001 (Password Guessing) — scripted SSH brute force against an owned Part 9 endpoint	Part 9 auditd auth/ <code>exec</code> telemetry; Part 14 Suricata if a matching signature fires	Correlation/baselining content, Parts 22–33 (authentication-volume correlation)	Playbook Library's credential-access/brute-force triage playbook
T1053.003 (Scheduled Task/Job: Cron) — scripted persistence via a cron entry on an owned endpoint	Part 9 auditd <code>exec</code> telemetry, plus a file watch on <code>/etc/cron.d</code>	Parts 22–33 (persistence-mechanism correlation)	Playbook Library's persistence-mechanism triage playbook
T1595/T1046/T1083/T1190 (naturally occurring, unscripted) — real internet scanning against the Part 15 honeynet	Part 15 honeynet session correlation	Parts 34–36 (threat hunting over captured session data)	Playbook Library's reconnaissance/scanning triage playbook

The third row is worth sitting with, because it's not a drill you script at all.

4.2 Real ground truth for free: the honeynet as an unscripted drill

[CONCEPT] If Part 15's honeynet is already built and running, it produces something a scripted drill can't: real, unsolicited attacker behavior against a target that exists specifically to attract it, already correlated and MITRE-tagged by the honeynet platform's own logic. You don't have to simulate T1595 (Active Scanning), T1046 (Network Service Discovery), T1083 (File and Directory Discovery), or T1190 (Exploit Public-Facing Application) against a honeynet — internet background scanning does it continuously, for free, and it's real attacker intent rather than a rehearsed script.

session_id	source_ip	status	mitre_techniques_json
d48b795e47c04c6fbb456b17ddd219b9	16.5.0.236	possible_success	["T1595", "T1046", "T1083", "T1190"]
5322cb1f62614593a6dc1e239f5209fc	198.235.24.116	possible_success	["T1595", "T1046", "T1083", "T1190"]
ba0ab1f38e11455f83e48298d7b45c1e	45.156.128.45	exploit_attempt	["T1595", "T1046", "T1190"]

Figure 19.2 — Real honeynet session correlation, unscripted. REAL LAB EXAMPLE. Excerpted from the author's own running CT103 honeynet correlation database (`attack_sessions` table), captured 2026-09-15. Each row is a real, unsolicited internet session against a sacrificial decoy address (`10.99.99.x`, not shown here — see Part 15 for the full addressing scheme), already tagged with MITRE technique IDs by the platform's own correlation logic and flagged `possible_success` when an exploit-shaped request was followed by continued activity from the same source. Source IPs shown are real external internet hosts; disclosing them carries no meaningful risk since the targets are sacrificial decoys by design.

Blind Spot A honeynet's real attacker sessions are genuine ground truth about what happens on the open internet, but they're not a substitute for a scripted drill, because you don't control when they happen or exactly what triggered the platform's `possible_success` judgment. Table 19.3's first two rows give you an exact start time from a `logger` marker and a known technique with no ambiguity about what ran; the honeynet row gives you real adversary behavior with no independent ground-truth timestamp beyond the platform's own `first_seen` value — you're trusting the same system you're trying to validate to also tell you when the test started. Use honeynet captures to hunt against and to validate that your correlation logic makes sensible judgment calls on real data; use scripted drills, not honeynet captures, when the thing you're actually measuring is detection latency.

5. Hands-on lab: running one drill end to end

[HANDS-ON LAB] Goal: run Table 19.3's first drill — a scripted SSH brute-force attempt against an owned Part 9 endpoint — and record whether the mapped detection fired and whether the mapped playbook's steps matched what you actually needed to do.

1. On the target Part 9 endpoint, confirm a disposable test account exists (reuse `labtestuser` from Part 9's §8 exercise, or create a new one).
2. From a second owned lab endpoint, inject the start marker: `logger -t purple_drill "scenario=2026-09-15-bruteforce stage=1 technique=T1110.001 status=start"`.
3. Run a bounded, scripted set of failed SSH logins against the target — 20 attempts within 60 seconds is enough to exceed most reasonable thresholds without generating an unmanageable log volume.
4. Inject the stop marker with `status=end` in place of `status=start`.
5. Search the Part 8 SIEM for the failed-auth burst on the target endpoint, and confirm its timestamps fall between the two markers.
6. Check whether a correlation rule from *Detection Engineering Handbook V2*'s Parts 22–33 content, if you've built one, fired against this burst.
7. If it fired, open the Playbook Library's credential-access/brute-force triage playbook and walk its steps against this drill as a rehearsal, not just a read-through — note any step that assumed a field or a tool you don't actually have in this lab.
8. If nothing fired, record that as a detection gap rather than treating a quiet drill as a successful one.

Validation Test Setup: Steps 1–4 above completed, with the Part 9 forwarding pipeline confirmed working. **Action:** Search the SIEM for auth-failure events on the target endpoint between the `start` and `end` markers. **Expected result:** A visible burst of failed-authentication events clustered inside that window, distinguishable from the low-volume baseline noise §2's generator produces on the same host — if the burst isn't visibly denser than the baseline, the two are indistinguishable to any threshold-based detection and step 6 will fail for a reason that has nothing to do with the detection's logic.

6. Recording drill results: the drill log

[SETUP] A drill that isn't recorded doesn't compound into anything — the value of running the same three or four drills every few months comes from comparing this run against the last one, not from any single run in isolation. Table 19.4 is a minimal template; keep it as a spreadsheet or a SIEM-indexed record, whichever makes it easier to actually update after a drill instead of intending to and not.

Table 19.4 — Drill log template. (CONCEPTUAL SAMPLE) The table below supports tracking drill outcomes over time rather than treating each drill as a one-off exercise with no record afterward.

Date	Scenario ID	MITRE ID(s)	Detection expected	Detection fired?	Time to detect	Playbook followed?	Outcome/notes
2026-09-15	2026-09-15-bruteforce	T1110.001	Auth-failure correlation rule	Yes	42s	Yes	Playbook step 3 assumed a field this lab's SIEM doesn't populate — flagged for playbook revision
—	—	—	—	—	—	—	—

What Would Change My Mind This part recommends running the noise generator (§2) for at least a full week before layering a scripted drill on top of it, on the grounds that a detection tuned against zero baseline noise will misrepresent its own false-positive rate. If a reader's detection logic is built entirely on absolute thresholds with no baselining component at all — a fixed "more than 10 failed logins in 5 minutes" rule with no comparison to recent history — that specific rule design wouldn't benefit from a noise baseline the way a statistical or adaptive-threshold rule would, and this recommendation would need a caveat for that case rather than a blanket week-long wait before every drill.

7. Where this closes the loop

[CONCEPT] Everything in this part exists to produce two things: telemetry with a realistic shape, and a record of whether a specific detection and a specific response procedure actually worked against it. Neither of those is useful sitting in this book — each one has a specific next step in a specific other volume.

- **Feed detection gaps back to *Detection Engineering Handbook V2*.** Every row in Table 19.4 where "Detection fired?" is No is a concrete input to that book's correlation and baselining content (its Parts 22–33) — not a vague "improve detections" note, but a specific technique, a specific telemetry source, and a specific timestamp window to build a rule against. Where a drill's target telemetry looks unusual or thin, that book's threat-hunting content (its Parts 34–36) is where you go hunting in it before assuming a rule is the right fix.
- **Rehearse *SOC Playbook Handbook's* playbooks against real drill conditions, not just read them.** Step 7 of §5's hands-on lab is the point of this cross-reference: a playbook step that assumes a log field, a tool, or a lookup your lab doesn't actually have is a gap in the playbook, not a gap in your lab, and it's far cheaper to find that during a scheduled drill than during a real incident. Record every mismatch in the drill log's outcome/notes column and route it back as a documented playbook revision.
- **Turn a repeatable drill log into a program, per the *SOC Manager's Operating Handbook*.** That book's Part 8 (assessment design) is where a series of Table 19.4 entries becomes a structured, graded exercise instead of an ad hoc one, and its Part 11 (onboarding/ramp-up) is where the exact same drill format becomes a way to bring a new analyst up to speed against a lab environment instead of a live production one. This part builds the drill; that book builds the program the drill lives inside of.
- **Feed dashboards.** Part 17's dashboard content can chart drill outcomes over time (detection-fire rate, time-to-detect trend) alongside the raw telemetry volume Parts 8–16 already produce — a drill log is itself a data source worth visualizing, not just a spreadsheet you check manually.

None of this closes the loop if §2–§6 never actually get run. A noise generator that's built but never scheduled, or a drill format that's designed but never executed against a real technique, leaves this part exactly as theoretical as the detections it was meant to validate.

Cross-references: Part 4 (network isolation this part's Safety Gate depends on), Part 6 (perimeter firewall enforcing segment boundaries), Part 7 (internal DNS noise target), Part 8 (SIEM ingest and search target throughout), Part 9 (Linux endpoint telemetry and forwarding pipeline this part's noise generator and drill both reuse), Part 10 (Windows endpoint equivalents for scenario stages run there), Part 13 (Zeek `http.log / dns.log` telemetry exercised by noise generation), Part 14 (Suricata alerts a scripted drill may trigger), Part 15 (honeynet correlation as unscripted ground truth), Part 17 (dashboards charting drill outcomes), Part 18 (the attack-simulation tooling this part's scenarios and drills are built from), Part 21 (retention planning for noise-generator log volume), Appendix A3 (reusable rule/config baselines this part's telemetry feeds), Appendix A4 (isolation pre-flight checklist referenced in this part's Safety Gate); *Detection Engineering Handbook V2*, Parts 22–33 (correlation/baselining that consumes this part's drill results) and Parts 34–36 (threat hunting over honeynet and drill telemetry); *SOC Playbook Handbook's* Playbook Library (the triage procedures rehearsed in §5 and §7); *SOC Manager's Operating Handbook*, Part 8 (assessment design) and Part 11 (onboarding/ramp-up) for turning this part's drill format into a repeatable program.

Section G — Operating, Scaling & Closing

neshbooy

Part 20 — Lab Maintenance, Patching, and Snapshot/Backup Strategy

Why this part exists

Every part before this one taught you to build something: a hypervisor, a firewall, a SIEM, endpoints, sensors, honeypots, feeds, dashboards, and attack-simulation tooling. This part is about what happens to all of that six months later, when a security advisory drops for a package half your infrastructure runs, or a config edit you're not sure about is one `systemctl restart` away from finding out the hard way. A home lab that gets built once and never maintained decays in two opposite directions at the same time. The infrastructure you rely on to be trustworthy — the firewall enforcing every Safety Gate in this book, the SIEM holding the evidence you're building a practice habit on — goes stale and quietly picks up real vulnerabilities you never intended. Meanwhile the practice targets you deliberately left vulnerable — a honeypot, a pinned old package staged for an exploit exercise — get patched out from under you by the exact same automated update mechanism that's protecting everything else.

This part is the discipline that keeps both halves of that tension resolved on purpose instead of by accident: a patch policy that treats "infrastructure" and "intentionally vulnerable target" as two different tiers with two different rules, and a snapshot/backup strategy that makes every risky change — a patch, a config edit, an attack-simulation run — reversible in minutes instead of costing you the whole lab. It does not cover storage sizing or retention policy for the data those snapshots and backups themselves consume; that's Part 21's job, cited where it matters below.

Safety Gate *Two things must hold before you touch anything in this part. First, any automated patching mechanism you enable (`unattended-upgrades`, a config-management run, a scheduled `apt upgrade` cron job) must be scoped to the infrastructure tier only — never let it run unscoped against a honeypot or deliberately vulnerable target host, or you will silently patch away the exact vulnerability that host exists to expose (see the Build Autopsy in §3). Second, any off-host backup destination and any patch-mirror or update source this part has you configure must route through the same perimeter-firewall egress rules Part 4 and Part 6 already built — a backup job or a package-update pull is not an exception to a segment's default-deny egress, and it must never be the reason a honeynet or attack-simulation segment (Parts 15 and 18) picks up a broader route than its own Safety Gate already permits. Before scheduling a single automated patch job or backup cron entry, confirm both against the Appendix A4 pre-flight checklist.*

1. Two patch regimes, one lab

[CONCEPT] Almost everything you've built in this book falls cleanly into one of two tiers, and the two tiers want opposite patch behavior. The **infrastructure tier** — the hypervisor (Part 5), the perimeter firewall (Part 6), internal DNS (Part 7), the SIEM (Part 8), most Linux and Windows lab endpoints (Parts 9

and 10), and the Zeek/Suricata sensor host (Parts 12–14) — should be patched roughly the way you'd patch anything you actually depend on: promptly, on a predictable cadence, because an unpatched piece of infrastructure is just a liability with no offsetting benefit. The **target tier** — honeypots and decoy services (Part 15) and any host deliberately left at a specific vulnerable version for an attack-simulation exercise (Part 18) — is the opposite case: its entire value comes from being a specific, known, often outdated version, and patching it on the same schedule as everything else destroys the thing it was built to be.

The mistake this section exists to head off is treating "patched" as a single global virtue that applies uniformly across a lab. It doesn't. Table 20.1 makes the split concrete, component by component.

The table below supports deciding which patch cadence applies to a given lab component before you enable any automation that touches it.

Table 20.1 — Lab component patch policy by tier.

Component	Tier	Patch cadence	What breaks if you get the tier wrong
Proxmox (or other) hypervisor host	Infrastructure	Monthly minor updates; security advisories promptly	An unpatched hypervisor is a single point of failure underneath every isolated segment above it — a host-level compromise collapses every segmentation boundary this book teaches at once
Perimeter firewall (pfSense/OPNsense)	Infrastructure	Within days of a security release	This is the control every Safety Gate box in this book depends on; an unpatched known CVE here is the one failure that defeats every other part's isolation claim simultaneously
SIEM platform (Wazuh/Elastic/Graylog)	Infrastructure	Monthly, snapshotted first (§4)	An unpatched ingest pipeline is both an attack surface and an integrity risk for the evidence this whole book teaches you to produce
Zeek/Suricata sensor host	Infrastructure	Monthly OS patches; rule/signature updates on their own faster cadence (daily/weekly)	A stale ruleset misses newly published signatures; a stale sensor OS accumulates its own attack surface on a box with a mirrored view of everything
Linux/Windows lab endpoints (Parts 9–10)	Infrastructure, unless pinned for a specific exercise	Monthly, or aligned to vendor patch cycles	An endpoint left unpatched "by default" rather than by deliberate pin is attack surface nobody chose, on a segment other parts assume is otherwise sound
Honeypot/decoy services (Part 15)	Target — deliberately vulnerable	Pinned; patch only when swapping to a new CVE/exercise	Patch it on the normal cadence and the vulnerability the honeypot exists to expose disappears silently — see §3's Build Autopsy
Attack-simulation tooling host (Part 18)	Infrastructure (the tool), pointed at target-tier hosts	Keep the tool itself current	An out-of-date attack-sim framework can fail a technique silently against a current OS build, and a failed test can look exactly like "the defense worked"
DNS/Pi-hole (Part 7)	Infrastructure	Monthly	Query-log telemetry (Part 7's own early-win example) depends on the resolver staying up and current, same as any other infrastructure service

2. Patching the infrastructure tier without breaking anything

[SETUP] For infrastructure-tier Linux hosts, `unattended-upgrades` gets you automatic security patching without needing to remember to do it. The commands below target Debian 12/Ubuntu 22.04 and 24.04 LTS, the same OS baseline Part 9 used for lab endpoints.

```
sudo apt install unattended-upgrades apt-listchanges
sudo dpkg-reconfigure --priority=low unattended-upgrades
```

Restrict it to security-origin updates only, and disable automatic reboots so a kernel update doesn't restart a SIEM VM mid-ingest without you knowing:

```
# /etc/apt/apt.conf.d/50unattended-upgrades
Unattended-Upgrade::Allowed-Origins {
    "${distro_id}:${distro_codename}-security";
};
Unattended-Upgrade::Remove-Unused-Dependencies "true";
Unattended-Upgrade::Automatic-Reboot "false";
```

Confirm the configuration is valid and see what it would do without actually applying anything:

```
sudo unattended-upgrade --dry-run --debug
```

The dry run should list the security-origin packages it would upgrade with no errors about malformed origin strings — that's the check to run once after editing the config, not something you need to repeat every month.

[COST/RESOURCE] Your snapshot safety net (§4) stops at the boundary of the hypervisor itself — you can snapshot a VM or container running on Proxmox, but you cannot snapshot Proxmox's own host OS the same way. A bad kernel update to the host is a different risk class than a bad patch inside a guest: there's no five-minute revert. The practical mitigation is to track Proxmox's own point releases rather than the bleeding edge, and to confirm the previous kernel is still selectable from GRUB before rebooting into a new one — `proxmox-boot-tool kernel list` shows what's installed and bootable. Treat a host-level kernel update as the one patch in this whole part where "read the release notes first" is not optional.

For pfSense/OPNsense, the update check lives in the GUI (System → Update on pfSense, System → Firmware on OPNsense); both platforms keep the previous firmware version available as a one-click rollback for a short window after an upgrade, which is the closest thing to a snapshot the firewall itself gets. Take the rollback window seriously — verify the firewall still passes the Part 6 Validation Test (the isolation rules still hold) before that window closes, not after.

Validation Test Setup: Infrastructure-tier host snapshotted per §4, patch batch applied per the commands above. **Action:** Run `systemctl status` (or the equivalent health check) on every service that host runs, and check the Part 8 SIEM for a fresh event from this host within 5 minutes of the patch completing. **Expected result:** Every service reports active/running with no restart loop, and at least one new event timestamped after the patch appears in the SIEM search. If either check fails, revert to the pre-patch snapshot from §4 before troubleshooting further — debugging a broken host live is strictly worse than reverting first and investigating the snapshot's before/after state at your own pace.

3. Keeping intentionally vulnerable targets vulnerable on purpose

[CONCEPT] A honeypot or a deliberately unpatched exercise target only teaches you anything for as long as it stays the specific version it was set up to be. The single most common way that stops being

true isn't an attacker — it's your own automation, patching a target-tier host the same way it patches everything else because nobody told it not to.

[SETUP] For a package-based target (a decoy SSH or HTTP service on a full Linux VM), the fix is two layers of the same control. First, `apt-mark hold` on the specific package whose version matters:

```
sudo apt-mark hold openssh-server
apt-mark showhold
```

Second, and more importantly, disable automated patching on that host entirely rather than trusting a hold to survive every future config-management run:

```
sudo systemctl disable --now unattended-upgrades.service
```

For a container-based decoy (a vulnerable web app image pulled from a public registry), the equivalent mistake is pulling by tag rather than digest — a `latest` or even a version tag can silently point at a rebuilt image with the underlying vulnerability patched. Pin by digest instead:

```
docker pull vulnerables/web-dvwa@sha256:<digest-you-verified-is-vulnerable>
```

Document the pin. A per-decoy README stating the exact package version or image digest and the CVE or exercise it represents is the difference between "this honeypot is vulnerable to X" as a checkable fact and as a fading assumption nobody re-verifies.

[SAFETY] Auditing which CVEs a pinned target-tier host is still exposed to sometimes tempts a reader to run a vulnerability scanner against it from a broader network host, for convenience. Don't. Run the scanner from inside the same isolated segment the honeypot already lives on — the same segment discipline Part 15's Safety Gate requires for everything else touching that host — never from a management host with a wider route. The inventory process itself must never become the thing that gives a decoy an unintended path out of its segment.

Build Autopsy — the honeypot that patched itself vulnerability-free

The plan: Reuse the same cloud-init base image and hardening baseline for a new SSH honeypot decoy that infrastructure-tier hosts already used elsewhere in the lab, on the reasoning that a consistent, patched baseline everywhere is strictly good practice.

Why it seemed reasonable: The base image's `unattended-upgrades` configuration had already been tuned and tested (per §2) on every other host in the lab with no problems, so reusing it for a new VM felt like reuse, not a new decision that needed separate thought.

How it failed: The decoy's entire purpose was an intentionally outdated OpenSSH version, targeted at a specific banner-based Suricata signature. `unattended-upgrades` pulled a routine security-origin OpenSSH point release within its first overnight run, silently closing the exact version gap the honeypot existed to expose. Nothing crashed and nothing alerted — the honeypot kept running, kept looking healthy, and simply stopped being interesting. The Suricata signature that used to fire on the old banner went quiet, and it took several weeks of "why hasn't this caught anything" before anyone checked the actual installed OpenSSH version against the one the exercise assumed.

The fix: Target-tier hosts never inherit the infrastructure-tier patch baseline wholesale. Every decoy or pinned exercise target gets `unattended-upgrades` disabled outright and its critical package(s) held explicitly, as its own first-boot step, documented in a per-decoy README — not as an exception someone remembers to apply, but as a default that has to be deliberately opted out of to get infrastructure-tier patching back.

4. Snapshot discipline before risky changes

[CONCEPT] A hypervisor snapshot is a point-in-time copy of a VM or container's disk state (and, optionally, its RAM state) that you can revert to without reinstalling anything. In this book, a snapshot is the thing that turns "I'm not sure this config edit is safe" from a source of anxiety into a non-event: take the snapshot, make the change, and if it's wrong, you're back to exactly where you started in minutes.

[SETUP] The commands below target Proxmox VE 8.x's `qm` (QEMU/KVM VMs) and `pct` (LXC containers) tooling, and VirtualBox 7.x's `VBoxManage`, matching the platforms Part 5 covers. Readers on Hyper-V or ESXi per Part 5's own comparison use that platform's checkpoint/snapshot equivalent — the naming-and-timing discipline below applies regardless of which hypervisor you're actually running.

```
# Proxmox VE, QEMU/KVM VM
qm snapshot <vmid> pre-patch-2026-09-15 --description "before monthly infra patch batch"
qm listsnapshot <vmid>
qm rollback <vmid> pre-patch-2026-09-15

# Proxmox VE, LXC container
pct snapshot <ctid> pre-patch-2026-09-15

# VirtualBox
VBoxManage snapshot "<vmname>" take "pre-patch-2026-09-15" --description "before monthly infra patch batch"
```

Confirm the snapshot exists and is restorable before you rely on it — `qm listsnapshot <vmid>` or its VirtualBox equivalent should list the snapshot immediately after creation; if it doesn't appear, don't proceed with the risky change on the assumption it silently worked.

Lab Note Name every snapshot with the date and the reason, not just "before update" or `snap3`. Six months into a running lab, a hypervisor with 40 unlabeled snapshots is a worse problem than the one they were supposed to protect against, because you can no longer tell which one is safe to delete without diffing state by hand. `pre-patch-2026-09-15-suricata-etopen-update` costs you 5 extra seconds to type and saves you an afternoon later.

5. Snapshot's limits: why you still need an offsite backup

[CONCEPT] A snapshot and a backup solve different failure modes, and conflating them is the second most common maintenance mistake in a home lab, right behind the patching-tier confusion in §3. A snapshot lives on the same storage pool as the VM it protects — it's fast to take and fast to revert, but it dies with that storage pool. A backup is a separate copy, ideally on physically different storage, that survives the loss of the host or pool the original lived on. Table 20.2 lays out what each layer actually protects against.

The table below supports deciding whether a given protection layer is sufficient for a specific risk, or whether you need the layer below it too.

Table 20.2 — Snapshot vs. backup: what each layer recovers from.

Protection layer	What it captures	Where it lives	Recovers from	Doesn't recover from
Hypervisor snapshot (<code>qm snapshot / pct snapshot / VBoxManage snapshot</code>)	Point-in-time disk state of one VM/container	Same storage pool as the running instance	A bad config edit, a failed patch, a broken rule set — revert in minutes	Storage pool failure, host disk corruption, accidental deletion of the whole instance
Off-host backup (<code>vzdump</code> /Proxmox Backup Server, or scheduled <code>rsync</code> to external storage)	Full VM/container image or file-level archive	A separate physical disk, or a target reachable only from the management VLAN	Host disk failure, accidental instance deletion, a local-storage failure that also destroys any snapshots on it	A config mistake or a logic bug preserved faithfully alongside the good data — a backup keeps whatever state existed at backup time, flaws included
Config-as-code source (Appendix A3 baselines, Sysmon/auditd configs, <code>.rules</code> files kept in git or a local repo)	The intended configuration itself, independent of any running instance	Wherever you keep source control — even a single local git repo is enough for one operator	Total lab loss — the fastest possible rebuild path, since it's declarative rather than a disk image	Runtime data: logs already ingested, honeypot session history, anything that only exists because a service ran and produced it

Engineering Reality On Proxmox, `qm snapshot` works for QEMU/KVM VMs on nearly any backing storage, but `pct snapshot` for LXC containers needs a storage type that actually supports it — ZFS, LVM-thin, or Btrfs. A container living on plain directory-backed storage returns an error the instant you try to snapshot it, not a slow warning during setup. If any of this book's LXC-based components — the DNS resolver in Part 7, an LXC-based lab endpoint from Part 9 — ended up on directory storage when you provisioned it, you won't discover the gap until the moment you actually need the snapshot, which is exactly the wrong moment to discover it. Run `pvesm status` and confirm the storage backing every container before you rely on §4's commands, not after.

Resource Reality A `qcow2` or LVM-thin snapshot doesn't cost you the VM's full provisioned disk size — it costs you the delta, every block that changes after the snapshot is taken. On a SIEM VM with active indexing, that delta grows fast: leaving a snapshot in place for a week under normal ingest can consume several GB from index churn alone, with the VM's nominal disk size never changing. Budget headroom for snapshot deltas the same way Part 9 budgeted headroom for `audit.log` growth, and delete a snapshot the moment its Validation Test passes and you're confident in the change. A snapshot isn't free storage sitting idle — it's a loan against your free space, with the delta as accruing interest.

6. Building the backup and restore pipeline

[SETUP] `vzdump` is Proxmox's built-in backup tool and the simplest starting point for a single-operator lab — it archives a whole VM or container to a target you specify, no separate backup server required.

```
vzdump <vmid> --mode snapshot --compress zstd --storage <backup-storage-target>
```

`--mode snapshot` takes the backup from a live snapshot rather than stopping the VM first, which matters for anything you don't want to take an outage window for (the SIEM, most infrastructure-tier hosts). If your lab grows past three or four hosts worth backing up, Proxmox Backup Server adds deduplication and incremental-forever backups on top of `vzdump`'s full-archive model — worth the extra VM at that scale, not worth it below it. For a smaller lab, a scheduled `rclone` or `rsync` job pushing the `vzdump` archive to an external USB disk or a NAS on the management VLAN gets you the same off-host protection with less new infrastructure to maintain.

Figure 20.1 traces the full path a risky change takes through this part's controls, from the snapshot taken before the change through validation, rollback if needed, and the scheduled backup that runs regardless of which path a given change took.

```
flowchart TD
    SNAP["Snapshot taken\n(qm/pct snapshot, dated + labeled)"]
    SNAP --> CHANGE{"What kind of change?"}
    CHANGE -->|"Infrastructure-tier patch (S2)"| PATCH["Apply OS/security patch"]
    CHANGE -->|"Target-tier exercise (Part 15/18/19)"| RUN["Run attack-sim / let honeypot be attacked"]
    PATCH --> VAL1["Validation Test:\nservices up, telemetry still flowing"]
    RUN --> VAL2["Validation Test:\nunexpected detection/alert fired"]
    VAL1 -->|"pass"| KEEP["Keep running"]
    VAL1 -->|"fail"| REVERT["Roll back to snapshot"]
    VAL2 -->|"pass"| KEEP
    VAL2 -->|"fail"| REVERT
    REVERT --> SNAP
    KEEP --> BACKUP["Scheduled off-host backup\n(vzdump/PBS or rclone)"]
    BACKUP -->|"management VLAN egress only"| DEST["Off-host backup target"]
    DEST -->|"quarterly, Table 20.3"| RESTORE["Restore test into scratch VM/container"]
```

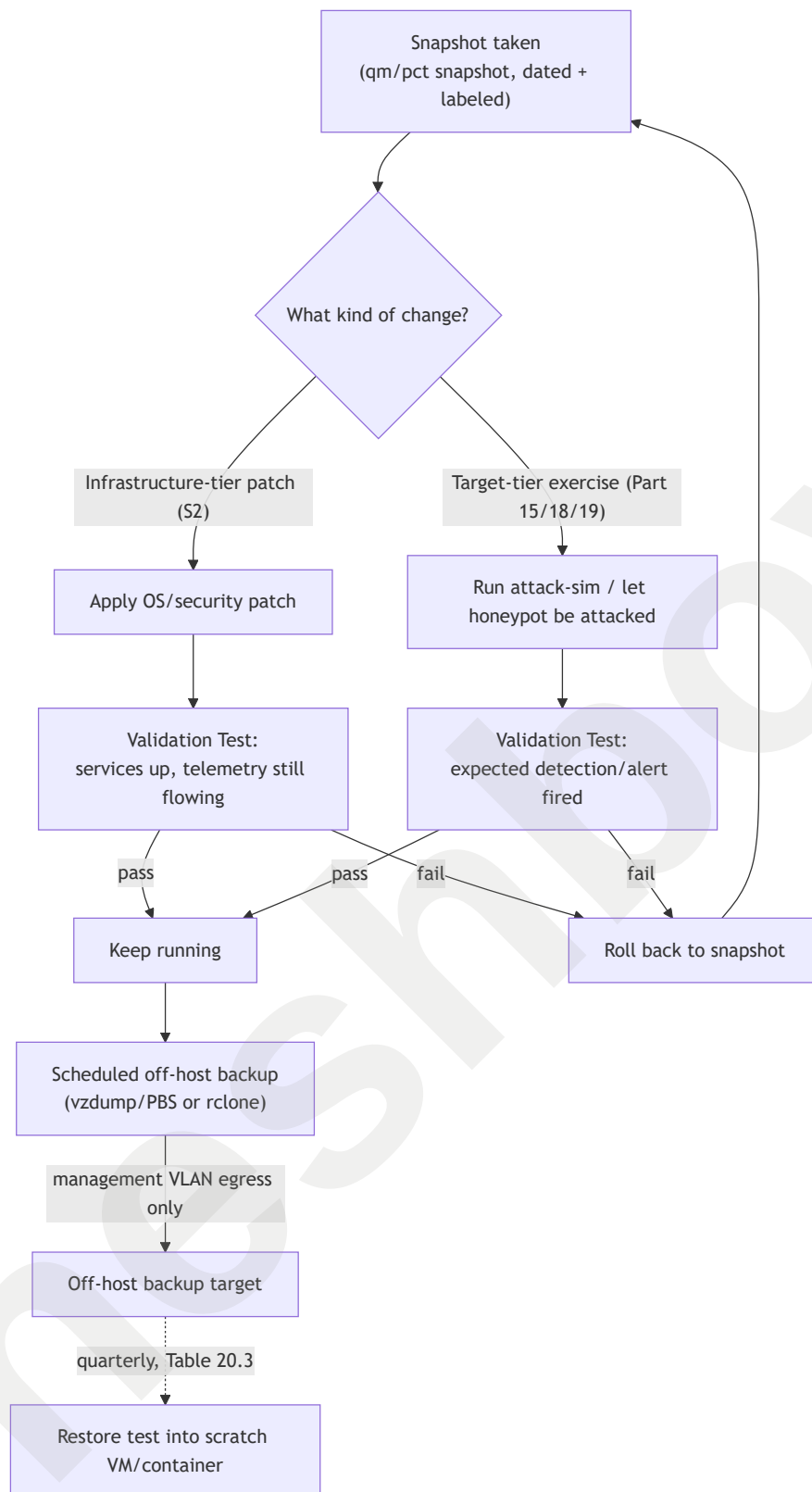


Figure 20.1 — Patch/exercise validation loop and backup pipeline. *CONCEPTUAL.* Illustrates how a snapshot taken before any infrastructure-tier patch or target-tier exercise feeds a validation-and-rollback loop, and how a passing change still flows into a scheduled, isolated off-host backup that itself gets a quarterly restore test. This is an architecture sketch of the intended discipline, not a capture from a running deployment.

Validation Test Setup: At least one completed off-host backup of an infrastructure-tier host (this section), and a scratch VM/container with no production role on the lab network. **Action:** Restore the backup into the scratch target, bring it up on an isolated network segment with no route to the live lab, and confirm the service starts. **Expected result:** The restored instance boots, the service reports active, and spot-checking one known log entry or config value confirms the backup is a real, usable recovery point rather than a file that merely exists on disk. Run this quarterly per Table 20.3 — a backup nobody has ever restored is a hope, not a plan.

7. A maintenance cadence you'll actually keep

[CONCEPT] None of the previous six sections matter if the discipline only gets applied once, right after reading this part, and then quietly lapses. Table 20.3 is the cadence this book recommends — short enough per task that skipping it has no good excuse, and frequent enough that a stale pin or a full disk gets caught before it becomes an outage.

The table below supports building a recurring maintenance schedule you can actually sustain as a single operator, rather than a one-time checklist you run once and forget.

Table 20.3 — Lab maintenance cadence checklist.

Cadence	Task	Tier	Why this cadence
Weekly	Check disk headroom on the sensor host, SIEM, and audit-log partitions	Infrastructure	A cheap check that catches Part 9's disk-fill Build Autopsy failure mode before it becomes an outage
Weekly	Confirm target-tier version pins haven't drifted (<code>apt-mark showhold</code> , a digest diff on pinned container images)	Target	Silent auto-update is the single most common way a honeypot stops being vulnerable — catching it weekly beats catching it never
Monthly	Apply OS/security patches to infrastructure-tier hosts, snapshot first	Infrastructure	Balances staying current against re-verifying isolation after every single patch
Monthly	Review Suricata/Zeek ruleset and SIEM version currency	Infrastructure	Stale detection content misses newly published signatures the same way a stale OS misses newly published exploits
Quarterly	Run a full restore test into a scratch VM/container from the most recent off-host backup	Infrastructure	The only way to know a backup works is to use it before an actual failure forces the question
Quarterly	Reconcile the snapshot inventory — delete stale ones, confirm the naming convention is still being followed	All tiers	Snapshot sprawl quietly consumes the disk budget Part 21 covers, without anyone ever deciding it should
Per exercise	Snapshot before, and confirm a restore point exists before, any Part 18/19 attack-simulation run	Target	The rollback that turns "the exercise broke something" into an afternoon, not a rebuild

Blind Spot Snapshots and backups protect you against breakage — a bad patch, a fat-fingered config edit, a failed exercise. They do nothing to protect you against a mistake or a compromise that gets faithfully preserved. If a honeypot's session-correlation logic (Part 15) has a bug that's been silently dropping every third session for a month, every backup taken during that month is a confidently preserved copy of bad data, not a safety net. This part's discipline catches breakage; it does not catch correctness, and the two are genuinely different problems that need different checks.

What Would Change My Mind This part recommends pinning target-tier packages and images manually (`apt-mark hold`, digest-pinned containers) rather than relying on a patch-management tool to distinguish infrastructure from target hosts automatically. If a patch-management tool built for home-lab scale shipped a genuinely reliable tag-based exclusion feature — patch everything except hosts tagged "target-tier," with no risk of a later config-management run silently overriding that tag — that would be worth adopting over the manual commands in §3. Nothing available did that reliably as of this writing.

8. Hands-on lab: patch, break, restore, verify

[HANDS-ON LAB]

Goal: confirm your own snapshot-and-revert loop actually works end to end on a disposable infrastructure-tier host, before you're relying on it under real pressure from an actual bad patch.

1. Pick a low-stakes infrastructure-tier host — the DNS/Pi-hole container from Part 7 or a spare Linux endpoint from Part 9 both work well.
2. Snapshot it per §4's naming convention: `pre-patch-<date>-hands-on-lab`.
3. Deliberately break something recoverable — truncate `/etc/rsyslog.conf` to zero bytes, or apply a real package upgrade you have reason to think might be disruptive.
4. Confirm the break: the forwarding service fails to restart, or telemetry visibly stops arriving in the Part 8 SIEM.
5. Roll back to the snapshot from step 2.
6. Confirm the service and telemetry are restored, and note how long the whole cycle took from break to confirmed-restored.

Expected result: the full cycle, from confirmed break to confirmed restore, should take single-digit minutes. If it takes longer, or the rollback doesn't fully restore the service, tighten the naming and snapshot-timing discipline in §4 now — this exercise is deliberately low-stakes precisely so the first time you find a gap in your own rollback process isn't during a real bad patch.

9. What maintenance discipline means for the other three books

[CONCEPT] Patching and maintenance windows aren't just an operational chore confined to this book — they generate their own telemetry, and if the rest of your stack doesn't know a maintenance window is happening, that telemetry becomes noise at best and a mistrained detection at worst.

- **Tell your baselines about the window, don't let them absorb it silently.** Detection Engineering Handbook V2, Part 31 (Baselining) treats maintenance windows explicitly as a standard input a baseline needs to account for — the burst of `exec`-keyed `auditd` events (Part 9) and Sysmon process creations (Part 10) a monthly patch batch generates looks, to an untuned baseline, exactly like a lateral-movement spree. Feed this part's cadence into that baselining logic rather than discovering months later that every patch cycle quietly inflates your false-positive rate.
- **Give alert-triage procedures a real maintenance-window exception, not a silent one.** SOC Playbook Handbook's escalation and alert-triage procedures assume, by default, that an unexpected burst of process-creation or configuration-change events is worth investigating. Cite this part's cadence (Table 20.3) explicitly in whatever suppression or annotation mechanism you build in that book's playbooks, so a planned patch batch doesn't get triaged with the same urgency you'd want applied to an actual, unplanned one — and so it doesn't get suppressed so broadly that a real incident during the same window gets missed.
- **Use a stable, restorable lab as a practice-drill platform, not a moving target.** SOC Manager's Operating Handbook, Part 8 (Interviewing & Technical Assessment Design) and Part 11 (Onboarding & Ramp-Up Programs) both assume you can run the same scenario repeatably against a known-good environment. That's only true if this part's restore strategy genuinely works — which is exactly what §8's hands-on lab and the quarterly restore test in Table 20.3 exist to prove before you rely on either for someone else's assessment or onboarding drill.

Cross-references: Part 4 (network isolation this part's Safety Gate depends on), Part 5 (hypervisor snapshot/checkpoint mechanics), Part 6 (perimeter firewall egress rules a backup or patch-mirror pull must stay inside), Part 7 through Part 10 (infrastructure-tier hosts this part's patch cadence applies to), Part 9 (the `audit.log` disk-growth precedent this part's Resource Reality reuses), Part 15 and Part 18 (target-tier honeypots and attack-simulation hosts this part's pinning discipline protects), Part 19 (purple-team drills that assume a stable, restorable lab), Part 21 (storage growth and retention that snapshot/backup disk cost feeds into), Appendix A3 (config-as-code baselines as the fastest rebuild path), Appendix A4 (pre-flight isolation checklist); Detection Engineering Handbook V2, Part 31 — Baselining (deh:part31); SOC Playbook Handbook's escalation and alert-triage procedures; SOC Manager's Operating Handbook, Part 8 — Interviewing & Technical Assessment Design and Part 11 — Onboarding & Ramp-Up Programs (manager:part08, manager:part11).

Part 21 — Storage Growth, Retention, and Long-Term Cost Management

Why this part exists

Every part in Sections B through F of this book adds something that writes to disk continuously: a SIEM index, a Zeek or Suricata log, a honeynet session record, a Sysmon or auditd event, a pre-change VM snapshot from Part 20. None of them, on its own, looks like a problem on day one. The problem shows up on day 90, when a filesystem that had 40% free space for two months is suddenly at 4% free with no single obvious cause — and the honest, embarrassing truth is that this already happened to the lab this book cites throughout. This part is where you build the habit of doing the arithmetic before the disk does it for you: what actually grows without bound, what's cheap to keep forever, what's expensive to keep for a week, and the specific point where the right answer is deleting data instead of buying another drive.

[CONCEPT] This part is deliberately not about backup strategy — Part 20 already covers snapshot discipline and restore paths for when a build breaks. This part is about the separate, slower-moving problem of a lab that works fine but is quietly consuming more disk every day it runs, until it doesn't.

Safety Gate *Retention and archiving in this part assume you're only ever moving data out of the isolated lab segment (Part 4), never opening a new path into it. Two conditions must hold before you attach any cold-storage target — a USB drive, a NAS share, a cloud backup bucket — to pull old logs or PCAP off a lab host: first, that target must not be simultaneously reachable from your home network and from inside the lab segment, because a NAS share mounted on both is a bridge, not a backup path, and it defeats every control Part 4 built regardless of how harmless the mount looks; second, archived honeynet captures and PCAP files can contain live exploit payloads, scanner shellcode, or malware samples your own decoys caught (Part 15) — extract and open them only on a host with no route back to anything you use for real work, and never on your everyday laptop out of convenience. Run the Appendix A4 pre-flight checklist against any new archive path before you use it for the first time, not just against the lab segment itself.*

1. What actually fills a home lab's disk

[CONCEPT] Four things eat disk in this book's lab, and they don't grow at the same rate or for the same reason. Confusing them is why "I'll just add more storage" is usually the wrong first response — it treats a policy problem (you never decided how long to keep something) as a capacity problem (you don't have enough room), and buying more room just delays the day you have to make the policy decision anyway.

- **SIEM indices** (Part 8) grow continuously and proportionally to how many endpoints you're forwarding from and how noisy each one is. This is the slowest, steadiest grower, and the one

with the most mature tooling for controlling it (§4).

- **Raw packet capture** (Part 12's mirror point, if you're saving full PCAP rather than just Zeek/Suricata's summaries) grows the fastest of anything in this book, because it stores every byte on the wire rather than a structured summary of what happened (§5).
- **Honeynet and endpoint logs** (Parts 9, 10, 15) are the cheapest, most compact category — they're already summaries, not raw captures — and the ones most worth keeping longer, because they're literally the evidence base for the hunting and purple-team content in the other three NESHBOY volumes (§6).
- **VM and container snapshots** (Part 20) don't grow with traffic at all — they grow with how much the live disk changes underneath them, and a snapshot chain nobody remembers to prune is one of the sneakiest full-disk causes in this whole book, because `df -h` on the host doesn't obviously point at "snapshots" the way it points at "the SIEM's data directory" (§7).

2. Real lab example: the host that went from 92% full to full

[CONCEPT] Part 3 already introduced the host this book partly evidences itself from: an HP EliteDesk 800 G1 desktop running Proxmox, 4 physical cores, 15GiB of RAM, hosting 13 guests. That part's Resource Reality box noted the host's root filesystem was at 92% full at a 2026-09-14 inspection. This part picks that thread up, because what happened next is the whole reason this part exists.

Resource Reality By 2026-09-15 — the day this part was drafted — that same root filesystem had gone from 92% full to completely full. No single service crashed outright in an obvious, loud way; instead, the honeynet's correlation backend and the DNS resolver's logging both started failing writes intermittently, in the quiet, easy-to-miss way a full filesystem fails rather than the loud way an OOM-killed process fails. The honest answer to "what exactly filled the last 8%" is that it wasn't instrumented at the time — nobody was running `du -sh` against the guest data directories on a schedule, so there's no clean breakdown of SIEM-equivalent index growth versus log growth versus snapshot sprawl to point to after the fact. That gap is itself the lesson this part is built around: you cannot right-size a retention policy for a filesystem you never metered, and "add more disk" fixes the symptom for exactly as long as it takes the same ungoverned growth to catch back up. Section 8 builds the monitoring this host didn't have.

[CONCEPT] Notice what didn't happen here: nobody needed a bigger SIEM index or a wider honeynet to hit this wall. A 4-core desktop with 15GiB of RAM and a modest guest count filled its own root filesystem within roughly a day of crossing 92%, on ordinary background growth from services that were already running, not from a burst of new load. That's the actual shape of the problem this part solves — not "will my lab eventually need more storage" (yes, eventually, everything does) but "will an unmonitored, unpoliced lab surprise you with a hard stop weeks before you expected one" (also yes, and it already has, on the lab this book is drawn from).

Build Autopsy — "the disk that kept needing one more terabyte"

The plan: Respond to "the SIEM VM is running low on disk" by attaching a bigger virtual disk to it every time the warning appeared, on the theory that storage is cheap and the alternative — figuring out a retention policy — is more work than it's worth for a single-operator lab.

Why it seemed reasonable: A 500GB SSD is genuinely inexpensive, the attach-and-extend-filesystem procedure takes ten minutes, and each individual expansion did solve the immediate problem. Nothing about any single instance of this looked like a mistake.

How it failed: Every expansion bought time proportional to the current growth rate, but the growth rate itself never changed, because nothing was ever deleted, rolled up, or archived — nothing enforced a ceiling. Each expansion also had to be manually repeated, on a schedule that kept getting shorter as more endpoints and a wider honeynet got added on top of the same unmanaged index. The underlying failure is that "buy more disk" treats an unbounded-growth problem as a one-time capacity problem, when it's actually a recurring policy problem that resurfaces on a shortening interval.

The fix: A retention policy (§3–4) that puts an actual ceiling on hot-tier growth, so a disk-size decision is made once per tier instead of being re-litigated every few weeks as an emergency.

3. Retention math: doing the arithmetic before your disk does it for you

[COST/RESOURCE] The table below is illustrative, not a measurement from a specific running lab — treat the ranges as a starting point for your own `du -sh` numbers once §8's monitoring is in place, not as a guarantee for your exact hardware and traffic mix.

Table 21.1 — Typical daily disk growth by lab component (CONCEPTUAL SAMPLE). Supports deciding which retention lever in §4–7 to pull first, by showing which components grow fastest relative to how compact their data actually is.

Component	Typical daily growth	Grows without bound?	Primary lever	Notes
SIEM event index (Sysmon + auditd + Zeek + Suricata combined feed)	50MB–500MB, scales with endpoint count	Yes, until policy caps it	ILM/index rollover (§4)	A default one-replica index roughly doubles this on disk — see §4's Engineering Reality
Raw PCAP (full packet capture, not just NSM summaries)	500MB–5GB+	Yes, fastest grower in the book	Ring-buffer rotation (§5)	Even a quiet lab segment generates this; an internet-facing honeynet decoy generates far more from background scanning alone
Honeynet session/probe logs (Part 15)	5MB–50MB	Yes, but slowly, and it's already compact	Long retention is usually affordable (§6)	Grows with how much scanning traffic your decoys attract, not with how much you did
Endpoint auditd/journald and Sysmon logs (Parts 9–10)	1MB–10MB per endpoint	Yes, slowly	Long retention is usually affordable (§6)	Cheapest category per byte of investigative value in this book
VM/container snapshots (Part 20)	0 at creation, then variable	Yes, tied to live-disk change rate, not time	Manual pruning (§7)	The only category that doesn't show up as "logs growing" in any dashboard

[COST/RESOURCE] The arithmetic that matters is simple: days until full = free space ÷ daily growth rate, summed across every row above that you're actually running. A lab with 200GB free and a combined growth rate of 6GB a day (a plausible total once PCAP retention and an active honeynet are both in the mix) has roughly 33 days before it hits zero — not months, and not a number most readers would guess without doing the division. Run this calculation for your own lab before reading further, using either Table 21.1's ranges or, better, your own §8 measurements.

What Would Change My Mind This part's working assumption, carried into §4's specific numbers, is that a single-operator lab following this book's scope (Sections B–F, no persistent Windows domain) rarely needs more than 30 days of hot-tier SIEM retention and a 72-hour rolling PCAP buffer to support the hunting and purple-team exercises Detection Engineering Handbook V2 and Part 19 of this book describe. If a reader reports a specific exercise from either source that genuinely required reaching back further than 30 days of hot data to reproduce or investigate — not "more retention would have been nice" but "the exercise's own instructions assume history this policy doesn't keep" — that specific number needs revision here, not just a footnote caveat.

4. SIEM index lifecycle: hot storage, rollover, and deletion

[CONCEPT] Part 8 left three SIEM choices on the table — Wazuh, the Elastic stack, or Graylog — and this part doesn't relitigate that choice. What it adds is the retention mechanism each one gives you, because "logs arrive and are searchable" (Part 8's stopping point) says nothing about how long they stay searchable before someone — you — has to decide.

[CONCEPT] The author's own running lab doesn't operate any of these SIEM platforms — it's the Linux-only honeynet and monitoring setup Part 9 and Part 15 draw real evidence from, with no Wazuh, Elastic, or Graylog instance behind it. Everything in this section is grounded in each platform's own published behavior, not a build the author has run end-to-end on this book's own hardware — treat the exact API syntax below as a documented starting point to verify against whatever version you actually installed in Part 8, not as a build-tested figure.

4.1 Elasticsearch/OpenSearch-family ILM (Wazuh's bundled indexer, or a standalone Elastic stack)

[SETUP] The commands below target the Index Lifecycle Management (ILM) API as documented for the Elasticsearch/OpenSearch 7.x–8.x lineage Wazuh's bundled indexer and a standalone Elastic stack both build on, per Part 8. Define a policy that keeps an index in a fast "hot" phase for 7 days, rolls it into a "warm" phase (still searchable, no longer accepting new writes) for another 23 days, then deletes it at 30 days total:

```
curl -X PUT "https://localhost:9200/_ilm/policy/lab-telemetry-policy" \
-H "Content-Type: application/json" \
-u "admin:<your-indexer-password>" \
-d '{
  "policy": {
    "phases": {
      "hot": { "actions": { "rollover": { "max_size": "20gb", "max_age": "7d" } } },
      "warm": { "min_age": "7d", "actions": { "shrink": { "number_of_shards": 1 } } },
      "delete": { "min_age": "30d", "actions": { "delete": {} } }
    }
  }
}'
```

Defining the policy alone doesn't attach it to anything: the `rollover` action operates on a write alias, not a fixed index name, so it only takes effect once an index template sets `index.lifecycle.name` to `lab-telemetry-policy` and `index.lifecycle.rollover_alias` to the alias your SIEM's data stream actually writes through — Part 8's install docs for whichever platform you chose cover that template step. Confirm the policy is attached and progressing with `GET _ilm/policy/lab-telemetry-policy` and, on an index using it, `GET <index-name>/_ilm/explain` — the explain output should show a `phase` field advancing from `hot` toward `delete` as the index ages, not stuck on `hot` indefinitely.

Engineering Reality ILM's `rollover` action only fires when an index crosses its configured `max_size` or `max_age` threshold — and in a home lab with modest ingest volume, an index can sit well under `max_size` for weeks. If your lab generates less than 20GB of events in 7 days (likely, per Table 21.1), the `max_age` trigger is what actually rolls it over, not the size trigger; if you set only a size threshold and skip `max_age`, a quiet lab's index may never roll over at all, and the delete phase that depends on it never fires either. Always pair a size threshold with an age threshold for a home-lab-scale policy, or the newest, most active index just keeps growing forever while you believe a retention policy is running.

4.2 Graylog's index set rotation and retention

[SETUP] Graylog exposes the same idea through its web UI rather than a REST policy document: under **System** → **Indices**, an index set's **Rotation strategy** (typically "Index Size" or "Index Time," matching Table 21.2's tiers) and **Retention strategy** ("Delete Index" or "Close Index," where closed indices stop accepting queries but stay on disk until manually reopened or deleted) together do what §4.1's ILM policy does for Elasticsearch. Set rotation to a 7-day time window and retention to delete after 4 rotated indices to land on the same 30-day-total policy as the ILM example above.

Validation Test Setup: either §4.1's ILM policy attached to a live index, or §4.2's Graylog rotation/retention strategy configured on an index set. **Action:** force a rollover early for testing — `POST <write-alias-name>/_rollover` (the alias, not the concrete backing index) for Elasticsearch/OpenSearch, or the **Rotate active write index now** button in Graylog's index set page. **Expected result:** a new index appears as the active write target, and the previous index shows the `warm` phase (ILM) or a "closed for writing" state (Graylog) rather than continuing to accept new events — confirming the policy actually executes before you wait 7 real days to find out it doesn't.

5. PCAP retention: the fastest way to fill a disk, and why you probably don't need much of it

[CONCEPT] Zeek and Suricata (Parts 13–14) already give you structured summaries of every connection and every rule match. Raw PCAP is different in kind, not just in size — it's every byte that crossed the wire, which is exactly why it's Table 21.1's fastest grower and exactly why most of this book's exercises don't actually need it kept for long. The honest case for raw PCAP in a home lab is narrow: reconstructing the literal payload of a specific session (extracting a file transferred in the clear, replaying an exact exploit byte sequence) after Zeek or Suricata already told you something worth looking closer at. That's a look-back-a-few-days use case, not a look-back-a-few-months one.

[SETUP] The commands below target `tcpdump` 4.9.x as shipped on Debian 12/Ubuntu 22.04, run on the NSM sensor VM from Part 12 against the same mirrored capture interface Zeek and Suricata already read. `-G` and `-W` together do not, on their own, build a self-sustaining ring buffer — without `-C`, `tcpdump` exits once it has written the 24th file instead of wrapping around and overwriting the oldest one, so a capture left running this way silently stops recording after 24 hours with no error anywhere:

```
sudo tcpdump -i eth1 -w /pcap/rolling/capture_%Y%m%d_%H%M%S.pcap -G 3600 -W 24
```

`-G 3600` rotates to a new file every 3600 seconds (one hour); `-W 24` stops `tcpdump` after the 24th file rather than deleting anything. Run it under a `systemd` unit with `Restart=always` so it relaunches the instant it exits at file 24, and add a separate cron job to actually enforce the 24-hour ceiling `tcpdump` itself doesn't:

```
# /etc/systemd/system/pcap-ring.service
[Unit]
Description=Rolling PCAP capture on the Part 12 mirror interface

[Service]
ExecStart=/usr/bin/tcpdump -i eth1 -w /pcap/rolling/capture_%Y%m%d_%H%M%S.pcap -G 3600 -W 24
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

```
# crontab -e -- deletes anything older than 24 hours; tcpdump's own -G/-W never does this
0 * * * * find /pcap/rolling -name 'capture_*.pcap' -mmin +1440 -delete
```

Confirm the loop is actually closed, not just running once and stopping, with `systemctl status pcap-ring` (expect `active (running)`, not `inactive (dead)` sitting at file 24) and `ls -la /pcap/rolling/ | wc -l` a few hours in — the count should stabilize at 24, not climb past it or freeze short of it.

Blind Spot A 24-hour PCAP ring buffer has an obvious, specific cost: if a purple-team drill (Part 19) or a honeynet capture (Part 15) turns out to be worth deeper packet-level analysis and you don't notice until the second day after it happened, the raw bytes are already gone — overwritten by the ring, not archived anywhere. Zeek's and Suricata's logs survive past that window and tell you the session happened, but neither one preserves the literal payload the way raw PCAP does. This ring buffer trades exactly that capability for disk space; it doesn't eliminate the need for payload-level analysis, it just narrows the window in which it's possible.

Lab Note If a specific session is worth keeping past the ring buffer's window — you just ran a Part 18 attack simulation and want the exact PCAP for later review — copy that one file (or use `tcpdump`'s BPF filter to extract just that session from the current ring file) to the archive tier from §6 within the same day, before the ring buffer overwrites it. This is a five-minute `cp` command done at the right moment instead of a capability you can never get back once the file's gone.

6. What's actually worth keeping long-term: honeynet and endpoint logs

[CONCEPT] Table 21.1's cheapest rows — honeynet session/probe logs and endpoint auditd/Sysmon/journald output — are also the ones with the best ratio of investigative value to disk cost in this entire book, because they're already the summarized, structured record of what happened rather than the raw bytes underneath it. A honeynet's correlated attack-session log (Part 15) at, say, 30MB a month costs you almost nothing to keep for a year; the same can't be said for a year of raw PCAP from the same segment.

[COST/RESOURCE] This is the retention tier where "keep it longer than you think you need to" is actually the right default, not a hoarding instinct to resist — the entire premise of Detection Engineering Handbook V2's threat-hunting content (its Parts 34–36) is having enough history to hunt across, and this book's Part 3 already flagged that a Tier-1 hardware budget forces short retention windows specifically here. If your hardware tier allows it, extend hot-or-warm retention for honeynet and endpoint logs well past the 30-day SIEM-index default in §4 — 90 days or more, if disk allows — before extending PCAP retention at all.

Table 21.2 — Recommended retention windows by data class. Supports deciding what changes between §4's SIEM-index default and a longer or shorter window for a specific data class, before you touch any single component's config.

Data class	Recommended hot/warm window	Cold-archive it?	Why
SIEM event index (mixed feed)	30 days	Selectively, only around a flagged finding	Bulk retention here is dominated by Windows/Sysmon volume, which is the least compact category
Raw PCAP	24–72 hours (rolling)	Only the specific session you flagged, same day	Fastest grower, lowest value once Zeek/Suricata have already summarized it
Honeynet session/probe logs	90+ days	Yes, indefinitely if disk allows	Cheap, compact, and the direct evidence base for hunting and correlation content elsewhere in the series
Endpoint auditd/journald/Sysmon logs	60–90 days	Yes, for anything tied to a documented exercise	Cheapest per-byte investigative value in the book
VM/container snapshots	Days, not weeks	No — snapshots aren't an archive format	See §7; a snapshot is a rollback point, not a retention record

7. Snapshot sprawl: Part 20's quiet disk-eater

[TROUBLESHOOTING] Part 20 tells you to snapshot before every risky change. It does not tell you, and this part will, that a snapshot nobody deletes keeps consuming more disk the longer it lives — not because the snapshot itself grows, but because a copy-on-write snapshot pins every block the live disk changes after it was taken, and a lab endpoint that's actively being used for exercises changes a lot of blocks over a few weeks. A snapshot taken before a config change three months ago, still sitting there because nobody remembered to remove it after confirming the change worked, can be pinning gigabytes of now-irrelevant delta data on a host where nobody's looking for it.

[SETUP] On a Proxmox VE 8.x/9.x host (Part 5), list what's actually pinned per guest and check the underlying storage's real usage against what the guest configs claim:

```
qm listsnapshot <vmid>
pvesm status
zfs list -t snapshot 2>/dev/null || lvs -a
```

`qm listsnapshot` shows every named snapshot for a given VM and how long ago it was taken; `pvesm status` shows real consumption per storage pool, which is the number that actually matters; the `zfs/lvs` fallback pair shows the storage-backend-level view depending on whether the pool is ZFS or LVM-thin, since a snapshot's real disk cost lives at that layer, not in the VM config.

[TROUBLESHOOTING] The fix is a habit, not a tool: delete a snapshot the moment you've confirmed the change it protected against actually worked, the same way you'd delete a backup you've confirmed you don't need — not "eventually," and not "when I notice the disk is full," because by the time `df -h` shows the problem, you're already in the situation Part 21's whole premise warns about.

8. Building a disk-growth monitoring loop

[SETUP] The gap that let \$2's host go from 92% to full without warning is a missing feedback loop — nothing was watching, so nothing could alert. The script below (targeting any Debian/Ubuntu host from Parts 5–14) checks root filesystem usage and writes a structured line to syslog, cheap enough to run via cron every 15 minutes:

```
#!/bin/bash
# /usr/local/bin/disk-growth-check.sh
THRESHOLD=85
USAGE=$(df --output=pcent / | tail -1 | tr -dc '0-9')
if [ "$USAGE" -ge "$THRESHOLD" ]; then
    logger -t disk-growth-check "WARNING: root filesystem at ${USAGE}% - threshold ${THRESHOLD}%"
fi
```

```
# crontab -e, on the host being monitored
*/15 * * * * /usr/local/bin/disk-growth-check.sh
```

Because the warning goes through `logger`, it lands in `journald`/syslog exactly like the baseline telemetry Part 9 already teaches you to forward — point the Part 8 SIEM at this host's syslog the same way, and disk usage becomes a searchable, alertable field alongside every other event source in the lab, instead of a number you only check when something already feels wrong.

Validation Test Setup: the script and cron entry above, installed on a lab host already forwarding syslog to the Part 8 SIEM. **Action:** temporarily lower `THRESHOLD` to a value below the host's current usage (for example, `THRESHOLD=1`), then wait for the next cron run or execute the script manually. **Expected result:** a `disk-growth-check` tagged warning entry appears in local syslog within seconds, and the same entry is searchable in the SIEM within the normal forwarding delay — confirming the loop actually closes end to end before you rely on it at a real threshold.

flowchart LR

```

SRC["Telemetry sources\n(SIEM feed, Part 8)"] --> HOT["Hot tier - SSD\nfull detail, 30 days"]
PCAP["Raw packet capture\n(Part 12 mirror port)"] --> RING["Rolling PCAP buffer\n24-72 hours,\nsize-capped"]
HOT --> POLICY{"Retention policy\ncheck (ILM / cron)"}
POLICY -- past hot window --> WARM["Warm tier - SSD\ncompressed rollup, 60-90 days"]
POLICY -- flagged finding --> COLD["Cold archive\noffline, one-way export"]
WARM --> KEEP{"Tied to a saved\nfinding or exercise?"}
KEEP -- Yes --> COLD
KEEP -- No --> GONE["Deleted"]
RING --> GONE
MON["Disk-usage monitor\nsection 8 script"] --> SRC
HOT --> MON
WARM --> MON
RING --> MON

```

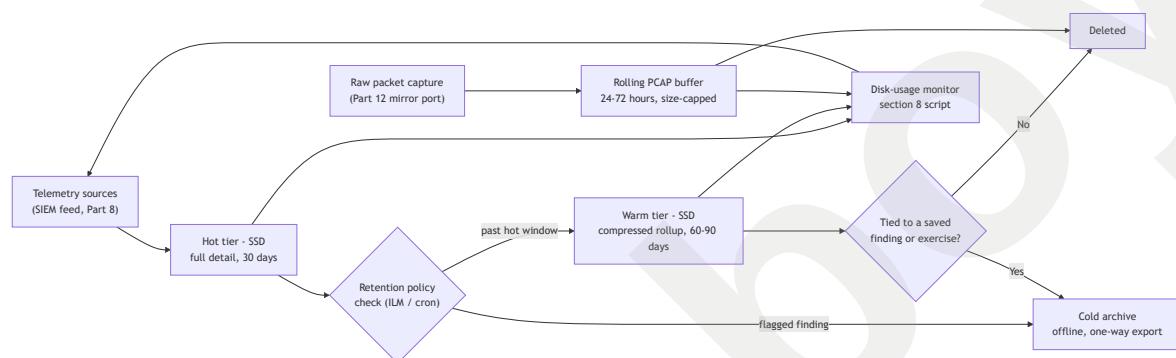


Figure 21.1 — Retention and disk-growth monitoring lifecycle. *CONCEPTUAL.* Illustrates how the retention levers built in §4–7 connect to the monitoring loop built in §8 — hot-tier telemetry and PCAP each move toward either a cold archive or deletion based on policy, while a disk-usage monitor feeds its own findings back into the SIEM as ordinary telemetry. This is an architecture sketch for the policy this part recommends, not a capture from a specific running build.

9. Buy more disk, or prune? A decision table

[COST/RESOURCE]

Every symptom in the table below has one right first response and one tempting wrong one — the wrong one is almost always "add capacity" when the real problem is "nothing has a ceiling."

Table 21.3 — Disk-growth symptom to response. Supports picking the right fix quickly instead of defaulting to "buy a bigger drive," which the Build Autopsy in §2 shows only delays the same problem.

Symptom	Likely cause	Right response	Wrong response
One VM's disk fills every few weeks, same VM each time	No retention policy on that VM's data (SIEM index or PCAP)	Apply §4 or §5's retention mechanism to that specific component	Extending that VM's virtual disk again
Host-level <code>pvesm status</code> usage climbs steadily with no new VMs added	Snapshot sprawl (§7)	Audit and prune snapshots older than their confirmed-working window	Adding a second storage pool
SIEM dashboards feel slow, disk usage looks fine	Disk I/O, not capacity — likely spinning disk under indexing load (Part 3, §5)	Move the active index to SSD; keep only cold archive on spinning disk	Buying more RAM
A specific honeynet decoy's log volume spiked overnight	Real scanning activity, not a misconfiguration	Confirm via the correlation logic in Part 15, then decide if it's worth extending that decoy's retention	Assuming it's a bug and restarting the service
Root filesystem crosses 85% with months of runway left at current growth	Normal, expected growth against a fixed disk	Nothing urgent — confirm §8's monitor is watching it and plan the next tier upgrade on your own schedule	Panicking and deleting data you haven't reviewed

[COST/RESOURCE] When a genuine capacity upgrade is the right call — a Tier-2 box (Part 3) whose SSD is simply full of retained data you've decided is worth keeping, not ungoverned growth — the incremental cost is modest at home-lab scale: a used 1TB SATA SSD for anything actively indexing runs roughly \$40–60, and a 2–4TB spinning HDD suitable for the cold-archive tier in Table 21.2 runs roughly \$20–30 per terabyte used. Buying that disk after a retention policy is in place buys you real, bounded headroom. Buying it before one is in place buys you the same conversation again in a few weeks, per §2's Build Autopsy.

10. What this retention policy feeds downstream

[CONCEPT] A retention policy isn't just a disk-management chore — it's a decision about how far back the rest of the NESHBOY series can reach into what your lab produced, so it's worth naming the handoff explicitly.

Detection Engineering Handbook V2's threat-hunting content (its Parts 34–36) assumes you have enough retained history to hunt across; §6's recommendation to extend honeynet and endpoint log retention well past the SIEM-index default exists specifically so a hunting exercise from that book isn't blocked by a policy this book set too conservatively. If a hunt asks you to look back 60 days and your policy only kept 30, the fix is this part, not that one.

SOC Playbook Handbook's playbooks assume the evidence an incident produced is still available when you work through it — a playbook that asks you to pull the original PCAP for a session your honeynet flagged three days ago fails, not because the playbook is wrong, but because §5's default 72-hour ring buffer already overwrote it. Decide your PCAP retention window with a specific playbook-practice cadence in mind, not just a disk-space instinct, if you plan to run Part 19's purple-team drills against Playbook Handbook procedures on a delay longer than a few days.

SOC Manager's Operating Handbook's budget-justification framing (its Part 20) is a near-exact mirror of the arithmetic in §3 and §9 here, just at a different scale: "we need X gigabytes a month to retain Y days of history for hunting and drills, costing \$Z, and here's what breaks below that" is the same sentence structure that book teaches for justifying a real SOC's storage line item to a real budget owner. The retention math you did in this part for your own lab is a small, low-stakes rehearsal of that exact argument — keep your numbers from §3 and §9 the same way Part 3 suggested keeping your hardware-tier numbers, because the next time you need to make this case for real, you'll already have run it once.

Cross-references: Part 3 (Hardware and Resource Budgeting) for the hardware tiers and the host this part's real-lab example continues from; Part 4 (Network Isolation and Segmentation Architecture) and Appendix A4 (The Safety & Isolation Pre-Flight Checklist) for the isolation controls this part's Safety Gate depends on when attaching any archive target; Part 5 (Choosing and Installing a Hypervisor) for the Proxmox commands in §7; Part 8 (Building the SIEM Platform) for the SIEM platform choice §4's retention mechanisms attach to; Part 9 (Linux Endpoints and auditd Deployment) and Part 10 (Windows Endpoints and Sysmon Deployment) for the endpoint telemetry sized in §3 and §6; Part 12 (Packet Capture and Traffic Visibility Fundamentals), Part 13 (Deploying Zeek), and Part 14 (Deploying Suricata) for the capture point §5's PCAP rotation reuses; Part 15 (Honeypots and HoneyNet Architecture) for the honeynet logs recommended for extended retention in §6; Part 18 (Safe Attack Simulation) and Part 19 (Generating Practice Telemetry and Purple-Team Drills) for the exercises §5's Lab Note and §10 assume you'll want to reach back for; Part 20 (Lab Maintenance, Patching, and Snapshot/Backup Strategy) for the snapshot discipline whose disk cost §7 covers; Detection Engineering Handbook V2, Parts 34–36 for the hunting depth §6 and §10 size retention against; SOC Playbook Handbook's Playbook Library for the evidence-availability assumption §10 flags; SOC Manager's Operating Handbook, Part 20 for the budget-justification argument this part's arithmetic rehearses.

Part 22 — From Lab to Practice: Closing Synthesis and the Series Map

Why this part exists

If you've built even half of what Parts 1 through 21 describe, you have something most security books never get you to: a running environment that produces real telemetry from real, if small, misbehavior — a Linux endpoint tagging `identity`-keyed file writes, a Windows host logging process creation, Zeek and Suricata watching a mirrored port, maybe a honeynet quietly logging unsolicited probes from the open internet. That's the entire promise this book made in Part 1, and by this point it's either kept or it isn't.

What this part does not do is teach you anything new to install. It closes the loop this book has been pointing at since Part 1's own closing section: telemetry sitting in a SIEM that nobody ever runs a Detection Test against, triages through a playbook, or turns into a repeatable drill is a lab that's built but not used — and "built but not used" is a worse outcome than never building it, because it costs you disk, power, and patching effort (Part 20) for nothing back. This chapter walks one complete path, start to finish, from a specific piece of telemetry this book's earlier parts can produce, through a Detection Test in *The Detection Engineering Handbook V2*, through a playbook in *SIGNAL TO ACTION: The Complete SOC Playbook Handbook*, to a repeatable Field Test drill structured per *The SOC Manager's Operating Handbook*. It then hands you the full cross-series lookup table this book has been building toward one hand-off at a time, part by part, so you have one place to find where any specific build output goes next.

Say plainly what this walkthrough is and isn't: it's a worked, illustrative path through the four volumes, built on the real `identity`-key exercise Part 9 already validated on the author's own lab-endpoint pattern — it is not a claim that the author ran an Atomic Red Team technique, wrote a formal Detection Test, worked a playbook, and scored a Field Test drill against this specific book's own environment end to end. Parts 15, 18, and 19 aren't written yet as this chapter goes to draft, and this closing synthesis doesn't get to borrow evidence from parts that don't exist. Where the walkthrough below touches something the author's real lab has actually produced — the honeynet's MITRE-tagged correlated sessions, cited from `BOOK-INDEX.md`'s own Part 15 evidence description — it says so and stays inside what's actually documented there.

Safety Gate *Nothing in this closing chapter loosens any isolation control built earlier in this book. The worked walkthrough in §3–§4 below reuses Part 9's disposable-account exercise, which must run only on a lab endpoint inside the isolated segment from Part 4, firewalled per Part 6, with the Appendix A4 pre-flight checklist already run — never on a host holding a real credential you use elsewhere. If your version of this walkthrough instead uses Part 18's attack-simulation tooling or touches Part 15's honeynet segment, that part's own (expanded) Safety Gate governs and this section doesn't substitute for it. Before you run anything below against your own lab, confirm the specific endpoint you're targeting is one you built for exactly this purpose and that its only network path is the one Part 4 and Part 6 designed — a closing chapter is not the place to get casual about a rule this book has stated as non-negotiable since Part 1.*

1. What "the lab is built" actually means at this point

[CONCEPT] "Built" doesn't mean every part in this book — it means enough of the pipeline exists end to end that a piece of telemetry can travel from an endpoint to a SIEM to a decision. At a minimum, that's a hypervisor (Part 5), a segmented network with a working firewall (Parts 4 and 6), a SIEM that can ingest and search (Part 8), and at least one endpoint producing telemetry into it (Part 9 or Part 10). Everything past that — Zeek and Suricata (Parts 13–14), threat-intel enrichment (Part 16), a honeynet (Part 15), dashboards (Part 17), attack-simulation tooling (Part 18) — adds more telemetry sources and more realism, but the loop this chapter closes works the same way with one endpoint as it does with fifteen.

That matters because a reader partway through this book, or a reader who retrofitted one component onto an existing setup per Part 1's second reading path, doesn't need to wait for Part 21's storage-retention content to be relevant before running this chapter's walkthrough once. The loop below is deliberately sized to the smallest lab this book can produce, not the largest.

[CONCEPT] It's also worth being honest about what "built" doesn't mean: it doesn't mean patched, monitored for its own health, or resourced for anything beyond light, human-driven use. Part 20's maintenance discipline and Part 21's retention policy are what keep a built lab usable six months from now instead of degrading quietly — this chapter assumes those are in place or on your list, not finished forever.

2. The closing loop, in one diagram

[CONCEPT] Before walking each step in prose, it's worth seeing the whole loop at once — one action generating telemetry in this book, three separate hand-offs to the other three NESHBOY volumes, and one feedback edge back into this book's own maintenance and expansion work.

flowchart TB

```
subgraph HLH["This book – SOC Home Lab Handbook"]
    EP["Lab endpoint\n(Part 9 or 10)\nforwarding to SIEM (Part 8)"]
    SIM["Known-cause action\n(Part 9 exercise reused here,\nor Part 18 attack-sim tooling)"]
    EP --> SIM
end
```

end

```
subgraph DEH["Detection Engineering Handbook V2"]
    DT["Detection Test\ndid the detection fire,\nfor the right reason?"]
end
```

end

```
subgraph PB["SOC Playbook Handbook"]
    TRI["Playbook Library\ntrriage the resulting alert"]
end
```

end

```
subgraph MGR["SOC Manager's Operating Handbook"]
    FT["Field Test framing (Part 8)\nscore it, formalize cadence"]
end
```

end

```
SIM -->|"event lands in SIEM"| DT
DT -->|"fired correctly"| TRI
DT -. ->|"fired wrong, or not at all"| TUNE["Back to Detection Engineering\nHandbook V2's own
tuning content"]
TRI --> FT
FT -->|"repeat on a schedule"| SIM
```

```
classDef here fill:#2b3a55,stroke:#7fa8d9,color:#fff
```

```
classDef other fill:#3a3a3a,stroke:#888,color:#eee
```

```
class EP,SIM here
```

```
class DT,TRI,FT,TUNE other
```

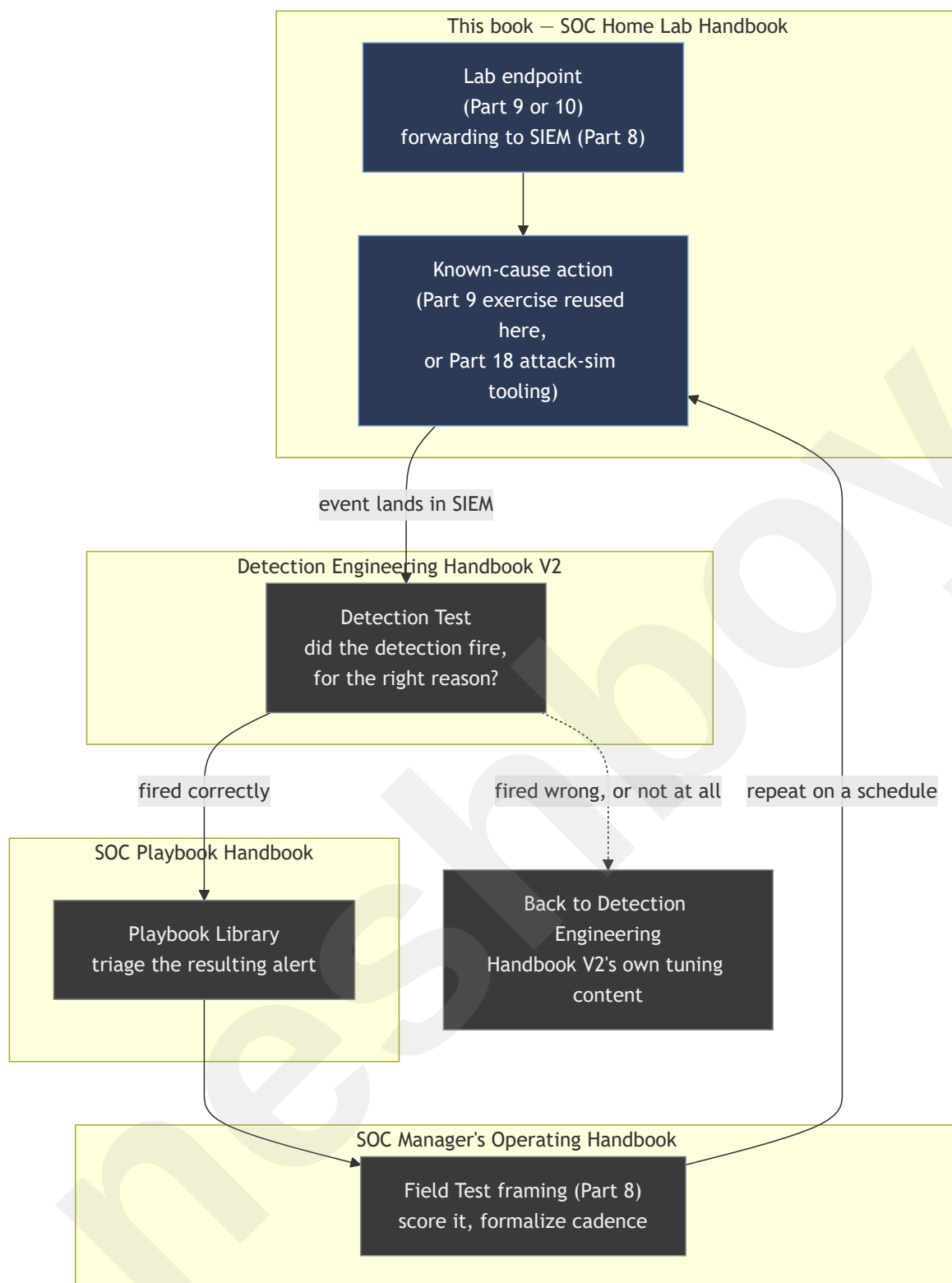


Figure 22.1 — The closing loop across all four NESHBOY volumes. CONCEPTUAL. Illustrates the hand-off sequence this chapter walks in prose: a lab endpoint built in this book produces telemetry from a known-cause action, a Detection Test in *The Detection Engineering Handbook V2* confirms whether a detection fired for the right reason, a fired alert gets worked through *SOC Playbook Handbook*'s Playbook Library, and *The SOC Manager's Operating Handbook*'s Field Test framing turns the whole run into a scored, repeatable drill that eventually schedules another pass through the same

loop. This is a process diagram, not a capture of a specific run — no claim is made that this exact sequence has been executed end to end against this book's own lab as of this writing.

The loop's honest failure path matters as much as its success path: if the Detection Test in §5 shows the detection didn't fire, or fired for the wrong reason, the loop doesn't continue to the playbook step — it goes back into Detection Engineering Handbook V2's own tuning content first. Working a playbook against an alert that never should have fired teaches you nothing about triage; it teaches you to distrust an alert that was simply built wrong.

3. Step one — pick a telemetry source and confirm it's still alive

[HANDS-ON LAB] Goal: before generating anything new, confirm the pipeline you're about to exercise is actually intact, since a stale or drifted lab is the single most common reason this chapter's walkthrough produces nothing.

1. Pick one endpoint you already built — Part 9's Linux endpoint is the easiest starting point if you have it, because its `identity` key (introduced in Part 9, tagging writes to `/etc/passwd`, `/etc/shadow`, and `/etc/group`) is already wired end to end from a real `useradd`/`userdel` exercise.
2. Confirm the endpoint is powered on and still forwarding: run `sudo systemctl status auditd rsyslog` on the endpoint itself.
3. Confirm the SIEM is still receiving from it — search for any event from that host in roughly the last 24 hours.

If step 3 comes back empty, stop here and fix the forwarding path (Part 9, §6) before doing anything below — generating a known-cause event against a pipeline that's already broken just gives you a second thing to debug at once.

Lab Note *This is the one exercise in this book worth running on a calendar, not just once. A lab left alone for a few weeks drifts — a certificate expires, a forwarder silently stops, a disk fills per Part 21's own warning — and the fastest way to catch that isn't watching dashboards passively, it's running this three-step check before every drill. Five minutes here saves you from discovering the break in the middle of §4, when it looks like the detection failed instead of the plumbing.*

4. Step two — generate known-cause telemetry

[HANDS-ON LAB] With the pipeline confirmed alive, generate one specific, known-cause event and note exactly what you did and when, so the Detection Test in §5 has a ground truth to check against. The commands below are the same disposable-account action Part 9's own Validation Test used — reused here for a different purpose: not confirming the audit pipeline works, but generating a real trigger to run a detection against.

This reuses Part 9's own disposable-account sequence verbatim, targeting the same Debian/Ubuntu `auditd` baseline Part 9 built — it is a real, run-it-as-shown command sequence standing in for a Part 18

attack-simulation technique, not an illustrative/invented sample.

```
# stand-in for a Part 18 attack-simulation technique; run only on the disposable
# lab endpoint from Part 9, per this chapter's Safety Gate.
sudo useradd -m closingdrill
sudo ausearch -k identity -ts recent
sudo userdel -r closingdrill
```

If you've built Part 18's attack-simulation tooling instead, run a scoped technique from it against the same endpoint rather than the account-creation stand-in above — MITRE ATT&CK's T1136.001 (Create Account: Local Account) is the technique this stand-in actually maps to, and Part 18's tooling likely has a named, reversible test for exactly that technique. Either way, the discipline is the same: one action, one timestamp, one expected event shape, so the next step has something concrete to confirm against.

[SAFETY] Whichever action you pick, it must be reversible and confined to the disposable lab endpoint — this chapter's Safety Gate restates that explicitly because a closing chapter is exactly the point in a book where a reader relaxes and runs "just one more thing" against whatever's closest to hand, which is precisely how isolation gets skipped in practice rather than in theory.

A Windows-endpoint reader running the equivalent exercise against Part 10's build would generate Event ID 4720 (A user account was created) instead of an `identity`-keyed auditd record — the loop below works identically either way; only the specific event shape changes.

5. Step three — run a Detection Test in Detection Engineering Handbook V2

[CONCEPT] This is the point where this book's job formally ends for this specific event, and *The Detection Engineering Handbook V2* picks up. That book's own Detection Test callout — the pattern this book's Validation Test callout is directly adapted from, per `BOOK-INDEX.md`'s provenance notes — asks one question with a checkable answer: given the event you just generated, does a detection rule fire, and does it fire for the reason you think it does?

Concretely, that means taking the `identity`-keyed event from §4 into whatever correlation or rule logic Detection Engineering Handbook V2's Parts 22–33 teach you to build on top of Part 9's raw telemetry, and checking three things: did an alert fire at all, did it fire within a reasonable window of the actual action, and does the alert's own fields (the account name, the host, the timestamp) match what you actually did rather than some other coincidental activity on the same host. A detection that fires on the right host at the wrong time, or the right time on the wrong host, has failed this test just as much as one that never fires — "it triggered" isn't the same claim as "it triggered for the right reason."

Validation Test Setup: The §4 action already run and confirmed present via `ausearch -k identity` (or the equivalent Event ID 4720 search), and a detection rule built per Detection Engineering Handbook V2's baselining/correlation content pointed at the same SIEM index.

Action: Search the SIEM for both the raw `identity`-keyed event and any alert your detection logic generated in the same window. **Expected result:** One alert, timestamped within roughly a minute of the raw event, carrying the same hostname and account name as the action in §4. Zero alerts, or an alert with mismatched fields, means the detection — not this chapter's walkthrough — needs the tuning content the other book owns; don't proceed to §6 with a detection you haven't confirmed is actually correct.

[CONCEPT] If your lab doesn't yet have a detection rule built against this specific telemetry, that's a legitimate stopping point for this pass of the loop, not a failure — Detection Engineering Handbook V2's Parts 22–33 are where you'd build one, and this chapter's job was only ever to hand you a confirmed, known-cause event to build and test that rule against.

6. Step four — work the alert through a SOC Playbook Handbook playbook

[CONCEPT] Once §5 confirms a real, correctly-fired alert, the next hand-off is to *SIGNAL TO ACTION*: *The Complete SOC Playbook Handbook's* Playbook Library — the volume that owns what an analyst actually does once an alert like this lands in a queue. This book stops at "the alert fired correctly"; it has no content on scoping, containment, or escalation decisions, and it shouldn't grow any, because that's a different, much deeper discipline with its own volume already covering it.

Concretely, the playbook exercise looks like this: treat the alert from §5 as if it landed in a real queue with no other context, and work it the way the relevant SOC Playbook Handbook playbook instructs — pull the account-creation event's surrounding context (what else happened on that host in the preceding few minutes), determine whether the account persisted or was cleaned up, and decide whether the activity, taken at face value with no foreknowledge that you caused it, would warrant escalation. This is also the point where a home-lab drill most resembles real analyst work, because the triage judgment itself doesn't care whether the underlying event was real or simulated — only the ground truth you're checking your judgment against does.

[TROUBLESHOOTING] A specific trap worth naming: because you know you caused the alert, it's tempting to skip straight to "yes, that's expected, close it" without actually working the playbook's steps. Resist that — the entire value of running a known-cause drill is practicing the triage motion under conditions where you can grade yourself against ground truth, and skipping the motion because you already know the answer defeats the exercise as thoroughly as skipping the isolation checklist defeats a honeypot.

7. Step five — formalize the loop as a repeatable Field Test drill

[CONCEPT] A single pass through §3–§6 is a useful exercise once. Repeating it on a schedule, with a way to compare this month's run against last month's, is what turns a home lab from "a thing I built"

into "a thing that keeps me sharp" — and that formalization is *The SOC Manager's Operating Handbook's* job, specifically its Part 8 Field Test framing, cited throughout this book's earlier parts as the destination for exactly this kind of drill.

Field Test framing (per `manager:part08`) asks you to fix, in advance, what a "pass" looks like — a maximum time from alert to triage decision, a specific set of context fields you're expected to pull, a defined escalation threshold — so that a repeated drill produces a comparable score instead of a vague sense of "that went fine." Table 22.2 below adapts that framing down to a single-operator scale, since the Manager's Handbook's own version assumes a team running the drill against each other, not one person running it against themselves.

Table 22.2 — Adapting Field Test framing to a solo-operator drill. The table below supports deciding what to keep, simplify, or drop from a team-scale Field Test when only one person is running it.

Drill element	Team-scale Field Test (Manager's Handbook)	Solo-operator adaptation (this book)
Scope/technique	Assigned per analyst, often blind	Self-selected from Table 22.1's telemetry sources, logged before running
Success criteria	Fixed rubric scored by a second reviewer	Fixed checklist (§5's Validation Test + §6's playbook steps), self-scored against a written answer key made before the run
Timing	Time-to-triage tracked per analyst, compared across the team	Time-to-triage tracked per run, compared against your own prior runs
Reporting	Rolled up to a manager for team-capability tracking	A dated log entry — technique run, detection fired (yes/no), triage time, one line on what you'd change
Cadence	Scheduled by the manager, often quarterly	Monthly is realistic for a hobbyist lab; tie it to Part 20's own patch-cadence reminder so both happen in the same sitting
Blind vs. known scope	Usually blind, to test real detection capability	Known on purpose here — the goal is confirming the pipeline and practicing triage motion, not testing whether you can surprise yourself

[CONCEPT] The last row is worth dwelling on. A team-scale Field Test often works best blind, because the point is testing whether an analyst who didn't design the detection can still work it correctly. A solo drill against your own lab can't be meaningfully blind — you already know what you did in §4 — so the adaptation's honest goal is narrower: confirm the pipeline stays alive over time (§3), and keep the triage motion itself from atrophying (§6), not simulate the experience of a real unknown alert. If you want that experience specifically, Part 15's honeynet is the better source, since its alerts are genuinely unsolicited and you don't know in advance what's going to show up in them.

Lab Note *Keep the answer-key log from Table 22.2's reporting row in something durable — a plain text file next to your Part 20 maintenance log is fine. Six months of "technique, detection fired, triage time" entries is the only real evidence you have that the lab is doing its job, and it's a more useful thing to point to in an interview or a portfolio than a screenshot of a dashboard with no history behind it.*

8. The full cross-series quick-reference table

[CONCEPT] Every part in this book has named its specific hand-off to one of the other three volumes at the point it applied. Table 22.1 collects those hand-offs into one lookup, organized by what this book's part actually produces — use it as the fast path when you've built something and want to know immediately where to take its output, and use Appendix A5 when the question is broader than "what does this specific part's telemetry feed into."

Table 22.1 — Cross-series quick reference: what to do with each part's output. The table below supports deciding which NESHBOY volume and part to open next once a given piece of this book's build is producing telemetry.

This book's part	What it produces	Detection Engineering Handbook V2	SOC Playbook Handbook	SOC Manager's Operating Handbook
Part 7 (Internal DNS)	DNS query/sinkhole logs	deh:part15 (DNS-based detection content)	Playbook Library — suspicious-domain triage	—
Part 9 (Linux/auditd)	identity, sudoers, sshd_config, exec - keyed events	deh:part03, deh:part31 (telemetry schema, baselining)	Linux logging technical-reference chapters	manager:part08 (drill design, this part)
Part 10 (Windows/Sysmon)	Sysmon Event ID 1 (Process Create) and related process/network telemetry	deh:part08, deh:part09 (Sysmon detection engineering)	Windows logging technical-reference chapters	manager:part08
Part 11 (domain lab problem)	Kerberos/AD-auth telemetry, if the Option B exercise is run	deh:part13 (Kerberos/AD detection logic)	Playbook Library — domain-authentication alert triage	manager:part20 (budget framing for AD-lab maintenance cost)
Part 13 (Zeek)	conn, dns, http, ssl, notice logs	deh:part14, deh:part15 (NSM detection content)	Playbook Library — network-anomaly triage	—
Part 14 (Suricata)	Rule-fired alerts	deh:part14, deh:part15	Playbook Library — signature-alert triage	—
Part 15 (honeynet)	Correlated, MITRE-tagged unsolicited attack sessions	Detection/hunt content on real attacker behavior, not simulated	Playbook Library — full incident triage as if the target were real	manager:part08 (blind, unscoped drill source)
Part 16 (threat intel)	Enriched/reputation-scored events	deh:part32 (threat-intel scoring judgment calls)	Playbook Library — enrichment-informed triage	—
Part 18 (attack simulation)	Known-cause technique execution, MITRE-tagged	This chapter's Detection Test pattern (§5)	This chapter's playbook pattern (§6)	manager:part08 (Field Test framing, this chapter's §7)
Part 19 (purple-team drills)	Paired action-plus-detection-plus-playbook runs	Direct input to the Detection Test loop	Direct input to the triage loop	Direct input to Field Test scoring

[CONCEPT] Two rows have an em dash in the SOC Manager's Operating Handbook column, and that's deliberate, not an omission — not every telemetry source in this book has a natural team-process hand-off. DNS query logs and NSM alerts are inputs to detections and triage; they don't, on their own, generate a distinct staffing or drill-design conversation the way a honeypot session or an attack-simulation run does. A blank cell in this table would look like an oversight; the dash says the gap was checked and is real.

9. What a solo home lab still can't teach you

[CONCEPT] This closing chapter's whole argument is that a well-built home lab can carry you meaningfully far into real SOC skill practice. It's worth being equally direct about where that argument stops, because overselling a hobbyist lab's realism is its own kind of dishonesty — the same discipline Part 11 applied to the Windows domain problem applies here to the whole book's closing claim.

Blind Spot *A solo home lab, however completely built, cannot teach you what it feels like to triage a real alert inside a queue of forty other alerts of unknown severity, half of them false positives, while a second analyst is also working the same incident and a manager is asking for a status update. Every drill in this chapter is run with full ground truth and no competing demands on your attention — that's what makes it useful for practicing a specific motion cleanly, and it's exactly what makes it a poor simulation of real SOC pressure, alert volume, or team coordination. The SOC Manager's Operating Handbook's own content on staffing and shift design exists because that pressure is a real, distinct problem this book's lab was never built to reproduce.*

[CONCEPT] A second, narrower gap: Part 15's honeynet is the one part of this book that generates genuinely unscoped, unsolicited attacker behavior, and even that has a ceiling — real opportunistic scanning traffic, not a targeted campaign against you specifically, and nothing resembling the scale or sophistication of what a real, staffed SOC's perimeter sees in a day. If your goal after this book is "practice against realistic adversary behavior at realistic volume," this lab gets you the mechanics and the triage motion; it doesn't get you the volume, and no single-operator hardware budget from Part 3 ever will.

10. Keeping the loop running

[CONCEPT] The loop in Figure 22.1 isn't a one-time closing exercise — it's the thing this whole book was built to make possible repeatedly, cheaply, on your own schedule. Three practical habits keep it running instead of becoming a thing you did once for this chapter and never again:

- Run the §3 pipeline check before every drill, not just the first one — it's five minutes, and it's the difference between debugging a real gap and debugging a stale lab.
- Log every drill per Table 22.2's reporting row, even the ones where nothing interesting happened — a string of "ran clean" entries is itself useful evidence that the pipeline has stayed healthy across Part 20's patching cycles.
- Expand the loop's inputs as you build more of this book, not just its outputs — a lab with only Part 9's Linux endpoint gets one drill variant; a lab with Part 15's honeynet and Part 18's attack-simulation tooling added gets a genuinely different, more valuable drill without any change to the loop itself in Figure 22.1.

What Would Change My Mind This chapter's central claim is that a single-operator home lab, run through the §3–§7 loop on a monthly cadence, meaningfully builds and maintains real SOC-relevant skill without needing a team, a budget, or a real employer's environment. If a reader ran this loop faithfully for six months and reported no measurable improvement in triage speed or detection-tuning judgment against their own Table 22.2 log — not against someone else's opinion of "meaningfully," against their own recorded numbers — that would be a direct, specific challenge to this chapter's argument, and it would need a documented revision here, not a quiet shrug that home labs are "just for fun."

11. Closing: the series map, one more time

[CONCEPT] This book opened, in Part 1, with a promise: it builds the environment; the other three volumes teach you what to do with what it produces. Twenty-two parts later, that promise is what this chapter has walked once, concretely, rather than just restated. The table in §8 above is this book's own version of the lookup Part 1 previewed and Appendix A5 expands exhaustively — treat Appendix A5 as the reference you return to as your lab grows past what §8's ten rows cover, and treat §8 as the fast path for the common case.

If you're finishing this book having built only a fraction of it — a SIEM and one Linux endpoint, say — that's still enough to run the loop in this chapter today. The rest of this book, and the three volumes it hands off to, are there for when one endpoint's worth of telemetry stops being enough to keep the loop interesting. Building the lab was never the point; running the loop in Figure 22.1 against it, repeatedly, is.

Cross-references: Part 1 (the series map and hand-off list this chapter closes out), Part 4 and Part 6 (isolation and firewall controls this chapter's Safety Gate depends on), Appendix A4 (pre-flight checklist referenced by this chapter's Safety Gate), Part 9 (the disposable-account exercise reused as this chapter's worked example), Part 10 (Windows-endpoint equivalent of the same exercise), Part 11 (the honesty standard this chapter's §9 applies to its own closing claim), Part 15, Part 18, and Part 19 (unscoped honeynet signal, attack-simulation tooling, and purple-team drill content this chapter's loop assumes exists once built), Part 20 (maintenance cadence this chapter ties its drill schedule to), Part 21 (retention policy assumed healthy for the pipeline check in §3), Appendix A5 (the exhaustive version of this chapter's §8 table); [deh:part03](#), [deh:part08](#), [deh:part09](#), [deh:part13](#), [deh:part14](#), [deh:part15](#), [deh:part31](#), [deh:part32](#), [deh:part33](#) (Detection Test and detection-logic content this chapter hands off to); SOC Playbook Handbook's Playbook Library and Linux/Windows logging technical-reference chapters (triage procedures this chapter hands off to); [manager:part08](#) (Field Test framing this chapter adapts in §7), [manager:part20](#) (budget framing referenced via Part 11).